Java Source Code Compilation

Total Files: 510

---

1. `common\ArrayUtils.java`

```
package leetcode.common;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collection;
import java.util.List;
import java.util.stream.Collectors;

public class ArrayUtils {

    public static int[] stringToIntArray(String str) {
        if (str == null || str.isEmpty()) {
            return new int[0];
        }

        // Remove the brackets and split the string by comma
        String[] strArray = str.replace("[", "").replace("]", "").split(",");

        // Convert each element to an integer
        int[] intArray = new int[strArray.length];
        for (int i = 0; i < strArray.length; i++) {
            intArray[i] = Integer.parseInt(strArray[i].trim());
        }

        return intArray;
    }

    public static List<Integer> stringToIntList(String str) {
        final int[] ints = stringToIntArray(str);
        if (ints.length == 0) {
            return List.of();
        }
        List<Integer> list = new ArrayList<>(ints.length);
        for (int anInt : ints) {
            list.add(anInt);
        }
        return list;
    }

    public static List<String> stringToStringList(String str) {
        String[] strArray = stringToStringArray(str);

        return Arrays.asList(strArray);
    }

    public static String[] stringToStringArray(String str) {
        if (str == null || str.isEmpty() || str.equals("[]")) {
            return new String[0];
        }

        return str.replaceAll("[\$$
```

```
\
$$\"]", "").split(",");
    }

    public static String stringArrayToString(String[] strArray) {
        return Arrays.stream(strArray).collect(Collectors.joining("\",\"", "[\"",
"\"]"));
    }

    public static String stringCollectionToString(Collection<String> stringCollection) {
        if (stringCollection.isEmpty()) {
            return "[]";
        }
        return stringCollection.stream().collect(Collectors.joining("\",\"", "[\"",
"\"]"));
    }

    public static String intArrayToString(int[] intArray) {
        return
Arrays.stream(intArray).mapToObj(String::valueOf).collect(Collectors.joining(",", "[",
"]"));
    }

    public static int[][] stringToInt2DArray(String str) {
        // Remove the outer brackets and split the string by "],["
        String strippedOuter = str.substring(1, str.length() - 1);
        if (strippedOuter.isEmpty()) {
            return new int[0][0];
        }
        strippedOuter = strippedOuter.replaceAll("\\s", "");
        String[] strRows = strippedOuter.split("\\],\$$
");

        // Convert each row string to an integer array
        int[][] matrix = new int[strRows.length][];
        for (int i = 0; i < strRows.length; i++) {
            matrix[i] = ArrayUtils.stringToIntArray("[" + strRows[i] + "]");
        }

        return matrix;
    }

    public static String int2DArrayToString(int[][] matrix) {
        return Arrays.stream(matrix)
                .map(ArrayUtils::intArrayToString)
                .collect(Collectors.joining(",", "[", "]"));
    }

    public static List<List<Integer>> stringToListOfLists(String str) {
        int[][] matrix = stringToInt2DArray(str);
        List<List<Integer>> matrixList = new ArrayList<>(matrix.length);
        for (int[] arr : matrix) {
            matrixList.add(Arrays.stream(arr).boxed().toList());
        }
```

```
        return matrixList;
    }

    public static String listOfListsToString(List<List<Integer>> matrixList) {
        String standardToString = matrixList.toString();
        return standardToString.replace(" ", "");
    }


    public static char[] stringToCharArray(String lettersStr) {
        return lettersStr.replaceAll("[\\[\
$$\",]", "").toCharArray();
    }

    public static String charArrayToString(char[] charArray) {
        StringBuilder sb = new StringBuilder();
        sb.append('[');
        for (int i = 0; i < charArray.length; i++) {
            if (i > 0) {
                sb.append(',');
            }
            char c = charArray[i];
            sb.append('"')
                    .append(c)
                    .append('"');
        }
        sb.append(']');
        return sb.toString();
    }

    public static char[][] stringToChar2DArray(String str) {
        // Remove the outer brackets and split the string by "],["
        String strippedOuter = str.substring(1, str.length() - 1);
        if (strippedOuter.isEmpty()) {
            return new char[0][0];
        }
        strippedOuter = strippedOuter.replaceAll("\\s", "");
        String[] strRows = strippedOuter.split("\\],\\[");

        // Convert each row string to a char array
        char[][] matrix = new char[strRows.length][];
        for (int i = 0; i < strRows.length; i++) {
            matrix[i] = ArrayUtils.stringToCharArray("[" + strRows[i] + "]");
        }

        return matrix;
    }

    public static String char2DArrayToString(char[][] matrix) {
        return Arrays.stream(matrix)
                .map(ArrayUtils::charArrayToString)
                .collect(Collectors.joining(",", "[", "]"));
    }
```

```
    public static double[] stringToDoubleArray(String str) {
        if (str == null || str.isEmpty()) {
            return new double[0];
        }

        // Remove the brackets and split the string by comma
        String[] strArray = str.replace("[", "").replace("]", "").split(",");

        // Convert each element to a double
        double[] doubleArray = new double[strArray.length];
        for (int i = 0; i < strArray.length; i++) {
            doubleArray[i] = Double.parseDouble(strArray[i].trim());
        }

        return doubleArray;
    }

}
```

2. `common\ListNode.java`

```
package leetcode.common;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class ListNode {

    public boolean safeToString = false;
    public int val;
    public ListNode next;

    public ListNode() {
    }

    public ListNode(int val) {
        this.val = val;
    }

    public ListNode(int val, ListNode next) {
        this.val = val;
        this.next = next;
    }

    public static ListNode of(String input) {
        int[] intArr = ArrayUtils.stringToIntArray(input);
        return of(intArr);
    }

    public static ListNode of(int... array) {
        ListNode head = new ListNode(array[0]);
        ListNode current = head;

        for (int i = 1; i < array.length; i++) {
            current.next = new ListNode(array[i]);
            current = current.next;
        }

        return head;
    }

    @Override
    public String toString() {
        if (safeToString) {
            return "[" +
                   val +
                   ",...]";
        }
        var head = this;
        List<Integer> values = new ArrayList<>();
        while (head != null) {
```

```
            values.add(head.val);
            head = head.next;
        }
        return "[" +
values.stream().map(String::valueOf).collect(Collectors.joining(",")) + "]";
    }

    @Override
    public boolean equals(Object obj) {
        if (obj == null || getClass() != obj.getClass()) {
            return false;
        }
        ListNode other = (ListNode) obj;
        ListNode current = this;
        ListNode otherCurrent = other;
        while (current != null && otherCurrent != null) {
            if (current.val != otherCurrent.val) {
                return false;
            }
            current = current.next;
            otherCurrent = otherCurrent.next;
        }
        return current == null && otherCurrent == null;
    }

    @Override
    public int hashCode() {
        return val;
    }

}
```

3. `common\Node.java`

```
package leetcode.common;

import java.util.ArrayList;
import java.util.List;

public class Node {
    public int val;
    public List<Node> neighbors;

    public Node() {
        val = 0;
        neighbors = new ArrayList<>();
    }

    public Node(int _val) {
        val = _val;
        neighbors = new ArrayList<>();
    }

    public Node(int _val, Node node1, Node node2) {
        val = _val;
        neighbors = new ArrayList<>();
        neighbors.add(node1);
        neighbors.add(node2);
    }
}
```

4. `common\TreeNode.java`

```
package leetcode.common;

import java.util.LinkedList;
import java.util.Queue;

public class TreeNode {

    public int val;
    public TreeNode left;
    public TreeNode right;

    TreeNode() {
    }

    public TreeNode(int val) {
        this.val = val;
    }

    public TreeNode(int val, TreeNode left, TreeNode right) {
        this.val = val;
        this.left = left;
        this.right = right;
    }

    public static TreeNode of(String treeStr) {
        if (treeStr == null || treeStr.isEmpty()) {
            return null;
        }

        String[] values = treeStr.replace("[", "").replace("]", "").split(",");
        if (values.length == 0 || values[0].equals("null") || values[0].isEmpty()) {
            return null;
        }

        TreeNode root = new TreeNode(Integer.parseInt(values[0]));
        Queue<TreeNode> queue = new LinkedList<>();
        queue.offer(root);

        for (int i = 1; i < values.length; i += 2) {
            TreeNode parent = queue.poll();

            if (!values[i].equals("null")) {
                TreeNode left = new TreeNode(Integer.parseInt(values[i]));
                parent.left = left;
                queue.offer(left);
            }

            if (i + 1 < values.length && !values[i + 1].equals("null")) {
                TreeNode right = new TreeNode(Integer.parseInt(values[i + 1]));
                parent.right = right;
                queue.offer(right);
```

```
            }
        }

        return root;
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder();
        sb.append("[");
        Queue<TreeNode> queue = new LinkedList<>();
        queue.offer(this);

        while (!queue.isEmpty()) {
            TreeNode node = queue.poll();
            if (node != null) {
                sb.append(node.val);
                queue.offer(node.left);
                queue.offer(node.right);
            } else {
                sb.append("null");
            }
            if (!queue.isEmpty()) {
                sb.append(",");
            }
        }

        // Remove trailing nulls
        while (sb.length() >= 5 && sb.lastIndexOf(",null") == sb.length() - 5) {
            sb.delete(sb.length() - 5, sb.length());
        }

        sb.append("]");
        return sb.toString();
    }

    private void printNode(TreeNode node, int totalSpaces, StringBuilder sb) {
        int spaces = totalSpaces * 2;
        sb.repeat(" ", spaces - 1);
        if (node != null) {
            sb.append(node.val);
        } else {
            sb.append(" ");
        }
        sb.repeat(" ", spaces);
    }

}
```

5. `Solution00001.java`

```
package leetcode;

import java.util.HashMap;
import java.util.Map;

/**
 * https://leetcode.com/problems/two-sum/
 * 1. Two Sum
 * Given an array of integers nums and an integer target, return indices of the two
numbers such that they add up to target.
 * You may assume that each input would have exactly one solution, and you may not use
the same element twice.
 * You can return the answer in any order.
 *
 * Example 1:
 *
 * Input: nums = [2,7,11,15], target = 9
 * Output: [0,1]
 * Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].
 * Example 2:
 *
 * Input: nums = [3,2,4], target = 6
 * Output: [1,2]
 * Example 3:
 *
 * Input: nums = [3,3], target = 6
 * Output: [0,1]
 *
 * Constraints:
 *
 * 2 <= nums.length <= 104
 * -109 <= nums[i] <= 109
 * -109 <= target <= 109
 * Only one valid answer exists.
 *
 * Follow-up: Can you come up with an algorithm that is less than O(n2) time complexity?
 */
public class Solution00001 {

    // Time: O(n^2)
    // Space: O(1)
    public int[] twoSumBruteForce(int[] nums, int target) {
        // It is guaranteed that array has at least two elements.

        for (int i = 0; i < nums.length; i++) {
            int current = nums[i];
            for (int j = i + 1; j < nums.length; j++) {
                int next = nums[j];
                if (current + next == target) {
                    return new int[]{i, j};
                }
```

```
            }
        }

        return new int[]{};
    }

    // Time: O(n)
    // Space: O(n)
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int current = nums[i];
            // current + complement = target
            int complement = target - current;
            if (map.containsKey(complement)) {
                return new int[] {map.get(complement), i};
            } else {
                map.put(current, i);
            }
        }
        return new int[] {};
    }

}
```

6. `Solution00002.java`

```
package leetcode;

import leetcode.common.ListNode;

/**
 * https://leetcode.com/problems/add-two-numbers/
 * 2. Add Two Numbers
 * <p>
 * You are given two non-empty linked lists representing two non-negative integers.
 * The digits are stored in reverse order, and each of their nodes contains a single
digit.
 * <p>
 * Add the two numbers and return the sum as a linked list.
 * <p>
 * You may assume the two numbers do not contain any leading zero,
 * except the number 0 itself.
 * <p>
 * <p>
 * Example 1:
 * <p>
 * <p>
 * Input: l1 = [2,4,3], l2 = [5,6,4]
 * Output: [7,0,8]
 * Explanation:
 * 342 + 465 = 807
 * <p>
 * Example 2:
 * <p>
 * Input: l1 = [0], l2 = [0]
 * Output: [0]
 * <p>
 * Example 3:
 * <p>
 * Input: l1 = [9,9,9,9,9,9,9], l2 = [9,9,9,9]
 * Output: [8,9,9,9,0,0,0,1]
 * <p>
 * <p>
 * Constraints:
 * <p>
 * The number of nodes in each linked list is in the range [1, 100].
 * <p>
 * 0 <= Node.val <= 9
 * <p>
 * It is guaranteed that the list represents a number that does not have leading zeros.
 */
public class Solution00002 {

    // Time: O(n)
    // Space: O(n)
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        // Create a dummy node to store the head of the result linked list.
```

```
        ListNode dummy = new ListNode();
        // Create a pointer to the current node in the result linked list.
        ListNode current = dummy;
        // Create a variable to store the carry.
        int carry = 0;

        // Iterate over the linked lists l1 and l2 until we reach the end of both lists.
        while (l1 != null || l2 != null) {
            // Create a variable to store the sum of the current nodes in l1 and l2.
            int sum = carry;
            // If the current node in l1 is not null, add its value to the sum.
            if (l1 != null) {
                sum += l1.val;
                // Move the pointer to the next node in l1.
                l1 = l1.next;
            }
            // If the current node in l2 is not null, add its value to the sum.
            if (l2 != null) {
                sum += l2.val;
                // Move the pointer to the next node in l2.
                l2 = l2.next;
            }
            // Update the carry.
            carry = sum / 10;
            // Update the current node in the result linked list.
            current.next = new ListNode(sum % 10);
            // Move the pointer to the next node in the result linked list.
            current = current.next;
        }

        // If the carry is not zero, add a new node to the result linked list.
        if (carry != 0) {
            current.next = new ListNode(carry);
        }

        // Return the head of the result linked list.
        return dummy.next;
    }

}
```

7. `Solution00003.java`

```
package leetcode;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * https://leetcode.com/problems/longest-substring-without-repeating-characters/
 * 3. Longest Substring Without Repeating Characters
 * <p>
 * Given a string s, find the length of the longest substring without repeating
characters.
 * <p>
 * Example 1:
 * <p>
 * Input: s = "abcabcbb"
 * Output: 3
 * <p>
 * Explanation: The answer is "abc", with the length of 3.
 * <p>
 * Example 2:
 * <p>
 * Input: s = "bbbbb"
 * Output: 1
 * <p>
 * Explanation: The answer is "b", with the length of 1.
 * <p>
 * Example 3:
 * <p>
 * Input: s = "pwwkew"
 * Output: 3
 * <p>
 * Explanation: The answer is "wke", with the length of 3.
 * Notice that the answer must be a substring, "pwke" is a subsequence and not a
substring.
 * <p>
 * Example 4:
 * <p>
 * Input: s = ""
 * Output: 0
 * <p>
 * Constraints:
 * <p>
 * 0 <= s.length <= 5 * 104
 * s consists of English letters, digits, symbols and spaces.
 */
public class Solution00003 {

    // Time: O(n)
    // Space: O(n)
```

```
    public int lengthOfLongestSubstring(String s) {
        int left = 0;
        int right = 0;

        Set<Character> seen = new HashSet<>();

        int result = 0;

        final int length = s.length();
        while (right < length) {
            Character rightChar = s.charAt(right);
            if (seen.contains(rightChar)) {
                seen.remove(s.charAt(left));
                left++;
            } else {
                seen.add(rightChar);
                right++;
                result = Math.max(result, right - left);
            }
        }

        return result;
    }

    // Time: O(n)
    // Space: O(n)
    public int lengthOfLongestSubstringMap(String s) {
        int left = 0;
        int right = 0;

        Map<Character, Integer> seen = new HashMap<>();

        int result = 0;

        final int length = s.length();
        while (right < length) {
            Character rightChar = s.charAt(right);
            int charSeen = seen.getOrDefault(rightChar, 0);
            if (charSeen == 0) {
                seen.put(rightChar, 1);
                right++;
                result = Math.max(result, right - left);
            } else {
                seen.put(s.charAt(left), 0);
                left++;
            }
        }

        return result;
    }

    // Time: O(n)
    // Space: O(1)
    public int lengthOfLongestSubstringIntArr(String s) {
```

```
        int left = 0;
        int right = 0;

        int[] seen = new int[128];

        int result = 0;

        final int length = s.length();
        while (right < length) {
            char rightChar = s.charAt(right);
            if (seen[rightChar] > 0) {
                seen[s.charAt(left)]--;
                left++;
            } else {
                seen[rightChar]++;
                right++;
                result = Math.max(result, right - left);
            }
        }

        return result;
    }
}
```

8. `Solution00004.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/median-of-two-sorted-arrays/
 * 4. Median of Two Sorted Arrays
 * Hard
 * <p>
 * Share
 * Given two sorted arrays nums1 and nums2 of size m and n respectively, return the
median of the two sorted arrays.
 * <p>
 * Follow up: The overall run time complexity should be O(log (m+n)).
 * <p>
 * <p>
 * <p>
 * Example 1:
 * <p>
 * Input: nums1 = [1,3], nums2 = [2]
 * Output: 2.00000
 * <p>
 * Explanation: merged array = [1,2,3] and median is 2.
 * <p>
 * Example 2:
 * <p>
 * Input: nums1 = [1,2], nums2 = [3,4]
 * Output: 2.50000
 * <p>
 * Explanation: merged array = [1,2,3,4] and median is (2 + 3) / 2 = 2.5.
 * <p>
 * Example 3:
 * <p>
 * Input: nums1 = [0,0], nums2 = [0,0]
 * Output: 0.00000
 * <p>
 * Example 4:
 * <p>
 * Input: nums1 = [], nums2 = [1]
 * Output: 1.00000
 * <p>
 * Example 5:
 * <p>
 * Input: nums1 = [2], nums2 = []
 * Output: 2.00000
 * <p>
 * <p>
 * <p>
 * Constraints:
 * <p>
 * nums1.length == m
 * nums2.length == n
 * 0 <= m <= 1000
```

```
 * 0 <= n <= 1000
 * 1 <= m + n <= 2000
 */
public class Solution00004 {

    // Time: O(log(min(m,n)))
    // Space: O(1)
    public double findMedianSortedArrays(int[] nums1, int[] nums2) {
        // make sure nums1 is shorter than nums2
        if (nums1.length > nums2.length) {
            return findMedianSortedArrays(nums2, nums1);
        }

        int m = nums1.length;
        int n = nums2.length;
        int total = m + n;

        int left = 0;
        int right = m;

        // Perform binary search on the shorter array
        while (left <= right) {
            // Calculate the partition point for the shorter array
            int partitionX = (left + right) / 2;
            // Calculate the partition point for the longer array
            int partitionY = (total + 1) / 2 - partitionX;

            // Calculate elements around the partition points
            int maxLeftX = partitionX == 0 ? Integer.MIN_VALUE : nums1[partitionX - 1];
            int minRightX = partitionX == m ? Integer.MAX_VALUE : nums1[partitionX];

            int maxLeftY = partitionY == 0 ? Integer.MIN_VALUE : nums2[partitionY - 1];
            int minRightY = partitionY == n ? Integer.MAX_VALUE : nums2[partitionY];

            // Compare elements around the partition points and adjust the binary search
range
            if (maxLeftX <= minRightY && maxLeftY <= minRightX) {
                // Calculate and return the median
                if (total % 2 == 0) {
                    // Even number of elements
                    return (Math.max(maxLeftX, maxLeftY) + Math.min(minRightX,
minRightY)) / 2.0;
                } else {
                    // Odd number of elements
                    return Math.max(maxLeftX, maxLeftY);
                }
            } else if (maxLeftX > minRightY) {
                // Move the partition point to the left
                right = partitionX - 1;
            } else {
                // Move the partition point to the right
                left = partitionX + 1;
            }
        }
```

```
        // Throw an exception if the input arrays do not allow a valid partition
        throw new IllegalArgumentException();
    }

}
```

9. `Solution00005.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/longest-palindromic-substring/
 * 5. Longest Palindromic Substring
 * Medium
 */
public class Solution00005 {

    // Time: O(n^3)
    // Space: O(1)
    public String longestPalindromeBruteforce(String s) {
        String result = "";
        for (int left = 0; left < s.length(); left++) {
            for (int right = left + 1; right <= s.length(); right++) {
                String substring = s.substring(left, right);
                if (isPalindrome(substring) && substring.length() > result.length()) {
                    result = substring;
                }
            }
        }

        return result;
    }

    private boolean isPalindrome(String s) {
        int left = 0;
        int right = s.length() - 1;

        while (left < right) {
            if (s.charAt(left) != s.charAt(right)) {
                return false;
            }

            left++;
            right--;
        }

        return true;
    }

    // Time: O(n^2)
    // Space: O(1)
    public String longestPalindromeExpandAroundCenter(String s) {
        int[] result = new int[]{0, 1}; // left index and length
        for (int center = 0; center < s.length(); center++) {
            // odd length
            int[] palindromeOdd = expandAroundCenter(s, center, center);
            if (palindromeOdd[1] > result[1]) {
                result = palindromeOdd;
            }
```

```
            // even length
            int[] palindromeEven = expandAroundCenter(s, center, center + 1);
            if (palindromeEven[1] > result[1]) {
                result = palindromeEven;
            }
        }

        return s.substring(result[0], result[0] + result[1]);
    }

    private int[] expandAroundCenter(String s, int left, int right) {
        while (left >= 0 && right < s.length()) {
            if (s.charAt(left) != s.charAt(right)) {
                break;
            }

            left--;
            right++;
        }

        final int outputLeft = left + 1;
        final int outputLength = right - left - 1;
        return new int[]{outputLeft, outputLength};
        // for left/right input 0, after first iteration left/right will be -1/1,
        // so to get left - we have to add 1, and get index 0 left + 1 = -1 + 1 = 0
        // and for the length - right - left - 1 = 1 - (-1) - 1 = 1
    }

    // Time: O(n^2)
    // Space: O(1)
    public String longestPalindromeDynamicProgramming(String s) {
        int resultLeft = 0;
        int resultLength = 1;
        boolean[][] dp = new boolean[s.length()][s.length()];

        for (int right = 0; right < s.length(); right++) {
            int left = right;
            dp[left][right] = true;
        }

        for (int right = 1; right < s.length(); right++) {
            int left = right - 1;
            if (s.charAt(left) == s.charAt(right)) {
                dp[left][right] = true;
                resultLength = 2;
                resultLeft = left;
            }
        }

        for (int right = 2; right < s.length(); right++) {
            for (int left = 0; left <= right; left++) {
                if (s.charAt(left) == s.charAt(right) && (right - left <= 2 || dp[left +
1][right - 1])) {
                    dp[left][right] = true;
```

```
                    if (right - left + 1 > resultLength) {
                        resultLength = right - left + 1;
                        resultLeft = left;
                    }
                }
            }
        }

        return s.substring(resultLeft, resultLeft + resultLength);
    }

}
```

10. `Solution00006.java`

```
package leetcode;

/**
 * 6. ZigZag Conversion
 * https://leetcode.com/problems/zigzag-conversion/
 * Medium
 */

public class Solution00006 {

    public String convert(String s, int numRows) {
        if (numRows == 1) {
            return s;
        }

        StringBuilder[] rows = new StringBuilder[numRows];
        for (int i = 0; i < numRows; i++) {
            rows[i] = new StringBuilder();
        }

        int row = 0;
        int direction = 1;
        for (char c : s.toCharArray()) {
            rows[row].append(c);
            if (row == 0) {
                direction = 1;
            } else if (row == numRows - 1) {
                direction = -1;
            }
            row += direction;
        }

        StringBuilder result = new StringBuilder();
        for (StringBuilder r : rows) {
            result.append(r);
        }

        return result.toString();
    }

    public String convertArr(String s, int numRows) {
        int len = s.length();

        if (numRows > len || numRows <= 1) {
            return s;
        }

        int index = 0;
        int diff = 2 * numRows - 2;
        char[] arr = new char[len];
```

```
        for (int row = 0; row < numRows; row++) {
            int diagDiff = diff - 2 * row;
            boolean isDiagExist = diagDiff < diff && diagDiff > 0;

            for (int col = row; col < len; col += diff) {
                arr[index] = s.charAt(col);
                index++;

                if (isDiagExist && col + diagDiff < len) {
                    arr[index] = s.charAt(col + diagDiff);
                    index++;
                }
            }
        }

        return new String(arr);
    }

}
```

11. `Solution00007.java`

```
package leetcode;

/**
 * 7. Reverse Integer
 * https://leetcode.com/problems/reverse-integer/
 * Medium
 */

public class Solution00007 {

    public int reverse(int x) {
        long reversed = 0L;
        while (x != 0) {
            int nextDigit = x % 10;
            reversed = reversed * 10 + nextDigit;
            x = x / 10;
        }
        if (reversed < Integer.MIN_VALUE || reversed > Integer.MAX_VALUE) {
            return 0;
        }
        return (int) reversed;
    }

}
```

12. `Solution00008.java`

```
package leetcode;

/**
 * 8. String to Integer (atoi)
 * https://leetcode.com/problems/string-to-integer-atoi/
 * Medium
 */

public class Solution00008 {

    public int myAtoi(String s) {
        int currentIndex = 0;
        int n = s.length();
        while (currentIndex < n && s.charAt(currentIndex) == ' ') {
            currentIndex++;
        }
        if (currentIndex == n) {
            return 0;
        }
        int sign = 1;
        if (s.charAt(currentIndex) == '+' || s.charAt(currentIndex) == '-') {
            sign = s.charAt(currentIndex) == '+' ? 1 : -1;
            currentIndex++;
        }
        long num = 0;
        while (currentIndex < n && Character.isDigit(s.charAt(currentIndex))) {
            final int currentDigit = s.charAt(currentIndex) - '0';
            num = num * 10 + currentDigit;
            if (num > Integer.MAX_VALUE) {
                return sign == 1 ? Integer.MAX_VALUE : Integer.MIN_VALUE;
            }
            currentIndex++;
        }
        return (int) num * sign;
    }

}
```

13. `Solution00011.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/container-with-most-water/
 * 11. Container With Most Water
 * Medium
 */
public class Solution00011 {

    // Time: O(n)
    // Space: O(1)
    public int maxArea(int[] height) {
        int max = 0;
        int left = 0;
        int right = height.length - 1;

        while (left < right) {
            // Calculate the area.
            int area = Math.min(height[left], height[right]) * (right - left);
            max = Math.max(max, area);

            // Move the pointer that is pointing to the smaller height.
            if (height[left] < height[right]) {
                left++;
            } else {
                right--;
            }
        }

        return max;
    }

    // Time: O(n^2)
    // Space: O(1)
    public int maxAreaBruteForce(int[] height) {
        int max = 0;

        for (int i = 0; i < height.length; i++) {
            int left = height[i];
            for (int j = i + 1; j < height.length; j++) {
                int right = height[j];
                int area = Math.min(left, right) * (j - i);
                max = Math.max(max, area);
            }
        }

        return max;
    }

}
```

14. `Solution00012.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/integer-to-roman/
 * 12. Integer to Roman
 * Medium
 */

public class Solution00012 {

    public String intToRoman(int num) {
        StringBuilder sb = new StringBuilder();
        int[] values = {1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1};
        String[] symbols = {"M", "CM", "D", "CD", "C", "XC", "L", "XL", "X", "IX", "V",
"IV", "I"};
        for (int i = 0; i < values.length; i++) {
            while (num >= values[i]) {
                num -= values[i];
                sb.append(symbols[i]);
            }
        }
        return sb.toString();
    }

}
```

15. `Solution00013.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/roman-to-integer/
 * 13. Roman to Integer
 * Easy
 */
public class Solution00013 {

    // Time: O(n)
    // Space: O(1)
    public int romanToInt(String s) {
        // Create a variable to store the result.
        int result = 0;

        // Create a variable to store the previous value.
        int prev = 0;

        // Iterate over the string.
        final int n = s.length();
        for (int i = 0; i < n; i++) {
            // Get the current character.
            char c = s.charAt(i);

            // Get the value of the current character.
            int value = getValue(c);

            // If the value of the current character is greater than the previous value,
            if (value > prev) {
                result = result + value - 2 * prev; // Eg. IV, at position V we already
have 1, and to add extra 3 to get 4 we need  + (5 - 2 * 1) = 4
            } else {
                result += value;
            }

            // Update the previous value.
            prev = value;
        }

        // Return the result.
        return result;
    }

    // Get the value of the character.
    private int getValue(char c) {
        return switch (c) {
            case 'I' -> 1;
            case 'V' -> 5;
            case 'X' -> 10;
            case 'L' -> 50;
            case 'C' -> 100;
```

```
            case 'D' -> 500;
            case 'M' -> 1000;
            default -> 0;
        };
    }
}
```

16. `Solution00014.java`

```
package leetcode;

/**
 * 14. Longest Common Prefix
 * https://leetcode.com/problems/longest-common-prefix/
 * Easy
 */

public class Solution00014 {

    public String longestCommonPrefix(String[] strs) {
        final int strLen = strs.length;
        if (strLen == 0) {
            return "";
        }

        StringBuilder prefixBuilder = new StringBuilder();
        final String firstString = strs[0];
        final int maxPrefix = firstString.length();
        for (int charIndex = 0; charIndex < maxPrefix; charIndex++) {
            char charFromFirstString = firstString.charAt(charIndex);
            for (int strIndex = 1; strIndex < strLen; strIndex++) {
                final String currentString = strs[strIndex];
                if (currentString.length() <= charIndex ||
currentString.charAt(charIndex) != charFromFirstString) {
                    return prefixBuilder.toString();
                }
            }
            prefixBuilder.append(charFromFirstString);
        }
        return prefixBuilder.toString();
    }

}
```

17. `Solution00015.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * https://leetcode.com/problems/3sum/
 * 15. 3Sum
 * <p>
 * Given an integer array nums, return all the triplets [nums[i], nums[j], nums[k]] such
that
 * i != j, i != k, and j != k, and nums[i] + nums[j] + nums[k] == 0.
 * <p>
 * Notice that the solution set must not contain duplicate triplets.
 * <p>
 * <p>
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [-1,0,1,2,-1,-4] Output: [[-1,-1,2],[-1,0,1]] Example 2:
 * <p>
 * Input: nums = [] Output: [] Example 3:
 * <p>
 * Input: nums = [0] Output: []
 * <p>
 * <p>
 * Constraints:
 * <p>
 * 0 <= nums.length <= 3000 -105 <= nums[i] <= 105
 */
public class Solution00015 {

    // Time: O(n^3)
    // Space: O(1)
    public List<List<Integer>> threeSumBrutforce(int[] nums) {
        // Create a list to store the result.
        List<List<Integer>> result = new ArrayList<>();

        // Check for invalid inputs.
        if (nums == null || nums.length < 3) {
            return result;
        }

        // Sort the array.
        Arrays.sort(nums);

        // Iterate over the array.
        for (int i = 0; i < nums.length - 2; i++) {
            // Skip duplicates.
            if (i > 0 && nums[i] == nums[i - 1]) {
```

```
            continue;
        }

        // Get the current element.
        int currentElement = nums[i];

        // Iterate over the array.
        for (int j = i + 1; j < nums.length - 1; j++) {
            // Get the current element.
            int currentElement2 = nums[j];

            // Iterate over the array.
            for (int k = j + 1; k < nums.length; k++) {
                // Get the current element.
                int currentElement3 = nums[k];

                // Check if the sum is equal to zero.
                if (currentElement + currentElement2 + currentElement3 == 0) {
                    // Create a list to store the triplet.
                    List<Integer> triplet = new ArrayList<>();

                    // Add the elements to the triplet.
                    triplet.add(currentElement);
                    triplet.add(currentElement2);
                    triplet.add(currentElement3);

                    // Add the triplet to the result.
                    result.add(triplet);
                }
            }
        }
    }

    return result;
}

// Time: O(n^2)
// Space: O(1)
public List<List<Integer>> threeSum(int[] nums) {
    // Create a list to store the result.
    List<List<Integer>> result = new ArrayList<>();

    // Check for invalid inputs.
    if (nums == null || nums.length < 3) {
        return result;
    }

    // Sort the array.
    Arrays.sort(nums);

    // Iterate over the array.
    for (int i = 0; i < nums.length - 2; i++) {
        // Skip duplicates.
        if (i > 0 && nums[i] == nums[i - 1]) {
```

```
                continue;
            }

            // Get the current element.
            int currentElement = nums[i];

            // Get the left and right pointers.
            int left = i + 1;
            int right = nums.length - 1;

            // Iterate over the array.
            while (left < right) {
                // Get the sum of the three elements.
                int sum = currentElement + nums[left] + nums[right];

                // Check if the sum is equal to zero.
                if (sum == 0) {
                    // Add the triplet to the result.
                    result.add(List.of(currentElement, nums[left], nums[right]));

                    // Skip duplicates.
                    while (left < right && nums[left] == nums[left + 1]) {
                        left++;
                    }

                    // Skip duplicates.
                    while (left < right && nums[right] == nums[right - 1]) {
                        right--;
                    }

                    // Move the left and right pointers.
                    left++;
                    right--;
                } else if (sum < 0) {
                    // Move the left pointer.
                    left++;
                } else {
                    // Move the right pointer.
                    right--;
                }
            }
        }

        return result;
    }

}
```

18. `Solution00019.java`

```
package leetcode;

import leetcode.common.ListNode;

/**
 * 19. Remove Nth Node From End of List
 * https://leetcode.com/problems/remove-nth-node-from-end-of-list/
 * <p>
 * Medium
 * <p>
 * 4752
 * <p>
 * 291
 * <p>
 * Add to List
 * <p>
 * Share
 * Given the head of a linked list, remove the nth node from the end of the list and
return its head.
 * <p>
 * Follow up: Could you do this in one pass?
 * <p>
 * Example 1:
 * <p>
 * <p>
 * Input: head = [1,2,3,4,5], n = 2
 * Output: [1,2,3,5]
 * Example 2:
 * <p>
 * Input: head = [1], n = 1
 * Output: []
 * Example 3:
 * <p>
 * Input: head = [1,2], n = 1
 * Output: [1]
 * <p>
 * Constraints:
 * <p>
 * The number of nodes in the list is sz.
 * 1 <= sz <= 30
 * 0 <= Node.val <= 100
 * 1 <= n <= sz
 */
public class Solution00019 {

    // Time: O(n)
    // Space: O(1)
    public ListNode removeNthFromEnd(ListNode head, int n) {
        // Create two pointers, left and right
        ListNode left = head;
        ListNode right = head;
```

```
        // Move right n nodes ahead of left
        for (int i = 0; i < n; i++) {
            right = right.next;
        }

        // If right is null, it means that the node to be removed is the head
        if (right == null) {
            return head.next;
        }

        // Move left and right one node at a time until right reaches the end of the list
        while (right.next != null) {
            left = left.next;
            right = right.next;
        }

        // Remove the node
        left.next = left.next.next;

        return head;
    }

    // Time: O(n)
    // Space: O(1)
    public ListNode removeNthFromEndAlt(ListNode head, int n) {
        // Create a dummy node to store the head of the list
        ListNode dummyHead = new ListNode(0, head);

        // Initialize two pointers, left and right, to the dummyHead and head
        // respectively
        ListNode left = dummyHead;
        ListNode right = head;

        // Move the right pointer n nodes ahead in the list
        for (int i = 0; i < n; i++) {
            right = right.next;
        }

        // Move both left and right pointers one node at a time until right reaches the
        // end of the list
        while (right != null) {
            left = left.next;
            right = right.next;
        }

        // At this point, the left pointer is at the node before the node to be removed
        // So, skip the next node of left pointer to remove the nth node from the end
        left.next = left.next.next;

        // Return the head of the modified list
        return dummyHead.next;
    }
```

```
}
```

19. `Solution00020.java`

```
package leetcode;

import java.util.Deque;
import java.util.LinkedList;

/**
 * https://leetcode.com/problems/valid-parentheses/
 * 20. Valid Parentheses
 * <p>
 * Given a string s containing just the characters '(', ')', '{', '}', '[' and ']',
 * determine if the input string is valid.
 * <p>
 * An input string is valid if:
 * <p>
 * 1. Open brackets must be closed by the same type of brackets.
 * 2. Open brackets must be closed in the correct order.
 * <p>
 * Example 1:
 * <p>
 * Input: s = "()" Output: true Example 2:
 * <p>
 * Input: s = "()[]{}" Output: true Example 3:
 * <p>
 * Input: s = "(]" Output: false Example 4:
 * <p>
 * Input: s = "([)]" Output: false Example 5:
 * <p>
 * Input: s = "{[]}" Output: true
 * <p>
 * Constraints:
 * <p>
 * 1 <= s.length <= 104 s consists of parentheses only '()[]{}'.
 */
public class Solution00020 {

    // Time: O(n)
    // Space: O(n)
    public boolean isValid(String s) {
        // Check for empty string.
        if (s.isEmpty()) {
            return true;
        }

        // Check for odd length.
        if (s.length() % 2 != 0) {
            return false;
        }

        // Create a stack.
        Deque<Character> stack = new LinkedList<>();
```

```
        // Iterate over the string.
        int n = s.length();
        for (int i = 0; i < n; i++) {
            char c = s.charAt(i);

            // If the character is an opening bracket, push it to the stack.
            if (c == '(' || c == '{' || c == '[') {
                stack.push(c);
            } else {
                // If the stack is empty, return false.
                if (stack.isEmpty()) {
                    return false;
                }

                // If the character is a closing bracket, pop the stack and compare.
                char top = stack.pop();

                if (c == ')' && top != '(') {
                    return false;
                }

                if (c == '}' && top != '{') {
                    return false;
                }

                if (c == ']' && top != '[') {
                    return false;
                }
            }
        }

        // If the stack is not empty, return false.
        return stack.isEmpty();
    }

}
```

20. `Solution00021.java`

```
package leetcode;

import leetcode.common.ListNode;

/**
 * https://leetcode.com/problems/merge-two-sorted-lists/
 * 21. Merge Two Sorted Lists
 * <p>
 * Merge two sorted linked lists and return it as a sorted list.
 * The list should be made by splicing together the nodes of the first two lists.
 * <p>
 * Example 1:
 * <p>
 * Input: l1 = [1,2,4], l2 = [1,3,4]
 * Output: [1,1,2,3,4,4]
 * <p>
 * Example 2:
 * <p>
 * Input: l1 = [], l2 = []
 * Output: []
 * <p>
 * Example 3:
 * <p>
 * Input: l1 = [], l2 = [0]
 * Output: [0]
 * <p>
 * Constraints:
 * <p>
 * The number of nodes in both lists is in the range [0, 50].
 * -100 <= Node.val <= 100
 * Both l1 and l2 are sorted in non-decreasing order.
 */
public class Solution00021 {

    // Time: O(n)
    // Space: O(1)
    public ListNode mergeTwoLists(ListNode list1, ListNode list2) {
        // Create a dummy node to store the head of the merged list.
        ListNode dummyHead = new ListNode();
        ListNode current = dummyHead;

        // Iterate over both lists and add the smaller node to the merged list.
        while (list1 != null && list2 != null) {
            if (list1.val <= list2.val) {
                // Add list1 to the result
                current.next = list1;

                // Move list1 forward
                list1 = list1.next;
            } else {
                // Add list2 to the result
```

```
                current.next = list2;

                // Move list2 forward
                list2 = list2.next;
            }

            // Move current forward
            current = current.next;
        }

        // Add remaining nodes
        current.next = list1 == null ? list2 : list1;

        // Return the head of the merged list.
        return dummyHead.next;
    }

}
```

21. `Solution00022.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * https://leetcode.com/problems/generate-parentheses/
 * 22. Generate Parentheses
 * <p>
 * Given n pairs of parentheses, write a function to generate all combinations of well-
formed parentheses.
 * <p>
 * <p>
 * <p>
 * Example 1:
 * <p>
 * Input: n = 3 Output: ["((()))","(()())","(())()","()(())","()()()"] Example 2:
 * <p>
 * Input: n = 1 Output: ["()"]
 * <p>
 * <p>
 * Constraints:
 * <p>
 * 1 <= n <= 8
 */
public class Solution00022 {

    // Time: O(4^n)
    // Space: O(n)
    public List<String> generateParenthesis(int n) {
        List<String> result = new ArrayList<>();
        generateParenthesis(result, "", n, n);
        return result;
    }

    private void generateParenthesis(List<String> result, String current, int remainOpen,
int remainClose) {
        // there are 4 possible combinations
        // Open parenthesis, valid combination:
        // Open parenthesis, invalid combination:
        // Close parenthesis, valid combination:
        // Close parenthesis, invalid combination:
        if (remainOpen == 0 && remainClose == 0) {
            result.add(current);
            return;
        }
        if (remainOpen > 0) {
            generateParenthesis(result, current + "(", remainOpen - 1, remainClose);
        }
```

```
        if (remainClose > 0 && remainClose > remainOpen) {
            generateParenthesis(result, current + ")", remainOpen, remainClose - 1);
        }
    }

    public List<String> generateParenthesisNonRecursive(int n) {
        List<String> res = new ArrayList<>();

        Queue<String> queue = new LinkedList<>();
        queue.add("");

        while (!queue.isEmpty()) {
            String str = queue.poll();

            if (str.length() == 2 * n) {
                if (isValid(str)) {
                    res.add(str);
                }
                continue;
            }

            queue.add(str + "(");
            queue.add(str + ")");
        }

        return res;
    }

    private boolean isValid(String str) {
        int open = 0;
        int close = 0;

        for (char c : str.toCharArray()) {
            if (c == '(') {
                open++;
            } else {
                close++;
            }

            if (close > open) {
                return false;
            }
        }

        return open == close;
    }

}
```

22. `Solution00023.java`

```
package leetcode;

import leetcode.common.ListNode;

import java.util.Comparator;
import java.util.PriorityQueue;

/**
 * https://leetcode.com/problems/merge-k-sorted-lists/
 * 23. Merge k Sorted Lists
 * <p>
 * You are given an array of k linked-lists lists, each linked-list is sorted in
ascending order.
 * <p>
 * Merge all the linked-lists into one sorted linked-list and return it.
 * <p>
 * Example 1:
 * <p>
 * Input: lists = [[1,4,5],[1,3,4],[2,6]]
 * Output: [1,1,2,3,4,4,5,6]
 * <p>
 * Example 2:
 * <p>
 * Input: lists = []
 * Output: []
 * <p>
 * Example 3:
 * <p>
 * Input: lists = [[]]
 * Output: []
 * <p>
 * Constraints:
 * <p>
 * k == lists.length
 * 0 <= k <= 10^4
 * 0 <= lists[i].length <= 500
 * -10^4 <= lists[i][j] <= 10^4
 * lists[i] is sorted in ascending order.
 * The sum of lists[i].length won't exceed 10^4.
 */
public class Solution00023 {

    // Time: O(n * log(k))
    // Space: O(k)
    public ListNode mergeKLists(ListNode[] lists) {
        // If the array is null or empty, return null.
        if (lists == null || lists.length == 0) {
            return null;
        }

        // Create a priority queue to store the first node of each list.
```

```
        PriorityQueue<ListNode> pq = new PriorityQueue<>(Comparator.comparingInt(a ->
a.val));
        for (ListNode list : lists) {
            if (list != null) {
                pq.add(list);
            }
        }

        // Create a dummy node to store the head of the merged list.
        ListNode dummyHead = new ListNode(0);

        ListNode current = dummyHead;
        while (!pq.isEmpty()) {
            // Get the node with the smallest value from the priority queue.
            ListNode node = pq.poll();
            current.next = node;
            current = current.next;

            // Add the next node of the list to the priority queue.
            if (node.next != null) {
                pq.add(node.next);
            }
        }

        return dummyHead.next;
    }

}
```

23. `Solution00025.java`

```
package leetcode;

import leetcode.common.ListNode;

/**
 * <a href="https://leetcode.com/problems/reverse-nodes-in-k-group/">
 * 25. Reverse Nodes in k-Group
 * </a>
 * <br/>
 * <div class="notranslate"><p>Given a linked list, reverse the nodes of a linked list
<em>k</em> at a time and return its modified list.</p>
 *
 * <p><em>k</em> is a positive integer and is less than or equal to the length of the
linked list. If the number of nodes is not a multiple of <em>k</em> then left-out nodes,
in the end, should remain as it is.</p>
 *
 * <p>You may&amp;nbsp;not alter the values in the list's nodes, only nodes themselves may be
changed.</p>
 *
 * <p><strong>Example 1:</strong></p>
 * <img alt="" src="https://assets.leetcode.com/uploads/2020/10/03/reverse_ex1.jpg"
style="width: 542px; height: 222px;">
 * <pre><strong>Input:</strong> head = [1,2,3,4,5], k = 2
 * <strong>Output:</strong> [2,1,4,3,5]
 * </pre>
 *
 * <p><strong>Example 2:</strong></p>
 * <img alt="" src="https://assets.leetcode.com/uploads/2020/10/03/reverse_ex2.jpg"
style="width: 542px; height: 222px;">
 * <pre><strong>Input:</strong> head = [1,2,3,4,5], k = 3
 * <strong>Output:</strong> [3,2,1,4,5]
 * </pre>
 *
 * <p><strong>Example 3:</strong></p>
 *
 * <pre><strong>Input:</strong> head = [1,2,3,4,5], k = 1
 * <strong>Output:</strong> [1,2,3,4,5]
 * </pre>
 *
 * <p><strong>Example 4:</strong></p>
 *
 * <pre><strong>Input:</strong> head = [1], k = 1
 * <strong>Output:</strong> [1]
 * </pre>
 */
public class Solution00025 {

    // Time: O(n)
    // Space: O(1)
    public ListNode reverseKGroupRecursive(ListNode head, int k) {
        if (head == null || k <= 1) {
```

```
            return head;
        }
        ListNode curr = head;
        int count = 0;
        while (curr != null && count != k) { // find the end of this group
            curr = curr.next;
            count++;
        }

        if (count == k) { // if the count is equal to k, reverse this group
            curr = reverseKGroupRecursive(curr, k); // reverse the rest of the list
            while (count-- > 0) { // reverse current group
                ListNode tmp = head.next; // tmp - next head in original list
                head.next = curr; // preappending "next head" to the reversed list
                curr = head; // move head of reversed list to a new node
                head = tmp; // move "next head" to the head of the remaining list
            }
            head = curr;
        }
        return head;
    }

    // Time: O(n)
    // Space: O(1)
    public ListNode reverseKGroupIterative(ListNode head, int k) {
        if (head == null || k <= 1) {
            return head;
        }
        ListNode dummy = new ListNode(0, head);
        ListNode groupPrev = dummy;

        while (true) {
            final ListNode kth = findK(k, groupPrev);
            if (kth == null) {
                break;
            }

            reverseGroup(groupPrev.next, kth.next);

            ListNode tmp = groupPrev.next;
            groupPrev.next = kth;
            groupPrev = tmp;
        }
        return dummy.next;
    }

    private static ListNode findK(int k, ListNode groupPrev) {
        ListNode kth = groupPrev;
        int count = 0;
        while (kth != null && count < k) {
            kth = kth.next;
            count++;
        }
        return kth;
```

```
    }

    private static void reverseGroup(ListNode start, ListNode end) {
        // reverse group
        ListNode prev = end;
        while (start != end) {
            ListNode tmp = start.next;
            start.next = prev;
            prev = start;
            start = tmp;
        }
    }

}
```

24. `Solution00026.java`

```
package leetcode;

/**
 * 26. Remove Duplicates from Sorted Array
 * [Easy]
 * https://leetcode.com/problems/remove-duplicates-from-sorted-array
 * <p>
 * Given an integer array nums sorted in non-decreasing order, remove the duplicates in-place such that each unique element appears only once. The relative order of the elements should be kept the same. Then return the number of unique elements in nums.
 * <p>
 * Consider the number of unique elements of nums to be k, to get accepted, you need to do the following things:
 * <p>
 * Change the array nums such that the first k elements of nums contain the unique elements in the order they were present in nums initially. The remaining elements of nums are not important as well as the size of nums.
 * Return k.
 * Custom Judge:
 * <p>
 * The judge will test your solution with the following code:
 * <p>
 * int[] nums = [...]; // Input array
 * int[] expectedNums = [...]; // The expected answer with correct length
 * <p>
 * int k = removeDuplicates(nums); // Calls your implementation
 * <p>
 * assert k == expectedNums.length;
 * for (int i = 0; i < k; i++) {
 * assert nums[i] == expectedNums[i];
 * }
 * If all assertions pass, then your solution will be accepted.
 * <p>
 * <p>
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [1,1,2]
 * Output: 2, nums = [1,2,_]
 * Explanation: Your function should return k = 2, with the first two elements of nums being 1 and 2 respectively.
 * It does not matter what you leave beyond the returned k (hence they are underscores).
 * Example 2:
 * <p>
 * Input: nums = [0,0,1,1,1,2,2,3,3,4]
 * Output: 5, nums = [0,1,2,3,4,_,_,_,_,_]
 * Explanation: Your function should return k = 5, with the first five elements of nums being 0, 1, 2, 3, and 4 respectively.
 * It does not matter what you leave beyond the returned k (hence they are underscores).
 * <p>
 * <p>
```

```
 * Constraints:
 * <p>
 * 1 <= nums.length <= 3 * 104
 * -100 <= nums[i] <= 100
 * nums is sorted in non-decreasing order.
 */

public class Solution00026 {

    // Time: O(n)
    // Space: O(1)
    public int removeDuplicates(int[] nums) {
        int newIndex = 1;
        int previousElement = nums[0];
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] != previousElement) {
                nums[newIndex] = nums[i];
                newIndex++;
            }
            previousElement = nums[i];
        }
        return newIndex;
    }

}
```

25. `Solution00027.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/remove-element/
 * 27. Remove Element
 * Easy
 * <p>
 * Given an integer array nums and an integer val, remove all occurrences of val in nums in-place. The order of the elements may be changed. Then return the number of elements in nums which are not equal to val.
 * <p>
 * Consider the number of elements in nums which are not equal to val be k, to get accepted, you need to do the following things:
 * <p>
 * Change the array nums such that the first k elements of nums contain the elements which are not equal to val. The remaining elements of nums are not important as well as the size of nums.
 * Return k.
 * Custom Judge:
 * <p>
 * The judge will test your solution with the following code:
 * <p>
 * int[] nums = [...]; // Input array
 * int val = ...; // Value to remove
 * int[] expectedNums = [...]; // The expected answer with correct length.
 * // It is sorted with no values equaling val.
 * <p>
 * int k = removeElement(nums, val); // Calls your implementation
 * <p>
 * assert k == expectedNums.length;
 * sort(nums, 0, k); // Sort the first k elements of nums
 * for (int i = 0; i < actualLength; i++) {
 * assert nums[i] == expectedNums[i];
 * }
 * If all assertions pass, then your solution will be accepted.
 * <p>
 * <p>
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [3,2,2,3], val = 3
 * Output: 2, nums = [2,2,_,_]
 * Explanation: Your function should return k = 2, with the first two elements of nums being 2.
 * It does not matter what you leave beyond the returned k (hence they are underscores).
 * Example 2:
 * <p>
 * Input: nums = [0,1,2,2,3,0,4,2], val = 2
 * Output: 5, nums = [0,1,4,0,3,_,_,_]
 * Explanation: Your function should return k = 5, with the first five elements of nums containing 0, 0, 1, 3, and 4.
```

```
 * Note that the five elements can be returned in any order.
 * It does not matter what you leave beyond the returned k (hence they are underscores).
 * <p>
 * <p>
 * Constraints:
 * <p>
 * 0 <= nums.length <= 100
 * 0 <= nums[i] <= 50
 * 0 <= val <= 100
 */
public class Solution00027 {

    // Time: O(n)
    // Space: O(1)
    public int removeElement(int[] nums, int val) {
        int newIndex = 0;
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] != val) {
                nums[newIndex] = nums[i];
                newIndex++;
            }
        }
        return newIndex;
    }

}
```

26. `Solution00028.java`

```
package leetcode;

/**
 * 28. Find the Index of the First Occurrence in a String
 * https://leetcode.com/problems/find-the-index-of-the-first-occurrence-in-a-
string/description/
 * Easy
 */

public class Solution00028 {
    public int strStr(String haystack, String needle) {
        int haystackLength = haystack.length();
        int needleLength = needle.length();
        for (int currentIndex = 0; currentIndex + needleLength <= haystackLength;
currentIndex++) {
            int matchingIndex = 0;
            while (matchingIndex < needleLength && haystack.charAt(currentIndex +
matchingIndex) == needle.charAt(matchingIndex)) {
                matchingIndex++;
            }
            if (matchingIndex == needleLength) {
                return currentIndex;
            }
        }
        return -1;
    }
}
```

27. `Solution00033.java`

```
package leetcode;

/**
 * 33. Search in Rotated Sorted Array
 * https://leetcode.com/problems/search-in-rotated-sorted-array/
 * Medium
 * <p>
 * You are given an integer array nums sorted in ascending order (with distinct values),
and an integer target.
 * <p>
 * Suppose that nums is rotated at some pivot unknown to you beforehand (i.e.,
[0,1,2,4,5,6,7] might
 * become [4,5,6,7,0,1,2]).
 * <p>
 * If target is found in the array return its index, otherwise, return -1.
 * <p>
 * <p>
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [4,5,6,7,0,1,2], target = 0 Output: 4 Example 2:
 * <p>
 * Input: nums = [4,5,6,7,0,1,2], target = 3 Output: -1 Example 3:
 * <p>
 * Input: nums = [1], target = 0 Output: -1
 * <p>
 * <p>
 * Constraints:
 * <p>
 * 1 <= nums.length <= 5000 -104 <= nums[i] <= 104 All values of nums are unique. nums is
guranteed to
 * be rotated at some pivot. -104 <= target <= 104
 */
public class Solution00033 {

    // Time: O(logn)
    // Space: O(1)
    public int search(int[] nums, int target) {
        // Create a variable to keep track of the left index.
        int left = 0;

        // Create a variable to keep track of the right index.
        int right = nums.length - 1;

        // Loop while the left index is less than or equal to the right index.
        while (left <= right) {
            // Create a variable to keep track of the middle index.
            int mid = left + (right - left) / 2;

            final int middleValue = nums[mid];
```

```
            if (middleValue == target) {
                // Return the middle index.
                return mid;
            }

            if (nums[left] <= middleValue) { // values are sorted in the left half
                if (target >= middleValue || // target is greater than the middle value
                    target < nums[left]) { // target is less than the left value
                    // Update the left index, values are in the right half.
                    left = mid + 1;
                } else {
                    // Update the right index, values are in the left half.
                    right = mid - 1;
                }
            } else { // values are sorted in the right half
                if (target <= middleValue || // target is less than the middle value
                    target > nums[right]) { // target is greater than the right value
                    // Update the right index, values are in the left half.
                    right = mid - 1;
                } else {
                    // Update the left index, values are in the right half.
                    left = mid + 1;
                }
            }
        }

        // Return -1.
        return -1;
    }

}
```

28. `Solution00035.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 35. Search Insert Position
 * https://leetcode.com/problems/search-insert-position/description/
 * Easy
 */

public class Solution00035 {

    public int searchInsert(int[] nums, int target) {
        int pos = Arrays.binarySearch(nums, target);
        return pos >= 0 ? pos : -pos - 1;
    }

}
```

29. `Solution00036.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/valid-sudoku/
 * 36. Valid Sudoku
 * <p>
 * Determine if a 9 x 9 Sudoku board is valid. Only the filled cells need to be validated
according to the following rules:
 * <p>
 * 1. Each row must contain the digits 1-9 without repetition.
 * 2. Each column must contain the digits 1-9 without repetition.
 * 3. Each of the nine 3 x 3 sub-boxes of the grid must contain the digits 1-9 without
repetition.
 * <p>
 * Note:
 * <p>
 * 1. A Sudoku board (partially filled) could be valid but is not necessarily solvable.
 * 2. Only the filled cells need to be validated according to the mentioned rules.
 * <p>
 * Example 1:
 * <p>
 * Input: board =
 * [["5","3",".",".","7",".",".",".","."],
 * ["6",".",".","1","9","5",".",".","."],
 * [".","9","8",".",".",".",".","6","."],
 * ["8",".",".",".","6",".",".",".","3"],
 * ["4",".",".","8",".","3",".",".","1"],
 * ["7",".",".",".","2",".",".",".","6"],
 * [".","6",".",".",".",".","2","8","."],
 * [".",".",".","4","1","9",".",".","5"],
 * [".",".",".",".","8",".",".","7","9"]]
 * Output: true
 * Example 2:
 * <p>
 * Input: board =
 * [["8","3",".",".","7",".",".",".","."],
 * ["6",".",".","1","9","5",".",".","."],
 * [".","9","8",".",".",".",".","6","."],
 * ["8",".",".",".","6",".",".",".","3"],
 * ["4",".",".","8",".","3",".",".","1"],
 * ["7",".",".",".","2",".",".",".","6"],
 * [".","6",".",".",".",".","2","8","."],
 * [".",".",".","4","1","9",".",".","5"],
 * [".",".",".",".","8",".",".","7","9"]]
 * Output: false
 * Explanation: Same as Example 1, except with the 5 in the top left corner being
modified to 8. Since there are two 8's in the top left 3x3 sub-box, it is invalid.
 * <p>
 * Constraints:
 * board.length == 9
 * board[i].length == 9
```

```
 * board[i][j] is a digit 1-9 or '.'.
 */
public class Solution00036 {

    // Time: O(n^2)
    // Space: O(n)
    public boolean isValidSudoku(char[][] board) {
        // Check for rows.
        for (int i = 0; i < 9; i++) {
            boolean[] visited = new boolean[9];
            for (int j = 0; j < 9; j++) {
                char current = board[i][j];
                if (current != '.') {
                    int index = current - '1';
                    if (visited[index]) {
                        return false;
                    }
                    visited[index] = true;
                }
            }
        }

        // Check for columns.
        for (int j = 0; j < 9; j++) {
            boolean[] visited = new boolean[9];
            for (int i = 0; i < 9; i++) {
                char current = board[i][j];
                if (current != '.') {
                    int index = current - '1';
                    if (visited[index]) {
                        return false;
                    }
                    visited[index] = true;
                }
            }
        }

        // Check for sub-boxes.
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                boolean[] visited = new boolean[9];
                int shiftI = i * 3;
                int shiftJ = j * 3;
                for (int k = 0; k < 3; k++) {
                    for (int l = 0; l < 3; l++) {
                        char current = board[shiftI + k][shiftJ + l];
                        if (current != '.') {
                            int index = current - '1';
                            if (visited[index]) {
                                return false;
                            }
                            visited[index] = true;
                        }
                    }
```

```
                }
            }
        }

        return true;
    }

    public boolean isValidSudokuSimplified(char[][] board) {
        for (int i = 0; i < 9; i++) {
            if (!isValidRow(board, i) || !isValidColumn(board, i) || !isValidBox(board,
i)) {
                return false;
            }
        }
        return true;
    }

    private boolean isValidRow(char[][] board, int row) {
        boolean[] visited = new boolean[9];
        for (int col = 0; col < 9; col++) {
            if (board[row][col] != '.') {
                int num = board[row][col] - '1';
                if (visited[num]) return false;
                visited[num] = true;
            }
        }
        return true;
    }

    private boolean isValidColumn(char[][] board, int col) {
        boolean[] visited = new boolean[9];
        for (int row = 0; row < 9; row++) {
            if (board[row][col] != '.') {
                int num = board[row][col] - '1';
                if (visited[num]) return false;
                visited[num] = true;
            }
        }
        return true;
    }

    private boolean isValidBox(char[][] board, int boxIndex) {
        // 0 1 2 - 0*3 - 0
        // 3 4 5 - 1*3 - 3
        // 6 7 8 - 2*3 - 6
        int rowIndexStart = boxIndex / 3 * 3;
        // 0 3 6 - 0*3 - 0
        // 1 4 7 - 1*3 - 3
        // 2 5 8 - 2*3 - 6
        int colIndexStart = boxIndex % 3 * 3;
        boolean[] visited = new boolean[9];
        for (int row = rowIndexStart; row < rowIndexStart + 3; row++) {
            for (int col = colIndexStart; col < colIndexStart + 3; col++) {
                if (board[row][col] != '.') {
```

```
                    int num = board[row][col] - '1';
                    if (visited[num]) return false;
                    visited[num] = true;
                }
            }
        }
        return true;
    }

}
```

30. `Solution00039.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.List;

/**
 * 39. Combination Sum
 * https://leetcode.com/problems/combination-sum/
 * <p>
 * Medium
 * <p>
 * Given an array of distinct integers candidates and a target integer target, return a
list of all unique combinations
 * of candidates where the chosen numbers sum to target. You may return the combinations
in any order.
 * <p>
 * The same number may be chosen from candidates an unlimited number of times. Two
combinations are unique if the
 * frequency of at least one of the chosen numbers is different.
 * <p>
 * It is guaranteed that the number of unique combinations that sum up to target is less
than 150 combinations for the
 * given input.
 * <p>
 * Example 1:
 * <p>
 * Input: candidates = [2,3,6,7], target = 7 Output: [[2,2,3],[7]] Explanation: 2 and 3
are candidates, and 2 + 2 + 3 =
 * 7. Note that 2 can be used multiple times. 7 is a candidate, and 7 = 7. These are the
only two combinations. Example
 * 2:
 * <p>
 * Input: candidates = [2,3,5], target = 8 Output: [[2,2,2,2],[2,3,3],[3,5]] Example 3:
 * <p>
 * Input: candidates = [2], target = 1 Output: [] Example 4:
 * <p>
 * Input: candidates = [1], target = 1 Output: [[1]] Example 5:
 * <p>
 * Input: candidates = [1], target = 2 Output: [[1,1]]
 * <p>
 * Constraints:
 * <p>
 * 1 <= candidates.length <= 30 1 <= candidates[i] <= 200 All elements of candidates are
distinct. 1 <= target <= 500
 */
public class Solution00039 {

    // Time: O(n * 2^n)
    // Space: O(n * 2^n)
    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        List<List<Integer>> result = new ArrayList<>();
```

```
        combinationSum(candidates, target, 0, new ArrayList<>(), result);
        return result;
    }

    private void combinationSum(int[] candidates, int target, int index,
                                List<Integer> current,
                                List<List<Integer>> result) {
        if (target == 0) {
            result.add(new ArrayList<>(current));
            return;
        }

        if (target < 0 || index == candidates.length) {
            return;
        }

        current.add(candidates[index]);
        combinationSum(candidates, target - candidates[index], index, current, result);

        current.remove(current.size() - 1);
        combinationSum(candidates, target, index + 1, current, result);
    }
}
```

31. `Solution00040.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 40. Combination Sum II
 * <a href="https://leetcode.com/problems/combination-sum-ii">Web link</a> |
 * Medium
 */

public class Solution00040 {

    public List<List<Integer>> combinationSum2(int[] candidates, int target) {
        List<List<Integer>> result = new ArrayList<>();
        Arrays.sort(candidates);
        findCombinations(candidates, target, 0, new ArrayList<>(), result);
        return result;
    }

    private void findCombinations(int[] candidates, int target, int currentIndex,
List<Integer> currentUsed, List<List<Integer>> result) {
        if (target == 0) {
            result.add(List.copyOf(currentUsed));
            return;
        }

        for (int i = currentIndex; i < candidates.length; i++) {
            if (i > currentIndex && candidates[i] == candidates[i - 1]) {
                continue;
            }
            if (candidates[i] > target) {
                break;
            }
            currentUsed.add(candidates[i]);
            findCombinations(candidates, target - candidates[i], i + 1, currentUsed,
result);
            currentUsed.removeLast();
        }
    }

}
```

32. `Solution00041.java`

```
package leetcode;

/**
 * 41. First Missing Positive
 * https://leetcode.com/problems/first-missing-positive/
 * Hard
 */

public class Solution00041 {

    public int firstMissingPositive(int[] nums) {
        int n = nums.length;
        int outOfRange = n + 1;

        // Replace all negative numbers and zeros with outOfRange to allow using them as
carrier, but ignore as candidates for solution
        for (int i = 0; i < n; i++) {
            if (nums[i] <= 0) {
                nums[i] = outOfRange;
            }
        }

        // Mark all positive numbers by changing the sign of the number at the index
equal to the number, but ignore out of range numbers
        for (int i = 0; i < n; i++) {
            var indexOfNegative = Math.abs(nums[i]) - 1;
            if (indexOfNegative >= 0 &&
                indexOfNegative < n &&
                nums[indexOfNegative] > 0) {
                nums[indexOfNegative] = -nums[indexOfNegative];
            }
        }

        // Find the first positive number, which is the first one that was not marked,
because it's index - +1
        for (int i = 0; i < n; i++) {
            if (nums[i] > 0) {
                return i + 1;
            }
        }

        return n + 1;
    }

}
```

33. `Solution00042.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/trapping-rain-water/
 * 42. Trapping Rain Water
 * <p>
 * Given n non-negative integers representing an elevation map where the width of each
bar is 1,
 * compute how much water it can trap after raining.
 * <p>
 * Example 1:
 * <p>
 * Input: height = [0,1,0,2,1,0,1,3,2,1,2,1] Output: 6 Explanation: The above elevation
map (black
 * section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of
rain water
 * (blue section) are being trapped. Example 2:
 * <p>
 * Input: height = [4,2,0,3,2,5] Output: 9
 * <p>
 * Constraints:
 * <p>
 * n == height.length 0 <= n <= 3 * 104 0 <= height[i] <= 105
 */
public class Solution00042 {

    // Time: O(n)
    // Space: O(1)
    public int trap(int[] height) {
        int result = 0;

        // Create two pointers.
        int left = 0;
        int right = height.length - 1;

        // Create two variables to keep track of the max height on the left and right.
        int leftMax = 0;
        int rightMax = 0;

        // Loop through the array.
        while (left < right) {
            // If the height on the left is less than the height on the right.
            if (height[left] < height[right]) {
                // If the height on the left is greater than the max height on the left.
                if (height[left] >= leftMax) {
                    // Update the max height on the left.
                    leftMax = height[left];
                } else {
                    // Add the water.
                    result += leftMax - height[left];
                }
```

```
                // Increment the left pointer.
                left++;
            } else {
                // If the height on the right is greater than the max height on the
right.
                if (height[right] >= rightMax) {
                    // Update the max height on the right.
                    rightMax = height[right];
                } else {
                    // Add the water.
                    result += rightMax - height[right];
                }
                // Decrement the right pointer.
                right--;
            }
        }

        return result;
    }

    // Time: O(n ^ 2)
    // Space: O(1)
    public int trapBruteForce(int[] height) {
        int result = 0;

        // Loop through the array.
        for (int i = 0; i < height.length; i++) {
            // Find the max height on the left.
            int leftMax = 0;
            for (int j = i; j >= 0; j--) {
                leftMax = Math.max(leftMax, height[j]);
            }

            // Find the max height on the right.
            int rightMax = 0;
            for (int j = i; j < height.length; j++) {
                rightMax = Math.max(rightMax, height[j]);
            }

            // Add the water.
            result += Math.min(leftMax, rightMax) - height[i];
        }

        return result;
    }

}
```

34. `Solution00045.java`

```
package leetcode;

import java.util.Arrays;

/**
 * https://leetcode.com/problems/jump-game-ii/description/
 * 45. Jump Game II
 * <p>
 * Given an array of non-negative integers nums, you are initially positioned at the
first index of the array. Each
 * element in the array represents your maximum jump length at that position. Your goal
is to reach the last index in
 * the minimum number of jumps. You can assume that you can always reach the last index.
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [2,3,1,1,4] Output: 2 Explanation: The minimum number of jumps to reach
the last index is 2. Jump 1
 * step from index 0 to 1, then 3 steps to the last index. Example 2:
 * <p>
 * Input: nums = [2,3,0,1,4] Output: 2
 * <p>
 * Constraints:
 * <p>
 * 1 <= nums.length <= 104 0 <= nums[i] <= 1000
 */
public class Solution00045 {

    // Time: O(n^2)
    // Space: O(n)
    public int jumpRecursive(int[] nums) {
        final int jumps = jumpRecursive(nums, 0);
        if (jumps == Integer.MAX_VALUE) {
            return -1;
        }
        return jumps;
    }

    private int jumpRecursive(int[] nums, int i) {
        // If we're at the last index, no more jumps are needed.
        if (i >= nums.length - 1) {
            return 0;
        }

        // If we're not at the last index, iterate from i + 1 to i + nums[i] and
recursively call jump for each index.
        int minJumps = Integer.MAX_VALUE;
        for (int j = i + 1; j <= i + nums[i] && j < nums.length; j++) {
            int jumps = jumpRecursive(nums, j);
            if (jumps != Integer.MAX_VALUE) {
                minJumps = Math.min(minJumps, jumps + 1);
```

```
            }
        }

        return minJumps;
    }

    // Time: O(n)
    // Space: O(1)
    public int jumpGreedy(int[] nums) {
        // If the array is null or with length <= 1, return 0.
        if (nums == null || nums.length <= 1) {
            return 0;
        }

        // Initialize the number of jumps to 0.
        int jumps = 0;

        // Initialize the current jump end to 0.
        int currentJumpEnd = 0;

        // Initialize the farthest jump to 0.
        int farthestJump = 0;

        // Iterate over the array of numbers.
        for (int i = 0; i < nums.length - 1; i++) {
            // Update the farthest jump to the maximum of the current farthest jump and
the index plus the number at the
            // index.
            farthestJump = Math.max(farthestJump, i + nums[i]);

            // If the current index is equal to the current jump end, increment the
number of jumps and update the
            // current jump end to the farthest jump.
            if (i == currentJumpEnd) {
                jumps++;
                if (farthestJump > currentJumpEnd) {
                    currentJumpEnd = farthestJump;
                } else {
                    return -1; // Cannot reach the end.
                }
            }
        }

        // Return the number of jumps.
        return jumps;
    }

    // Time: O(n^2)
    // Space: O(n)
    public int jumpDp(int[] nums) {
        // If the array is null or with length <= 1, return 0.
        if (nums == null || nums.length <= 1) {
            return 0;
        }
```

```
        int n = nums.length;
        int[] jumps = new int[n];
        Arrays.fill(jumps, Integer.MAX_VALUE);
        jumps[0] = 0;

        for (int i = 1; i < n; i++) {
            for (int j = 0; j < i; j++) {
                if (j + nums[j] >= i) {
                    if (jumps[j] == Integer.MAX_VALUE) {
                        return -1;
                    }
                    jumps[i] = Math.min(jumps[i], jumps[j] + 1);
                }
            }
        }

        return jumps[n - 1];
    }

}
```

35. `Solution00046.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 46. Permutations
 * https://leetcode.com/problems/permutations/
 * Medium
 */
public class Solution00046 {

    public List<List<Integer>> permute(int[] nums) {
        List<Integer> numsList = Arrays.stream(nums).boxed().toList();
        return permute(numsList);
    }

    public List<List<Integer>> permute(List<Integer> numsList) {
        if (numsList.isEmpty()) {
            return List.of(List.of());
        }

        final int first = numsList.getFirst();
        List<List<Integer>> permuteNext = permute(numsList.subList(1, numsList.size()));
        List<List<Integer>> result = new ArrayList<>();
        for (List<Integer> perm : permuteNext) {
            for (int i = 0; i <= perm.size(); i++) {
                List<Integer> newPerm = new ArrayList<>(perm);
                newPerm.add(i, first);
                result.add(newPerm);
            }
        }
        return result;
    }


}
```

36. `Solution00048.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/rotate-image/
 * 48. Rotate Image
 * <p>
 * You are given an n x n 2D matrix representing an image, rotate the image by 90 degrees
(clockwise).
 * <p>
 * You have to rotate the image in-place, which means you have to modify the input 2D
matrix directly.
 * DO NOT allocate another 2D matrix and do the rotation.
 * <p>
 * Example 1:
 * <p>
 * Input: matrix = [[1,2,3],
 * [4,5,6],
 * [7,8,9]]
 * Output: [[7,4,1],
 * [8,5,2],
 * [9,6,3]]
 * <p>
 * Example 2:
 * <p>
 * Input: matrix = [[5,1,9,11],
 * [2,4,8,10],
 * [13,3,6,7],
 * [15,14,12,16]]
 * Output: [[15,13,2,5],
 * [14,3,4,1],
 * [12,6,8,9],
 * [16,7,10,11]]
 * <p>
 * Example 3:
 * <p>
 * Input: matrix = [[1]]
 * Output: [[1]]
 * <p>
 * Example 4:
 * <p>
 * Input: matrix = [[1,2],
 * [3,4]]
 * Output: [[3,1],
 * [4,2]]
 * <p>
 * Constraints:
 * <p>
 * matrix.length == n
 * matrix[i].length == n
 * 1 <= n <= 20
 * -1000 <= matrix[i][j] <= 1000
```

```
 * <p>
 * Hide Hint #1
 * <p>
 * The easiest solution would use additional memory and two passes.
 * <p>
 * Hide Hint #2
 * <p>
 * Can we do it in one pass?
 * <p>
 * Hide Hint #3
 * <p>
 * Related problem: Rotate Array
 * <p>
 * Hide Hint #4
 * <p>
 * First reverse up to down, then swap the symmetry
 */
public class Solution00048 {

    // Time: O(n)
    // Space: O(1)
    public void rotate(int[][] matrix) {
        // Get the size of the matrix.
        int n = matrix.length;

        // Reverse the matrix.
        for (int i = 0; i < n / 2; i++) {
            int[] temp = matrix[i];
            matrix[i] = matrix[n - i - 1];
            matrix[n - i - 1] = temp;
        }

        // Swap the symmetry.
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                int temp = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = temp;
            }
        }
    }

}
```

37. `Solution00049.java`

```
package leetcode;

import java.util.*;

/**
 * https://leetcode.com/problems/group-anagrams/
 * 49. Group Anagrams
 * <p>
 * Given an array of strings strs, group the anagrams together. You can return the answer
in any order.
 * <p>
 * An Anagram is a word or phrase formed by rearranging the letters of a different word
or phrase,
 * typically using all the original letters exactly once.
 * <p>
 * Example 1:
 * <p>
 * Input: strs = ["eat","tea","tan","ate","nat","bat"] Output: [["bat"],["nat","tan"],
["ate","eat","tea"]] Example 2:
 * <p>
 * Input: strs = [""] Output: [[""]] Example 3:
 * <p>
 * Input: strs = ["a"] Output: [["a"]]
 * <p>
 * Constraints:
 * <p>
 * 1 <= strs.length <= 104 0 <= strs[i].length <= 100 strs[i] consists of lower-case
English letters.
 */
public class Solution00049 {

    // Time: O(n * k log k)
    // Space: O(n * k)
    // runtime Beats 98.84%of users with Java
    // memory Beats 88.47%of users with Java
    public List<List<String>> groupAnagrams(String[] strs) {
        // Check for empty array.
        if (strs.length == 0) {
            return new ArrayList<>();
        }

        // Create a map of sorted string to list of strings.
        Map<String, List<String>> map = new HashMap<>();

        // Iterate over the array.
        for (String str : strs) {
            // Convert the string to char array.
            char[] chars = str.toCharArray();

            // Sort the char array.
            Arrays.sort(chars);
```

```
            // Convert the char array to string.
            String key = String.valueOf(chars);

            // Add the string to the map.
            map.putIfAbsent(key, new ArrayList<>());
            map.get(key).add(str);
        }

        // Return the map values.
        return new ArrayList<>(map.values());
    }

    // Time: O(n * k)
    // Space: O(n * k)
    // runtime Beats 16.11%of users with Java
    // memory Beats 19.19%of users with Java
    public List<List<String>> groupAnagramsCounting(String[] strs) {
        // Check for empty array.
        if (strs.length == 0) {
            return new ArrayList<>();
        }

        // Create a map of sorted string to list of strings.
        Map<String, List<String>> map = new HashMap<>();

        // Iterate over the array.
        for (String str : strs) {
            // Create an array of 26 characters.
            int[] count = new int[26];

            // Iterate over the string.
            for (char c : str.toCharArray()) {
                // Increment the count of the character.
                count[c - 'a']++;
            }

            // Add the string to the map.
            String key = Arrays.toString(count);
            map.putIfAbsent(key, new ArrayList<>());
            map.get(key).add(str);
        }

        // Return the map values.
        return new ArrayList<>(map.values());
    }

}
```

38. `Solution00053.java`

```
package leetcode;

/** https://leetcode.com/problems/maximum-subarray/
 * 53. Maximum Subarray
 * <p>
 * Given an integer array nums, find the contiguous subarray (containing at least one
number) which has the largest
 * sum and return its sum.
 * <p>
 * A subarray is a contiguous part of an array.
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [-2,1,-3,4,-1,2,1,-5,4] Output: 6 Explanation: [4,-1,2,1] has the
largest sum = 6. Example 2:
 * <p>
 * Input: nums = [1] Output: 1 Example 3:
 * <p>
 * Input: nums = [5,4,-1,7,8] Output: 23
 * <p>
 * Constraints:
 * <p>
 * 1 <= nums.length <= 3 * 104 -105 <= nums[i] <= 105
 */
public class Solution00053 {

    // Time: O(n)
    // Space: O(1)
    public int maxSubArray(int[] nums) {
        // Initialize the max sum to the first element.
        int maxSum = nums[0];

        // Initialize the current sum to the first element.
        int currentSum = nums[0];

        // Iterate over the array starting from the second element.
        for (int i = 1; i < nums.length; i++) {
            // Update the current sum
            currentSum = Math.max(nums[i], currentSum + nums[i]);

            // Update the max sum.
            maxSum = Math.max(maxSum, currentSum);
        }

        // Return the max sum.
        return maxSum;
    }

}
```

39. `Solution00054.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.List;

/**
 * https://leetcode.com/problems/spiral-matrix/
 * 54. Spiral Matrix
 * <p>
 * Given an m x n matrix, return all elements of the matrix in spiral order.
 * <p>
 * Example 1:
 * <p>
 * Input: matrix = [[1,2,3],
 * [4,5,6],
 * [7,8,9]]
 * Output: [1,2,3,6,9,8,7,4,5]
 * <p>
 * Example 2:
 * <p>
 * Input: matrix = [[1,2,3,4],
 * [5,6,7,8],
 * [9,10,11,12]]
 * Output: [1,2,3,4,8,12,11,10,9,5,6,7]
 * <p>
 * Constraints:
 * <p>
 * m == matrix.length
 * n == matrix[i].length
 * 1 <= m, n <= 10
 * -100 <= matrix[i][j] <= 100
 */
public class Solution00054 {

    // Time: O(n)
    // Space: O(n)
    public List<Integer> spiralOrder(int[][] matrix) {
        int rows = matrix.length;
        int columns = matrix[0].length;
        final int matrixSize = rows * columns;
        List<Integer> result = new ArrayList<>(matrixSize);

        // Initialize the boundaries of the matrix.
        int top = 0;
        int bottom = rows - 1;
        int left = 0;
        int right = columns - 1;

        // Iterate through the matrix.
        while (true) {
            // first from left to right.
```

```
            if (top <= bottom) {
                for (int i = left; i <= right; i++) {
                    result.add(matrix[top][i]);
                }
            }
            // Increment the top boundary.
            top++;

            // second from top to bottom.
            if (left <= right) {
                for (int i = top; i <= bottom; i++) {
                    result.add(matrix[i][right]);
                }
            }
            // Decrement the right boundary.
            right--;

            // third from right to left.
            if (top <= bottom) { // for last iteration - only left to right should be
done (not right to left
                for (int i = right; i >= left; i--) {
                    result.add(matrix[bottom][i]);
                }
            }
            // Decrement the bottom boundary.
            bottom--;

            // last bottom to top.
            if (left <= right) { // for last iteration - only top to bottom should be
done (not bottom to top)
                for (int i = bottom; i >= top; i--) {
                    result.add(matrix[i][left]);
                }
            }
            // Increment the left boundary.
            left++;

            if (result.size() == matrixSize) {
                break;
            }
        }

        return result;
    }

}
```

40. `Solution00055.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/jump-game/
 * 55. Jump Game
 * <p>
 * Given an array of non-negative integers nums, you are initially positioned at the
first index of the array.
 * <p>
 * Each element in the array represents your maximum jump length at that position.
Determine if you are able to reach
 * the last index.
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [2,3,1,1,4] Output: true Explanation: Jump 1 step from index 0 to 1,
then 3 steps to the last index.
 * <p>
 * Example 2:
 * <p>
 * Input: nums = [3,2,1,0,4] Output: false Explanation: You will always arrive at index 3
no matter what. Its maximum
 * jump length is 0, which makes it impossible to reach the last index.
 * <p>
 * Constraints:
 * <p>
 * 1 <= nums.length <= 3 * 104 0 <= nums[i] <= 105
 */
public class Solution00055 {

    // Time: O(n)
    // Space: O(1)
    public boolean canJump(int[] nums) {
        // Initialize the last reachable index to the last index.
        int lastReachableIndex = nums.length - 1;

        // Iterate over the array backwards.
        for (int i = nums.length - 1; i >= 0; i--) {
            // If the current index plus the value at the current index is greater than
or equal to the last reachable
            // index, then the last reachable index is the current index.
            if (i + nums[i] >= lastReachableIndex) {
                lastReachableIndex = i;
            }
        }

        // Return whether the last reachable index is the first index.
        return lastReachableIndex == 0;
    }

    public boolean canJumpDirect(int[] nums) {
```

```
        int pos = 0;
        int remainingJumps = 0;
        while (pos < nums.length - 1) {
            int possibleJumps = Math.max(nums[pos], remainingJumps);
            if (possibleJumps <= 0) {
                return false;
            }
            remainingJumps = possibleJumps - 1;
            pos++;
        }
        return true;
    }

    public boolean canJumpBruteforce(int[] nums) {
        return canJumpFromPosition(nums, 0);
    }

    private boolean canJumpFromPosition(int[] nums, int i) {
        // If the current index is the last index, then return true.
        if (i == nums.length - 1) {
            return true;
        }

        // Iterate over the possible jump lengths.
        for (int j = 1; j <= nums[i]; j++) {
            // If the current index plus the jump length is less than the length of the
array, then recursively call
            // canJumpFromPosition with the current index plus the jump length.
            if (i + j < nums.length) {
                if (canJumpFromPosition(nums, i + j)) {
                    return true;
                }
            }
        }

        // Return false.
        return false;
    }

}
```

41. `Solution00056.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;

/**
 * https://leetcode.com/problems/merge-intervals/
 * 56. Merge Intervals
 * <p>
 * Given an array of intervals where intervals[i] = [starti, endi],
 * merge all overlapping intervals,
 * and return an array of the non-overlapping intervals
 * that cover all the intervals in the input.
 * <p>
 * Example 1:
 * <p>
 * Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
 * Output: [[1,6],[8,10],[15,18]]
 * <p>
 * Explanation: Since intervals [1,3] and [2,6] overlaps,
 * merge them into [1,6].
 * <p>
 * Example 2:
 * <p>
 * Input: intervals = [[1,4],[4,5]]
 * Output: [[1,5]]
 * <p>
 * Explanation: Intervals [1,4] and [4,5] are considered overlapping.
 * <p>
 * Constraints:
 * <p>
 * 1 <= intervals.length <= 10^4
 * intervals[i].length == 2
 * 0 <= starti <= endi <= 10^4
 */
public class Solution00056 {

    // Time: O(n * log(n))
    // Space: O(n)
    public int[][] merge(int[][] intervals) {
        // If the array is null or empty, return null.
        if (intervals.length <= 1) {
            return intervals;
        }

        // Sort the array by the start time of each interval.
        Arrays.sort(intervals, Comparator.comparingInt(interval -> interval[0]));

        // Create a list to store the merged intervals.
```

```
        List<int[]> mergedIntervals = new ArrayList<>(intervals.length);

        // Add the first interval to the list.
        int[] lastInterval = intervals[0];
        mergedIntervals.add(lastInterval);

        // Iterate over the array of intervals.
        for (int i = 1; i < intervals.length; i++) {
            // Get the current interval.
            int[] currentInterval = intervals[i];

            // If the start time of the current interval is less than or equal to the end
time of the last interval,
            // merge the intervals.
            if (currentInterval[0] <= lastInterval[1]) {
                lastInterval[1] = Math.max(lastInterval[1], currentInterval[1]);
            } else {
                // Otherwise, add the current interval to the list.
                lastInterval = currentInterval;
                mergedIntervals.add(currentInterval);
            }
        }

        // Return the list as an array.
        return mergedIntervals.toArray(new int[][]{});
    }

}
```

42. `Solution00057.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.List;

/**
 * https://leetcode.com/problems/insert-interval/
 * 57. Insert Interval
 * <p>
 * Given a set of non-overlapping intervals, insert a new interval into the intervals
(merge if necessary).
 * <p>
 * You may assume that the intervals were initially sorted according to their start
times.
 * <p>
 * <p>
 * <p>
 * Example 1:
 * <p>
 * Input: intervals = [[1,3],[6,9]], newInterval = [2,5]
 * Output: [[1,5],[6,9]]
 * <p>
 * Example 2:
 * <p>
 * Input: intervals = [[1,2],[3,5],[6,7],[8,10],[12,16]], newInterval = [4,8]
 * Output: [[1,2],[3,10],[12,16]]
 * <p>
 * Explanation: Because the new interval [4,8] overlaps with [3,5],[6,7],[8,10].
 * <p>
 * Example 3:
 * <p>
 * Input: intervals = [], newInterval = [5,7]
 * Output: [[5,7]]
 * <p>
 * Example 4:
 * <p>
 * Input: intervals = [[1,5]], newInterval = [2,3]
 * Output: [[1,5]]
 * <p>
 * Example 5:
 * <p>
 * Input: intervals = [[1,5]], newInterval = [2,7]
 * Output: [[1,7]]
 * <p>
 * <p>
 * <p>
 * Constraints:
 * <p>
 * 0 <= intervals.length <= 104
 * intervals[i].length == 2
 * 0 <= intervals[i][0] <= intervals[i][1] <= 105
```

```
 * intervals is sorted by intervals[i][0] in ascending order.
 * newInterval.length == 2
 * 0 <= newInterval[0] <= newInterval[1] <= 105
 */
public class Solution00057 {

    // Time: O(n)
    // Space: O(n)
    public int[][] insert(int[][] intervals, int[] newInterval) {
        List<int[]> result = new ArrayList<>();
        int currentIndex = 0;

        // Add all the intervals ending before newInterval starts
        while (currentIndex < intervals.length && intervals[currentIndex][1] <
newInterval[0]) {
            result.add(intervals[currentIndex++]);
        }

        // Merge all overlapping intervals to one considering newInterval
        while (currentIndex < intervals.length && intervals[currentIndex][0] <=
newInterval[1]) {
            newInterval[0] = Math.min(newInterval[0], intervals[currentIndex][0]);
            newInterval[1] = Math.max(newInterval[1], intervals[currentIndex][1]);
            currentIndex++;
        }
        result.add(newInterval); // Add the union of intervals we got

        // Add all the rest
        while (currentIndex < intervals.length) {
            result.add(intervals[currentIndex++]);
        }

        return result.toArray(new int[][]{});
    }

}
```

43. `Solution00058.java`

```
package leetcode;

/**
 * 58. Length of Last Word
 * https://leetcode.com/problems/length-of-last-word/
 * Easy
 */

public class Solution00058 {

    public int lengthOfLastWord(String s) {
        int n = s.length();
        int end = n - 1;
        while (end >= 0 && s.charAt(end) == ' ') {
            end--;
        }
        int start = end;
        while (start >= 0 && s.charAt(start) != ' ') {
            start--;
        }
        return end - start;
    }

}
```

44. `Solution00066.java`

```
package leetcode;

/**
 * 66. Plus One
 * <a href="https://leetcode.com/problems/plus-one">Web link</a>
 * Easy
 */

public class Solution00066 {

    public int[] plusOne(int[] digits) {
        int n = digits.length;
        for (int i = n - 1; i >= 0; i--) {
            if (digits[i] != 9) {
                digits[i] += 1;
                return digits;
            } else {
                digits[i] = 0;
            }
        }
        int[] newArr = new int[n + 1];
        newArr[0] = 1;
        System.arraycopy(digits, 0, newArr, 1, n);
        return newArr;
    }

}
```

45. `Solution00070.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/climbing-stairs/
 * 70. Climbing Stairs
 *
 * You are climbing a staircase. It takes n steps to reach the top.
 * Each time you can either climb 1 or 2 steps.
 * In how many distinct ways can you climb to the top?
 *
 * Example 1:
 *
 * Input: n = 2
 * Output: 2
 *
 * Explanation: There are two ways to climb to the top.
 * 1. 1 step + 1 step
 * 2. 2 steps
 *
 * Example 2:
 *
 * Input: n = 3
 * Output: 3
 *
 * Explanation: There are three ways to climb to the top.
 * 1. 1 step + 1 step + 1 step
 * 2. 1 step + 2 steps
 * 3. 2 steps + 1 step

 * Constraints:
 *
 * 1 <= n <= 45
 */
public class Solution00070 {

    // Time: O(n)
    // Space: O(1)
    public int climbStairs(int n) {
        if (n <= 2) {
            return n;
        }

        int first = 1;
        int second = 2;

        for (int i = 3; i <= n; i++) {
            int tmp = first;
            first = second;
            second = second + tmp;
        }
```

```
        return second;
    }

    // Time: O(n)
    // Space: O(n)
    public int climbStairsFirstApproach(int n) {
        if (n <= 2) {
            return n;
        }

        int[] dp = new int[n + 1];

        // base cases
        dp[1] = 1;
        dp[2] = 2;

        // recursive cases
        for (int i = 3; i <= n; i++) {
            dp[i] = dp[i - 2] + dp[i - 1];
        }

        return dp[n];
    }

}
```

46. `Solution00073.java`

```
package leetcode;

import java.util.Arrays;

/**
 * https://leetcode.com/problems/set-matrix-zeroes/
 * 73. Set Matrix Zeroes
 * <p>
 * Given an m x n integer matrix matrix, if an element is 0, set its entire row and
column to 0's,
 * and return the matrix.
 * <p>
 * You must do it in place.
 * <p>
 * Example 1:
 * <p>
 * Input: matrix = [[1,1,1],
 * [1,0,1],
 * [1,1,1]]
 * Output: [[1,0,1],
 * [0,0,0],
 * [1,0,1]]
 * <p>
 * Example 2:
 * <p>
 * Input: matrix = [[0,1,2,0],
 * [3,4,5,2],
 * [1,3,1,5]]
 * Output: [[0,0,0,0],
 * [0,4,5,0],
 * [0,3,1,0]]
 * <p>
 * Constraints:
 * <p>
 * m == matrix.length
 * n == matrix[0].length
 * 1 <= m, n <= 200
 * -231 <= matrix[i][j] <= 231 - 1
 * <p>
 * Follow up:
 * <p>
 * A straightforward solution using O(mn) space is probably a bad idea.
 * A simple improvement uses O(m + n) space, but still not the best solution.
 * Could you devise a constant space solution?
 */
public class Solution00073 {

    // Time: O(m * n)
    // Space: O(1)
    public void setZeroes(int[][] matrix) {
        boolean firstRowHasZero = false;
```

```
        boolean firstColumnHasZero = false;

        // Check if first row has zero
        for (int cell : matrix[0]) {
            if (cell == 0) {
                firstRowHasZero = true;
                break;
            }
        }

        // Check if first column has zero
        for (int[] row : matrix) {
            if (row[0] == 0) {
                firstColumnHasZero = true;
                break;
            }
        }

        // Check for zeros
        for (int row = 1; row < matrix.length; row++) {
            for (int column = 1; column < matrix[row].length; column++) {
                if (matrix[row][column] == 0) {
                    // Set first cell of the row and column to zero
                    matrix[row][0] = 0;
                    matrix[0][column] = 0;
                }
            }
        }

        // Nullify rows based on values in first column
        for (int row = 1; row < matrix.length; row++) {
            if (matrix[row][0] == 0) {
                nullifyRow(matrix, row);
            }
        }

        // Nullify columns based on values in first row
        for (int column = 1; column < matrix[0].length; column++) {
            if (matrix[0][column] == 0) {
                nullifyColumn(matrix, column);
            }
        }

        // Nullify first row
        if (firstRowHasZero) {
            nullifyRow(matrix, 0);
        }

        // Nullify first column
        if (firstColumnHasZero) {
            nullifyColumn(matrix, 0);
        }
    }
```

```
    private void nullifyColumn(int[][] matrix, int column) {
        for (int row = 0; row < matrix.length; row++) {
            matrix[row][column] = 0;
        }
    }

    private void nullifyRow(int[][] matrix, int row) {
        Arrays.fill(matrix[row], 0);
    }

}
```

47. `Solution00074.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/search-a-2d-matrix/
 * 74. Search a 2D Matrix
 * <p>
 * Write an efficient algorithm that searches for a value in an m x n matrix. This matrix
has the following properties:
 * <p>
 * 1. Integers in each row are sorted from left to right.
 * 2. The first integer of each row is greater than the last integer of the previous row.
 * <p>
 * Example 1:
 * <p>
 * Input: matrix = [ [1,3,5,7], [10,11,16,20], [23,30,34,50] ] target = 3 Output: true
Example 2:
 * <p>
 * Input: matrix = [ [1,3,5,7], [10,11,16,20], [23,30,34,50] ] target = 13 Output: false
Example 3:
 * <p>
 * Input: matrix = [] target = 0 Output: false
 * <p>
 * <p>
 * Constraints:
 * <p>
 * m == matrix.length n == matrix[i].length 0 <= m, n <= 100 -104 <= matrix[i][j], target
<= 104
 */
public class Solution00074 {

    // Time: O(logm + logn)
    // Space: O(1)
    public boolean searchMatrix(int[][] matrix, int target) {
        int top = 0;
        int bottom = matrix.length - 1;
        while (top <= bottom) {
            int mid = top + (bottom - top) / 2;
            int[] midRow = matrix[mid];
            if (midRow[0] <= target && target <= midRow[midRow.length - 1]) {
                return binarySearch(midRow, target);
            } else if (midRow[0] > target) {
                bottom = mid - 1;
            } else {
                top = mid + 1;
            }
        }
        return false;
    }

    private boolean binarySearch(int[] nums, int target) {
        // Create a variable to keep track of the left index.
```

```
        int left = 0;

        // Create a variable to keep track of the right index.
        int right = nums.length - 1;

        // Loop while the left index is less than or equal to the right index.
        while (left <= right) {
            // Create a variable to keep track of the middle index.
            int mid = left + (right - left) / 2;

            final int middleValue = nums[mid];

            // If the middle element is less than the target.
            if (middleValue < target) {
                // Update the left index.
                left = mid + 1;
            } else if (middleValue > target) {
                // Update the right index.
                right = mid - 1;
            } else {
                // Return the middle index.
                return true;
            }
        }

        // Return false.
        return false;
    }

}
```

48. `Solution00075.java`

```
package leetcode;

/**
 * 75. Sort Colors
 * <a href="https://leetcode.com/problems/sort-colors/">Web link</a>
 * Medium
 */

public class Solution00075 {

    public void sortColors(int[] nums) {
        int[] buckets = new int[3];
        for (int num : nums) {
            buckets[num]++;
        }
        int pos = 0;
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < buckets[i]; j++) {
                nums[pos++] = i;
            }
        }
    }

}
```

49. `Solution00076.java`

```
package leetcode;

import java.util.HashMap;
import java.util.Map;

/**
 * https://leetcode.com/problems/minimum-window-substring/
 * 76. Minimum Window Substring
 * <p>
 * Given two strings s and t of lengths m and n respectively,
 * return the minimum window substring of s such that every character in t (including
duplicates) is included in the window.
 * If there is no such substring, return the empty string "".
 * <p>
 * The testcases will be generated such that the answer is unique.
 * <p>
 * A substring is a contiguous sequence of characters within the string.
 * <p>
 * Example 1:
 * <p>
 * Input: s = "ADOBECODEBANC", t = "ABC"
 * Output: "BANC"
 * Explanation: The minimum window substring "BANC" includes 'A', 'B', and 'C' from
string t.
 * Example 2:
 * <p>
 * Input: s = "a", t = "a"
 * Output: "a"
 * Explanation: The entire string s is the minimum window.
 * Example 3:
 * <p>
 * Input: s = "a", t = "aa"
 * Output: ""
 * Explanation: Both 'a's from t must be included in the window.
 * Since the largest window of s only has one 'a', return empty string.
 * <p>
 * <p>
 * Constraints:
 * <p>
 * m == s.length
 * n == t.length
 * 1 <= m, n <= 105
 * s and t consist of uppercase and lowercase English letters.
 * <p>
 * <p>
 * Follow up: Could you find an algorithm that runs in O(m + n) time?
 */
public class Solution00076 {

    // Time: O(n)
    // Space: O(n)
```

```
    public String minWindow(String s, String t) {
        // Create an array to store the start and end indices of the minimum window
substring.
        int[] result = new int[2];
        // Create a hash map to store the count of each character in the string t.
        Map<Character, Integer> charsOfT = new HashMap<>();
        // Create a hash map to store the count of each character in the current window
of the string s.
        Map<Character, Integer> windowOfS = new HashMap<>();

        // Iterate over the characters in the string t and add them to the hash map
charsOfT, with a count of 1 for each character.
        t.chars().forEach(ch -> charsOfT.merge((char) ch, 1, Integer::sum));

        // Initialize the minimum window length.
        int min = Integer.MAX_VALUE;

        // Initialize the number of distinct characters in the string t that need to be
present in the minimum window substring.
        int need = charsOfT.size();

        // Initialize the number of distinct characters in the current window of the
string s that are present in the string t.
        int have = 0;

        // Initialize the start and end indices of the current window of the string s.
        int left = 0;
        final int length = s.length();

        // Iterate over the characters in the string s.
        for (int right = 0; right < length; right++) {
            // Get the current character in the string s.
            final char rightChar = s.charAt(right);

            // Check if the current character is present in the string t.
            Integer countInT = charsOfT.get(rightChar);
            if (countInT == null) {
                // Continue to the next iteration.
                continue;
            }

            // Update the count of the current character in the current window of the
string s.
            windowOfS.merge(rightChar, 1, Integer::sum);

            // Check if the count of the current character in the current window of the
string s is equal to the count of that character in the string t.
            if (countInT.equals(windowOfS.get(rightChar))) {
                // Increment the have variable.
                have++;
            }

            // While the have variable is equal to the need variable, iterate.
            while (have == need) {
```

```
                // Calculate the length of the current window.
                final int minCandidate = right - left + 1;

                // Check if the length of the current window is less than the minimum
window length.
                if (minCandidate < min) {
                    // Update the minimum window length.
                    min = minCandidate;

                    // Update the start and end indices of the minimum window substring.
                    result[0] = left;
                    result[1] = right;
                }

                // Get the character at the left index of the current window.
                final char leftChar = s.charAt(left);

                // Check if the character at the left index of the current window is
present in the string t.
                Integer countInT2 = charsOfT.get(leftChar);
                if (countInT2 != null) {
                    // Update the count of the character at the left index of the current
window in the current window of the string s.
                    windowOfS.merge(leftChar, -1, Integer::sum);

                    // Check if the count of the character at the left index of the
current window in the current window of the string s is less than the count of that
character in the string t.
                    if (windowOfS.get(leftChar) < countInT2) {
                        // Decrement the have variable.
                        have--;
                    }
                }

                // Increment the left index of the current window.
                left++;
            }
        }

        // Return the minimum window substring, or an empty string if no minimum window
substring is found.
        return min == Integer.MAX_VALUE ? "" : s.substring(result[0], result[1] + 1);
    }

    // Time: O(n)
    // Space: O(n)
    public String minWindowArr(String s, String t) {
        int[] need = new int[128];
        int total = 0;
        for (char c : t.toCharArray()) {
            need[c]++;
            total++;
        }
        int left = 0;
```

```
        int right = 0;
        int valid = 0;
        int winStart = 0;
        int winLength = Integer.MAX_VALUE;
        int[] window = new int[128];
        int strLen = s.length();
        while (right < strLen) {
            char c = s.charAt(right);
            right++;
            if (need[c] > 0) {
                window[c]++;
                if (window[c] <= need[c]) {
                    valid++;
                }
            }
            while (valid == total) {
                if (right - left < winLength) {
                    winStart = left;
                    winLength = right - left;
                }
                char d = s.charAt(left);
                left++;
                if (need[d] > 0) {
                    if (window[d] <= need[d]) {
                        valid--;
                    }
                    window[d]--;
                }
            }
        }
        return winLength == Integer.MAX_VALUE ? "" : s.substring(winStart, winStart +
winLength);
    }

}
```

50. `Solution00078.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.List;

/**
 * https://leetcode.com/problems/subsets/
 * 78. Subsets
 * <p>
 * Given an integer array nums of unique elements, return all possible subsets (the power
set).
 * <p>
 * The solution set must not contain duplicate subsets. Return the solution in any order.
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [1,2,3]
 * Output: [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]
 * <p>
 * Example 2:
 * <p>
 * Input: nums = [0]
 * Output: [[],[0]]
 * <p>
 * Constraints:
 * <p>
 * 1 <= nums.length <= 10
 * -10 <= nums[i] <= 10
 * All the numbers of nums are unique.
 */
public class Solution00078 {

    // Time: O(n * 2^n)
    // Space: O(n * 2^n)
    public List<List<Integer>> subsets(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        subsets(nums, 0, new ArrayList<>(), result);
        return result;
    }

    private void subsets(int[] nums, int index, List<Integer> current,
List<List<Integer>> result) {
        if (index == nums.length) {
            result.add(new ArrayList<>(current));
            return;
        }

        current.add(nums[index]);
        subsets(nums, index + 1, current, result);

        current.remove(current.size() - 1);
```

```
        subsets(nums, index + 1, current, result);
    }
}
```

51. `Solution00079.java`

```
package leetcode;

/**
 * 79. Word Search
 * https://leetcode.com/problems/word-search/
 * Medium
 */

public class Solution00079 {

    public boolean exist(char[][] board, String word) {
        int m = board.length;
        int n = board[0].length;
        boolean[][] visited = new boolean[m][n];
        char[] wordChars = word.toCharArray();
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                boolean flag = dfs(board, visited, wordChars, i, j, 0);
                if (flag) {
                    return true;
                }
            }
        }
        return false;
    }

    private boolean dfs(char[][] board, boolean[][] visited, char[] wordChars, int i, int
j, int index) {
        if (index == wordChars.length) {
            return true;
        }
        if (i < 0 || i >= board.length ||
            j < 0 || j >= board[0].length ||
            visited[i][j] ||
            board[i][j] != wordChars[index]) {
            return false;
        }
        visited[i][j] = true;
        boolean flag = dfs(board, visited, wordChars, i - 1, j, index + 1)
                       || dfs(board, visited, wordChars, i + 1, j, index + 1)
                       || dfs(board, visited, wordChars, i, j - 1, index + 1)
                       || dfs(board, visited, wordChars, i, j + 1, index + 1);
        visited[i][j] = false;
        return flag;
    }

}
```

52. `Solution00080.java`

```
package leetcode;

/**
 * 80. Remove Duplicates from Sorted Array II
 * https://leetcode.com/problems/remove-duplicates-from-sorted-array-ii/
 * Medium
 * <p>
 * Given an integer array nums sorted in non-decreasing order, remove some duplicates in-place such that each unique element appears at most twice. The relative order of the elements should be kept the same.
 * <p>
 * Since it is impossible to change the length of the array in some languages, you must instead have the result be placed in the first part of the array nums. More formally, if there are k elements after removing the duplicates, then the first k elements of nums should hold the final result. It does not matter what you leave beyond the first k elements.
 * <p>
 * Return k after placing the final result in the first k slots of nums.
 * <p>
 * Do not allocate extra space for another array. You must do this by modifying the input array in-place with O(1) extra memory.
 * <p>
 * Custom Judge:
 * <p>
 * The judge will test your solution with the following code:
 * <p>
 * int[] nums = [...]; // Input array
 * int[] expectedNums = [...]; // The expected answer with correct length
 * <p>
 * int k = removeDuplicates(nums); // Calls your implementation
 * <p>
 * assert k == expectedNums.length;
 * for (int i = 0; i < k; i++) {
 * assert nums[i] == expectedNums[i];
 * }
 * If all assertions pass, then your solution will be accepted.
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [1,1,1,2,2,3]
 * Output: 5, nums = [1,1,2,2,3,_]
 * Explanation: Your function should return k = 5, with the first five elements of nums being 1, 1, 2, 2 and 3 respectively.
 * It does not matter what you leave beyond the returned k (hence they are underscores).
 * Example 2:
 * <p>
 * Input: nums = [0,0,1,1,1,1,2,3,3]
 * Output: 7, nums = [0,0,1,1,2,3,3,_,_]
 * Explanation: Your function should return k = 7, with the first seven elements of nums being 0, 0, 1, 1, 2, 3 and 3 respectively.
 * It does not matter what you leave beyond the returned k (hence they are underscores).
```

```
 * <p>
 * Constraints:
 * <p>
 * 1 <= nums.length <= 3 * 104
 * -104 <= nums[i] <= 104
 * nums is sorted in non-decreasing order.
 */
public class Solution00080 {

    // Time: O(n)
    // Space: O(1)
    public int removeDuplicates(int[] nums) {
        int n = nums.length;
        if (n <= 2) {
            return n;
        }
        int newIndex = 2;
        for (int i = 2; i < n; i++) {
            if (nums[i] != nums[newIndex - 2]) {
                nums[newIndex] = nums[i];
                newIndex++;
            }
        }
        return newIndex;
    }

}
```

53. `Solution00084.java`

```
package leetcode;

import java.util.Deque;
import java.util.LinkedList;

/**
 * https://leetcode.com/problems/largest-rectangle-in-histogram/
 * 84. Largest Rectangle in Histogram
 * <p>
 * Given an array of integers heights representing the histogram's bar height where the
width of each bar is 1,
 * return the area of the largest rectangle in the histogram.
 * <p>
 * <p>
 * <p>
 * Example 1:
 * <p>
 * <p>
 * Input: heights = [2,1,5,6,2,3] Output: 10 Explanation: The above is a histogram where
width of each bar is 1. The
 * largest rectangle is shown in the red area, which has an area = 10 units. Example 2:
 * <p>
 * <p>
 * Input: heights = [2,4] Output: 4
 * <p>
 * <p>
 * Constraints:
 * <p>
 * 1 <= heights.length <= 105 0 <= heights[i] <= 104
 */
public class Solution00084 {

    // Time: O(n)
    // Space: O(n)
    public int largestRectangleArea(int[] heights) {
        int maxArea = 0;
        Deque<Integer> stack = new LinkedList<>(); // Store the indexes

        for (int i = 0; i < heights.length; i++) {
            while (!stack.isEmpty() && heights[i] < heights[stack.peek()]) {
                maxArea = Math.max(maxArea, getArea(heights, stack, i));
            }
            stack.push(i);
        }

        // Clear the remaining elements in the stack
        while (!stack.isEmpty()) {
            maxArea = Math.max(maxArea, getArea(heights, stack, heights.length));
        }

        return maxArea;
```

```
    }

    // Time: O(n^2)
    // Space: O(1)
    public int largestRectangleAreaBruteForce(int[] heights) {
        int maxArea = 0;
        for (int i = 0; i < heights.length; i++) {
            int minHeight = heights[i];
            maxArea = Math.max(maxArea, minHeight);
            for (int j = i + 1; j < heights.length; j++) {
                minHeight = Math.min(minHeight, heights[j]);
                int area = minHeight * (j - i + 1);
                maxArea = Math.max(maxArea, area);
            }
        }
        return maxArea;
    }

    private static int getArea(int[] heights, Deque<Integer> stack, int positionEnd) {
        // While the stack is not empty and the current height is less than or equal to
the previous height
        final int indexFromStack = stack.pop();
        int height = heights[indexFromStack];  // Get the height at the index from the
stack
        // Calculate the width of the rectangle:
        final int width;
        if (stack.isEmpty()) {
            // current height is the smallest height, so the width is full length until
current position
            width = positionEnd;
        } else {
            width = positionEnd - stack.peek() - 1;
        }
        return height * width;
    }

}
```

54. `Solution00085.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 85. Maximal Rectangle
 * https://leetcode.com/problems/maximal-rectangle/
 * Hard
 */

public class Solution00085 {
    public int maximalRectangle(char[][] matrix) {
        if (matrix.length == 0) {
            return 0;
        }

        int n = matrix[0].length;

        int[] left = new int[n];
        int[] right = new int[n];
        int[] height = new int[n];

        Arrays.fill(right, n);

        int maxArea = 0;
        for (char[] horizontalLine : matrix) {
            int curLeft = 0;
            int curRight = n;
            for (int j = 0; j < n; j++) {
                if (horizontalLine[j] == '1') {
                    height[j]++;
                } else {
                    height[j] = 0;
                }
            }
            for (int j = 0; j < n; j++) {
                if (horizontalLine[j] == '1') {
                    left[j] = Math.max(left[j], curLeft);
                } else {
                    left[j] = 0;
                    curLeft = j + 1;
                }
            }
            for (int j = n - 1; j >= 0; j--) {
                if (horizontalLine[j] == '1') {
                    right[j] = Math.min(right[j], curRight);
                } else {
                    right[j] = n;
                    curRight = j;
                }
            }
```

```
            for (int j = 0; j < n; j++) {
                maxArea = Math.max(maxArea, (right[j] - left[j]) * height[j]);
            }
        }
        return maxArea;
    }
}
```

55. `Solution00088.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/merge-sorted-array/
 * 88. Merge Sorted Array
 * <p>
 * Given two sorted integer arrays nums1 and nums2, merge nums2 into nums1 as one sorted
array.
 * <p>
 * Example 1:
 * <p>
 * Input: nums1 = [1,2,3,0,0,0], m = 3, nums2 = [2,5,6] Output: [1,2,2,3,5,6] Example 2:
 * <p>
 * Input: nums1 = [1], m = 1, nums2 = [] Output: [1]
 * <p>
 * Constraints:
 * <p>
 * nums1.length == m + n nums2.length == n 0 <= m, n <= 200 -109 <= nums1[i], nums2[i] <=
109
 */
public class Solution00088 {

    // Time: O(n)
    // Space: O(1)
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        // Start from the end of the array.
        int lastIndexNum1 = m - 1;
        int lastIndexNum2 = n - 1;
        int lastIndexResult = m + n - 1;

        // While there are elements in both.
        while (lastIndexNum1 >= 0 && lastIndexNum2 >= 0) {
            if (nums1[lastIndexNum1] >= nums2[lastIndexNum2]) {
                nums1[lastIndexResult] = nums1[lastIndexNum1];
                lastIndexNum1--;
            } else { // nums1[lastIndexNum1] < nums2[lastIndexNum2]
                nums1[lastIndexResult] = nums2[lastIndexNum2];
                lastIndexNum2--;
            }
            lastIndexResult--;
        }

        // If there are still elements in nums2, then copy them to nums1.
        // if remaining elements are in num1 - they are already in target location
        if (lastIndexNum2 >= 0) {
            System.arraycopy(nums2, 0, nums1, 0, lastIndexNum2 + 1);
        }
    }

    public void mergeSimpler(int[] nums1, int m, int[] nums2, int n) {
        // Start from the end of the array.
```

```
        int lastIndexNum1 = m - 1;
        int lastIndexNum2 = n - 1;
        int lastIndexResult = m + n - 1;

        // While there are elements in both.
        while (lastIndexNum2 >= 0) {
            if (lastIndexNum1 >= 0 && nums1[lastIndexNum1] > nums2[lastIndexNum2]) {
                nums1[lastIndexResult--] = nums1[lastIndexNum1--];
            } else { // nums1[lastIndexNum1] < nums2[lastIndexNum2] || lastIndexNum1 < 0
                nums1[lastIndexResult--] = nums2[lastIndexNum2--];
            }
        }
    }

}
```

56. `Solution00091.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/decode-ways/
 * 91. Decode Ways
 * Medium
 */
public class Solution00091 {

    // Time: O(2^n)
    // Space: O(n)
    public int numDecodingsRecursion(String s) {
        if (s == null || s.isEmpty()) {
            return 0;
        }

        return numDecodings(0, s);
    }

    private int numDecodings(int p, String s) {
        int n = s.length();
        if (p == n) {
            return 1;
        }

        // If the current character is '0', return 0.
        if (s.charAt(p) == '0') {
            return 0;
        }

        int res = numDecodings(p + 1, s);

        if (p < n - 1 &&
            (s.charAt(p) == '1' || (s.charAt(p) == '2' && s.charAt(p + 1) <= '6'))) {
            res += numDecodings(p + 2, s);
        }

        return res;
    }

    // Time: O(n)
    // Space: O(n)
    public int numDecodingsDp(String s) {
        // Check for invalid inputs.
        if (s == null || s.isEmpty()) {
            return 0;
        }
        int n = s.length();

        int[] dp = new int[n + 1];
        dp[0] = 1;
```

```
        dp[1] = s.charAt(0) == '0' ? 0 : 1;

        // Iterate over the string.
        for (int i = 2; i <= n; i++) {
            // Get the current character.
            char currentChar = s.charAt(i - 1);

            // Get the previous character.
            char previousChar = s.charAt(i - 2);

            // If the current character is 0, then we can't decode it.
            dp[i] = currentChar == '0' ? 0 : dp[i - 1];

            // If the previous character is 1 or 2 and the current character is less than
or equal to 6,
            // then we can decode it.
            if (previousChar == '1' || (previousChar == '2' && currentChar <= '6')) {
                dp[i] += dp[i - 2];
            }
        }

        // Return the number of ways to decode the string.
        return dp[n];
    }

    // Time: O(n)
    // Space: O(1)
    public int numDecodings(String s) {
        // Check for invalid inputs.
        if (s == null || s.isEmpty()) {
            return 0;
        }
        int n = s.length();

        int currentWay = s.charAt(0) == '0' ? 0 : 1;
        int previousWay = currentWay;
        int beforePreviousWay = currentWay;

        // Iterate over the string.
        for (int i = 1; i < n; i++) {
            // Get the current character.
            char currentChar = s.charAt(i);

            // Get the previous character.
            char previousChar = s.charAt(i - 1);

            // If the current character is 0, then we can't decode it.
            currentWay = currentChar == '0' ? 0 : previousWay;

            // If the previous character is 1 or 2 and the current character is less than
or equal to 6,
            // then we can decode it.
            if (previousChar == '1' || (previousChar == '2' && currentChar <= '6')) {
                currentWay += beforePreviousWay;
```

```
            }

            beforePreviousWay = previousWay;
            previousWay = currentWay;
        }

        // Return the number of ways to decode the string.
        return currentWay;
    }

}
```

57. `Solution00092.java`

```
package leetcode;


import leetcode.common.ListNode;

/**
 * 92. Reverse Linked List II
 * https://leetcode.com/problems/reverse-linked-list-ii/
 * Medium
 */

public class Solution00092 {

    // Time: O(n)
    // Space: O(1)
    public ListNode reverseBetween(ListNode head, int left, int right) {
        // Create a dummy node to store the head of the reversed list.
        ListNode dummyHead = new ListNode();
        dummyHead.next = head;

        // Create a pointer to the node before the sublist to be reversed.
        ListNode beforeSublist = dummyHead;
        for (int i = 0; i < left - 1; i++) {
            beforeSublist = beforeSublist.next;
        }

        // Create a pointer to the start of the sublist to be reversed.
        ListNode sublistStart = beforeSublist.next;

        // Create a pointer to the node after the sublist to be reversed.
        ListNode sublistEnd = sublistStart;
        for (int i = left; i < right; i++) {
            sublistEnd = sublistEnd.next;
        }

        // Create a pointer to the node after the sublist to be reversed.
        ListNode afterSublist = sublistEnd.next;

        // Break the sublist to be reversed from the rest of the list.
        sublistEnd.next = null;

        // Reverse the sublist to be reversed.
        reverse(sublistStart);

        // Reconnect the reversed sublist to the rest of the list.
        beforeSublist.next = sublistEnd;
        sublistStart.next = afterSublist;

        return dummyHead.next;
    }
```

```
    private void reverse(ListNode head) {
        ListNode prev = null;
        ListNode current = head;
        while (current != null) {
            ListNode next = current.next;
            current.next = prev;
            prev = current;
            current = next;
        }
    }

}
```

58. `Solution00098.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.LinkedList;
import java.util.Queue;

/**
 * https://leetcode.com/problems/validate-binary-search-tree/
 * 98. Validate Binary Search Tree
 * Medium
 * Given the root of a binary tree, determine if it is a valid binary search tree (BST).
 * A valid BST is defined as follows:
 * 1. The left subtree of a node contains only nodes with keys less than the node's key.
 * 2. The right subtree of a node contains only nodes with keys greater than the node's
key.
 * 3. Both the left and right subtrees must also be binary search trees.
 * Example 1:
 * Input: root = [2,1,3]
 * Output: true
 * Example 2:
 * Input: root = [5,1,4,null,null,3,6]
 * Output: false
 * Explanation: The root node's value is 5 but its right child's value is 4.
 * Constraints:
 * The number of nodes in the tree is in the range [1, 10^4].
 * -2^31 <= Node.val <= 2^31 - 1
 */
public class Solution00098 {

    // Time O(n)
    // Space O(n)
    public boolean isValidBstDfs(TreeNode root) {
        return isValidBstDfs(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }

    private boolean isValidBstDfs(TreeNode currentNode, long min, long max) {
        if (currentNode == null) {
            return true;
        }

        if (currentNode.val <= min) {
            return false;
        }

        if (currentNode.val >= max) {
            return false;
        }

        return isValidBstDfs(currentNode.left, min, currentNode.val) &&
isValidBstDfs(currentNode.right, currentNode.val, max);
```

```
    }

    public boolean isValidBstBfs(TreeNode root) {
        if (root == null) {
            return true;
        }

        Queue<QueueData> queue = new LinkedList<>();
        queue.add(new QueueData(root, Long.MIN_VALUE, Long.MAX_VALUE));

        while (!queue.isEmpty()) {
            QueueData queueData = queue.poll();
            TreeNode currentNode = queueData.node;
            long min = queueData.min;
            long max = queueData.max;

            if (currentNode.val <= min) {
                return false;
            }

            if (currentNode.val >= max) {
                return false;
            }

            if (currentNode.left != null) {
                queue.add(new QueueData(currentNode.left, min, currentNode.val));
            }

            if (currentNode.right != null) {
                queue.add(new QueueData(currentNode.right, currentNode.val, max));
            }
        }

        return true;
    }

    private static class QueueData {
        TreeNode node;
        long min;
        long max;

        QueueData(TreeNode node, long min, long max) {
            this.node = node;
            this.min = min;
            this.max = max;
        }
    }

}
```

59. `Solution00100.java`

```
package leetcode;

import javafx.util.Pair;
import leetcode.common.TreeNode;

import java.util.LinkedList;
import java.util.Queue;

/**
 * https://leetcode.com/problems/same-tree/
 * 100. Same Tree
 * <p>
 * Given the roots of two binary trees p and q, write a function to check if they are the
same or not.
 * <p>
 * Two binary trees are considered the same if they are structurally identical,
 * and the nodes have the same value.
 * <p>
 * Example 1:
 * <p>
 * Input: p = [1,2,3], q = [1,2,3]
 * Output: true
 * <p>
 * Example 2:
 * <p>
 * Input: p = [1,2], q = [1,null,2]
 * Output: false
 * <p>
 * Example 3:
 * <p>
 * Input: p = [1,2,1], q = [1,1,2]
 * Output: false
 * <p>
 * Constraints:
 * <p>
 * The number of nodes in both trees is in the range [0, 100].
 * -104 <= Node.val <= 104
 */
public class Solution00100 {

    // Time: O(n)
    // Space: O(n)
    public boolean isSameTree(TreeNode p, TreeNode q) {
        // Create a queue to store the nodes that need to be compared.
        Queue<Pair<TreeNode, TreeNode>> queue = new LinkedList<>();

        // Add the root nodes of the two trees to the queue.
        queue.add(new Pair<>(p, q));

        // While the queue is not empty, do the following:
        while (!queue.isEmpty()) {
```

```
            // Remove the next pair of nodes from the queue.
            Pair<TreeNode, TreeNode> nodes = queue.poll();
            p = nodes.getKey();
            q = nodes.getValue();

            // If both nodes are null, then the trees are the same.
            if (p == null && q == null) {
                continue;
            }

            // If either node is null, then the trees are not the same.
            if (p == null || q == null) {
                return false;
            }

            // If the values of the nodes are not the same, then the trees are not the
same.
            if (p.val != q.val) {
                return false;
            }

            // Add the left and right subtrees of the current nodes to the queue.
            queue.add(new Pair<>(p.left, q.left));
            queue.add(new Pair<>(p.right, q.right));
        }

        // If the queue is empty, then the trees are the same.
        return true;
    }

}
```

60. `Solution00102.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * <a href="https://leetcode.com/problems/binary-tree-level-order-traversal/">
 * 102. Binary Tree Level Order Traversal
 * </a>
 * <p>
 * Given the root of a binary tree, return the level order traversal of its nodes'
values. (i.e., from left to right,
 * level by level).
 * <p>
 * Example 1:
 * <p>
 * Input: root = [3,9,20,null,null,15,7]
 * Output: [[3],[9,20],[15,7]]
 * <p>
 * Example 2:
 * <p>
 * Input: root = [1]
 * Output: [[1]]
 * <p>
 * Example 3:
 * <p>
 * Input: root = []
 * Output: []
 * <p>
 * <p>
 * Constraints:
 * <p>
 * The number of nodes in the tree is in the range [0, 2000].
 * -1000 <= Node.val <= 1000
 * <p>
 * <p>
 * Follow-up: Recursive solution is trivial, could you do it iteratively?
 */
public class Solution00102 {

    // Time O(n)
    // Space O(n)
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> result = new ArrayList<>();
        if (root == null) return List.of();

        Queue<TreeNode> queue = new LinkedList<>();
```

```
        queue.offer(root);

        while (!queue.isEmpty()) {
            int size = queue.size();
            List<Integer> level = new ArrayList<>(size);
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                level.add(node.val);

                if (node.left != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
            result.add(level);
        }

        return result;
    }
}
```

61. `Solution00104.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.Deque;
import java.util.LinkedList;
import java.util.Queue;

/**
 * https://leetcode.com/problems/maximum-depth-of-binary-tree/
 * 104. Maximum Depth of Binary Tree
 * <p>
 * Given the root of a binary tree, return its maximum depth.
 * <p>
 * A binary tree's maximum depth is the number of nodes along
 * the longest path from the root node down to the farthest leaf node.
 * <p>
 * Example 1:
 * <p>
 * Input: root = [3,9,20,null,null,15,7]
 * Output: 3
 * <p>
 * Example 2:
 * <p>
 * Input: root = [1,null,2]
 * Output: 2
 * <p>
 * Example 3:
 * <p>
 * Input: root = []
 * Output: 0
 * <p>
 * Example 4:
 * <p>
 * Input: root = [0]
 * Output: 1
 * <p>
 * Constraints:
 * <p>
 * The number of nodes in the tree is in the range [0, 104].
 * -100 <= Node.val <= 100
 */
public class Solution00104 {

    // Time: O(n)
    // Space: O(n)
    public int maxDepthRecursiveDfs(TreeNode root) {
        if (root == null) {
            return 0;
        }
```

```
        return 1 + Math.max(maxDepthRecursiveDfs(root.left),
maxDepthRecursiveDfs(root.right));
    }

    // Time: O(n)
    // Space: O(n)
    public int maxDepthIterativeBfs(TreeNode root) {
        if (root == null) {
            return 0;
        }

        // Create a queue to store the nodes that need to be visited.
        Queue<TreeNode> queue = new LinkedList<>();

        // Add the root node to the queue.
        queue.add(root);

        // Create a variable to store the maximum depth.
        int maxDepth = 0;

        // While the queue is not empty, do the following:
        while (!queue.isEmpty()) {
            // Remove the next node from the queue.
            final int queueSize = queue.size();
            for (int i = 0; i < queueSize; i++) {
                TreeNode node = queue.remove();
                if (node.left != null) {
                    // Add the left child of the node to the queue.
                    queue.add(node.left);
                }
                if (node.right != null) {
                    // Add the right child of the node to the queue.
                    queue.add(node.right);
                }
            }

            maxDepth++;
        }

        // Return the maximum depth.
        return maxDepth;
    }

    // Time: O(n)
    // Space: O(n)
    public int maxDepthIterativeDfs(TreeNode root) {
        if (root == null) {
            return 0;
        }

        // Create a stack to store the nodes that need to be visited.
        Deque<TreeNode> stack = new LinkedList<>();

        // Create a stack to store the depths of the nodes that need to be visited.
```

```
        Deque<Integer> depths = new LinkedList<>();

        // Add the root node to the stack.
        stack.push(root);

        // Add the depth of the root node to the stack.
        depths.push(1);

        // Create a variable to store the maximum depth.
        int maxDepth = 1;

        // While the stack is not empty, do the following:
        while (!stack.isEmpty()) {
            // Remove the next node from the stack.
            TreeNode node = stack.pop();
            // Remove the depth of the next node from the stack.
            int depth = depths.pop();

            // If the depth of the next node is greater than the maximum depth, update
the maximum depth.
            if (depth > maxDepth) {
                maxDepth = depth;
            }

            // If the left child of the next node is not null, do the following:
            if (node.left != null) {
                // Add the left child of the next node to the stack.
                stack.push(node.left);
                // Add the depth of the left child of the next node to the stack.
                depths.push(depth + 1);
            }

            // If the right child of the next node is not null, do the following:
            if (node.right != null) {
                // Add the right child of the next node to the stack.
                stack.push(node.right);
                // Add the depth of the right child of the next node to the stack.
                depths.push(depth + 1);
            }
        }

        // Return the maximum depth.
        return maxDepth;
    }

}
```

62. `Solution00105.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.HashMap;
import java.util.Map;

/**
 * https://leetcode.com/problems/construct-binary-tree-from-preorder-and-inorder-
traversal/
 * 105. Construct Binary Tree from Preorder and Inorder Traversal
 * Medium
 * <p>
 * Share
 * Given two integer arrays preorder and inorder where preorder is the preorder traversal
of a binary tree and inorder
 * is the inorder traversal of the same tree, construct and return the binary tree.
 * <p>
 * Example 1:
 * <p>
 * Input: preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]
 * Output: [3,9,20,null,null,15,7]
 * <p>
 * Example 2:
 * <p>
 * Input: preorder = [-1], inorder = [-1]
 * Output: [-1]
 * <p>
 * Constraints:
 * <p>
 * 1 <= preorder.length <= 3000
 * inorder.length == preorder.length
 * -3000 <= preorder[i], inorder[i] <= 3000
 * preorder and inorder consist of unique values.
 * Each value of inorder also appears in preorder.
 * preorder is guaranteed to be the preorder traversal of the tree.
 * inorder is guaranteed to be the inorder traversal of the tree.
 */
public class Solution00105 {

    // Time: O(n)
    // Space: O(n)
    public TreeNode buildTree(int[] preorder, int[] inorder) {
        return buildTree(preorder, inorder, 0, 0, inorder.length - 1);
    }

    private TreeNode buildTree(int[] preorder, int[] inorder, int preStart, int inStart,
int inEnd) {
        if (preStart >= preorder.length || inStart > inEnd) {
            return null;
        }
```

```
        TreeNode root = new TreeNode(preorder[preStart]);
        int inIndex = inStart;
        while (inIndex <= inEnd && inorder[inIndex] != preorder[preStart]) {
            inIndex++;
        }

        root.left = buildTree(preorder, inorder, preStart + 1, inStart, inIndex - 1);
        final int inPreDiff = inIndex - inStart;
        root.right = buildTree(preorder, inorder, preStart + inPreDiff + 1, inIndex + 1,
inEnd);
        return root;
    }

    // Time: O(n)
    // Space: O(n)
    public TreeNode buildTreeWithMap(int[] preorder, int[] inorder) {
        final int arrLen = inorder.length;
        final Map<Integer, Integer> inOrderMap = new HashMap<>();
        for (int i = 0; i < arrLen; i++) {
            inOrderMap.put(inorder[i], i);
        }

        return buildTreeWithMap(preorder, 0, arrLen - 1,
                inOrderMap, 0, arrLen - 1
        );
    }

    private TreeNode buildTreeWithMap(int[] preorder, int preStart, int preEnd,
                                      Map<Integer, Integer> inOrderMap, int inStart, int
inEnd) {
        if (preStart > preEnd || inStart > inEnd) {
            return null;
        }

        // create the root node, which is the first node in preorder
        TreeNode root = new TreeNode(preorder[preStart]);

        // find the root node in in_order
        int inRoot = inOrderMap.get(root.val);

        // now , all the left side values in in_order is left tree, so calculate the
index
        int leftNodeIndexIn = inRoot - inStart;

        // set the left node
        root.left = buildTreeWithMap(preorder, preStart + 1, preStart + leftNodeIndexIn,
                inOrderMap, inStart, inRoot - 1);

        //set the right node
        root.right = buildTreeWithMap(preorder, preStart + leftNodeIndexIn + 1, preEnd,
                inOrderMap, inRoot + 1, inEnd);

        return root;
```

```
    }

}
```

63. `Solution00110.java`

```
package leetcode;

import leetcode.common.TreeNode;

/**
 * https://leetcode.com/problems/balanced-binary-tree/
 * 110. Balanced Binary Tree
 * <p>
 * Given a binary tree, determine if it is height-balanced.
 * <p>
 * For this problem, a height-balanced binary tree is defined as:
 * <p>
 * 1. a binary tree in which the left and right subtrees of every node differ in height
by no more than 1.
 * <p>
 * Example 1:
 * <p>
 * Input: root = [3,9,20,null,null,15,7]
 * Output: true
 * <p>
 * Example 2:
 * <p>
 * Input: root = [1,2,2,3,3,null,null,4,4]
 * Output: false
 * <p>
 * Example 3:
 * <p>
 * Input: root = []
 * Output: true
 * <p>
 * Constraints:
 * <p>
 * The number of nodes in the tree is in the range [0, 5000].
 * -104 <= Node.val <= 104
 */
public class Solution00110 {

    private static final int UNBALANCED = -1;

    // Time: O(n^2)
    // Space: O(n)
    public boolean isBalanced(TreeNode root) {
        if (root == null) {
            return true;
        }
        int leftDepth = maxDepthRecursiveDfs(root.left);
        int rightDepth = maxDepthRecursiveDfs(root.right);
        return Math.abs(leftDepth - rightDepth) <= 1
               && isBalanced(root.left)
               && isBalanced(root.right);
    }
```

```
    // Time: O(n)
    // Space: O(n)
    public int maxDepthRecursiveDfs(TreeNode root) {
        if (root == null) {
            return 0;
        }

        return 1 + Math.max(maxDepthRecursiveDfs(root.left),
maxDepthRecursiveDfs(root.right));
    }

    // Time: O(n)
    // Space: O(n)
    public boolean isBalancedOptimized(TreeNode root) {
        return dfs(root) != UNBALANCED;
    }

    private int dfs(TreeNode root) {
        if (root == null) {
            return 0;
        }

        int leftDepth = dfs(root.left);
        if (leftDepth == UNBALANCED) {
            return UNBALANCED;
        }

        int rightDepth = dfs(root.right);
        if (rightDepth == UNBALANCED) {
            return UNBALANCED;
        }

        if (Math.abs(leftDepth - rightDepth) > 1) {
            return UNBALANCED;
        }

        return 1 + Math.max(leftDepth, rightDepth);
    }

}
```

64. `Solution00118.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.List;

/**
 * 118. Pascal's Triangle
 * https://leetcode.com/problems/pascals-triangle/
 * Easy
 */

public class Solution00118 {

    public List<List<Integer>> generate(int numRows) {
        List<List<Integer>> result = new ArrayList<>();

        List<Integer> baseRow = new ArrayList<>();
        baseRow.add(1);
        result.add(baseRow);

        List<Integer> prevRow = baseRow;
        for (int i = 2; i <= numRows; i++) {
            int newSize = prevRow.size() + 1;
            List<Integer> newRow = new ArrayList<>(newSize);
            newRow.add(1);
            for (int j = 1; j < newSize - 1; j++) {
                newRow.add(prevRow.get(j - 1) + prevRow.get(j));
            }
            newRow.add(1);
            result.add(newRow);
            prevRow = newRow;
        }

        return result;
    }

}
```

65. `Solution00119.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 119. Pascal's Triangle II
 * https://leetcode.com/problems/pascals-triangle-ii/
 * Easy
 */

public class Solution00119 {

    public List<Integer> getRowOptimized(int rowIndex) {
        Integer[] result = new Integer[rowIndex + 1];
        result[0] = 1;
        result[rowIndex] = 1;

        for (int i = 1; i <= rowIndex / 2; i++) {
            int previousValue = result[i - 1];
            long multiplier = rowIndex - i + 1L;
            long product = previousValue * multiplier;
            long divisionResult = product / i;
            result[i] = (int) divisionResult;
            result[rowIndex - i] = (int) divisionResult;
        }

        return Arrays.asList(result);
    }

    public List<Integer> getRow(int rowIndex) {
        int numRows = rowIndex + 1;
        List<Integer> baseRow = new ArrayList<>();
        baseRow.add(1);

        List<Integer> actualRow = baseRow;
        for (int i = 2; i <= numRows; i++) {
            int newSize = actualRow.size() + 1;
            List<Integer> newRow = new ArrayList<>(newSize);
            newRow.add(1);
            for (int j = 1; j < newSize - 1; j++) {
                newRow.add(actualRow.get(j - 1) + actualRow.get(j));
            }
            newRow.add(1);
            actualRow = newRow;
        }

        return actualRow;
    }
```

```
}
```

66. `Solution00121.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/best-time-to-buy-and-sell-stock/
 * 121. Best Time to Buy and Sell Stock
 *
 * You are given an array prices where prices[i] is the price of a given stock on the ith
day.
 * You want to maximize your profit by choosing a single day to buy one stock and
choosing a different day in the future to sell that stock.

 * Return the maximum profit you can achieve from this transaction. If you cannot achieve
any profit, return 0.

 * Example 1:

 * Input: prices = [7,1,5,3,6,4]
 * Output: 5
 * Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5.
 * Note that buying on day 2 and selling on day 1 is not allowed because you must buy
before you sell.
 * Example 2:

 * Input: prices = [7,6,4,3,1]
 * Output: 0
 * Explanation: In this case, no transactions are done and the max profit = 0.

 * Constraints:

 * 1 <= prices.length <= 105
 * 0 <= prices[i] <= 104
 */
public class Solution00121 {

    // Time: O(n)
    // Space: O(1)
    public int maxProfit(int[] prices) {
        int maxProfit = 0;

        int left = 0;
        int right = 1;

        final int length = prices.length;

        while (right < length) {
            if (prices[left] >= prices[right]) {
                left = right;
                right = left + 1;
            } else {
                maxProfit = Math.max(maxProfit, prices[right] - prices[left]);
                right++;
```

```
            }
        }

        return maxProfit;
    }

}
```

67. `Solution00122.java`

```
package leetcode;

/**
 * 122. Best Time to Buy and Sell Stock II
 * https://leetcode.com/problems/best-time-to-buy-and-sell-stock-ii/
 * Medium
 */
public class Solution00122 {

    // Time: O(n)
    // Space: O(1)
    public int maxProfit(int[] prices) {
        int maxProfit = 0;
        for (int i = 1; i < prices.length; i++) {
            if (prices[i] > prices[i - 1]) {
                maxProfit += (prices[i] - prices[i - 1]);
            }
        }
        return maxProfit;
    }

}
```

68. `Solution00124.java`

```
package leetcode;

import leetcode.common.TreeNode;

/**
 * https://leetcode.com/problems/binary-tree-maximum-path-sum/
 * 124. Binary Tree Maximum Path Sum
 * Hard
 * <p>
 * Share
 * Given a non-empty binary tree, find the maximum path sum.
 * <p>
 * For this problem, a path is defined as any node sequence from some starting node to
any node
 * in the tree along the parent-child connections. The path must contain at least one
node and
 * does not need to go through the root.
 * <p>
 * <p>
 * <p>
 * Example 1:
 * <p>
 * <p>
 * Input: root = [1,2,3]
 * Output: 6
 * Example 2:
 * <p>
 * <p>
 * Input: root = [-10,9,20,null,null,15,7]
 * Output: 42
 * <p>
 * <p>
 * Constraints:
 * <p>
 * The number of nodes in the tree is in the range [0, 3 * 104].
 * -1000 <= Node.val <= 1000
 */
public class Solution00124 {

    public int maxPathSum(TreeNode root) {
        TraverseResult traverseResult = new TraverseResult();
        traverseResult.max = Integer.MIN_VALUE;
        traverse(root, traverseResult);
        return traverseResult.max;
    }

    private int traverse(TreeNode root, TraverseResult traverseResult) {
        if (root == null) {
            return 0;
        }
```

```
        int left = Math.max(0, traverse(root.left, traverseResult));
        int right = Math.max(0, traverse(root.right, traverseResult));
        traverseResult.max = Math.max(traverseResult.max, left + right + root.val);
        return Math.max(left, right) + root.val;
    }

    static class TraverseResult {
        int max;
    }

}
```

69. `Solution00125.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/valid-palindrome/
 * 125. Valid Palindrome
 * A phrase is a palindrome if, after converting all uppercase letters into lowercase
letters and removing all non-alphanumeric characters, it reads the same forward and
backward. Alphanumeric characters include letters and numbers.
 * <p>
 * Given a string s, return true if it is a palindrome, or false otherwise.
 * <p>
 * Example 1:
 * <p>
 * Input: s = "A man, a plan, a canal: Panama"
 * Output: true
 * Explanation: "amanaplanacanalpanama" is a palindrome.
 * Example 2:
 * <p>
 * Input: s = "race a car"
 * Output: false
 * Explanation: "raceacar" is not a palindrome.
 * Example 3:
 * <p>
 * Input: s = " "
 * Output: true
 * Explanation: s is an empty string "" after removing non-alphanumeric characters.
 * Since an empty string reads the same forward and backward, it is a palindrome.
 * <p>
 * Constraints:
 * <p>
 * 1 <= s.length <= 2 * 105
 * s consists only of printable ASCII characters.
 */
public class Solution00125 {

    // Time: O(n)
    // Space: O(1)
    public boolean isPalindrome(String s) {
        int start = 0;
        int end = s.length() - 1;

        while (start < end) {
            char startChar = s.charAt(start);
            char endChar = s.charAt(end);

            if (!Character.isLetterOrDigit(startChar)) {
                start++;
            } else if (!Character.isLetterOrDigit(endChar)) {
                end--;
            } else {
                if (Character.toLowerCase(startChar) != Character.toLowerCase(endChar)) {
```

```
                    return false;
                }
                start++;
                end--;
            }
        }
        return true;
    }

}
```

70. `Solution00128.java`

```
package leetcode;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

/**
 * https://leetcode.com/problems/longest-consecutive-sequence/
 * 128. Longest Consecutive Sequence
 * <p>
 * Given an unsorted array of integers nums, return the length of the longest consecutive
elements sequence.
 * <p>
 * You must write an algorithm that runs in O(n) time.
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [100,4,200,1,3,2]
 * Output: 4
 * Explanation: The longest consecutive elements sequence is [1, 2, 3, 4].
 * Therefore its length is 4.
 * Example 2:
 * <p>
 * Input: nums = [0,3,7,2,5,8,4,6,0,1]
 * Output: 9
 * <p>
 * <p>
 * Constraints:
 * <p>
 * 0 <= nums.length <= 105
 * -109 <= nums[i] <= 109
 */
public class Solution00128 {

    // Time: O(n)
    // Space: O(n)
    public int longestConsecutive(int[] nums) {
        // put all the numbers in the set
        Set<Integer> set = new HashSet<>();
        for (int num : nums) {
            set.add(num);
        }
        int max = 0;
        for (int num : nums) {
            // if num - 1 is not in the set, then num is the start of the sequence
            if (!set.contains(num - 1)) {
                int current = num;
                int currentMax = 1;
                // find the end of the sequence
                while (set.contains(current + 1)) {
                    current++;
```

```
                    currentMax++;
                }
                // update the max
                max = Math.max(max, currentMax);
            }
            // if the max is greater than half of the array, then we can return the max
            if (max > nums.length / 2) {
                return max;
            }
        }
        return max;
    }

    // Time: O(n log n)
    // Space: O(1)
    public int longestConsecutiveSorting(int[] nums) {
        if (nums.length == 0) {
            return 0;
        }
        Arrays.sort(nums);
        int max = 1;
        int currentMax = 1;
        for (int i = 1; i < nums.length; i++) {
            int current = nums[i];
            int prev = nums[i - 1];
            if (current == prev + 1) {
                currentMax++;
            } else if (current != prev) {
                max = Math.max(max, currentMax);
                currentMax = 1;
            }
        }
        return Math.max(max, currentMax);
    }

}
```

71. `Solution00129.java`

```
package leetcode;

import leetcode.common.TreeNode;

/**
 * 129. Sum Root to Leaf Numbers
 * https://leetcode.com/problems/sum-root-to-leaf-numbers/
 * Medium
 */

public class Solution00129 {
    public int sumNumbers(TreeNode root) {
        return sumNumbers(root, 0);
    }

    private int sumNumbers(TreeNode root, int sum) {
        if (root == null) {
            return 0;
        }
        sum = sum * 10 + root.val;
        if (root.left == null && root.right == null) {
            return sum;
        }
        return sumNumbers(root.left, sum) + sumNumbers(root.right, sum);
    }
}
```

72. `Solution00131.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.List;

/**
 * 131. Palindrome Partitioning
 * <a href="https://leetcode.com/problems/palindrome-partitioning">Web link</a> |
 * Medium
 */

public class Solution00131 {

    public List<List<String>> partition(String s) {
        List<List<String>> result = new ArrayList<>();
        dfs(s, 0, new ArrayList<>(), result);
        return result;
    }

    private void dfs(String s, int start, List<String> path, List<List<String>> result) {
        if (start == s.length()) {
            result.add(new ArrayList<>(path));
            return;
        }

        for (int i = start; i < s.length(); i++) {
            if (isPalindrome(s, start, i)) {
                path.add(s.substring(start, i + 1));
                dfs(s, i + 1, path, result);
                path.removeLast();
            }
        }
    }

    private boolean isPalindrome(String s, int start, int end) {
        while (start < end) {
            if (s.charAt(start++) != s.charAt(end--)) {
                return false;
            }
        }
        return true;
    }

}
```

73. `Solution00133.java`

```
package leetcode;

import leetcode.common.Node;

import java.util.HashMap;
import java.util.LinkedList;
import java.util.Map;
import java.util.Queue;

/**
 * https://leetcode.com/problems/clone-graph/
 * 133. Clone Graph
 * <p>
 * Given a reference of a node in a connected undirected graph.
 * <p>
 * Return a deep copy (clone) of the graph.
 * <p>
 * Each node in the graph contains a value (int) and a list (List[Node]) of its
neighbors.
 * <p>
 * class Node {
 * public int val;
 * public List<Node> neighbors;
 * }
 * <p>
 * Test case format:
 * <p>
 * For simplicity, each node's value is the same as the node's index (1-indexed).
 * For example, the first node with val == 1, the second node with val == 2, and so on.
 * The graph is represented in the test case using an adjacency list.
 * <p>
 * An adjacency list is a collection of unordered lists used to represent a finite graph.
 * Each list describes the set of neighbors of a node in the graph.
 * <p>
 * The given node will always be the first node with val = 1.
 * You must return the copy of the given node as a reference to the cloned graph.
 * <p>
 * Example 1:
 * <p>
 * Input: adjList = [[2,4],[1,3],[2,4],[1,3]]
 * Output: [[2,4],[1,3],[2,4],[1,3]]
 * <p>
 * Explanation: There are 4 nodes in the graph.
 * 1st node (val = 1) has neighbors 2 and 4.
 * 2nd node (val = 2) has neighbors 1 and 3.
 * 3rd node (val = 3) has neighbors 2 and 4.
 * 4th node (val = 4) has neighbors 1 and 3.
 * <p>
 * Example 2:
 * <p>
 * Input: adjList = [[]]
```

```
 * Output: [[]]
 * <p>
 * Explanation: Note that the input contains one empty list.
 * The graph consists of only one node with val = 1 and it does not have any neighbors.
 * <p>
 * Example 3:
 * <p>
 * Input: adjList = []
 * Output: []
 * <p>
 * Explanation: This an empty graph, it does not have any nodes.
 *
 * Constraints:
 *
 * The number of nodes in the graph is in the range [0, 100].
 * 1 <= Node.val <= 100
 * Node.val is unique for each node.
 * There are no repeated edges and no self-loops in the graph.
 * The Graph is connected and all nodes can be visited starting from the given node.
 */
public class Solution00133 {

    // Time: O(n)
    // Space: O(n)
    public Node cloneGraphBfs(Node node) {
        if (node == null) {
            return null;
        }
        Map<Node, Node> clonedNodes = new HashMap<>();
        Queue<Node> queue = new LinkedList<>();

        queue.add(node);
        clonedNodes.put(node, new Node(node.val));

        while (!queue.isEmpty()) {
            Node currentNode = queue.poll();
            Node clonedCurrentNode = clonedNodes.get(currentNode);

            for (Node neighbor : currentNode.neighbors) {
                if (!clonedNodes.containsKey(neighbor)) {
                    clonedNodes.put(neighbor, new Node(neighbor.val));
                    queue.add(neighbor);
                }

                clonedCurrentNode.neighbors.add(clonedNodes.get(neighbor));
            }
        }

        return clonedNodes.get(node);
    }

    // Time: O(n)
    // Space: O(n)
    public Node cloneGraphDfs(Node node) {
```

```
        if (node == null) {
            return null;
        }
        Map<Node, Node> clonedNodes = new HashMap<>();
        return cloneGraphDfs(node, clonedNodes);
    }

    private Node cloneGraphDfs(Node node, Map<Node, Node> clonedNodes) {
        final Node clonedNode = clonedNodes.get(node);
        if (clonedNode != null) {
            return clonedNode;
        }

        Node newClonedNode = new Node(node.val);
        clonedNodes.put(node, newClonedNode);

        for (Node neighbor : node.neighbors) {
            newClonedNode.neighbors.add(cloneGraphDfs(neighbor, clonedNodes));
        }

        return newClonedNode;
    }

}
```

74. `Solution00134.java`

```
package leetcode;

/**
 * 134. Gas Station
 * https://leetcode.com/problems/gas-station/
 * Medium
 */

public class Solution00134 {

    public int canCompleteCircuit(int[] gas, int[] cost) {
        int n = gas.length;
        int total = 0;
        int tank = 0;
        int start = 0;

        for (int i = 0; i < n; i++) {
            int consume = gas[i] - cost[i];
            tank += consume;
            total += consume;

            if (tank < 0) {
                start = i + 1;
                tank = 0;
            }
        }

        return total >= 0 ? start : -1;
    }

}
```

75. `Solution00138.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * https://leetcode.com/problems/copy-list-with-random-pointer/
 * 138. Copy List with Random Pointer
 * Medium
 */
public class Solution00138 {

    // Time: O(n)
    // Space: O(n)
    public Node copyRandomList(Node head) {
        // Create a hash map to store the mapping from the original node to the copied
node.
        Map<Node, Node> map = new HashMap<>();
        // Create a dummy node to be the head of the copied linked list.
        Node dummy = new Node(0);
        // Create a pointer to the dummy node.
        Node curr = dummy;
        // Create a pointer to the head of the original linked list.
        Node original = head;
        // Iterate over the nodes in the original linked list.
        while (original != null) {
            // Create a new node with the same value as the current node in the original
linked list.
            Node copy = new Node(original.val);
            // Add the mapping from the current node in the original linked list to the
new node to the hash map.
            map.put(original, copy);
            // Set the next pointer of the current node in the copied linked list to the
new node.
            curr.next = copy;
            // Move the pointer to the current node in the original linked list to the
next node.
            original = original.next;
            // Move the pointer to the current node in the copied linked list to the next
node.
            curr = curr.next;
        }
        // Reset the pointer to the head of the original linked list.
        original = head;
        // Reset the pointer to the head of the copied linked list.
        curr = dummy.next;
        // Iterate over the nodes in the original linked list.
        while (original != null) {
            // Set the random pointer of the current node in the copied linked list to
```

```
the node in the hash map
            // that the random pointer of the current node in the original linked list
points to.
            curr.random = map.get(original.random);
            // Move the pointer to the current node in the original linked list to the
next node.
            original = original.next;
            // Move the pointer to the current node in the copied linked list to the next
node.
            curr = curr.next;
        }
        // Return the head of the copied linked list.
        return dummy.next;
    }

    // Time: O(n)
    // Space: O(1)
    public Node copyRandomListInterweavingNodes(Node head) {
        // Create a pointer to the head of the original linked list.
        Node original = head;
        // Iterate over the nodes in the original linked list.
        while (original != null) {
            // Create a new node with the same value as the current node in the original
linked list.
            Node copy = new Node(original.val);
            // Set the next pointer of the new node to the next pointer of the current
node in the original linked list.
            copy.next = original.next;
            // Set the next pointer of the current node in the original linked list to
the new node.
            original.next = copy;
            // Move the pointer to the current node in the original linked list to the
next node.
            original = original.next.next;
        }
        // Reset the pointer to the head of the original linked list.
        original = head;
        // Iterate over the nodes in the original linked list.
        while (original != null) {
            // Set the random pointer of the new node to the random pointer of the
current node in the original linked list.
            original.next.random = original.random == null ? null : original.random.next;
            // Move the pointer to the current node in the original linked list to the
next node.
            original = original.next.next;
        }
        // Create a dummy node to be the head of the copied linked list.
        Node dummy = new Node(0);
        // Create a pointer to the dummy node.
        Node curr = dummy;
        // Reset the pointer to the head of the original linked list.
        original = head;
        // Iterate over the nodes in the original linked list.
        while (original != null) {
```

```
            // Set the next pointer of the current node in the copied linked list to the
next pointer of the current node in the original linked list.
            curr.next = original.next;
            // Move the pointer to the current node in the original linked list to the
next node.
            original = original.next.next;
            // Move the pointer to the current node in the copied linked list to the next
node.
            curr = curr.next;
        }
        // Return the head of the copied linked list.
        return dummy.next;
    }

    static class Node {
        int val;
        Node next;
        Node random;

        public Node(int val) {
            this.val = val;
            this.next = null;
            this.random = null;
        }

        @Override
        public String toString() {
            var head = this;
            List<List<Integer>> values = new ArrayList<>();
            while (head != null) {
                List<Integer> value = new ArrayList<>();
                value.add(head.val);
                value.add(head.random == null ? null : head.random.val);
                values.add(value);
                head = head.next;
            }
            return values.toString();
        }
    }

}
```

76. `Solution00140.java`

```
package leetcode;

import java.util.*;

/**
 * 140. Word Break II
 * https://leetcode.com/problems/word-break-ii/
 * Hard
 */

public class Solution00140 {

    public List<String> wordBreak(String s, List<String> wordDict) {
        Set<String> wordSet = new HashSet<>(wordDict);
        Map<Integer, List<String>> memo = new HashMap<>();
        return wordBreak(s, wordSet, 0, memo);
    }

    private List<String> wordBreak(String s, Set<String> wordSet, int start, Map<Integer,
List<String>> memo) {
        if (memo.containsKey(start)) {
            return memo.get(start);
        }

        List<String> result = new ArrayList<>();
        if (start == s.length()) {
            result.add("");
        }

        for (int end = start + 1; end <= s.length(); end++) {
            String word = s.substring(start, end);
            if (wordSet.contains(word)) {
                List<String> nextWords = wordBreak(s, wordSet, end, memo);
                for (String nextWord : nextWords) {
                    result.add(word + (nextWord.isEmpty() ? "" : " ") + nextWord);
                }
            }
        }

        memo.put(start, result);
        return result;
    }

}
```

77. `Solution00141.java`

```
package leetcode;

import leetcode.common.ListNode;

/**
 * https://leetcode.com/problems/linked-list-cycle/
 * 141. Linked List Cycle
 * <p>
 * Given head, the head of a linked list, determine if the linked list has a cycle in it.
 * <p>
 * There is a cycle in a linked list if there is some node in the list that can be
reached again by continuously following the next pointer.
 * Internally, pos is used to denote the index of the node that tail's next pointer is
connected to.
 * Note that pos is not passed as a parameter.
 * <p>
 * Return true if there is a cycle in the linked list. Otherwise, return false.
 * <p>
 * Example 1:
 * <p>
 * Input: head = [3,2,0,-4], pos = 1
 * Output: true
 * Explanation: There is a cycle in the linked list, where the tail connects to the 1st
node (0-indexed).
 * <p>
 * Example 2:
 * <p>
 * Input: head = [1,2], pos = 0
 * Output: true
 * Explanation: There is a cycle in the linked list, where the tail connects to the 0th
node.
 * <p>
 * Example 3:
 * <p>
 * Input: head = [1], pos = -1
 * Output: false
 * Explanation: There is no cycle in the linked list.
 * <p>
 * Constraints:
 * <p>
 * The number of the nodes in the list is in the range [0, 104].
 * -105 <= Node.val <= 105
 * pos is -1 or a valid index in the linked-list.
 * <p>
 * Follow up: Can you solve it using O(1) (i.e. constant) memory?
 */
public class Solution00141 {

    // Time: O(n)
    // Space: O(1)
    public boolean hasCycle(ListNode head) {
```

```
        ListNode slowPointer = head;
        ListNode fastPointer = head;

        do {
            if (fastPointer == null || fastPointer.next == null) {
                return false;
            }

            slowPointer = slowPointer.next;
            fastPointer = fastPointer.next.next;
        } while (slowPointer != fastPointer);

        return true;
    }

}
```

78. `Solution00143.java`

```
package leetcode;

import leetcode.common.ListNode;

/**
 * 143. Reorder List
 * https://leetcode.com/problems/reorder-list/
 * <p>
 * Medium
 * <p>
 * 2516
 * <p>
 * 131
 * <p>
 * Add to List
 * <p>
 * Share
 * You are given the head of a singly linked-list. The list can be represented as:
 * <p>
 * L0 → L1 → … → Ln - 1 → Ln
 * Reorder the list to be on the following form:
 * <p>
 * L0 → Ln → L1 → Ln - 1 → L2 → Ln - 2 → …
 * You may not modify the values in the list's nodes. Only nodes themselves may be
changed.
 * <p>
 * Example 1:
 * <p>
 * Input: head = [1,2,3,4]
 * Output: [1,4,2,3]
 * Example 2:
 * <p>
 * Input: head = [1,2,3,4,5]
 * Output: [1,5,2,4,3]
 * <p>
 * Constraints:
 * <p>
 * The number of nodes in the list is in the range [1, 5 * 104].
 * 1 <= Node.val <= 1000
 * <p>
 * Follow up: Can you solve the problem without modifying the list's values?
 */
public class Solution00143 {

    // Time: O(n)
    // Space: O(1)
    public void reorderList(ListNode head) {
        // Find the middle of the list and split it into two lists
        ListNode middle = findMiddleAndSplit(head);

        // Reverse the second half of the list
```

```
        ListNode reversed = reverse(middle);

        // Merge the two lists
        merge(head, reversed);
    }

    private ListNode findMiddleAndSplit(ListNode head) {
        // Create two pointers, slow and fast
        ListNode slow = head;
        ListNode fast = head.next;

        // Move slow one node at a time and fast two nodes at a time
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        // remember the middle node before splitting the list
        ListNode middle = slow.next;
        // Split the list
        slow.next = null;

        return middle;
    }

    private ListNode reverse(ListNode head) {
        // Create a dummy node to store the head of the reversed list
        ListNode dummyHead = new ListNode();

        // Iterate over the list and add each node to the front of the reversed list
        while (head != null) {
            // Store the next node
            ListNode next = head.next;

            // Add the current node to the front of the reversed list
            head.next = dummyHead.next;
            dummyHead.next = head;

            // Move head forward
            head = next;
        }

        // Return the head of the reversed list
        return dummyHead.next;
    }

    private void merge(ListNode list1, ListNode list2) {
        // Create a dummy node to store the head of the merged list
        ListNode dummyHead = new ListNode();
        ListNode current = dummyHead;

        boolean addList1 = true;
        // Iterate over both lists and add from each list one element at a time
        while (list1 != null && list2 != null) {
```

```
            if (addList1) {
                // Add list1 to the result
                current.next = list1;

                // Move list1 forward
                list1 = list1.next;
                addList1 = false;
            } else {
                // Add list2 to the result
                current.next = list2;

                // Move list2 forward
                list2 = list2.next;
                addList1 = true;
            }
            // Move current forward
            current = current.next;
        }

        // Add remaining nodes
        current.next = list1 == null ? list2 : list1;
    }

}
```

79. `Solution00145.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.ArrayList;
import java.util.List;

/**
 * 145. Binary Tree Postorder Traversal
 * <a href="https://leetcode.com/problems/binary-tree-postorder-traversal">Web link</a>
 * Easy
 */

public class Solution00145 {

    public List<Integer> postorderTraversal(TreeNode root) {
        final List<Integer> list = new ArrayList<>();
        postorderTraversal(root, list);
        return list;
    }

    private void postorderTraversal(TreeNode root, List<Integer> list) {
        if (root == null) return;
        postorderTraversal(root.left, list);
        postorderTraversal(root.right, list);
        list.add(root.val);
    }

}
```

80. `Solution00146.java`

```
package leetcode;

import java.util.HashMap;
import java.util.Map;

/**
 * <a href="https://leetcode.com/problems/lru-cache/">
 * 146. LRU Cache
 * </a>
 * <br/>
 * <div class="content__u3I1 question-content__JfgR">
 * <div><p>Design a data structure that follows the constraints of a <a
href="https://en.wikipedia.org/wiki/Cache_replacement_policies#LRU" target="_blank">Least
Recently Used (LRU) cache</a>.</p>
 *
 * <p>Implement the <code>LRUCache</code> class:</p>
 *
 * <ul>
 *      <li><code>LRUCache(int capacity)</code> Initialize the LRU cache with
<em>positive</em> size <code>capacity</code>.</li>
 *      <li><code>int get(int key)</code> Return the value of the <code>key</code> if the
key exists, otherwise return <code>-1</code>.</li>
 *      <li><code>void put(int key, int value)</code>&amp;nbsp;Update the value of the
<code>key</code> if the <code>key</code> exists. Otherwise, add the <code>key-
value</code> pair to the cache. If the number of keys exceeds the <code>capacity</code>
from this operation, <strong>evict</strong> the least recently used key.</li>
 * </ul>
 *
 * <p><strong>Follow up:</strong><br />
 * Could you do <code>get</code> and <code>put</code> in <code>O(1)</code> time
complexity?</p>
 *
 * <p>&amp;nbsp;</p>
 * <p><strong>Example 1:</strong></p>
 *
 * <pre>
 * <strong>Input</strong>
 * ["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]
 * [[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]
 * <strong>Output</strong>
 * [null, null, null, 1, null, -1, null, -1, 3, 4]
 *
 * <strong>Explanation</strong>
 * LRUCache lRUCache = new LRUCache(2);
 * lRUCache.put(1, 1); // cache is {1=1}
 * lRUCache.put(2, 2); // cache is {1=1, 2=2}
 * lRUCache.get(1);    // return 1
 * lRUCache.put(3, 3); // LRU key was 2, evicts key 2, cache is {1=1, 3=3}
 * lRUCache.get(2);    // returns -1 (not found)
 * lRUCache.put(4, 4); // LRU key was 1, evicts key 1, cache is {4=4, 3=3}
 * lRUCache.get(1);    // return -1 (not found)
```

```
 * lRUCache.get(3);    // return 3
 * lRUCache.get(4);    // return 4
 * </pre>
 *
 * <p> </p>
 * <p><strong>Constraints:</strong></p>
 *
 * <ul>
 *      <li><code>1 &lt;= capacity &lt;= 3000</code></li>
 *      <li><code>0 &lt;= key &lt;= 10<sup>4</sup></code></li>
 *      <li><code>0 &lt;= value &lt;= 10<sup>5</sup></code></li>
 *      <li>At most <code>2 * 10<sup>5</sup></code> calls will be made to
<code>get</code> and <code>put</code>.</li>
 * </ul>
 * </div>
 * </div>
 */
public class Solution00146 {

    // Time: O(1)
    // Space: O(n)
    public static class LRUCache {

        private final int capacity;
        private final Map<Integer, Node> map;
        private final Node dummyMost;
        private final Node dummyLeast;

        public LRUCache(int capacity) {
            this.capacity = capacity;
            this.map = new HashMap<>();
            this.dummyMost = new Node();
            this.dummyLeast = new Node();
            dummyMost.previous = dummyLeast;
            dummyLeast.next = dummyMost;
        }

        public int get(int key) {
            Node node = map.get(key);
            if (node == null) {
                return -1;
            }

            remove(node);
            add(node);

            return node.value;
        }

        private void remove(Node node) {
            Node savedNext = node.next;
            Node savedPrevious = node.previous;
            savedNext.previous = savedPrevious;
            savedPrevious.next = savedNext;
```

```
        }

        private void add(Node node) {
            Node oldMost = dummyMost.previous;
            // link dummyMost to new most
            dummyMost.previous = node;
            node.next = dummyMost;
            // link current most to previous most
            node.previous = oldMost;
            oldMost.next = node;
        }

        public void put(int key, int value) {
            Node node = map.get(key);
            if (node != null) {
                remove(node);
            } else if (map.size() == capacity) {
                final Node least = dummyLeast.next;
                remove(least);
                map.remove(least.key);
            }

            Node newMost = new Node(key, value);
            add(newMost);
            map.put(key, newMost);
        }

        private static class Node {

            private final int key;
            private int value;
            private Node previous;
            private Node next;

            public Node() {
                this.key = -1;
            }

            public Node(int key, int value) {
                this.key = key;
                this.value = value;
            }

        }

    }
}
```

81. `Solution00150.java`

```
package leetcode;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * https://leetcode.com/problems/evaluate-reverse-polish-notation/
 * 150. Evaluate Reverse Polish Notation
 * <p>
 * Evaluate the value of an arithmetic expression in Reverse Polish Notation.
 * <p>
 * Valid operators are +, -, *, and /. Each operand may be an integer or another
expression.
 * <p>
 * Note that division between two integers should truncate toward zero.
 * <p>
 * It is guaranteed that the given RPN expression is always valid. That means the
expression would always evaluate to
 * a result, and there will not be any division by zero operation.
 * <p>
 * Example 1:
 * <p>
 * Input: tokens = ["2","1","+","3","*"] Output: 9 Explanation: ((2 + 1) * 3) = 9 Example
2:
 * <p>
 * Input: tokens = ["4","13","5","/","+"] Output: 6 Explanation: (4 + (13 / 5)) = 6
Example 3:
 * <p>
 * Input: tokens = ["10","6","9","3","+","-11","*","/","*","17","+","5","+"] Output: 22
Explanation: ((10 * (6 / ((9 +
 * 3) * -11))) + 17) + 5 = ((10 * (6 / (12 * -11))) + 17) + 5 = ((10 * (6 / -132)) + 17)
+ 5 = ((10 * 0) + 17) + 5 =
 * (0 + 17) + 5 = 17 + 5 = 22
 * <p>
 * <p>
 * Constraints:
 * <p>
 * 1 <= tokens.length <= 104 tokens[i] is either an operator: "+", "-", "*", or "/", or
an integer in the range [-200,
 * 200].
 * <p>
 * <p>
 * <p>
 * Follow up: Could you solve it without using stack?
 */
public class Solution00150 {

    // Time: O(n)
    // Space: O(n)
    public int evalRPN(String[] tokens) {
        // Create a stack.
```

```
        Deque<Integer> stack = new ArrayDeque<>();
        int num1;
        int num2;

        // Loop through the tokens.
        for (String token : tokens) {
            // If the token is an operator.
            final char firstChar = token.charAt(0);
            if (Character.isDigit(firstChar) || token.length() > 1) {
                stack.push(Integer.parseInt(token));
            } else {
                num2 = stack.pop();
                num1 = stack.pop();
                switch (firstChar) {
                    case '+' -> stack.push(num1 + num2);
                    case '-' -> stack.push(num1 - num2);
                    case '*' -> stack.push(num1 * num2);
                    case '/' -> stack.push(num1 / num2);
                }
            }
        }

        // Return the top element from the stack.
        return stack.pop();
    }

}
```

82. `Solution00152.java`

```
package leetcode;

/**
 * 152. Maximum Product Subarray
 * https://leetcode.com/problems/maximum-product-subarray/description/
 * Medium
 */
public class Solution00152 {

    public int maxProduct(int[] nums) {
        int n = nums.length;
        int[] dpMax = new int[n + 1];
        int[] dpMin = new int[n + 1];
        dpMax[0] = 1;
        dpMin[0] = 1;
        int max = nums[0];
        for (int i = 0; i < n; i++) {
            int currentValue = nums[i];
            int maxAndValue = dpMax[i] * currentValue;
            int minAndValue = dpMin[i] * currentValue;
            int currentMax = Math.max(currentValue, Math.max(maxAndValue, minAndValue));
            int currentMin = Math.min(currentValue, Math.min(maxAndValue, minAndValue));
            max = Math.max(max, currentMax);
            dpMax[i + 1] = currentMax;
            dpMin[i + 1] = currentMin;
        }
        return max;
    }

    public int maxProductOptimized(int[] nums) {
        int n = nums.length;
        int max = nums[0];
        int maxSoFar = max;
        int minSoFar = max;
        for (int i = 1; i < n; i++) {
            int currentValue = nums[i];
            int maxAndValue = maxSoFar * currentValue;
            int minAndValue = minSoFar * currentValue;
            maxSoFar = Math.max(currentValue, Math.max(maxAndValue, minAndValue));
            minSoFar = Math.min(currentValue, Math.min(maxAndValue, minAndValue));
            max = Math.max(max, maxSoFar);
        }
        return max;
    }

}
```

83. `Solution00153.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/find-minimum-in-rotated-sorted-array/
 * 153. Find Minimum in Rotated Sorted Array
 * <p>
 * Suppose an array sorted in ascending order is rotated at some pivot unknown to you
beforehand.
 * <p>
 * (i.e., 0 1 2 4 5 6 7 might become 4 5 6 7 0 1 2).
 * <p>
 * Find the minimum element.
 * <p>
 * You may assume no duplicate exists in the array.
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [3,4,5,1,2]
 * Output: 1
 * Explanation: The original array was [1,2,3,4,5] rotated 3 times.
 * Example 2:
 * <p>
 * Input: nums = [4,5,6,7,0,1,2]
 * Output: 0
 * Explanation: The original array was [0,1,2,4,5,6,7] and it was rotated 4 times.
 * Example 3:
 * <p>
 * Input: nums = [11,13,15,17]
 * Output: 11
 * Explanation: The original array was [11,13,15,17] and it was rotated 4 times.
 */
public class Solution00153 {

    // Time: O(logn)
    // Space: O(1)
    public int findMin(int[] nums) {
        // Create a variable to keep track of the left index.
        int left = 0;

        // Create a variable to keep track of the right index.
        int right = nums.length - 1;

        // Loop while the left index is less than the right index.
        while (left < right) {
            // Create a variable to keep track of the middle index.
            int mid = left + (right - left) / 2;

            // If the middle element is less than the right element.
            if (nums[mid] < nums[right]) {
                // Update the right index.
                right = mid;
```

```
            } else {
                // Update the left index.
                left = mid + 1;
            }
        }

        // Return the element at the left index.
        return nums[left];
    }

}
```

84. `Solution00155.java`

```
package leetcode;

import java.util.Deque;
import java.util.LinkedList;

/**
 * https://leetcode.com/problems/min-stack/
 * 155. Min Stack
 * <p>
 * Design a stack that supports push, pop, top, and retrieving the minimum element in
constant time.
 * <p>
 * Implement the MinStack class:
 * <p>
 * 1. MinStack() initializes the stack object.
 * 2. void push(val) pushes the element val onto the stack.
 * 3. void pop() removes the element on the top of the stack.
 * 4. int top() gets the top element of the stack.
 * 5. int getMin() retrieves the minimum element in the stack.
 * <p>
 * <p>
 * Example 1:
 * <p>
 * Input
 * ["MinStack","push","push","push","getMin","pop","top","getMin"]
 * [[],[-2],[0],[-3],[],[],[],[]]
 * <p>
 * Output
 * [null,null,null,null,-3,null,0,-2]
 * <p>
 * Explanation
 * MinStack minStack = new MinStack();
 * minStack.push(-2);
 * // stack: [-2]
 * minStack.push(0);
 * // stack: [-2, 0]
 * minStack.push(-3);
 * // stack: [-2, 0, -3]
 * minStack.getMin(); // return -3
 * // stack: [-2, 0, -3]
 * minStack.pop();
 * // stack: [-2, 0]
 * minStack.top();    // return 0
 * // stack: [-2, 0]
 * minStack.getMin(); // return -2
 * // stack: [-2, 0]
 * <p>
 * <p>
 * Constraints:
 * <p>
 * -231 <= val <= 231 - 1
```

```
 * Methods pop, top and getMin operations will always be called on non-empty stacks.
 * At most 3 * 104 calls will be made to push, pop, top, and getMin.
 */
public class Solution00155 {

    static class Node {
        int val;
        int min;
        Node next;

        public Node(int val, int min, Node next) {
            this.val = val;
            this.min = min;
            this.next = next;
        }
    }

    public interface MinStack {
        void push(int val);

        void pop();

        int top();

        int getMin();
    }

    // Time: O(1)
    // Space: O(n)
    class MinStackImpl implements MinStack {

        private Node head;

        @Override
        public void push(int val) {
            if (head == null) {
                head = new Node(val, val, null);
            } else {
                head = new Node(val, Math.min(val, head.min), head);
            }
        }

        @Override
        public void pop() {
            head = head.next;
        }

        @Override
        public int top() {
            return head.val;
        }

        @Override
        public int getMin() {
```

```
            return head.min;
        }

    }

    class MinStackUsing2Deques implements MinStack {

        private Deque<Integer> stack;
        private Deque<Integer> minStack;

        public MinStackUsing2Deques() {
            stack = new LinkedList<>();
            minStack = new LinkedList<>();
        }

        @Override
        public void push(int val) {
            stack.push(val);

            if (minStack.isEmpty()) {
                minStack.push(val);
            } else {
                minStack.push(Math.min(val, minStack.peek()));
            }
        }

        @Override
        public void pop() {
            stack.pop();
            minStack.pop();
        }

        @Override
        public int top() {
            return stack.peek();
        }

        @Override
        public int getMin() {
            return minStack.peek();
        }

    }

}
```

85. `Solution00165.java`

```
package leetcode;

/**
 * 165. Compare Version Numbers
 * <a href="https://leetcode.com/problems/compare-version-numbers">Web link</a>
 * Medium
 */

public class Solution00165 {

    public int compareVersion(String version1, String version2) {
        String[] version1Parts = version1.split("\\.");
        String[] version2Parts = version2.split("\\.");
        int maxLength = Math.max(version1Parts.length, version2Parts.length);
        for (int i = 0; i < maxLength; i++) {
            int version1Part = i < version1Parts.length ?
Integer.parseInt(version1Parts[i]) : 0;
            int version2Part = i < version2Parts.length ?
Integer.parseInt(version2Parts[i]) : 0;
            if (version1Part < version2Part) {
                return -1;
            } else if (version1Part > version2Part) {
                return 1;
            }
        }
        return 0;
    }

}
```

86. `Solution00167.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/two-sum-ii-input-array-is-sorted/
 * 167. Two Sum II - Input array is sorted
 * <p>
 * Given an array of integers numbers that is already sorted in non-decreasing order,
find two numbers such that they add up to a specific target number.
 * <p>
 * Return the indices of the two numbers (1-indexed) as an integer array answer of size
2,
 * where 1 <= answer[0] < answer[1] <= numbers.length.
 * <p>
 * You may assume that each input would have exactly one solution and you may not use the
same element twice.
 * <p>
 * <p>
 * <p>
 * Example 1:
 * <p>
 * Input: numbers = [2,7,11,15], target = 9 Output: [1,2] Explanation: The sum of 2 and 7
is 9. Therefore index1 = 1, index2 = 2.
 * Example 2:
 * <p>
 * Input: numbers = [2,3,4], target = 6 Output: [1,3] Example 3:
 * <p>
 * Input: numbers = [-1,0], target = -1 Output: [1,2]
 * <p>
 * <p>
 * Constraints:
 * <p>
 * 2 <= numbers.length <= 3 * 104 -1000 <= numbers[i] <= 1000 numbers is sorted in non-
decreasing order. -1000 <= target <= 1000
 */
public class Solution00167 {

    // Time: O(n^2)
    // Space: O(1)
    public int[] twoSumBruteForce(int[] nums, int target) {
        // Loop through the array.
        for (int i = 0; i < nums.length; i++) {
            // Loop through the array again.
            for (int j = i + 1; j < nums.length; j++) {
                // If the sum of the two numbers is equal to the target, return the
indices.
                if (nums[i] + nums[j] == target) {
                    return new int[]{i + 1, j + 1};
                }
            }
        }
```

```
        return new int[]{};
    }

    // Time: O(n)
    // Space: O(1)
    public int[] twoSum(int[] numbers, int target) {
        // Create two pointers.
        int left = 0;
        int right = numbers.length - 1;

        // Loop through the array.
        while (left < right) {
            // Get the sum of the two numbers.
            int sum = numbers[left] + numbers[right];

            // If the sum is equal to the target, return the indices.
            if (sum == target) {
                return new int[]{left + 1, right + 1};
            }

            // If the sum is greater than the target, decrement the right pointer.
            if (sum > target) {
                right--;
            } else {
                // If the sum is less than the target, increment the left pointer.
                left++;
            }
        }

        return new int[]{};
    }
}
```

87. `Solution00169.java`

```
package leetcode;

import java.util.HashMap;
import java.util.Map;

/**
 * 169. Majority Element
 * https://leetcode.com/problems/majority-element
 * Easy
 */
public class Solution00169 {

    // Time: O(n)
    // Space: O(1)
    public int majorityElement(int[] nums) {
        int count = 0;
        int candidate = 0;

        for (int num : nums) {
            if (count == 0) {
                candidate = num;
            }

            count += (num == candidate) ? 1 : -1;
        }

        return candidate;
    }

    // Time: O(n)
    // Space: O(n)
    public int majorityElementMap(int[] nums) {
        int n = nums.length;
        Map<Integer, Integer> freqMap = new HashMap<>();
        for (int num : nums) {
            freqMap.merge(num, 1, Integer::sum);
        }

        int n2 = n / 2;
        for (Map.Entry<Integer, Integer> entry : freqMap.entrySet()) {
            if (entry.getValue() > n2) {
                return entry.getKey();
            }
        }

        return 0;
    }

}
```

88. `Solution00179.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 179. Largest Number
 * <a href="https://leetcode.com/problems/largest-number">Web link</a> |
 * Medium
 */

public class Solution00179 {

    public String largestNumber(int[] nums) {
        int n = nums.length;
        String[] strNums = new String[n];
        for (int i = 0; i < n; i++) {
            strNums[i] = String.valueOf(nums[i]);
        }
        Arrays.sort(strNums, (a, b) -> (b + a).compareTo(a + b));
        if (strNums[0].equals("0")) {
            return "0";
        }
        StringBuilder sb = new StringBuilder();
        for (String str : strNums) {
            sb.append(str);
        }
        return sb.toString();
    }

}
```

89. `Solution00189.java`

```
package leetcode;

/**
 * 189. Rotate Array
 * https://leetcode.com/problems/rotate-array/
 * #medium
 * Given an integer array nums, rotate the array to the right by k steps, where k is non-
negative.
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [1,2,3,4,5,6,7], k = 3
 * Output: [5,6,7,1,2,3,4]
 * Explanation:
 * rotate 1 steps to the right: [7,1,2,3,4,5,6]
 * rotate 2 steps to the right: [6,7,1,2,3,4,5]
 * rotate 3 steps to the right: [5,6,7,1,2,3,4]
 * Example 2:
 * <p>
 * Input: nums = [-1,-100,3,99], k = 2
 * Output: [3,99,-1,-100]
 * Explanation:
 * rotate 1 steps to the right: [99,-1,-100,3]
 * rotate 2 steps to the right: [3,99,-1,-100]
 * <p>
 * Constraints:
 * <p>
 * 1 <= nums.length <= 105
 * -231 <= nums[i] <= 231 - 1
 * 0 <= k <= 105
 * <p>
 * Follow up:
 * <p>
 * Try to come up with as many solutions as you can. There are at least three different
ways to solve this problem.
 * Could you do it in-place with O(1) extra space?
 */
public class Solution00189 {

    /**
     * 1. Brute force
     * Time complexity: O(n*k)
     * Space complexity: O(1)
     */
    public void rotateBruteforce(int[] nums, int k) {
        // Get the length of the array
        int n = nums.length;

        // Calculate the effective rotation steps, as k could be larger than n
        k %= n;
```

```
        // If no rotation is needed, return immediately
        if (k == 0) {
            return;
        }

        // Create a temporary array to hold the last k elements of the original array
        int[] tmp = new int[k];

        // Copy the last k elements from the original array to the temporary array
        System.arraycopy(nums, n - k, tmp, 0, k);

        // Shift the first n-k elements of the original array to the right by k positions
        System.arraycopy(nums, 0, nums, k, n - k);

        // Copy the temporary array (which holds the original last k elements) to the
beginning of the array
        System.arraycopy(tmp, 0, nums, 0, k);
    }

    /**
     * 2. Cyclic replacement
     * Time complexity: O(n)
     * Space complexity: O(1)
     */
    public void rotateCyclicReplacement(int[] nums, int k) {
        // Get the length of the array
        int n = nums.length;

        // Calculate the effective rotation steps, as k could be larger than n
        k %= n;

        // If no rotation is needed, return immediately
        if (k == 0) {
            return;
        }

        // Initialize a counter to keep track of the number of elements that have been
moved
        int count = 0;

        // Iterate over the array, starting from the first element
        for (int startIndex = 0; count < n; startIndex++) {
            // Initialize the currentTargetIndex index and the value to be moved
            int currentTargetIndex = startIndex;
            int prevValue = nums[startIndex];

            // Keep moving the values in the array until we return to the startIndex
index
            do {
                // Calculate the index of the nextTargetIndex value to be moved
                int nextTargetIndex = (currentTargetIndex + k) % n;

                // Swap the nextTargetIndex value with the previous value
                int tmp = nums[nextTargetIndex];
```

```
                nums[nextTargetIndex] = prevValue;
                prevValue = tmp;

                // Update the currentTargetIndex index and increment the counter
                currentTargetIndex = nextTargetIndex;
                count++;
            } while (startIndex != currentTargetIndex); // Continue until we have moved
all n elements
        }
    }

}
```

90. `Solution00191.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/number-of-1-bits/
 * 191. Number of 1 Bits
 * <p>
 * Write a function that takes an unsigned integer and returns the number of '1' bits it
has (also known as the Hamming weight).
 * <p>
 * Note:
 * <p>
 * Note that in some languages, such as Java, there is no unsigned integer type. In this
case, the input will be given as a signed integer type.
 * It should not affect your implementation, as the integer's internal binary
representation is the same, whether it is signed or unsigned.
 * In Java, the compiler represents the signed integers using 2's complement notation.
 * Therefore, in Example 3, the input represents the signed integer. -3.
 * <p>
 * <p>
 * Example 1:
 * <p>
 * Input: n = 00000000000000000000000000001011
 * Output: 3
 * Explanation: The input binary string 00000000000000000000000000001011 has a total of
three '1' bits.
 * Example 2:
 * <p>
 * Input: n = 00000000000000000000000010000000
 * Output: 1
 * Explanation: The input binary string 00000000000000000000000010000000 has a total of
one '1' bit.
 * Example 3:
 * <p>
 * Input: n = 11111111111111111111111111111101
 * Output: 31
 * Explanation: The input binary string 11111111111111111111111111111101 has a total of
thirty one '1' bits.
 */
public class Solution00191 {

    // Time: O(1)
    // Space: O(1)
    public int hammingWeight(int n) {
        int count = 0;
        while (n != 0) {
            // n & (n-1) flips the least significant 1-bit in n to 0
            n &= (n - 1);
            count++;
        }

        return count;
```

```
    }

}
```

91. `Solution00198.java`

```
package leetcode;

/**
 * 198. House Robber
 * https://leetcode.com/problems/house-robber/
 * <p>
 * Easy
 * <p>
 * You are a professional robber planning to rob houses along a street. Each house has a
certain amount of money
 * stashed, the only constraint stopping you from robbing each of them is that adjacent
houses have security system
 * connected and it will automatically contact the police if two adjacent houses were
broken into on the same night.
 * <p>
 * Given a list of non-negative integers representing the amount of money of each house,
determine the maximum
 * amount of money you can rob tonight without alerting the police.
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [1,2,3,1]
 * <p>
 * Output: 4
 * <p>
 * Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3). Total amount
you can rob = 1 + 3 = 4.
 * <p>
 * Example 2:
 * <p>
 * Input: nums = [2,7,9,3,1]
 * <p>
 * Output: 12
 * <p>
 * Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money =
1). Total amount you
 * can rob = 2 + 9 + 1 = 12.
 * <p>
 * Constraints:
 * <p>
 * 0 <= nums.length <= 100
 * <p>
 * 0 <= nums[i] <= 400
 */
public class Solution00198 {

    // Time: O(n)
    // Space: O(1)
    public int rob(int[] nums) {
        int max1 = 0;
        int max2 = 0;
```

```
        for (int num : nums) {
            int temp = max1;
            max1 = max2;
            max2 = Math.max(temp + num, max2);
        }

        return max2;
    }

    // Time: O(n)
    // Space: O(n)
    public int robDp(int[] nums) {
        if (nums.length == 0) {
            return 0;
        }
        if (nums.length == 1) {
            return nums[0];
        }

        int[] dp = new int[nums.length];
        dp[0] = nums[0];
        dp[1] = Math.max(nums[0], nums[1]);

        for (int i = 2; i < nums.length; i++) {
            dp[i] = Math.max(dp[i - 1], dp[i - 2] + nums[i]);
        }

        return dp[nums.length - 1];
    }

    // Time: O(2^n)
    // Space: O(n)
    public int robBruteForce(int[] nums) {
        return robBruteForce(nums, 0);
    }

    private int robBruteForce(int[] nums, int index) {
        if (index >= nums.length) {
            return 0;
        }
        int robIncludeCurrent = nums[index] + robBruteForce(nums, index + 2);
        int robExcludeCurrent = robBruteForce(nums, index + 1);
        return Math.max(robIncludeCurrent, robExcludeCurrent);
    }

}
```

92. `Solution00199.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * https://leetcode.com/problems/binary-tree-right-side-view/
 * 199. Binary Tree Right Side View
 * Medium
 * Given the root of a binary tree, imagine yourself standing on the right side of it,
return the values of the nodes you can see ordered from top to bottom.
 * Example 1:
 * Input: root = [1,2,3,null,5,null,4]
 * Output: [1,3,4]
 * Example 2:
 * Input: root = [1,null,3]
 * Output: [1,3]
 * Example 3:
 * Input: root = []
 * Output: []
 * Constraints:
 * The number of nodes in the tree is in the range [0, 100].
 * -100 <= Node.val <= 100
 */
public class Solution00199 {

    // Time O(n)
    // Space O(n)
    public List<Integer> rightSideViewBfs(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        if (root == null) return result;

        Queue<TreeNode> queue = new LinkedList<>();
        queue.offer(root);
        while (!queue.isEmpty()) {
            int size = queue.size();
            TreeNode node = null;
            for (int i = 0; i < size; i++) {
                node = queue.poll();
                if (node.left != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
            result.add(node.val);
        }
        return result;
    }
```

```
}
```

93. `Solution00200.java`

```
package leetcode;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 200. Number of Islands
 * https://leetcode.com/problems/number-of-islands/
 * <p>
 * Medium
 * <p>
 * Given a 2d grid map of '1's (land) and '0's (water), count the number of islands. An
island is surrounded
 * by water and is formed by connecting adjacent lands horizontally or vertically. You
may assume all four
 * edges of the grid are all surrounded by water.
 * <p>
 * Example 1:
 * <p>
 * Input: 11110 11010 11000 00000
 * <p>
 * Output: 1
 * <p>
 * Example 2:
 * <p>
 * Input: 11000 11000 00100 00011
 * <p>
 * Output: 3
 */
public class Solution00200 {

    // Time: O(n * m)
    // Space: O(n * m)
    public int numIslandsDfsBased(char[][] grid) {
        int islands = 0;
        boolean[][] visited = new boolean[grid.length][grid[0].length];

        for (int row = 0; row < grid.length; row++) {
            for (int col = 0; col < grid[0].length; col++)
                if (grid[row][col] == '1' && !visited[row][col]) {
                    islands++;
                    dfs(grid, visited, row, col);
                }
        }

        return islands;
    }

    private void dfs(char[][] grid, boolean[][] visited, int row, int col) {
        if (row < 0 || row >= grid.length
            || col < 0 || col >= grid[0].length
```

```
            || grid[row][col] == '0'
            || visited[row][col]) {
            return;
        }

        visited[row][col] = true;
        dfs(grid, visited, row - 1, col);
        dfs(grid, visited, row + 1, col);
        dfs(grid, visited, row, col - 1);
        dfs(grid, visited, row, col + 1);
    }

    // Time: O(n * m)
    // Space: O(n * m)
    public int numIslandsBfsBased(char[][] grid) {
        int islands = 0;
        boolean[][] visited = new boolean[grid.length][grid[0].length];

        for (int row = 0; row < grid.length; row++) {
            for (int col = 0; col < grid[0].length; col++)
                if (grid[row][col] == '1' && !visited[row][col]) {
                    islands++;
                    bfs(grid, visited, row, col);
                }
        }

        return islands;
    }

    private void bfs(char[][] grid, boolean[][] visited, int row, int col) {
        Queue<int[]> queue = new LinkedList<>();
        queue.offer(new int[]{row, col});

        while (!queue.isEmpty()) {
            int[] cell = queue.poll();
            row = cell[0];
            col = cell[1];

            if (row < 0 || row >= grid.length
                || col < 0 || col >= grid[0].length
                || grid[row][col] == '0' || visited[row][col]) {
                continue;
            }

            visited[row][col] = true;
            queue.offer(new int[]{row - 1, col});
            queue.offer(new int[]{row + 1, col});
            queue.offer(new int[]{row, col - 1});
            queue.offer(new int[]{row, col + 1});
        }
    }

}
```

94. `Solution00201.java`

```
package leetcode;

/**
 * 201. Bitwise AND of Numbers Range
 * https://leetcode.com/problems/bitwise-and-of-numbers-range/
 * Medium
 */
public class Solution00201 {

    // Time: O(n)
    // Space: O(1)
    public int rangeBitwiseAnd(int left, int right) {
        // Initialize a shift counter
        int shift = 0;

        // Keep shifting the bits of both numbers to the right until they become equal
        // This is done because the bitwise AND of all numbers in the range is just the
common prefix of the binary representations of the left and right numbers
        while (left < right) {
            left >>= 1;  // Shift the bits of the left number to the right
            right >>= 1; // Shift the bits of the right number to the right
            shift++;     // Increment the shift counter
        }

        // Once the numbers are equal, shift the bits of the left number to the left by
the shift counter
        // This is done to restore the original number of bits in the left number
        return left << shift;
    }

}
```

95. `Solution00205.java`

```
package leetcode;

/**
 * 205. Isomorphic Strings
 * https://leetcode.com/problems/isomorphic-strings/
 * Easy
 */

public class Solution00205 {

    public boolean isIsomorphic(String s, String t) {
        int[] sourceCharCount = new int[128];
        int[] targetCharCount = new int[128];
        int stringLength = s.length();
        for (int index = 0; index < stringLength; index++) {
            char sourceChar = s.charAt(index);
            char targetChar = t.charAt(index);
            if (sourceCharCount[sourceChar] != targetCharCount[targetChar]) {
                return false;
            }
            sourceCharCount[sourceChar] = index + 1;
            targetCharCount[targetChar] = index + 1;
        }
        return true;
    }

}
```

96. `Solution00206.java`

```
package leetcode;

import leetcode.common.ListNode;

/**
 * https://leetcode.com/problems/reverse-linked-list/
 * 206. Reverse Linked List
 * <p>
 * Given the head of a singly linked list, reverse the list, and return the reversed
list.
 * <p>
 * Example 1:
 * <p>
 * Input: head = [1,2,3,4,5]
 * Output: [5,4,3,2,1]
 * <p>
 * Example 2:
 * <p>
 * Input: head = [1,2]
 * Output: [2,1]
 * <p>
 * Example 3:
 * <p>
 * Input: head = []
 * Output: []
 * <p>
 * Constraints:
 * <p>
 * The number of nodes in the list is the range [0, 5000].
 * -5000 <= Node.val <= 5000
 * <p>
 * Follow up: A linked list can be reversed either iteratively or recursively. Could you
implement both?
 */
public class Solution00206 {

    // Time: O(n)
    // Space: O(1)
    public ListNode reverseList(ListNode head) {
        ListNode previous = null;
        ListNode current = head;

        while (current != null) {
            // Save next node
            ListNode next = current.next;

            // Reverse current node
            current.next = previous;

            // Move pointers
            previous = current;
```

```
            current = next;
        }

        return previous;
    }

    // Time: O(n)
    // Space: O(n)
    public ListNode reverseListRecursive(ListNode head) {
        if (head == null || head.next == null) {
            return head;
        }

        ListNode reversed = reverseListRecursive(head.next);

        head.next.next = head;
        head.next = null;

        return reversed;
    }

}
```

97. `Solution00207.java`

```
package leetcode;

import java.util.*;

/**
 * https://leetcode.com/problems/course-schedule/
 * 207. Course Schedule
 * Medium
 */
public class Solution00207 {

    // Time complexity: O(V + E)
    // Space complexity: O(V)
    public boolean canFinish(int numCourses, int[][] prerequisites) {
        // Create a map to store the prerequisites for each course.
        Map<Integer, List<Integer>> graph = new HashMap<>();

        // Create an array to store the in-degree of each course.
        int[] inDegree = new int[numCourses];

        // Iterate over all of the prerequisites and add them to the map and the in-
degree array.
        for (int[] prerequisite : prerequisites) {
            int course = prerequisite[0];
            int prerequisiteCourse = prerequisite[1];

            graph.computeIfAbsent(prerequisiteCourse, k -> new ArrayList<>
()).add(course);
            inDegree[course]++;
        }

        // Create a queue to store the courses that have no prerequisites.
        Queue<Integer> queue = new LinkedList<>();

        // Add all of the courses that have no prerequisites to the queue.
        for (int i = 0; i < numCourses; i++) {
            if (inDegree[i] == 0) {
                queue.add(i);
            }
        }

        // Initialize a counter to track the number of courses that have been completed.
        int count = 0;

        // While the queue is not empty, do the following:
        while (!queue.isEmpty()) {

            // Remove a course from the queue.
            int course = queue.poll();

            // Increment the counter.
```

```
            count++;

            // Iterate over all of the prerequisites of the course.
            if (graph.containsKey(course)) {
                for (int nextCourse : graph.get(course)) {

                    // Decrement the in-degree of the prerequisite course.
                    inDegree[nextCourse]--;

                    // If the in-degree of the prerequisite course is now 0, then add it
to the queue.
                    if (inDegree[nextCourse] == 0) {
                        queue.add(nextCourse);
                    }
                }
            }
        }

        // If all of the courses have been completed, then return true. Otherwise, return
false.
        return count == numCourses;
    }

}
```

98. `Solution00208.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/implement-trie-prefix-tree/
 * 208. Implement Trie (Prefix Tree)
 * <p>
 * A trie (pronounced as "try") or prefix tree is a tree data structure used to
efficiently store and retrieve keys
 * in a dataset of strings. There are various applications of this data structure, such
as autocomplete and spellchecker.
 * <p>
 * Implement the Trie class:
 * <p>
 * 1. Trie() Initializes the trie object.
 * 2. void insert(String word) Inserts the string word into the trie.
 * 3. boolean search(String word) Returns true if the string word is in the trie (i.e.,
was inserted before),
 * and false otherwise.
 * 4. boolean startsWith(String prefix) Returns true if there is a previously inserted
string word that has the prefix
 * prefix, and false otherwise.
 * <p>
 * Example 1:
 * <p>
 * Input
 * ["Trie", "insert", "search", "search", "startsWith", "insert", "search"]
 * [[], ["apple"], ["apple"], ["app"], ["app"], ["app"], ["app"]]
 * <p>
 * Output
 * [null, null, true, false, true, null, true]
 * <p>
 * Explanation
 * Trie trie = new Trie();
 * trie.insert("apple");
 * trie.search("apple");   // return True
 * trie.search("app");     // return False
 * trie.startsWith("app"); // return True
 * trie.insert("app");
 * trie.search("app");     // return True
 * <p>
 * Constraints:
 * <p>
 * 1. 1 <= word.length, prefix.length <= 2000
 * 2. word and prefix consist only of lowercase English letters.
 * 3. At most 3 * 104 calls in total will be made to insert, search, and startsWith.
 */
public class Solution00208 {

    public static class Trie {

        private final TrieNode root;
```

```
/**
 * Initialize your data structure here.
 */
public Trie() {
    root = new TrieNode();
}

// Time: O(m), where m is the key length
// Space: O(m)
public void insert(String word) {
    TrieNode node = root;

    for (char c : word.toCharArray()) {
        int index = c - 'a';
        if (node.children[index] == null) {
            node.children[index] = new TrieNode();
        }
        node = node.children[index];
    }

    node.isEndOfWord = true;
}

// Time: O(m), where m is the key length
// Space: O(1)
public boolean search(String word) {
    TrieNode node = root;

    for (char c : word.toCharArray()) {
        node = node.children[c - 'a'];
        if (node == null) {
            return false;
        }
    }

    return node.isEndOfWord;
}

// Time: O(m), where m is the key length
// Space: O(1)
public boolean startsWith(String prefix) {
    TrieNode node = root;

    for (char c : prefix.toCharArray()) {
        node = node.children[c - 'a'];
        if (node == null) {
            return false;
        }
    }

    return true;
}
```

```
    }

    private static class TrieNode {
        private static final int ALPHABET_SIZE = 26;
        private final TrieNode[] children = new TrieNode[ALPHABET_SIZE];
        private boolean isEndOfWord;
    }

}
```

99. `Solution00211.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/design-add-and-search-words-data-structure/
 * 211. Design Add and Search Words Data Structure
 * Medium
 * <p>
 * Design a data structure that supports adding new words and finding if a string matches
any previously added string.
 * Implement the WordDictionary class:
 * WordDictionary() Initializes the object.
 * void addWord(word) Adds word to the data structure, it can be matched later.
 * bool search(word) Returns true if there is any string in the data structure that
matches word or false otherwise.
 * word may contain dots '.' where dots can be matched with any letter.
 * Example:
 * Input
 * ["WordDictionary","addWord","addWord","addWord","search","search","search","search"]
 * [[],["bad"],["dad"],["mad"],["pad"],["bad"],[".ad"],["b.."]]
 * Output
 * [null,null,null,null,false,true,true,true]
 * Explanation
 * WordDictionary wordDictionary = new WordDictionary();
 * wordDictionary.addWord("bad");
 * wordDictionary.addWord("dad");
 * wordDictionary.addWord("mad");
 * wordDictionary.search("pad"); // return False
 * wordDictionary.search("bad"); // return True
 * wordDictionary.search(".ad"); // return True
 * wordDictionary.search("b.."); // return True
 * Constraints:
 * Constraints:
 * 1 <= word.length <= 25
 * word in addWord consists of lowercase English letters.
 * word in search consist of '.' or lowercase English letters.
 * There will be at most 2 dots in word for search queries.
 * At most 104 calls will be made to addWord and search.
 */
public class Solution00211 {

    static class WordDictionary {

        private static final int ALPHABET_SIZE = 26;
        private final Node root;

        /**
         * Initialize your data structure here.
         */
        public WordDictionary() {
            root = new Node();
        }
```

```
        public void addWord(String word) {
            Node node = root;
            for (char c : word.toCharArray()) {
                int i = c - 'a';
                if (node.next[i] == null) {
                    node.next[i] = new Node();
                }
                node = node.next[i];
            }
            node.isWord = true;
        }

        public boolean search(String word) {
            return search(root, word, 0);
        }

        private boolean search(Node node, String word, int i) {
            if (node == null) {
                return false;
            }
            if (i == word.length()) {
                return node.isWord;
            }
            char c = word.charAt(i);
            if (c == '.') {
                for (Node next : node.next) {
                    if (search(next, word, i + 1)) {
                        return true;
                    }
                }
                return false;
            } else {
                return search(node.next[c - 'a'], word, i + 1);
            }
        }

        private static class Node {
            private final Node[] next = new Node[ALPHABET_SIZE];
            private boolean isWord;
        }

    }

}
```

100. `Solution00212.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.List;

/**
 * 212. Word Search II
 * https://leetcode.com/problems/word-search-ii
 * <p>
 * Given an m x n board of characters and a list of strings words, return all words on
the board.
 * <p>
 * Each word must be constructed from letters of sequentially adjacent cells, where
adjacent cells are horizontally or
 * vertically neighboring. The same letter cell may not be used more than once in a word.
 * <p>
 * Constraints:
 * <p>
 * m == board.length
 * n == board[i].length
 * 1 <= m, n <= 12
 * board[i][j] is a lowercase English letter.
 * 1 <= words.length <= 3 * 10^4
 * 1 <= words[i].length <= 10
 * words[i] consists of lowercase English letters.
 * All the strings of words are unique.
 */
public class Solution00212 {

    // Time: O(m * n * 4^k)
    // Space: O(k)
    public List<String> findWords(char[][] board, String[] words) {
        // Build a Trie from the given words
        TrieNode root = buildTrie(words);

        // Initialize a list to store the result
        List<String> result = new ArrayList<>();

        // Iterate over each cell in the board
        for (int i = 0; i < board.length; i++) {
            for (int j = 0; j < board[0].length; j++) {
                // Perform a depth-first search from each cell
                dfs(board, i, j, root, words, result);
            }
        }

        // Return the list of words found on the board
        return result;
    }

    private void dfs(char[][] board, int i, int j, TrieNode p, String[] words,
```

```
List<String> result) {
        // Boundary check for board
        if (i < 0 || i >= board.length || j < 0 || j >= board[0].length) {
            return;
        }

        // Get the character at the current board position
        char c = board[i][j];

        // If the character is '#' (indicating it's already visited) or there's no
corresponding TrieNode, return
        if (c == '#' || p.next[c - 'a'] == null) {
            return;
        }

        // Move to the next TrieNode corresponding to the character
        p = p.next[c - 'a'];

        // If the TrieNode has a word index >= 0 (indicating we found a word),
        // add it to the result and set the word index to -1 to avoid duplicates
        if (p.wordIndex >= 0) {
            result.add(words[p.wordIndex]);
            p.wordIndex = -1;   // avoid duplicate
        }

        // Mark the current board position as visited by setting it to '#'
        board[i][j] = '#';

        // Iterate over all four directions
        // Move up
        if (i - 1 >= 0) {
            dfs(board, i - 1, j, p, words, result);
        }

        // Move down
        if (i + 1 < board.length) {
            dfs(board, i + 1, j, p, words, result);
        }

        // Move left
        if (j - 1 >= 0) {
            dfs(board, i, j - 1, p, words, result);
        }

        // Move right
        if (j + 1 < board[0].length) {
            dfs(board, i, j + 1, p, words, result);
        }

        // After the DFS, revert the current board position back to its original
character
        board[i][j] = c;
    }
```

```
    private TrieNode buildTrie(String[] words) {
        // Create a new TrieNode to serve as the root of the Trie
        TrieNode root = new TrieNode();

        // Iterate over each word in the input array
        for (int wordIndex = 0; wordIndex < words.length; wordIndex++) {
            String word = words[wordIndex];
            // Start from the root for each word
            TrieNode p = root;

            // Iterate over each character in the word
            for (char c : word.toCharArray()) {
                // Convert the character to an index. 'a' becomes 0, 'b' becomes 1, etc.
                int i = c - 'a';

                // If the TrieNode for this character does not exist, create a new one
                if (p.next[i] == null) {
                    p.next[i] = new TrieNode();
                }

                // Move to the next TrieNode
                p = p.next[i];
            }

            // After all characters of the word have been processed, set the word at the
last TrieNode
            p.wordIndex = wordIndex;
        }

        // Return the root of the Trie
        return root;
    }

    private static class TrieNode {
        // An array of TrieNode references, each corresponding to a possible next
character in a word.
        // 'a' corresponds to index 0, 'b' to index 1, ..., 'z' to index 25.
        TrieNode[] next = new TrieNode[26];

        // The index of word at this TrieNode, if any. -1 indicates no word ends at this
TrieNode.
        int wordIndex = -1;
    }

}
```

101. `Solution00213.java`

```
package leetcode;

/**
 * 213. House Robber II
 * https://leetcode.com/problems/house-robber-ii/description/
 * Medium
 */
public class Solution00213 {

    public int rob(int[] nums) {
        if (nums.length == 1) {
            return nums[0];
        }
        int robFromFirst = rob(nums, 0, nums.length - 1);
        int robFromSecond = rob(nums, 1, nums.length - 1);
        return Math.max(robFromFirst, robFromSecond);
    }

    private int rob(int[] nums, int indexStart, int size) {
        int[] dp = new int[nums.length + 1];
        dp[indexStart] = 0;
        dp[indexStart + 1] = nums[indexStart];
        final int indexEnd = indexStart + size;
        for (int i = indexStart + 2; i <= indexEnd; i++) {
            int skipCurrent = dp[i - 1];
            int includeCurrent = dp[i - 2] + nums[i - 1];
            dp[i] = Math.max(skipCurrent, includeCurrent);
        }
        return dp[indexEnd];
    }

    public int robOptimized(int[] nums) {
        if (nums.length == 1) {
            return nums[0];
        }
        int robFromFirst = robOptimized(nums, 0, nums.length - 1);
        int robFromSecond = robOptimized(nums, 1, nums.length - 1);
        return Math.max(robFromFirst, robFromSecond);
    }

    private int robOptimized(int[] nums, int indexStart, int size) {
        int previous = 0;
        int current = nums[indexStart];
        final int indexEnd = indexStart + size;
        for (int i = indexStart + 2; i <= indexEnd; i++) {
            int skipCurrent = current;
            int includeCurrent = previous + nums[i - 1];
            previous = current;
            current = Math.max(skipCurrent, includeCurrent);
        }
        return current;
```

```
    }

}
```

102. `Solution00214.java`

```
package leetcode;

/**
 * 214. Shortest Palindrome
 * <a href="https://leetcode.com/problems/shortest-palindrome">Web link</a>
 * Hard
 */

public class Solution00214 {

    public String shortestPalindrome(String s) {
        int n = s.length();
        if (n == 0) return "";
        String rev = new StringBuilder(s).reverse().toString();
        for (int i = 0; i < rev.length(); i++) {
            if (s.startsWith(rev.substring(i))) {
                return rev.substring(0, i) + s;
            }
        }
        return rev + s;
    }

}
```

103. `Solution00217.java`

```
package leetcode;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

/**
 * https://leetcode.com/problems/contains-duplicate/
 * 217. Contains Duplicate
 * Easy
 */
public class Solution00217 {

    // Time: O(n^2)
    // Space: O(1)
    public boolean containsDuplicateBrutforce(int[] nums) {
        for (int i = 0; i < nums.length; i++) {
            for (int j = i+1; j < nums.length; j++) {
                if (nums[j] == nums[i]) {
                    return true;
                }
            }
        }
        return false;
    }

    // Time: O(n log n)
    // Space: O(1)
    public boolean containsDuplicateSorted(int[] nums) {
        Arrays.sort(nums);
        for (int i = 0; i < nums.length-1; i++) {
            if (nums[i+1] == nums[i]) {
                return true;
            }
        }
        return false;
    }

    // Time: O(n)
    // Space: O(n)
    public boolean containsDuplicate(int[] nums) {
        Set<Integer> set = new HashSet<>();
        for (int num: nums) {
            if (!set.add(num)) {
                return true;
            }
        }

        return false;
    }
```

```
}
```

104. `Solution00226.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.LinkedList;
import java.util.Queue;

/**
 * https://leetcode.com/problems/invert-binary-tree/
 * 226. Invert Binary Tree
 * <p>
 * Given the root of a binary tree, invert the tree, and return its root.
 * <p>
 * <p>
 * Example 1:
 * <p>
 * <p>
 * Input: root = [4,2,7,1,3,6,9]
 * Output: [4,7,2,9,6,3,1]
 * Example 2:
 * <p>
 * <p>
 * Input: root = [2,1,3]
 * Output: [2,3,1]
 * Example 3:
 * <p>
 * Input: root = []
 * Output: []
 * <p>
 * <p>
 * Constraints:
 * <p>
 * The number of nodes in the tree is in the range [0, 100].
 * -100 <= Node.val <= 100
 */
public class Solution00226 {

    // Time: O(n)
    // Space: O(n)
    public TreeNode invertTree(TreeNode root) {
        // If the root is null, return null.
        if (root == null) {
            return null;
        }

        // save left subtrees
        TreeNode tmpLeft = root.left;

        // Invert the left subtree of the root.
        root.left = invertTree(root.right);
        // Invert the right subtree of the root.
```

```
        root.right = invertTree(tmpLeft);

        // Return the root.
        return root;
    }

    // Time: O(n)
    // Space: O(n)
    public TreeNode invertTreeIterative(TreeNode root) {
        // If the root is null, return null.
        if (root == null) {
            return null;
        }
        // Create a queue to store the nodes that need to be inverted.
        Queue<TreeNode> queue = new LinkedList<>();
        // Add the root to the queue.
        queue.add(root);
        // While the queue is not empty, do the following:
        while (!queue.isEmpty()) {
            // Remove the first node from the queue.
            TreeNode current = queue.remove();

            // save left subtrees
            TreeNode tmpLeft = current.left;

            // Invert the left subtree of the current node.
            current.left = current.right;
            // Invert the right subtree of the current node.
            current.right = tmpLeft;

            // If the left subtree of the current node is not null, add it to the queue.
            if (current.left != null) {
                queue.add(current.left);
            }
            // If the right subtree of the current node is not null, add it to the queue.
            if (current.right != null) {
                queue.add(current.right);
            }
        }
        // Return the root.
        return root;
    }
}
```

105. `Solution00230.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.Deque;
import java.util.LinkedList;

/**
 * https://leetcode.com/problems/kth-smallest-element-in-a-bst/
 * 230. Kth Smallest Element in a BST
 * #medium
 * Given the root of a binary search tree, and an integer k, return the kth (1-indexed)
smallest element in the tree.
 * Example 1:
 * Input: root = [3,1,4,null,2], k = 1
 * Output: 1
 * Example 2:
 * Input: root = [5,3,6,2,4,null,null,1], k = 3
 * Output: 3
 * Constraints:
 * The number of nodes in the tree is n.
 * 1 <= k <= n <= 104
 * 0 <= Node.val <= 104
 * Follow up: If the BST is modified often (i.e., we can do insert and delete operations)
and you need to find
 * the kth smallest frequently, how would you optimize?
 */
public class Solution00230 {

    // Time O(n)
    // Space O(n)
    public int kthSmallestRecursive(TreeNode root, int k) {
        TraverseData traverseData = new TraverseData();
        inorder(root, k, traverseData);
        return traverseData.result;
    }

    private void inorder(TreeNode root, int k, TraverseData traverseData) {
        if (root == null) {
            return;
        }
        inorder(root.left, k, traverseData);
        traverseData.nodeOrder++;
        if (traverseData.nodeOrder == k) {
            traverseData.result = root.val;
            return;
        }
        inorder(root.right, k, traverseData);
    }

    // Time O(n)
```

```
    // Space O(n)
    public int kthSmallestIterative(TreeNode root, int k) {
        int nodeOrder = 0;
        Deque<TreeNode> stack = new LinkedList<>();
        TreeNode current = root;
        while (current != null || !stack.isEmpty()) {
            // Traverse left subtree
            while (current != null) {
                stack.push(current);
                current = current.left;
            }
            // Pop the top node from the stack
            current = stack.pop();
            // Decrement k by 1
            nodeOrder++;
            // If k is 0, we have found the kth smallest element
            if (nodeOrder == k) {
                return current.val;
            }
            // Otherwise, set current to the right child of the popped node
            current = current.right;
        }
        return -1;
    }

    private static class TraverseData {
        int nodeOrder;
        int result;
    }

}
```

106. `Solution00231.java`

```
package leetcode;

/**
 * 231. Power of Two
 * https://leetcode.com/problems/power-of-two/
 * Easy
 */
public class Solution00231 {

    // Time: O(1)
    // Space: O(1)
    public boolean isPowerOfTwoBinary(int n) {
        return n > 0 && (n & (n - 1)) == 0;
    }

    // Time: O(1)
    // Space: O(1)
    public boolean isPowerOfTwoBitCount(int n) {
        return n > 0 && (Integer.bitCount(n) == 1);
    }

}
```

107. `Solution00232.java`

```
package leetcode;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * https://leetcode.com/problems/implement-queue-using-stacks/
 * 232. Implement Queue using Stacks
 * Easy
 * Implement a first in first out (FIFO) queue using only two stacks. The implemented
queue should support all the functions of a normal queue (push, peek, pop, and empty).
 * Implement the MyQueue class:
 * void push(int x) Pushes element x to the back of the queue.
 * int pop() Removes the element from the front of the queue and returns it.
 * int peek() Returns the element at the front of the queue.
 * boolean empty() Returns true if the queue is empty, false otherwise.
 * Notes:
 * You must use only standard operations of a stack, which means only push to top,
peek/pop from top, size, and is empty operations are valid.
 * Depending on your language, the stack may not be supported natively.
 * You may simulate a stack using a list or deque (double-ended queue) as long as you use
only a stack's standard operations.
 * Follow-up: Can you implement the queue such that each operation is amortized O(1) time
complexity? In other words,
 * performing n operations will take overall O(n) time even if one of those operations
may take longer.
 * Example 1:
 * Input
 * ["MyQueue", "push", "push", "peek", "pop", "empty"]
 * [[], [1], [2], [], [], []]
 * Output
 * [null, null, null, 1, 1, false]
 * Explanation
 * MyQueue myQueue = new MyQueue();
 * myQueue.push(1); // queue is: [1]
 * myQueue.push(2); // queue is: [1, 2] (leftmost is front of the queue)
 * myQueue.peek(); // return 1
 * myQueue.pop(); // return 1, queue is [2]
 * myQueue.empty(); // return false
 * Constraints:
 * 1 <= x <= 9
 * At most 100 calls will be made to push, pop, peek, and empty.
 * All the calls to pop and peek are valid.
 */
public class Solution00232 {

    public static class MyQueue {

        private Deque<Integer> stack1;
        private Deque<Integer> stack2;
```

```
        public MyQueue() {
            stack1 = new ArrayDeque<>();
            stack2 = new ArrayDeque<>();
        }

        public void push(int x) {
            stack1.push(x);
        }

        public int pop() {
            fillStack2IfEmpty();
            return stack2.pop();
        }

        private void fillStack2IfEmpty() {
            if (stack2.isEmpty()) {
                while (!stack1.isEmpty()) {
                    stack2.push(stack1.pop());
                }
            }
        }

        public int peek() {
            fillStack2IfEmpty();
            return stack2.peek();
        }

        public boolean empty() {
            return stack1.isEmpty() && stack2.isEmpty();
        }

    }

/**
 * Your MyQueue object will be instantiated and called as such:
 * MyQueue obj = new MyQueue();
 * obj.push(x);
 * int param_2 = obj.pop();
 * int param_3 = obj.peek();
 * boolean param_4 = obj.empty();
 */

}
```

108. `Solution00234.java`

```
package leetcode;

import leetcode.common.ListNode;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * 234. Palindrome Linked List
 * https://leetcode.com/problems/palindrome-linked-list/
 * Easy
 */

public class Solution00234 {

    public boolean isPalindrome(ListNode head) {
        if (head == null) {
            return true;
        }

        ListNode slow = head;
        ListNode fast = head;

        // Find the middle of the list
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        // Reverse the second half of the list
        ListNode curr = slow;
        ListNode right = null;
        while (curr != null) {
            ListNode next = curr.next;
            curr.next = right;
            right = curr;
            curr = next;
        }

        // Compare the first half and the reversed second half
        ListNode left = head;
        while (right != null) {
            if (left.val != right.val) {
                return false;
            }
            left = left.next;
            right = right.next;
        }

        return true;
    }
```

```
    public boolean isPalindromeStack(ListNode head) {
        if (head == null) {
            return true;
        }

        Deque<Integer> stack = new ArrayDeque<>();
        ListNode curr = head;
        while (curr != null) {
            stack.push(curr.val);
            curr = curr.next;
        }
        curr = head;
        while (curr != null) {
            if (curr.val != stack.pop()) {
                return false;
            }
            curr = curr.next;
        }

        return true;
    }

}
```

109. `Solution00235.java`

```
package leetcode;

import leetcode.common.TreeNode;

/**
 * <a href="https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-search-tree/">
 * 235. Lowest Common Ancestor of a Binary Search Tree
 * </a>
 * <br/>
 * <div class="notranslate"><p>Given a binary search tree (BST), find the lowest common ancestor (LCA) of two given nodes in the BST.</p>
 *
 * <p>According to the <a href="https://en.wikipedia.org/wiki/Lowest_common_ancestor" target="_blank">definition of LCA on Wikipedia</a>: “The lowest common ancestor is defined between two nodes <code>p</code> and <code>q</code> as the lowest node in <code>T</code> that has both <code>p</code> and <code>q</code> as descendants (where we allow <b>a node to be a descendant of itself</b>).”</p>
 *
 * <p><b>Example 1:</b></p>
 * <img alt="" src="https://assets.leetcode.com/uploads/2018/12/14/binarysearchtree_improved.png" style="width: 200px; height: 190px;">
 * <pre><b>Input:</b> root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8
 * <b>Output:</b> 6
 * <b>Explanation:</b> The LCA of nodes 2 and 8 is 6.
 * </pre>
 *
 * <p><b>Example 2:</b></p>
 * <img alt="" src="https://assets.leetcode.com/uploads/2018/12/14/binarysearchtree_improved.png" style="width: 200px; height: 190px;">
 * <pre><b>Input:</b> root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 4
 * <b>Output:</b> 2
 * <b>Explanation:</b> The LCA of nodes 2 and 4 is 2, since a node can be a descendant of itself according to the LCA definition.
 * </pre>
 *
 * <p><b>Example 3:</b
 * </p>
 * <pre><b>Input:</b> root = [2,1], p = 2, q = 1
 * <b>Output:</b> 2
 * </pre>
 * </div>
 * <p><strong>Constraints:</strong></p>
 * <ul>
 * <li>The number of nodes in the tree is in the range <code>[2, 10<sup>5</sup>]</code>.
</li>
 * <li><code>-10<sup>9</sup> &lt;= Node.val &lt;= 10<sup>9</sup></code></li>
 * <li>All <code>Node.val</code> are <strong>unique</strong>.</li>
 * <li><code>p != q</code></li>
```

```
 * <li><code>p</code> and <code>q</code> will exist in the BST.</li>
 * </ul>
 * </div>
 */
public class Solution00235 {

    // Time: O(n)
    // Space: O(1)
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        TreeNode current = root;
        while (current != null) {
            if (current.val > p.val && current.val > q.val) {
                current = current.left;
            } else if (current.val < p.val && current.val < q.val) {
                current = current.right;
            } else {
                return current;
            }
        }
        return null;
    }

    // Time: O(n)
    // Space: O(1)
    public TreeNode lowestCommonAncestorOptimized(TreeNode root, TreeNode p, TreeNode q)
{
        int small = Math.min(p.val, q.val);
        int large = Math.max(p.val, q.val);
        TreeNode current = root;
        while (current != null) {
            if (current.val > large) {
                current = current.left;
            } else if (current.val < small) {
                current = current.right;
            } else {
                return current;
            }
        }
        return null;
    }


}
```

110. `Solution00237.java`

```
package leetcode;

import leetcode.common.ListNode;

/**
 * 237. Delete Node in a Linked List
 * https://leetcode.com/problems/delete-node-in-a-linked-list/
 * Medium
 */

public class Solution00237 {

    public void deleteNode(ListNode node) {
        node.val = node.next.val;
        node.next = node.next.next;
    }

}
```

111. `Solution00238.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/product-of-array-except-self/
 * 238. Product of Array Except Self
 *
 * Given an integer array nums, return an array answer such that answer[i] is equal to
the product of all the elements of nums except nums[i].
 *
 * The product of any prefix or suffix of nums is guaranteed to fit in a 32-bit integer.
 *
 *
 *
 * Example 1:
 *
 * Input: nums = [1,2,3,4]
 * Output: [24,12,8,6]
 * Example 2:
 *
 * Input: nums = [-1,1,0,-3,3]
 * Output: [0,0,9,0,0]
 *
 *
 * Constraints:
 *
 * 2 <= nums.length <= 105
 * -30 <= nums[i] <= 30
 * The product of any prefix or suffix of nums is guaranteed to fit in a 32-bit integer.
 *
 *
 * Follow up:
 *
 * Could you solve it in O(n) time complexity and without using division?
 * Could you solve it with O(1) constant space complexity? (The output array does not
count as extra space
 */
public class Solution00238 {

    // Time: O(n * n)
    // Space: O(n)
    public int[] productExceptSelfBruteforce(int[] nums) {
        int[] result = new int[nums.length];

        // Iterate over the array.
        for (int i = 0; i < nums.length; i++) {
            // Calculate the product of all elements except the current one.
            int product = 1;
            for (int j = 0; j < nums.length; j++) {
                if (i != j) {
                    product *= nums[j];
                }
```

```
            }
            result[i] = product;
        }

        return result;
    }

    // Time: O(n)
    // Space: O(1)
    public int[] productExceptSelf(int[] nums) {
        int[] result = new int[nums.length];

        // product of all elements to the left of i
        result[0] = 1;
        for (int i = 1; i < nums.length; i++) {
            result[i] = result[i-1] * nums[i-1];
        }

        // product of all elements to the right of i
        int rightProducts = 1;
        for (int i = nums.length - 2; i >= 0; i--) {
            rightProducts *= nums[i+1];
            result[i] *= rightProducts; // multiply products to the left by the product
to the right
        }

        return result;
    }

    // Time: O(n)
    // Space: O(n)
    public int[] productExceptSelf2Arrays(int[] nums) {
        int[] leftProducts = new int[nums.length];
        int[] rightProducts = new int[nums.length];
        int[] result = new int[nums.length];

        // product of all elements to the left of i
        leftProducts[0] = 1;
        for (int i = 1; i < nums.length; i++) {
            leftProducts[i] = leftProducts[i - 1] * nums[i - 1];
        }

        // product of all elements to the right of i
        rightProducts[nums.length - 1] = 1;
        for (int i = nums.length - 2; i >= 0; i--) {
            rightProducts[i] = rightProducts[i + 1] * nums[i + 1];
        }

        // multiply products to the left by the product to the right
        for (int i = 0; i < nums.length; i++) {
            result[i] = leftProducts[i] * rightProducts[i];
        }

        return result;
```

```
    }
}
```

112. `Solution00239.java`

```
package leetcode;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * https://leetcode.com/problems/sliding-window-maximum/
 * 239. Sliding Window Maximum
 * <p>
 * You are given an array of integers nums,
 * there is a sliding window of size k which is moving from the very left of the array to
the very right.
 * You can only see the k numbers in the window.
 * Each time the sliding window moves right by one position.
 * <p>
 * Return the max sliding window.
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [1,3,-1,-3,5,3,6,7], k = 3
 * Output: [3,3,5,5,6,7]
 * Explanation:
 * <p>
 * Window position                Max
 * ---------------               -----
 * [1  3  -1] -3  5  3  6  7       3
 * 1 [3  -1  -3] 5  3  6  7        3
 * 1  3 [-1  -3  5] 3  6  7        5
 * 1  3  -1 [-3  5  3] 6  7        5
 * 1  3  -1  -3 [5  3  6] 7        6
 * 1  3  -1  -3  5 [3  6  7]       7
 * Example 2:
 * <p>
 * Input: nums = [1], k = 1
 * Output: [1]
 * Example 3:
 * <p>
 * Input: nums = [1,-1], k = 1
 * Output: [1,-1]
 * Example 4:
 * <p>
 * Input: nums = [9,11], k = 2
 * Output: [11]
 * Example 5:
 * <p>
 * Input: nums = [4,-2], k = 2
 * Output: [4]
 * <p>
 * <p>
 * Constraints:
 * <p>
```

```
 * 1 <= nums.length <= 105
 * -104 <= nums[i] <= 104
 * 1 <= k <= nums.length
 */
public class Solution00239 {

    // Time: O(n)
    // Space: O(n)
    public int[] maxSlidingWindow(int[] nums, int k) {
        final int numsLength = nums.length;
        // Create an array to store the maximum values of the sliding window.
        int[] result = new int[numsLength - k + 1];
        // Create a queue to store the indices of the numbers in the sliding window.
        int[] queue = new int[numsLength];
        // Create a pointer to keep track of the index of the first element in the queue.
        int front = 0;
        // Create a pointer to keep track of the index of the last element in the queue.
        int rear = -1;

        // Iterate over the numbers in the array nums.
        for (int i = 0; i < numsLength; i++) {
            // If the queue is not empty and the index of the first element in the queue
is outside of the sliding window,
            // remove the first element from the queue.
            if (front <= rear && i - queue[front] >= k) {
                front++;
            }

            // While the queue is not empty and the number at the index of the last
element in the queue is less than
            // the current number, remove the last element from the queue.
            while (front <= rear && nums[queue[rear]] < nums[i]) {
                rear--;
            }

            // Add the index of the current number to the queue.
            queue[++rear] = i;

            // If the index of the first element in the queue is within the sliding
window,
            // add the number at the index of the first element in the queue to the array
result.
            final int resultIndex = i - k + 1;
            if (resultIndex >= 0) {
                result[resultIndex] = nums[queue[front]];
            }
        }

        // Return the array result.
        return result;
    }

    // Time: O(n)
    // Space: O(n)
```

```
    public int[] maxSlidingWindowDeque(int[] nums, int k) {
        // Create an array to store the maximum values of the sliding window.
        final int numsLength = nums.length;
        int[] result = new int[numsLength - k + 1];
        // Create a deque to store the indices of the numbers in the sliding window.
        Deque<Integer> deque = new ArrayDeque<>();

        // Iterate over the numbers in the array nums.
        for (int i = 0; i < numsLength; i++) {
            // If the deque is not empty and the index of the first element in the deque
is outside of the sliding window,
            // remove the first element from the deque.
            if (!deque.isEmpty() && i - deque.peekFirst() >= k) {
                deque.pollFirst();
            }

            // While the deque is not empty and the number at the index of the last
element in the deque is less than
            // the current number, remove the last element from the deque.
            while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i]) {
                deque.pollLast();
            }

            // Add the index of the current number to the deque.
            deque.offerLast(i);

            // If the index of the first element in the deque is within the sliding
window,
            // add the number at the index of the first element in the deque to the array
result.
            final int resultIndex = i - k + 1;
            if (resultIndex >= 0) {
                result[resultIndex] = nums[deque.peekFirst()];
            }
        }

        // Return the array result.
        return result;
    }

}
```

113. `Solution00241.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 241. Different Ways to Add Parentheses
 * <a href="https://leetcode.com/problems/different-ways-to-add-parentheses">Web link</a>
 * Medium
 */

public class Solution00241 {

    public List<Integer> diffWaysToCompute(String expression) {
        Map<String, List<Integer>> map = new HashMap<>();
        return solveTopDown(expression, map);
    }

    private List<Integer> solveTopDown(String expression, Map<String, List<Integer>> map)
{
        if (map.containsKey(expression)) {
            return map.get(expression);
        }
        List<Integer> ans = new ArrayList<>();
        for (int i = 0; i < expression.length(); i++) {
            char c = expression.charAt(i);
            if (c == '+' || c == '-' || c == '*') {
                List<Integer> left = solveTopDown(expression.substring(0, i), map);
                List<Integer> right = solveTopDown(expression.substring(i + 1), map);
                for (int l : left) {
                    for (int r : right) {
                        if (c == '+') ans.add(l + r);
                        else if (c == '-') ans.add(l - r);
                        else ans.add(l * r);
                    }
                }
            }
        }
        if (ans.isEmpty()) {
            ans.add(Integer.parseInt(expression));
        }
        map.put(expression, ans);
        return ans;
    }

}
```

114. `Solution00242.java`

```
package leetcode;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

/**
 * https://leetcode.com/problems/valid-anagram/
 * 242. Valid Anagram
 * <p>
 * Given two strings s and t, return true if t is an anagram of s, and false otherwise.
 * <p>
 * Example 1:
 * <p>
 * Input: s = "anagram", t = "nagaram" Output: true Example 2:
 * <p>
 * Input: s = "rat", t = "car" Output: false
 * <p>
 * Constraints:
 * <p>
 * 1 <= s.length, t.length <= 5 * 104 s and t consist of lowercase English letters.
 * <p>
 * Follow up: What if the inputs contain Unicode characters? How would you adapt your
solution to such a case?
 */
public class Solution00242 {

    // Time: O(n log n)
    // Space: O(n)
    public boolean isAnagramSorting(String s, String t) {
        // Check for different lengths, strings not empty by constraints.
        if (s.length() != t.length()) {
            return false;
        }

        // Convert the strings to char arrays.
        char[] sChars = s.toCharArray();
        char[] tChars = t.toCharArray();

        // Sort the char arrays.
        Arrays.sort(sChars);
        Arrays.sort(tChars);

        // Compare the sorted char arrays.
        return Arrays.equals(sChars, tChars);
    }

    // Time: O(n)
    // Space: O(1)
    public boolean isAnagramMap(String s, String t) {
        // Check for different lengths, strings not empty by constraints.
```

```
        if (s.length() != t.length()) {
            return false;
        }

        // Create a map for storing frequencies
        Map<Character, Integer> freqMap = new HashMap<>();

        // Iterate over the first string and increment the count of each character.
        for (int i = 0; i < s.length(); i++) {
            char currentChar = s.charAt(i);
            freqMap.put(currentChar, freqMap.getOrDefault(currentChar, 0) + 1);
        }

        // Iterate over the second string and decrement the count of each character.
        for (int i = 0; i < t.length(); i++) {
            char currentChar = t.charAt(i);
            freqMap.put(currentChar, freqMap.getOrDefault(currentChar, 0) - 1);
        }

        // Iterate over the counts array and check if all counts are 0.
        for (int value : freqMap.values()) {
            if (value != 0) {
                return false;
            }
        }

        return true;
    }

    // Time: O(n)
    // Space: O(1)
    public boolean isAnagram(String s, String t) {
        // Check for different lengths, strings not empty by constraints.
        if (s.length() != t.length()) {
            return false;
        }

        // Create an array to store the counts of each character.
        int[] freqArr = new int[26];

        char currentChar;
        int position;

        // Iterate over the first string and increment the count of each character.
        for (int i = 0; i < s.length(); i++) {
            currentChar = s.charAt(i);
            position = currentChar - 'a';
            freqArr[position]++;
        }

        // Iterate over the second string and decrement the count of each character.
        for (int i = 0; i < t.length(); i++) {
            currentChar = t.charAt(i);
            position = currentChar - 'a';
```

```
            freqArr[position]--;
            if (freqArr[position] < 0) {
                return false;
            }
        }

        // Iterate over the frequency array and check if all counts are 0.
        for (int count : freqArr) {
            if (count != 0) {
                return false;
            }
        }

        return true;
    }

}
```

115. `Solution00260.java`

```
package leetcode;

/**
 * 260. Single Number III
 * <a href="https://leetcode.com/problems/single-number-iii">Web link</a> |
 * Medium
 */

public class Solution00260 {
    public int[] singleNumber(int[] nums) {
        int xor = 0;
        for (int num : nums) {
            xor ^= num;
        }
        int mask = xor & -xor;
        int[] result = new int[2];
        for (int num : nums) {
            if ((num & mask) == 0) {
                result[0] ^= num;
            } else {
                result[1] ^= num;
            }
        }
        return result;
    }
}
```

116. `Solution00264.java`

```
package leetcode;

import java.util.HashSet;
import java.util.PriorityQueue;
import java.util.Queue;
import java.util.Set;

/**
 * 264. Ugly Number II
 * <a href="https://leetcode.com/problems/ugly-number-ii">Web link</a> |
 * Medium
 */

public class Solution00264 {

    public int nthUglyNumber_PriorityQueue(int n) {
        long num = 1;
        Queue<Long> pq = new PriorityQueue<>();
        Set<Long> visited = new HashSet<>();
        addIfNotPresent(pq, num, visited);
        for (int i = 0; i < n; i++) {
            num = pq.poll();
            addIfNotPresent(pq, num * 2, visited);
            addIfNotPresent(pq, num * 3, visited);
            addIfNotPresent(pq, num * 5, visited);
        }
        return (int) num;
    }

    private static void addIfNotPresent(Queue<Long> pq, long num, Set<Long> visited) {
        if (visited.add(num)) {
            pq.add(num);
        }
    }

    public int nthUglyNumber_Dp(int n) {
        int[] dp = new int[n];
        dp[0] = 1;

        int i2 = 0;
        int i3 = 0;
        int i5 = 0;

        for (int i = 1; i < n; i++) {
            int num2 = dp[i2] * 2;
            int num3 = dp[i3] * 3;
            int num5 = dp[i5] * 5;

            int nextMin = Math.min(num2, Math.min(num3, num5));
            dp[i] = nextMin;
```

```
            if (nextMin == num2) {
                i2++;
            }
            if (nextMin == num3) {
                i3++;
            }
            if (nextMin == num5) {
                i5++;
            }
        }

        return dp[n - 1];
    }

}
```

117. `Solution00268.java`

```
package leetcode;

/**
 * Problem: 268. Missing Number
 * Source: https://leetcode.com/problems/missing-number/
 * Easy
 */

public class Solution00268 {

    // Time: O(n)
    // Space: O(1)
    public int missingNumber(int[] nums) {
        int n = nums.length;
        int result = n;
        for (int i = 0; i < n; i++) {
            result = result + i - nums[i];
        }
        return result;
    }

}
```

118. `Solution00271.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.List;

/**
 * https://leetcode.com/problems/encode-and-decode-strings/
 * 271. Encode and Decode Strings
 * Medium
 * <p>
 * Design an algorithm to encode a list of strings to a string. The encoded string is
then sent over the network and is
 * decoded back to the original list of strings.
 * <p>
 * Machine 1 (sender) has the function:
 * <p>
 * string encode(vector<string> strs) { // ... your code return encoded_string; } Machine
2 (receiver) has the function:
 * <p>
 * vector<string> decode(string s) { //... your code return strs; }
 * <p>
 * So Machine 1 does:
 * <p>
 * string encoded_string = encode(strs); and Machine 2 does:
 * <p>
 * vector<string> strs2 = decode(encoded_string); strs2 in Machine 2 should be the same
as strs in Machine 1.
 * <p>
 * Implement the encode and decode methods.
 * <p>
 * <p>
 * <p>
 * Example 1:
 * <p>
 * Input: dummy_input = ["Hello","World"] Output: ["Hello","World"] Explanation: Machine
1: Codec encoder = new
 * Codec(); String msg = encoder.encode(strs); // send encoded string over the network to
machine 2
 * <p>
 * Machine 2: Codec decoder = new Codec(); String[] strs = decoder.decode(msg); // return
["Hello","World"].
 * <p>
 * <p>
 * Constraints:
 * <p>
 * 1 <= strs.length <= 200 0 <= strs[i].length <= 200 strs[i] contains any possible
characters out of 256 valid ASCII
 * characters. Calls encode() and decode() as many times as needed. You must not
implement any functionality of
 * Built-in String library or Built-in Codec library, like
 * <p>
```

```
 * String.split(). Note: This is a companion problem to the System Design problem: Design TinyURL.
 */
public class Solution00271 {

    // Time: O(n)
    // Space: O(n)
    public class Codec {

        private static final char SEPARATOR = ';';

        // Encodes a list of strings to a single string.
        public String encode(String[] strs) {
            // Create a string builder.
            StringBuilder sb = new StringBuilder();

            // Iterate over the array.
            for (String str : strs) {
                // Append the length of the string.
                sb.append(str.length());
                // Append a delimiter.
                sb.append(SEPARATOR);
                // Append the string.
                sb.append(str);
            }

            // Return the string.
            return sb.toString();
        }

        // Decodes a single string to a list of strings.
        public String[] decode(String s) {
            // Create a list of strings.
            List<String> list = new ArrayList<>();

            // Iterate over the string.
            int n = s.length();
            int pointer1 = 0;
            int pointer2 = pointer1 + 1;
            while (pointer2 < n - 1) {
                // Get the length of the string.
                if (s.charAt(pointer2) == SEPARATOR) {
                    int len = Integer.parseInt(s.substring(pointer1, pointer2));
                    String str = s.substring(pointer2 + 1, pointer2 + 1 + len);
                    list.add(str);
                    pointer1 = pointer2 + 1 + len;
                    pointer2 = pointer1 + 1;
                } else {
                    pointer2++;
                }
            }

            // Return the list.
            return list.toArray(new String[0]);
```

```
        }
    }

}
```

119. `Solution00273.java`

```
package leetcode;

/**
 * 273. Integer to English Words
 * <a href="https://leetcode.com/problems/integer-to-english-words">Web link</a>
 * Hard
 */

public class Solution00273 {

    private static final String[] ONES = {"Zero ", "One ", "Two ", "Three ", "Four ",
"Five ", "Six ", "Seven ", "Eight ", "Nine "};
    private static final String[] TENS = {"", "", "Twenty ", "Thirty ", "Forty ", "Fifty
", "Sixty ", "Seventy ", "Eighty ", "Ninety "};
    private static final String[] TEENS = {"Ten ", "Eleven ", "Twelve ", "Thirteen ",
"Fourteen ", "Fifteen ", "Sixteen ", "Seventeen ", "Eighteen ", "Nineteen "};

    public String numberToWordsStructured(int num) {
        if (num == 0) {
            return "Zero";
        }

        final StringBuilder sb = new StringBuilder();
        if (num >= 1_000_000_000) {
            sb.append(numberToWordsStructured(num / 1_000_000_000));
            sb.append(" Billion ");
            num %= 1_000_000_000;
        }
        if (num >= 1_000_000) {
            sb.append(numberToWordsStructured(num / 1_000_000));
            sb.append(" Million ");
            num %= 1_000_000;
        }
        if (num >= 1_000) {
            sb.append(numberToWordsStructured(num / 1_000));
            sb.append(" Thousand ");
            num %= 1_000;
        }
        if (num >= 100) {
            sb.append(ONES[num / 100]);
            sb.append("Hundred ");
            num %= 100;
        }
        if (num >= 20) {
            sb.append(TENS[num / 10]);
            num %= 10;
        } else if (num >= 10) {
            sb.append(TEENS[num - 10]);
            num = 0;
        }
        if (num > 0) {
```

```
            sb.append(ONES[num]);
        }
        sb.setLength(sb.length() - 1);
        return sb.toString();
    }

    public String numberToWords(int num) {
        if (num < 1_000) {
            return get3DigitString(num, true, "");
        } else if (num < 1_000_000) {
            return lessThanMillion(num, true);
        } else if (num < 1_000_000_000) {
            return lessThanBillion(num, true);
        } else {
            return lessThanTrillion(num);
        }
    }

    private String lessThanTrillion(int num) {
        int last9Digits = num % 1_000_000_000;
        int first3Digits = num / 1_000_000_000;
        return get3DigitString(first3Digits, true, " Billion") +
               lessThanBillion(last9Digits, false);
    }

    private String lessThanBillion(int num, boolean firstSection) {
        int last6Digits = num % 1_000_000;
        int first3Digits = num / 1_000_000;
        return get3DigitString(first3Digits, firstSection, " Million") +
               lessThanMillion(last6Digits, false);
    }

    private String lessThanMillion(int num, boolean firstSection) {
        int last3Digits = num % 1000;
        int first3Digits = num / 1000;
        return get3DigitString(first3Digits, firstSection, " Thousand") +
               get3DigitString(last3Digits, false, "");
    }

    private String get3DigitString(int num, boolean firstSection, String suffix) {
        StringBuilder sb = new StringBuilder();
        int hundred = num / 100;
        if (hundred > 0) {
            if (!firstSection) {
                sb.append(" ");
            }
            sb.append(lessThanTwenty(hundred)).append(" Hundred");
        }
        int lastTwoDigits = num % 100;
        if (lastTwoDigits == 0) {
            if (hundred > 0) {
                return sb
                        .append(suffix)
                        .toString();
```

```
            } else {
                if (firstSection) {
                    return "Zero";
                } else {
                    return "";
                }
            }
        }
        if (lastTwoDigits > 0) {
            if (hundred > 0 || !firstSection) {
                sb.append(" ");
            }
        }
        if (lastTwoDigits < 20) {
            return sb.append(lessThanTwenty(lastTwoDigits))
                    .append(suffix)
                    .toString();
        }
        sb.append(lessThanHundred(lastTwoDigits / 10 * 10));
        int lastOnes = lastTwoDigits % 10;
        if (lastOnes == 0) {
            return sb
                    .append(suffix)
                    .toString();
        }
        return sb.append(" ")
                .append(lessThanTwenty(lastOnes))
                .append(suffix)
                .toString();
    }

    private String lessThanHundred(int num) {
        return switch (num) {
            case 20 -> "Twenty";
            case 30 -> "Thirty";
            case 40 -> "Forty";
            case 50 -> "Fifty";
            case 60 -> "Sixty";
            case 70 -> "Seventy";
            case 80 -> "Eighty";
            case 90 -> "Ninety";
            default -> "";
        };
    }

    private String lessThanTwenty(int num) {
        return switch (num) {
            case 1 -> "One";
            case 2 -> "Two";
            case 3 -> "Three";
            case 4 -> "Four";
            case 5 -> "Five";
            case 6 -> "Six";
            case 7 -> "Seven";
```

```
            case 8 -> "Eight";
            case 9 -> "Nine";
            case 10 -> "Ten";
            case 11 -> "Eleven";
            case 12 -> "Twelve";
            case 13 -> "Thirteen";
            case 14 -> "Fourteen";
            case 15 -> "Fifteen";
            case 16 -> "Sixteen";
            case 17 -> "Seventeen";
            case 18 -> "Eighteen";
            case 19 -> "Nineteen";
            default -> "";
        };
    }

}
```

120. `Solution00274.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 274. H-Index
 * <a href="https://leetcode.com/problems/h-index">Web link</a> |
 * Medium
 */

public class Solution00274 {

    // Time: O(n)
    // Space: O(n)
    public int hIndex(int[] citations) {
        int totalPapers = citations.length;
        int[] papersWithCitations = new int[totalPapers + 1];

        for (int citationCount : citations) {
            if (citationCount > totalPapers) {
                papersWithCitations[totalPapers]++;
            } else {
                papersWithCitations[citationCount]++;
            }
        }

        int hIndexCandidate = totalPapers;
        int totalPapersWithAtLeastHCitations = papersWithCitations[totalPapers];

        while (hIndexCandidate > totalPapersWithAtLeastHCitations) {
            totalPapersWithAtLeastHCitations += papersWithCitations[--hIndexCandidate];
        }

        return hIndexCandidate;
    }

    public int hIndexSorting(int[] citations) {
        Arrays.sort(citations);
        int totalPapers = citations.length;
        int result = 0;
        for (int i = 0; i < totalPapers; i++) {
            if (citations[i] >= totalPapers - i) {
                result = totalPapers - i;
                break;
            }
        }
        return result;
    }

}
```

121. `Solution00279.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/perfect-squares/
 * 279. Perfect Squares
 * Medium
 */
public class Solution00279 {

    // Time complexity: O(n)
    // Space complexity: O(n)
    public int numSquaresDP(int n) {
        // Initialize dynamic programming array
        int[] dp = new int[n + 1];

        // Iterate through each number up to n
        for (int currentNum = 1; currentNum <= n; currentNum++) {
            // Initially set the minimum number of squares for currentNum to be
currentNum itself
            // because square of 1 is 1, we can always get target using squares of 1
            dp[currentNum] = currentNum;

            // Iterate through each square number up to currentNum
            for (int square = 1, squareOfSquare = 1; squareOfSquare <= currentNum;
square++, squareOfSquare = square * square) {
                // Calculate the remaining number after subtracting the square number
                int remaining = currentNum - squareOfSquare;

                // Calculate the minimum number of squares for currentNum
                int minSquares = dp[remaining] + 1;

                // Update the minimum number of squares for currentNum
                dp[currentNum] = Math.min(dp[currentNum], minSquares);
            }
        }

        // Return the minimum number of squares for n
        return dp[n];
    }

}
```

122. `Solution00286.java`

```
package leetcode;

import java.util.ArrayDeque;
import java.util.Queue;

/**
 * 286. Walls and Gates
 * <a href="https://leetcode.com/problems/walls-and-gates">Web link</a>
 * Medium
 */

public class Solution00286 {

    static final int GATE = 0;
    static final int WALL = -1;

    public void wallsAndGates(int[][] rooms) {
        int rowCount = rooms.length;
        int colCount = rooms[0].length;

        boolean[][] visited = new boolean[rowCount][colCount];
        Queue<Room> deque = new ArrayDeque<>();

        for (int row = 0; row < rowCount; row++) {
            for (int col = 0; col < colCount; col++) {
                if (rooms[row][col] == GATE) {
                    deque.offer(new Room(row, col));
                    visited[row][col] = true;
                }
            }
        }

        int dist = 0;
        while (!deque.isEmpty()) {
            int numToPop = deque.size();
            for (int i = 0; i < numToPop; i++) {
                Room room = deque.poll();
                final int row = room.row;
                final int col = room.col;

                rooms[row][col] = dist;

                addRoom(rooms, row + 1, col, deque, visited);
                addRoom(rooms, row - 1, col, deque, visited);
                addRoom(rooms, row, col + 1, deque, visited);
                addRoom(rooms, row, col - 1, deque, visited);

            }
            dist++;
        }
```

```
    }

    private void addRoom(int[][] rooms, int row, int col, Queue<Room> deque, boolean[][]
visited) {
        if (row < 0 || row >= rooms.length ||
            col < 0 || col >= rooms[0].length ||
            rooms[row][col] == WALL ||
            visited[row][col]) {
            return;
        }

        deque.offer(new Room(row, col));
        visited[row][col] = true;
    }

    private static class Room {
        private final int row;
        private final int col;

        public Room(int row, int col) {
            this.row = row;
            this.col = col;
        }
    }

}
```

123. `Solution00287.java`

```
package leetcode;

import java.util.HashSet;
import java.util.Set;

/**
 * <a href="https://leetcode.com/problems/find-the-duplicate-number/">
 * 287. Find the Duplicate Number
 * </a>
 * <br/>
 * <div class="content__u3I1 question-content__JfgR">
 * <div><p>Given an array of integers <code>nums</code> containing <code>n +
1</code> integers where each integer is in the range <code>[1, n]</code> inclusive.</p>
 *
 * <p>There is only <strong>one repeated number</strong> in <code>nums</code>, return
<em>this repeated number</em>.</p>
 *
 * <p>You must solve the problem <strong>without</strong> modifying the array
<code>nums</code> and uses only constant extra space.</p>
 *
 * <p><strong>Example 1:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> nums = [1,3,4,2,2]
 * <strong>Output:</strong> 2
 * </pre>
 *
 * <p><strong>Example 2:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> nums = [3,1,3,4,2]
 * <strong>Output:</strong> 3
 * </pre>
 *
 * <p><strong>Example 3:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> nums = [1,1]
 * <strong>Output:</strong> 1
 * </pre>
 *
 * <p><strong>Example 4:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> nums = [1,1,2]
 * <strong>Output:</strong> 1
 * </pre>
 * <p>
 * Constraints:
 * <p>
 * 1 <= n <= 105
```

```
 * nums.length == n + 1
 * 1 <= nums[i] <= n
 * All the integers in nums appear only once except for precisely one integer which
appears two or more times.
 * <p>
 * <p>
 * Follow up:
 * <p>
 * How can we prove that at least one duplicate number must exist in nums?
 * Can you solve the problem in linear runtime complexity?
 */
public class Solution00287 {

    // Time: O(n^2)
    // Space: O(1)
    public int findDuplicateBruteforce(int[] nums) {
        for (int i = 0; i < nums.length; i++) {
            int current = nums[i];
            for (int j = i + 1; j < nums.length; j++) {
                if (current == nums[j]) {
                    return current;
                }
            }
        }

        return -1;
    }

    // Time: O(n)
    // Space: O(n)
    public int findDuplicateArray(int[] nums) {
        boolean[] visited = new boolean[nums.length];
        for (int num : nums) {
            if (visited[num]) {
                return num;
            }
            visited[num] = true;
        }

        return -1;
    }

    // Time: O(n)
    // Space: O(n)
    public int findDuplicateSet(int[] nums) {
        Set<Integer> set = new HashSet<>();
        for (int num : nums) {
            if (set.contains(num)) {
                return num;
            }

            set.add(num);
        }
```

```
        return -1;
    }

    // Time: O(n)
    // Space: O(1)
    // Floyd's Tortoise and Hare (Cycle Detection), youtube
https://www.youtube.com/watch?v=PvrxZaH_eZ4
    public int findDuplicateFloyd(int[] nums) {
        int slowPointer = nums[0];
        int fastPointer = nums[0];

        do {
            slowPointer = nums[slowPointer];
            fastPointer = nums[nums[fastPointer]];
        } while (slowPointer != fastPointer);

        slowPointer = nums[0];
        while (slowPointer != fastPointer) {
            slowPointer = nums[slowPointer];
            fastPointer = nums[fastPointer];
        }

        return slowPointer;
    }

}
```

124. `Solution00297.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * https://leetcode.com/problems/serialize-and-deserialize-binary-tree/
 * 297. Serialize and Deserialize Binary Tree
 * Hard
 * <p>
 * Serialization is the process of converting a data structure or object into a sequence
of bits so that it can be
 * stored in a file or memory buffer, or transmitted across a network connection link to
be reconstructed later in
 * the same or another computer environment.
 * <p>
 * Design an algorithm to serialize and deserialize a binary tree. There is no
restriction on how your
 * serialization/deserialization algorithm should work. You just need to ensure that a
binary tree can be serialized
 * to a string and this string can be deserialized to the original tree structure.
 * <p>
 * Clarification: The input/output format is the same as how LeetCode serializes a binary
tree. You do not necessarily
 * need to follow this format, so please be creative and come up with different
approaches yourself.
 * <p>
 * Example 1:
 * Input: root = [1,2,3,null,null,4,5]
 * Output: [1,2,3,null,null,4,5]
 * Example 2:
 * Input: root = []
 * Output: []
 * Example 3:
 * Input: root = [1]
 * Output: [1]
 * Example 4:
 * Input: root = [1,2]
 * Output: [1,2]
 * Constraints:
 * The number of nodes in the tree is in the range [0, 104].
 * -1000 <= Node.val <= 1000
 */
public class Solution00297 {

    static interface Codec {
        // Encodes a tree to a single string.
```

```
        String serialize(TreeNode root);

        // Decodes your encoded data to tree.
        TreeNode deserialize(String data);
    }

    static class CodecBfs implements Codec {

        @Override
        public String serialize(TreeNode root) {
            if (root == null) {
                return "";
            }
            Queue<TreeNode> queue = new LinkedList<>();
            StringBuilder sb = new StringBuilder();
            queue.offer(root);
            while (!queue.isEmpty()) {
                TreeNode node = queue.poll();
                if (node == null) {
                    sb.append("null,");
                    continue;
                }
                sb.append(node.val).append(",");
                queue.offer(node.left);
                queue.offer(node.right);
            }
            return sb.toString();
        }

        // Decodes your encoded data to tree.
        public TreeNode deserialize(String data) {
            if ("".equals(data)) {
                return null;
            }
            String[] nodes = data.split(",");
            Queue<TreeNode> queue = new LinkedList<>();
            TreeNode root = new TreeNode(Integer.parseInt(nodes[0]));
            queue.offer(root);
            int index = 1;
            while (index < nodes.length) {
                TreeNode parent = queue.poll();
                int leftIndex = index;
                final String leftValue = nodes[leftIndex];
                if (!"null".equals(leftValue)) {
                    TreeNode left = new TreeNode(Integer.parseInt(leftValue));
                    parent.left = left;
                    queue.offer(left);
                }
                index++;
                int rightIndex = index;
                final String rightValue = nodes[rightIndex];
                if (!"null".equals(rightValue)) {
                    TreeNode right = new TreeNode(Integer.parseInt(rightValue));
                    parent.right = right;
```

```
                queue.offer(right);
            }
            index++;
        }
        return root;
    }

}

static class CodecDfs implements Codec {

    @Override
    public String serialize(TreeNode root) {
        if (root == null) return "";
        List<String> result = new ArrayList<>();
        serializeDfs(root, result);
        return String.join(",", result);
    }

    @Override
    public TreeNode deserialize(String data) {
        if ("".equals(data)) {
            return null;
        }
        IndexHolder indexHolder = new IndexHolder();
        String[] nodes = data.split(",");
        return deserializeDfs(nodes, indexHolder);
    }

    private void serializeDfs(TreeNode root, List<String> result) {
        if (root == null) {
            result.add("null");
            return;
        }
        result.add(String.valueOf(root.val));
        serializeDfs(root.left, result);
        serializeDfs(root.right, result);
    }

    private TreeNode deserializeDfs(String[] nodes, IndexHolder indexHolder) {
        if ("null".equals(nodes[indexHolder.index])) {
            indexHolder.index++;
            return null;
        }
        TreeNode root = new TreeNode(Integer.parseInt(nodes[indexHolder.index]));
        indexHolder.index++;
        root.left = deserializeDfs(nodes, indexHolder);
        root.right = deserializeDfs(nodes, indexHolder);
        return root;
    }

    private static class IndexHolder {
        int index = 0;
    }
```

```
    }

}
```

125. `Solution00300.java`

```
package leetcode;

import java.util.Arrays;

/**
 * <a href="https://leetcode.com/problems/longest-increasing-subsequence/">
 * 300. Longest Increasing Subsequence
 * </a>
 * <p>
 * Given an integer array nums, return the length of the longest strictly increasing
subsequence.
 * <p>
 * A subsequence is a sequence that can be derived from an array by deleting some or no
elements without changing the
 * order of the remaining elements. For example, [3,6,2,7] is a subsequence of the array
[0,3,1,6,2,2,7].
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [10,9,2,5,3,7,101,18]
 * Output: 4
 * Explanation: The longest increasing subsequence is [2,3,7,101], therefore the length
is 4.
 * <p>
 * Example 2:
 * <p>
 * Input: nums = [0,1,0,3,2,3]
 * Output: 4
 * <p>
 * Example 3:
 * <p>
 * Input: nums = [7,7,7,7,7,7,7]
 * Output: 1
 * <p>
 * <p>
 * Constraints:
 * <p>
 * 1 <= nums.length <= 2500
 * -104 <= nums[i] <= 104
 * <p>
 * <p>
 * Follow up:
 * <p>
 * Could you come up with the O(n2) solution?
 * Could you improve it to O(n log(n)) time complexity?
 */
public class Solution00300 {

    // Time O(n^2)
    // Space O(n)
    public int lengthOfLIS(int[] nums) {
```

```
        int[] dp = new int[nums.length];
        Arrays.fill(dp, 1);

        for (int i = 0; i < nums.length; i++) {
            for (int j = 0; j < i; j++) {
                if (nums[j] < nums[i]) {
                    dp[i] = Math.max(dp[j] + 1, dp[i]);
                }
            }
        }

        int max = 0;
        for (int i : dp) {
            max = Math.max(max, i);
        }
        return max;
    }

}
```

126. `Solution00310.java`

```
package leetcode;

import java.util.*;

/**
 * 310. Minimum Height Trees
 * <a href="https://leetcode.com/problems/minimum-height-trees">Web link</a> |
 * Medium
 */

public class Solution00310 {

    public List<Integer> findMinHeightTrees(int n, int[][] edges) {
        if (n == 1) {
            return List.of(0);
        }

        int[] degree = new int[n];
        Map<Integer, List<Integer>> adjacencyList = new HashMap<>();
        for (int[] edge : edges) {
            degree[edge[0]]++;
            degree[edge[1]]++;
            adjacencyList.computeIfAbsent(edge[0], x -> new ArrayList<>()).add(edge[1]);
            adjacencyList.computeIfAbsent(edge[1], x -> new ArrayList<>()).add(edge[0]);
        }

        Queue<Integer> leaves = new LinkedList<>();
        for (int i = 0; i < degree.length; i++) {
            if (degree[i] == 1) {
                leaves.add(i);
            }
        }

        int remainingNodes = n;
        while (remainingNodes > 2) {
            int size = leaves.size();
            remainingNodes -= size;
            for (int i = 0; i < size; i++) {
                int leaf = leaves.poll();
                for (int neighbor : adjacencyList.get(leaf)) {
                    if (--degree[neighbor] == 1) {
                        leaves.add(neighbor);
                    }
                }
            }
        }

        return new ArrayList<>(leaves);
    }

    public List<Integer> findMinHeightTreesMoreEfficient(int n, int[][] edges) {
```

```
        if (edges.length == 0 || n == 1) {
            return List.of(0);
        }
        List<Integer>[] adj = new List[n];
        int[] indegree = new int[n];

        for (int i = 0; i < n; i++) {
            adj[i] = new ArrayList<>();
        }

        for (int[] e : edges) {
            int v = e[0];
            int u = e[1];

            adj[v].add(u);
            adj[u].add(v);
            indegree[u]++;
            indegree[v]++;
        }

        Deque<Integer> queue = new ArrayDeque<>();

        for (int i = 0; i < n; ++i) {
            if (indegree[i] == 1) {
                queue.add(i);
            }
        }

        while (n > 2) {
            int size = queue.size();
            n -= size;

            while (size-- > 0) {
                //remove
                int rem = queue.remove();

                //mark*
                //work
                for (int i : adj[rem]) {
                    indegree[i]--;
                    if (indegree[i] == 1) {
                        queue.add(i);
                    }
                }
                //add*
            }
        }

        List<Integer> res = new ArrayList<>();
        while (!queue.isEmpty()) {
            res.add(queue.pop());
        }
```

```
        return res;
    }

}
```

127. `Solution00322.java`

```
package leetcode;

import java.util.Arrays;

/**
 * https://leetcode.com/problems/coin-change/
 * 322. Coin Change
 * <p>
 * You are given an integer array coins representing coins of different denominations
 * and an integer amount representing a total amount of money.
 * <p>
 * Return the fewest number of coins that you need to make up that amount.
 * If that amount of money cannot be made up by any combination of the coins, return -1.
 * <p>
 * You may assume that you have an infinite number of each kind of coin.
 * <p>
 * Example 1:
 * <p>
 * Input: coins = [1,2,5], amount = 11
 * Output: 3
 * <p>
 * Explanation: 11 = 5 + 5 + 1
 * <p>
 * Example 2:
 * <p>
 * Input: coins = [2], amount = 3
 * Output: -1
 * <p>
 * Example 3:
 * <p>
 * Input: coins = [1], amount = 0
 * Output: 0
 * <p>
 * Example 4:
 * <p>
 * Input: coins = [1], amount = 1
 * Output: 1
 * <p>
 * Example 5:
 * <p>
 * Input: coins = [1], amount = 2
 * Output: 2
 * <p>
 * Constraints:
 * <p>
 * 1 <= coins.length <= 12
 * 1 <= coins[i] <= 231 - 1
 * 0 <= amount <= 104
 */
public class Solution00322 {
```

```
    // Time: O(n * m)
    // Space: O(n)
    public int coinChange(int[] coins, int amount) {
        // Create a `dp` array to store the minimum number of coins needed to make up to
the given amount.
        int[] dp = new int[amount + 1];
        // Fill the `dp` array with the values that is more than the max int value.
        Arrays.fill(dp, Integer.MAX_VALUE);
        dp[0] = 0; // The minimum number of coins needed to make up 0 is 0.

        // Iterate over the possible amounts from 1 to the given amount.
        for (int i = 1; i <= amount; i++) {
            // For each amount, check if any of the coins can be used to make up the
amount.
            for (int coin : coins) {
                // If the amount is greater than or equal to the coin
                if (i >= coin && dp[i - coin] != Integer.MAX_VALUE) {
                    dp[i] = Math.min(dp[i], dp[i - coin] + 1);
                }
            }
        }

        if (dp[amount] == Integer.MAX_VALUE) {
            // If the minimum number of coins needed to make up the amount is still max
value, then there is no solution
            // for the given amount.
            return -1;
        } else {
            // Return the minimum number of coins needed to make up the given amount.
            return dp[amount];
        }
    }

}
```

128. `Solution00323.java`

```
package leetcode;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * https://leetcode.com/problems/number-of-connected-components-in-an-undirected-graph/
 * 323. Number of Connected Components in an Undirected Graph
 * <p>
 * You have a graph of n nodes. You are given an integer n and an array edges where
edges[i] = [ai, bi] indicates
 * that there is an edge between ai and bi in the graph.
 * <p>
 * Return the number of connected components in the graph.
 * <p>
 * Example 1:
 * <p>
 * Input: n = 5, edges = [[0,1],[1,2],[3,4]]
 * Output: 2
 * <p>
 * Example 2:
 * <p>
 * Input: n = 5, edges = [[0,1],[1,2],[2,3],[3,4]]
 * Output: 1
 * <p>
 * Constraints:
 * <p>
 * 1 <= n <= 2000
 * 1 <= edges.length <= 5000
 * edges[i].length == 2
 * 0 <= ai <= bi < n
 * ai != bi
 * There are no repeated edges.
 */
public class Solution00323 {

    // Time: O(n + m)
    // Space: O(n)
    public int countComponents(int n, int[][] edges) {
        // Create a map to store the edges.
        Map<Integer, Set<Integer>> edgesMap = new HashMap<>();

        // Iterate over the edges and add them to the map.
        for (int[] edge : edges) {
            int node1 = edge[0];
            int node2 = edge[1];

            if (!edgesMap.containsKey(node1)) {
                edgesMap.put(node1, new HashSet<>());
```

```
            }

            if (!edgesMap.containsKey(node2)) {
                edgesMap.put(node2, new HashSet<>());
            }

            edgesMap.get(node1).add(node2);
            edgesMap.get(node2).add(node1);
        }

        // Create an array to store the visited nodes.
        boolean[] visited = new boolean[n];

        // Create a variable to store the number of connected components.
        int numComponents = 0;

        // Iterate over the nodes and count the number of connected components.
        for (int i = 0; i < n; i++) {
            if (!visited[i]) {
                numComponents++;
                dfs(edgesMap, visited, i);
            }
        }

        return numComponents;
    }

    // Depth-first search to visit all nodes in the connected component.
    private void dfs(Map<Integer, Set<Integer>> edgesMap, boolean[] visited, int node) {
        visited[node] = true;

        for (int neighbor : edgesMap.get(node)) {
            if (!visited[neighbor]) {
                dfs(edgesMap, visited, neighbor);
            }
        }
    }

}
```

129. `Solution00330.java`

```
package leetcode;

/**
 * 330. Patching Array
 * <a href="https://leetcode.com/problems/patching-array/">Web link</a> |
 * Hard
 */

public class Solution00330 {

    public int minPatches(int[] nums, int n) {
        final int numsLen = nums.length;
        int patches = 0;
        int index = 0;
        long numberToReachNext = 1;
        while (numberToReachNext <= n) {
            if (index < numsLen && nums[index] <= numberToReachNext) {
                numberToReachNext += nums[index];
                index++;
            } else {
                numberToReachNext += numberToReachNext;
                patches++;
            }
        }
        return patches;
    }

}
```

130. `Solution00338.java`

```
package leetcode;

/**
 * 338. Counting Bits
 * <a href="https://leetcode.com/problems/counting-bits">Web link</a> |
 * Easy
 */

public class Solution00338 {
    public int[] countBits(int n) {
        int[] result = new int[n + 1];
        for (int i = 1; i <= n; i++) {
            // i & (i - 1) removes the rightmost 1-bit
            result[i] = result[i & (i - 1)] + 1;
        }
        return result;
    }

    public int[] countBitsClear(int n) {
        int[] result = new int[n + 1];
        for (int i = 0; i <= n; i++) {
            result[i] = Integer.bitCount(i);
        }
        return result;
    }

}
```

131. `Solution00344.java`

```
package leetcode;

/**
 * 344. Reverse String
 * https://leetcode.com/problems/reverse-string/description/
 * Easy
 */

public class Solution00344 {

    public void reverseString(char[] s) {
        int start = 0;
        int end = s.length - 1;
        while (start < end) {
            char tmp = s[start];
            s[start] = s[end];
            s[end] = tmp;
            start++;
            end--;
        }
    }

}
```

132. `Solution00347.java`

```
package leetcode;

import java.util.Arrays;
import java.util.Comparator;
import java.util.HashMap;
import java.util.Map;
import java.util.stream.Collectors;

/**
 * https://leetcode.com/problems/top-k-frequent-elements/
 * 347. Top K Frequent Elements
 * <p>
 * Given an integer array nums and an integer k, return the k most frequent elements. You
may return the answer in any order.
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [1,1,1,2,2,3], k = 2
 * Output: [1,2]
 * <p>
 * Example 2:
 * <p>
 * Input: nums = [1], k = 1
 * Output: [1]
 * <p>
 * Constraints:
 * <p>
 * 1 <= nums.length <= 105
 * k is in the range [1, the number of unique elements in the array].
 * It is guaranteed that the answer is unique.
 * <p>
 * Follow up: Your algorithm's time complexity must be better than O(n log n), where n is
the array's size.
 */
public class Solution00347 {

    // Time: O(n log n)
    // Space: O(n)
    public int[] topKFrequentStream(int[] nums, int k) {
        return Arrays.stream(nums)
                .boxed()
                .collect(Collectors.groupingBy(i -> i, Collectors.counting()))
                .entrySet()
                .stream()
                .sorted(Map.Entry.comparingByValue(Comparator.reverseOrder()))
                .limit(k)
                .map(Map.Entry::getKey)
                .mapToInt(Integer::intValue)
                .toArray();
    }
```

```
    // Time: O(n log n)
    // Space: O(n)
    public int[] topKFrequentInSteps(int[] nums, int k) {
        // Create a map to keep track of the frequency of each number.
        Map<Integer, Integer> frequencyMap = new HashMap<>();
        for (int num : nums) {
            frequencyMap.merge(num, 1, Integer::sum);
        }

        return frequencyMap.entrySet()
                .stream()
                .sorted(Map.Entry.comparingByValue(Comparator.reverseOrder())) // Sort
the map by value in descending order
                .limit(k)  // limit the results to k.
                .mapToInt(entry -> entry.getKey()) // convert to int
                .toArray(); // convert to array
    }

}
```

133. `Solution00349.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 349. Intersection of Two Arrays
 * <a href="https://leetcode.com/problems/intersection-of-two-arrays">Web link</a>
 * Easy
 */

public class Solution00349 {

    // Time: O(n + m)
    // Space: O(n + m)
    public int[] intersection(int[] nums1, int[] nums2) {
        Set<Integer> set1 = new HashSet<>();
        for (int n : nums1) {
            set1.add(n);
        }
        Set<Integer> set2 = new HashSet<>();
        for (int n : nums2) {
            set2.add(n);
        }
        set1.retainAll(set2);
        return set1.stream()
                .mapToInt(Integer::intValue)
                .toArray();
    }

    // Time: O(n + m)
    // Space: O(n + m)
    public int[] intersectionImproved(int[] nums1, int[] nums2) {
        Set<Integer> set1 = new HashSet<>(nums1.length);
        for (int n : nums1) {
            set1.add(n);
        }
        List<Integer> result = new ArrayList<>(set1.size());
        for (int n : nums2) {
            if (set1.contains(n)) {
                result.add(n);
                set1.remove(n);
            }
        }
        int[] arr = new int[result.size()];
        for (int index = 0; index < result.size(); index++) {
            arr[index] = result.get(index);
        }
        return arr;
```

```
    }
}
```

134. `Solution00350.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 350. Intersection of Two Arrays II
 * <a href="https://leetcode.com/problems/intersection-of-two-arrays-ii">Web link</a>
 * Easy
 */

public class Solution00350 {

    public int[] intersect(int[] nums1, int[] nums2) {
        if (nums1.length > nums2.length) {
            return intersect(nums2, nums1);
        }
        Arrays.sort(nums1);
        Arrays.sort(nums2);
        int pos1 = 0;
        int pos2 = 0;
        int posResult = 0;
        int[] resultMax = new int[nums1.length];
        while (pos1 < nums1.length && pos2 < nums2.length) {
            if (nums1[pos1] == nums2[pos2]) {
                resultMax[posResult] = nums1[pos1];
                ++pos1;
                ++pos2;
                ++posResult;
            } else if (nums1[pos1] < nums2[pos2]) {
                ++pos1;
            } else {
                ++pos2;
            }
        }
        int[] resultPrecise = new int[posResult];
        System.arraycopy(resultMax, 0, resultPrecise, 0, posResult);
        return resultPrecise;
    }

}
```

135. `Solution00368.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;

/**
 * https://leetcode.com/problems/largest-divisible-subset/
 * 368. Largest Divisible Subset
 * Medium
 */
public class Solution00368 {

    // Time complexity: O(n^2)
    // Space complexity: O(n)
    public List<Integer> largestDivisibleSubset(int[] nums) {
        Arrays.sort(nums);
        int n = nums.length;
        int[] dp = new int[n];
        int[] prev = new int[n];
        int max = 0;
        int index = -1;
        for (int i = 0; i < n; i++) {
            dp[i] = 1;
            prev[i] = -1;
            for (int j = i - 1; j >= 0; j--) {
                if ((nums[i] % nums[j] == 0) &&
                    (dp[j] + 1 > dp[i])) {
                    dp[i] = dp[j] + 1;
                    prev[i] = j;
                }
            }
            if (dp[i] > max) {
                max = dp[i];
                index = i;
            }
        }
        List<Integer> result = new ArrayList<>();
        while (index != -1) {
            result.add(nums[index]);
            index = prev[index];
        }
        Collections.reverse(result);
        return result;
    }

}
```

136. `Solution00371.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/sum-of-two-integers/
 * 371. Sum of Two Integers
 *
 * Given two integers a and b, return the sum of the two integers without using the
operators + and -.
 *
 *
 *
 * Example 1:
 *
 * Input: a = 1, b = 2
 * Output: 3
 * Example 2:
 *
 * Input: a = 2, b = 3
 * Output: 5
 *
 *
 * Constraints:
 *
 * -1000 <= a, b <= 1000
 */
public class Solution00371 {

    // Time: O(1)
    // Space: O(1)
    public int getSum(int a, int b) {
        int carry;
        while (b != 0) {
            // carry now contains common set bits of a and b, shifted left by 1
            carry = (a & b) << 1;

            // Sum of bits of a and b where at least one of the bits is not set
            a = a ^ b;

            b = carry;
        }

        return a;
    }

}
```

137. `Solution00380.java`

```
package leetcode;

import java.util.*;

/**
 * https://leetcode-cn.com/problems/insert-delete-getrandom-o1/
 * 380. Insert Delete GetRandom O(1)
 * Medium
 * <p>
 * Design a data structure that supports all following operations in average O(1) time.
 * <p>
 * insert(val): Inserts an item val to the set if not already present.
 * remove(val): Removes an item val from the set if present.
 * getRandom(): Returns a random element from current set of elements.
 * Each element must have the same probability of being returned.
 * <p>
 * Example:
 * // Init an empty set.
 * RandomizedSet randomSet = new RandomizedSet();
 * // Inserts 1 to the set. Returns true as 1 was inserted successfully.
 * randomSet.insert(1);
 * // Returns false as 2 does not exist in the set.
 * randomSet.remove(2);
 * // Inserts 2 to the set, returns true. Set now contains [1,2].
 * randomSet.insert(2);
 * // getRandom should return either 1 or 2 randomly.
 * randomSet.getRandom();
 * // Removes 1 from the set, returns true. Set now contains [2].
 * randomSet.remove(1);
 * // 2 was already in the set, so return false.
 * randomSet.insert(2);
 * // Since 2 is the only number in the set, getRandom always return 2.
 * randomSet.getRandom();
 */
public class Solution00380 {

    public static class RandomizedSet {

        private final List<Integer> list;
        private final Map<Integer, Integer> map;
        private final Random random;

        /**
         * Initialize your data structure here.
         */
        public RandomizedSet() {
            list = new ArrayList<>();
            map = new HashMap<>();
            random = new Random();
        }
```

```
        /**
         * Inserts a value to the set. Returns true if the set did not already contain
the specified element.
         */
        public boolean insert(int val) {
            if (map.containsKey(val)) {
                return false;
            }

            list.add(val);
            map.put(val, list.size() - 1);
            return true;
        }

        /**
         * Removes a value from the set. Returns true if the set contained the specified
element.
         */
        public boolean remove(int val) {
            // Check if the value exists in the map
            if (!map.containsKey(val)) {
                // If it doesn't exist, return false
                return false;
            }

            // Get the index of the value in the list
            int index = map.get(val);

            // Get the last element in the list
            int lastElement = list.get(list.size() - 1);

            // Replace the element at the index with the last element
            list.set(index, lastElement);

            // Update the map to reflect the new position of the last element
            map.put(lastElement, index);

            // Remove the last element from the list
            list.remove(list.size() - 1);

            // Remove the value from the map
            map.remove(val);

            // If we've reached this point, the removal was successful, so return true
            return true;
        }

        /**
         * Get a random element from the set.
         */
        public int getRandom() {
            return list.get(random.nextInt(list.size()));
        }
```

```
    }

}
```

138. `Solution00386.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.List;

/**
 * 386. Lexicographical Numbers
 * <a href="https://leetcode.com/problems/lexicographical-numbers">Web link</a>
 * Medium
 */

public class Solution00386 {

    public List<Integer> lexicalOrder(int n) {
        List<Integer> result = new ArrayList<>(n);
        int current = 1;
        while (result.size() < n) {
            result.add(current);
            if (current * 10 <= n) {
                current *= 10;
            } else {
                while (current == n || current % 10 == 9) {
                    current /= 10;
                }
                current++;
            }
        }
        return result;
    }

}
```

139. `Solution00387.java`

```
package leetcode;

import java.util.HashMap;
import java.util.Map;

/**
 * https://leetcode.com/problems/first-unique-character-in-a-string/
 * 387. First Unique Character in a String
 * Easy
 * <p>
 * Given a string s, find the first non-repeating character in it and return its index.
If it does not exist, return -1.
 * <p>
 * Example 1:
 * <p>
 * Input: s = "leetcode"
 * Output: 0
 * Example 2:
 * <p>
 * Input: s = "loveleetcode"
 * Output: 2
 * Example 3:
 * <p>
 * Input: s = "aabb"
 * Output: -1
 * <p>
 * Constraints:
 * <p>
 * 1 <= s.length <= 105
 * s consists of only lowercase English letters.
 */
public class Solution00387 {

    // Time complexity: O(n)
    // Space complexity: O(1)
    public int firstUniqCharArray(String s) {
        int[] count = new int[26];
        for (char c : s.toCharArray()) {
            count[c - 'a']++;
        }
        for (int i = 0; i < s.length(); i++) {
            if (count[s.charAt(i) - 'a'] == 1) {
                return i;
            }
        }
        return -1;
    }

    // Time complexity: O(n)
    // Space complexity: O(1)
    public int firstUniqMap(String s) {
```

```
        Map<Character, Integer> charCountMap = new HashMap<>();
        for (char c : s.toCharArray()) {
            charCountMap.merge(c, 1, Integer::sum);
        }
        for (int i = 0; i < s.length(); i++) {
            if (charCountMap.get(s.charAt(i)) == 1) {
                return i;
            }
        }
        return -1;
    }

}
```

140. `Solution00392.java`

```
package leetcode;

/**
 * 392. Is Subsequence
 * <a href="https://leetcode.com/problems/is-subsequence">Web link</a>
 * Easy
 */

public class Solution00392 {

    public boolean isSubsequence(String s, String t) {
        int positionT = 0;
        int positionS = 0;
        final int sLen = s.length();
        final int tLen = t.length();
        while (positionS < sLen && positionT < tLen) {
            if (s.charAt(positionS) == t.charAt(positionT)) {
                positionS++;
            }
            positionT++;
        }
        return positionS == sLen;
    }

}
```

141. `Solution00402.java`

```
package leetcode;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * https://leetcode.com/problems/remove-k-digits/
 * 402. Remove K Digits
 * Medium
 */

public class Solution00402 {

    public String removeKdigits(String num, int k) {
        int n = num.length();
        if (n == k) {
            return "0";
        }
        StringBuilder sb = new StringBuilder();
        for (char digit : num.toCharArray()) {
            while (k > 0 &&
                   !sb.isEmpty() &&
                   sb.charAt(sb.length() - 1) > digit) {
                sb.deleteCharAt(sb.length() - 1);
                k--;
            }
            sb.append(digit);
        }
        while (k > 0) {
            sb.deleteCharAt(sb.length() - 1);
            k--;
        }
        while (!sb.isEmpty() && sb.charAt(0) == '0') {
            sb.deleteCharAt(0);
        }
        return sb.isEmpty() ? "0" : sb.toString();
    }

    public String removeKdigitsStack(String num, int k) {
        if (k >= num.length()) {
            return "0";
        }
        int n = num.length();
        Deque<Character> stack = new ArrayDeque<>();
        for (int i = 0; i < n; i++) {
            char ch = num.charAt(i);
            while (k > 0 && !stack.isEmpty() && stack.peek() > ch) {
                stack.pop();
                k--;
            }
            stack.push(ch);
```

```
        }
        while (!stack.isEmpty() && k > 0) {
            stack.pop();
            k--;
        }
        StringBuilder str = new StringBuilder();
        while (!stack.isEmpty()) {
            str.append(stack.pop());
        }
        str.reverse();
        while (str.length() > 1 && str.charAt(0) == '0')
            str.deleteCharAt(0);

        return str.toString();
    }

}
```

142. `Solution00404.java`

```
package leetcode;

import leetcode.common.TreeNode;

/**
 * 404. Sum of Left Leaves
 * https://leetcode.com/problems/sum-of-left-leaves/
 * Easy
 */

public class Solution00404 {

    public int sumOfLeftLeaves(TreeNode root) {
        return sumOfLeftLeaves(root, false);
    }

    private int sumOfLeftLeaves(TreeNode root, boolean isLeft) {
        if (root == null) {
            return 0;
        }
        if (root.left == null && root.right == null) {
            if (isLeft) {
                return root.val;
            } else {
                return 0;
            }
        }
        return sumOfLeftLeaves(root.left, true) + sumOfLeftLeaves(root.right, false);
    }

}
```

143. `Solution00407.java`

```
/**
 * 407. Trapping Rain Water II
 * <a href="https://leetcode.com/problems/trapping-rain-water-ii">Web link</a> |
 * Hard
 */

package leetcode;

import java.util.Comparator;
import java.util.PriorityQueue;

public class Solution00407 {

    public int trapRainWater(int[][] heightMap) {
        int[] dx = {-1, 0, 1, 0};
        int[] dy = {0, 1, 0, -1};

        int m = heightMap.length;
        int n = heightMap[0].length;

        PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a ->
a[0]));
        boolean[][] visited = new boolean[m][n];

        // Add border cells
        for (int j = 0; j < n; j++) {
            pq.offer(new int[]{heightMap[0][j], 0, j});
            pq.offer(new int[]{heightMap[m - 1][j], m - 1, j});
            visited[0][j] = visited[m - 1][j] = true;
        }

        for (int i = 1; i < m - 1; i++) {
            pq.offer(new int[]{heightMap[i][0], i, 0});
            pq.offer(new int[]{heightMap[i][n - 1], i, n - 1});
            visited[i][0] = visited[i][n - 1] = true;
        }

        int water = 0;
        while (!pq.isEmpty()) {
            int[] curr = pq.poll();
            int height = curr[0];
            int row = curr[1];
            int col = curr[2];

            for (int k = 0; k < 4; k++) {
                int newRow = row + dx[k];
                int newCol = col + dy[k];

                if (newRow < 0 || newRow >= m || newCol < 0 || newCol >= n ||
                        visited[newRow][newCol]) continue;
```

```
                if (heightMap[newRow][newCol] < height) {
                    water += height - heightMap[newRow][newCol];
                    pq.offer(new int[]{height, newRow, newCol});
                } else {
                    pq.offer(new int[]{heightMap[newRow][newCol], newRow, newCol});
                }
                visited[newRow][newCol] = true;
            }
        }
        return water;
    }

}
```

144. `Solution00409.java`

```
package leetcode;

import java.util.HashMap;
import java.util.Map;

/**
 * 409. Longest Palindrome
 * <a href="https://leetcode.com/problems/longest-palindrome">Web link</a> |
 * Easy
 */

public class Solution00409 {

    public int longestPalindrome(String s) {
        Map<Character, Integer> countMap = new HashMap<>();
        for (int i = 0; i < s.length(); i++) {
            final char currentChar = s.charAt(i);
            countMap.put(currentChar, countMap.getOrDefault(currentChar, 0) + 1);
        }
        int result = 0;
        boolean hasOdd = false;
        for (Integer coun : countMap.values()) {
            result += coun;
            if (coun % 2 == 1) {
                result--;
                hasOdd = true;
            }
        }
        if (hasOdd) {
            result++;
        }
        return result;
    }

}
```

145. `Solution00417.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/** https://leetcode.com/problems/pacific-atlantic-water-flow/
 * 417. Pacific Atlantic Water Flow
 * <p>
 * You are given an m x n integer matrix heights representing the height of each unit
cell in a continent.
 * The Pacific ocean touches the continent's left and top edges, and the Atlantic ocean
touches the continent's
 * right and bottom edges.
 * <p>
 * Water can only flow in four directions: up, down, left, and right. Water flows from a
cell to an adjacent one
 * with an equal or lower height.
 * <p>
 * Return a list of grid coordinates where water can flow to both the Pacific and
Atlantic oceans.
 * <p>
 * Example 1:
 * <p>
 * Input: heights = [[1,2,2,3,5],
 * [3,2,3,4,4],
 * [2,4,5,3,1],
 * [6,7,1,4,5],
 * [5,1,1,2,4]]
 * Output: [[0,4],[1,3],[1,4],[2,2],[3,0],[3,1],[4,0]]
 * <p>
 * Example 2:
 * <p>
 * Input: heights = [[2,1],[1,2]]
 * Output: [[0,0],[0,1],[1,0],[1,1]]
 * <p>
 * Constraints:
 * <p>
 * m == heights.length
 * n == heights[i].length
 * 1 <= m, n <= 200
 * 1 <= heights[i][j] <= 105
 */
public class Solution00417 {

    // Time: O(n * m)
    // Space: O(n * m)
    public List<List<Integer>> pacificAtlantic(int[][] heights) {
        // Create a list to store the result.
        List<List<Integer>> result = new ArrayList<>();
```

```
        // Check for invalid inputs.
        if (heights == null || heights.length == 0 || heights[0].length == 0) {
            return result;
        }

        // Get the number of rows and columns.
        int rows = heights.length;
        int cols = heights[0].length;

        // Create a boolean matrix to store the nodes that can reach the Pacific.
        boolean[][] canReachPacific = new boolean[rows][cols];

        // Create a boolean matrix to store the nodes that can reach the Atlantic.
        boolean[][] canReachAtlantic = new boolean[rows][cols];

        // Create a queue to store the nodes that can reach the Pacific.
        Queue<int[]> pacificQueue = new LinkedList<>();

        // Create a queue to store the nodes that can reach the Atlantic.
        Queue<int[]> atlanticQueue = new LinkedList<>();

        // Iterate over the rows and add the nodes that can reach the Pacific to the
queue.
        for (int row = 0; row < rows; row++) {
            pacificQueue.offer(new int[]{row, 0});
        }

        // Iterate over the columns and add the nodes that can reach the Pacific to the
queue.
        for (int col = 0; col < cols; col++) {
            pacificQueue.offer(new int[]{0, col});
        }

        // Iterate over the rows and add the nodes that can reach the Atlantic to the
queue.
        for (int row = 0; row < rows; row++) {
            atlanticQueue.offer(new int[]{row, cols - 1});
        }

        // Iterate over the columns and add the nodes
        // that can reach the Atlantic to the queue.
        for (int col = 0; col < cols; col++) {
            atlanticQueue.offer(new int[]{rows - 1, col});
        }

        // BFS to find the nodes that can reach the Pacific.
        bfs(heights, pacificQueue, canReachPacific);

        // BFS to find the nodes that can reach the Atlantic.
        bfs(heights, atlanticQueue, canReachAtlantic);

        // Iterate over the rows.
        for (int row = 0; row < rows; row++) {
```

```
            // Iterate over the columns.
            for (int col = 0; col < cols; col++) {
                // If the node can reach both the Pacific and the Atlantic,
                // add it to the result.
                if (canReachPacific[row][col] && canReachAtlantic[row][col]) {
                    result.add(List.of(row, col));
                }
            }
        }

        return result;
    }

    // BFS to find the nodes that can reach the ocean.
    private void bfs(int[][] heights, Queue<int[]> queue, boolean[][] canReachOcean) {
        // Get the number of rows and columns.
        int rows = heights.length;
        int cols = heights[0].length;

        // Create a boolean matrix to store the visited nodes.
        boolean[][] visited = new boolean[rows][cols];

        // Iterate while the queue is not empty.
        while (!queue.isEmpty()) {
            // Get the current node.
            int[] curr = queue.poll();

            // Get the row and column of the current node.
            int row = curr[0];
            int col = curr[1];

            // Mark the current node as visited.
            visited[row][col] = true;

            // Add the current node to the queue if it can reach the ocean.
            canReachOcean[row][col] = true;

            // Create an array to store the directions.
            int[][] directions = {{0, 1}, {0, -1}, {1, 0}, {-1, 0}};

            // Iterate over the directions.
            for (int[] direction : directions) {
                // Get the next row and column.
                int nextRow = row + direction[0];
                int nextCol = col + direction[1];

                // Check if the next row and column are valid.
                if (nextRow < 0 || nextRow >= rows || nextCol < 0 || nextCol >= cols) {
                    continue;
                }

                // Check if the next node is visited.
                if (visited[nextRow][nextCol]) {
                    continue;
```

```
                }

                // Check if the next node is higher than the current node.
                if (heights[nextRow][nextCol] < heights[row][col]) {
                    continue;
                }

                // Add the next node to the queue.
                queue.offer(new int[]{nextRow, nextCol});
            }
        }
    }

}
```

146. `Solution00424.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/longest-repeating-character-replacement/
 * 424. Longest Repeating Character Replacement
 * <p>
 * You are given a string s and an integer k. You can choose any character of the string
and change it to any other uppercase English character.
 * You can perform this operation at most k times.
 * <p>
 * Return the length of the longest substring containing the same letter you can get
after performing the above operations.
 * <p>
 * Example 1:
 * <p>
 * Input: s = "ABAB", k = 2
 * Output: 4
 * Explanation: Replace the two 'A's with two 'B's or vice versa.
 * <p>
 * Example 2:
 * <p>
 * Input: s = "AABABBA", k = 1
 * Output: 4
 * Explanation: Replace the one 'A' in the middle with 'B' and form "AABBBBA".
 * The substring "BBBB" has the longest repeating letters, which is 4.
 * <p>
 * Constraints:
 * <p>
 * 1 <= s.length <= 105
 * s consists of only uppercase English letters.
 * 0 <= k <= s.length
 */
public class Solution00424 {

    // Time: O(n)
    // Space: O(1)
    public int characterReplacement(String s, int k) {
        int[] counts = new int[26]; // An array to keep track of character counts

        int left = 0; // The left pointer of the sliding window
        int right = 0; // The right pointer of the sliding window
        int maxCount = 0; // Maximum character count within the window

        while (right < s.length()) {
            char rightChar = s.charAt(right);
            counts[rightChar - 'A']++; // Increment the count for the current character
            maxCount = Math.max(maxCount, counts[rightChar - 'A']); // Update the
maxCount

            // Check if we need to shrink the window
            final int currentSlidingWindow = right - left;
```

```
            final int maxSlidingWindow = maxCount + k;
            if (currentSlidingWindow + 1 > maxSlidingWindow) {
                // Check if the window needs to be shrunk.
                // The condition checks if the number of characters that need replacement
is greater than 'k'.
                // If it's true, shrink the window.
                counts[s.charAt(left) - 'A']--;
                left++;
            }

            right++; // Move the right pointer to expand the window
        }

        // Return the length of the longest valid substring, ensuring it doesn't exceed
the length of the input string.
        return Math.min(s.length(), maxCount + k);
    }

}
```

147. `Solution00432.java`

```
package leetcode;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 432. All O`one Data Structure
 * <a href="https://leetcode.com/problems/all-oone-data-structure">Web link</a>
 * Hard
 */

public class Solution00432 {

    static class AllOne {

        private final Node head;
        private final Node tail;
        private final Map<String, Node> map;

        public AllOne() {
            head = new Node(0);
            tail = new Node(0);
            map = new HashMap<>();
            head.next = tail;
            tail.prev = head;
        }

        public void inc(String key) {
            if (map.containsKey(key)) {
                Node node = map.get(key);
                int freq = node.freq;
                node.keys.remove(key);

                Node nextNode = node.next;
                if (nextNode == tail || nextNode.freq != freq + 1) {
                    Node newNode = new Node(freq + 1);
                    newNode.keys.add(key);
                    newNode.prev = node;
                    newNode.next = nextNode;
                    node.next = newNode;
                    nextNode.prev = newNode;
                    map.put(key, newNode);
                } else {
                    nextNode.keys.add(key);
                    map.put(key, nextNode);
                }
                if (node.keys.isEmpty()) {
                    deleteNode(node);
                }
```

```
            } else {
                Node firstNode = head.next;
                if (firstNode == tail || firstNode.freq > 1) {
                    Node newNode = new Node(1);
                    newNode.keys.add(key);
                    newNode.prev = head;
                    newNode.next = firstNode;
                    head.next = newNode;
                    firstNode.prev = newNode;
                    map.put(key, newNode);
                } else {
                    firstNode.keys.add(key);
                    map.put(key, firstNode);
                }
            }
        }

        public void dec(String key) {
            if (!map.containsKey(key)) {
                return;
            }
            Node node = map.get(key);
            int freq = node.freq;
            node.keys.remove(key);

            if (freq > 1) {
                Node prevNode = node.prev;
                if (prevNode == head || prevNode.freq != freq - 1) {
                    Node newNode = new Node(freq - 1);
                    newNode.keys.add(key);
                    newNode.prev = prevNode;
                    newNode.next = node;
                    node.prev = newNode;
                    prevNode.next = newNode;
                    map.put(key, newNode);
                } else {
                    prevNode.keys.add(key);
                    map.put(key, prevNode);
                }
            } else {
                map.remove(key);
            }
            if (node.keys.isEmpty()) {
                deleteNode(node);
            }
        }

        public String getMaxKey() {
            if (tail.prev == head) {
                return "";
            }
            return tail.prev.keys.iterator().next();
        }
```

```
        public String getMinKey() {
            if (head.next == tail) {
                return "";
            }
            return head.next.keys.iterator().next();
        }

        private void deleteNode(Node node) {
            Node prevNode = node.prev;
            Node nextNode = node.next;
            prevNode.next = nextNode;
            nextNode.prev = prevNode;
        }

    }

    private static class Node {
        private final int freq;
        private Node prev;
        private Node next;
        private final Set<String> keys;

        public Node(int freq) {
            this.freq = freq;
            this.keys = new HashSet<>();
        }
    }

}
```

148. `Solution00435.java`

```
package leetcode;

import java.util.Arrays;
import java.util.Comparator;

/**
 * 435. Non-overlapping Intervals
 * https://leetcode.com/problems/non-overlapping-intervals/
 * Medium
 * <p>
 * Given an array of intervals intervals where intervals[i] = [starti, endi], return the
minimum number of intervals
 * you need to remove to make the rest of the intervals non-overlapping.
 * <p>
 * Example 1:
 * <p>
 * Input: intervals = [[1,2],[2,3],[3,4],[1,3]] Output: 1 Explanation: [1,3] can be
removed and the rest of the
 * intervals are non-overlapping. Example 2:
 * <p>
 * Input: intervals = [[1,2],[1,2],[1,2]] Output: 2 Explanation: You need to remove two
[1,2] to make the rest of the
 * intervals non-overlapping. Example 3:
 * <p>
 * Input: intervals = [[1,2],[2,3]] Output: 0 Explanation: You don't need to remove any
of the intervals since they're
 * already non-overlapping.
 * <p>
 * Constraints:
 * <p>
 * 1 <= intervals.length <= 2 * 104 intervals[i].length == 2 -2 * 104 <= starti < endi <=
2 * 104
 */
public class Solution00435 {

    // Time: O(n * log(n))
    // Space: O(1)
    public int eraseOverlapIntervalsSortByEnd(int[][] intervals) {
        // If the array is null or empty, return 0.
        if (intervals.length <= 1) {
            return 0;
        }

        // Sort the array by the end time of each interval.
        Arrays.sort(intervals, Comparator.comparingInt(interval -> interval[1]));

        // Initialize the number of intervals to remove to 0.
        int intervalsToRemove = 0;

        // Initialize the end time of the last interval to the end time of the first
interval.
```

```
        int lastIntervalEnd = intervals[0][1];

        // Iterate over the array of intervals.
        for (int i = 1; i < intervals.length; i++) {
            // Get the current interval.
            int[] currentInterval = intervals[i];

            // If the start time of the current interval is less than or equal to the end
time of the last interval,
            // increment the number of intervals to remove.
            if (currentInterval[0] < lastIntervalEnd) {
                intervalsToRemove++;
            } else {
                // Otherwise, update the end time of the last interval.
                lastIntervalEnd = currentInterval[1];
            }
        }

        // Return the number of intervals to remove.
        return intervalsToRemove;
    }

    // Time: O(n * log(n))
    // Space: O(1)
    public int eraseOverlapIntervalsSortByStart(int[][] intervals) {
        if (intervals.length <= 1) {
            return 0;
        }
        Arrays.sort(intervals, Comparator.comparingInt(interval -> interval[0]));

        int lastIntervalEnd = intervals[0][1];
        int intervalsToRemove = 0;
        for (int i = 1; i < intervals.length; i++) {
            int[] currentInterval = intervals[i];
            if (currentInterval[0] >= lastIntervalEnd) {
                // non overlaping, update last interval
                lastIntervalEnd = currentInterval[1];
            } else {
                // as last - should be used the one with smaller end, so we have less
overlaps
                // the one with bigger end - should be removed and not used as last
                if (currentInterval[1] < lastIntervalEnd) {
                    lastIntervalEnd = currentInterval[1];
                }
                intervalsToRemove++;
            }
        }

        return intervalsToRemove;
    }

}
```

149. `Solution00440.java`

```
package leetcode;

/**
 * 440. K-th Smallest in Lexicographical Order
 * <a href="https://leetcode.com/problems/k-th-smallest-in-lexicographical-order">Web
link</a>
 * Hard
 */

public class Solution00440 {

    public int findKthNumber(int n, int k) {
        long current = 1;
        k--;
        while (k > 0) {
            int steps = calculateSteps(n, current);
            if (steps <= k) {
                current++;
                k -= steps;
            } else {
                current *= 10;
                k--;
            }
        }
        return (int) current;
    }

    private int calculateSteps(int n, long current) {
        long next = current + 1;
        long steps = 0;
        while (current <= n) {
            steps += Math.min(n + 1, next) - current;
            current *= 10;
            next *= 10;
        }
        return (int) steps;
    }

}
```

150. `Solution00442.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.List;

/**
 * 442. Find All Duplicates in an Array
 * https://leetcode.com/problems/find-all-duplicates-in-an-array/
 * Medium
 */

public class Solution00442 {

    public List<Integer> findDuplicates(int[] nums) {
        List<Integer> result = new ArrayList<>();
        for (int num : nums) {
            int index = Math.abs(num) - 1;
            if (nums[index] < 0) {
                result.add(index + 1);
            }
            nums[index] = -nums[index];
        }
        return result;
    }

}
```

151. `Solution00446.java`

```
package leetcode;

import java.util.HashMap;
import java.util.Map;

/**
 * https://leetcode.com/problems/arithmetic-slices-ii-subsequence/
 * 446. Arithmetic Slices II - Subsequence
 * Hard
 * Given an integer array nums, return the number of all the arithmetic subsequences of
nums.
 * A sequence of numbers is called arithmetic if it consists of at least three elements
and if the difference between any two consecutive elements is the same.
 * For example, [1, 3, 5, 7, 9], [7, 7, 7, 7], and [3, -1, -5, -9] are arithmetic
sequences.
 * For example, [1, 1, 2, 5, 7] is not an arithmetic sequence.
 * A subsequence of an array is a sequence that can be formed by removing some elements
(possibly none) of the array.
 * For example, [2,5,10] is a subsequence of [1,2,1,2,4,1,5,10].
 * The answer is guaranteed to fit in 32-bit integer.
 * Example 1:
 * Input: nums = [2,4,6,8,10]
 * Output: 7
 * Explanation: All arithmetic subsequence slices are:
 * [2,4,6]
 * [4,6,8]
 * [6,8,10]
 * [2,4,6,8]
 * [4,6,8,10]
 * [2,4,6,8,10]
 * [2,6,10]
 * Example 2:
 * Input: nums = [7,7,7,7,7]
 * Output: 16
 * Explanation: Any subsequence of this array is arithmetic.
 * Constraints:
 * 1  <= nums.length <= 1000
 * -231 <= nums[i] <= 231 - 1
 */
public class Solution00446 {

    // Time: O(n^2)
    // Space: O(n^2)
    public int numberOfArithmeticSlices(int[] nums) {
        int result = 0;
        int n = nums.length;
        Map<Integer, Integer>[] dp = new Map[n];

        for (int i = 0; i < n; i++) {
            dp[i] = new HashMap<>();
```

```
            for (int j = 0; j < i; j++) {
                long diff = (long) nums[i] - nums[j];
                if (diff <= Integer.MIN_VALUE || diff > Integer.MAX_VALUE) {
                    continue;
                }

                int d = (int) diff;
                int c1 = dp[i].getOrDefault(d, 0);
                int c2 = dp[j].getOrDefault(d, 0);
                result += c2;
                dp[i].put(d, c1 + c2 + 1);
            }
        }

        return result;
    }

}
```

152. `Solution00451.java`

```
package leetcode;

import java.util.HashMap;
import java.util.Map;

/**
 * https://leetcode.com/problems/sort-characters-by-frequency/
 * 451. Sort Characters By Frequency
 * Medium
 * <p>
 * Given a string s, sort it in decreasing order based on the frequency of the
characters. The frequency of a character is the number of times it appears in the string.
 * <p>
 * Return the sorted string. If there are multiple answers, return any of them.
 * <p>
 * Example 1:
 * <p>
 * Input: s = "tree"
 * Output: "eert"
 * Explanation: 'e' appears twice while 'r' and 't' both appear once.
 * So 'e' must appear before both 'r' and 't'. Therefore "eetr" is also a valid answer.
 * Example 2:
 * <p>
 * Input: s = "cccaaa"
 * Output: "aaaccc"
 * Explanation: Both 'c' and 'a' appear three times, so both "cccaaa" and "aaaccc" are
valid answers.
 * Note that "cacaca" is incorrect, as the same characters must be together.
 * Example 3:
 * <p>
 * Input: s = "Aabb"
 * Output: "bbAa"
 * Explanation: "bbaA" is also a valid answer, but "Aabb" is incorrect.
 * Note that 'A' and 'a' are treated as two different characters.
 * <p>
 * Constraints:
 * <p>
 * 1 <= s.length <= 5 * 105
 * s consists of uppercase and lowercase English letters and digits.
 */

public class Solution00451 {

    // Time complexity: O(n)
    // Space complexity: O(1)
    public String frequencySortArray(String s) {
        int[] count = new int[128];
        for (char c : s.toCharArray()) {
            count[c]++;
        }
        StringBuilder sb = new StringBuilder();
```

```
        while (sb.length() < s.length()) {
            int max = 0;
            for (int i = 0; i < count.length; i++) {
                if (count[i] > count[max]) {
                    max = i;
                }
            }
            if (count[max] > 0) {
                sb.repeat(max, count[max]);
                count[max] = 0;
            }
        }
        return sb.toString();
    }

    // Time complexity: O(n)
    // Space complexity: O(1)
    public String frequencySortMap(String s) {
        Map<Character, Integer> count = new HashMap<>();
        for (char c : s.toCharArray()) {
            count.merge(c, 1, Integer::sum);
        }

        StringBuilder sb = new StringBuilder();
        count.entrySet()
                .stream()
                .sorted(Map.Entry.<Character, Integer>comparingByValue().reversed())
                .forEachOrdered(entry -> sb.repeat(entry.getKey(), entry.getValue()));

        return sb.toString();
    }

}
```

153. `Solution00452.java`

```
package leetcode;

import java.util.Arrays;
import java.util.Comparator;

/**
 * 452. Minimum Number of Arrows to Burst Balloons
 * https://leetcode.com/problems/minimum-number-of-arrows-to-burst-balloons/
 * Medium
 */

public class Solution00452 {

    public int findMinArrowShots(int[][] points) {
        if (points.length == 0) {
            return 0;
        }

        Arrays.sort(points, Comparator.comparingInt(a -> a[1]));

        int arrows = 1;
        int end = points[0][1];

        for (int i = 1; i < points.length; i++) {
            if (points[i][0] > end) {
                arrows++;
                end = points[i][1];
            }
        }

        return arrows;
    }
}
```

154. `Solution00455.java`

```
package leetcode;

import java.util.Arrays;

/**
 * <a href="https://leetcode.com/problems/assign-cookies/">
 * 455. Assign Cookies
 * </a>
 * <br/>
 * <div class="content__u3I1 question-content__JfgR">
 * <div><p>Assume you are an awesome parent and want to give your children some cookies.
But, you should give each child at most one cookie.</p>
 *
 * <p>Each child <code>i</code> has a greed factor <code>g[i]</code>, which is the
minimum size of a cookie that the child will be content with; and each cookie
<code>j</code> has a size <code>s[j]</code>. If <code>s[j] &gt;= g[i]</code>, we can
assign the cookie <code>j</code> to the child <code>i</code>, and the child
<code>i</code> will be content. Your goal is to maximize the number of your content
children and output the maximum number.</p>
 *
 * <p> </p>
 * <p><strong>Example 1:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> g = [1,2,3], s = [1,1]
 * <strong>Output:</strong> 1
 * <strong>Explanation:</strong> You have 3 children and 2 cookies. The greed factors of
3 children are 1, 2, 3.
 * And even though you have 2 cookies, since their size is both 1, you could only make
the child whose greed factor is 1 content.
 * You need to output 1.
 * </pre>
 *
 * <p><strong>Example 2:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> g = [1,2], s = [1,2,3]
 * <strong>Output:</strong> 2
 * <strong>Explanation:</strong> You have 2 children and 3 cookies. The greed factors of
2 children are 1, 2.
 * You have 3 cookies and their sizes are big enough to gratify all of the children,
 * You need to output 2.
 * </pre>
 *
 * <p> </p>
 * <p><strong>Constraints:</strong></p>
 * <p>
 * 1 <= g.length <= 3 * 104
 * 0 <= s.length <= 3 * 104
 * 1 <= g[i], s[j] <= 231 - 1
 */
```

```
public class Solution00455 {

    // O(n·logn+m·logm)
    // Space: O(logn+logm)
    public int findContentChildren(int[] g, int[] s) {
        int result = 0;
        int childIndex = 0;
        int cookieIndex = 0;

        if (s.length == 0) {
            return 0;
        }

        Arrays.sort(g);
        Arrays.sort(s);

        while (childIndex < g.length && cookieIndex < s.length) {
            int greed = g[childIndex];
            int cookieSize = s[cookieIndex];
            if (greed <= cookieSize) {
                result++;
                childIndex++;
                cookieIndex++;
            } else {
                cookieIndex++;
            }
        }
        return result;
    }
}
```

155. `Solution00463.java`

```
package leetcode;

/**
 * 463. Island Perimeter
 * <a href="https://leetcode.com/problems/island-perimeter">Web link</a>
 * Easy
 */

public class Solution00463 {

    public int islandPerimeter(int[][] grid) {
        int perimeter = 0;
        int rows = grid.length;
        int cols = grid[0].length;
        for (int row = 0; row < rows; row++) {
            for (int col = 0; col < cols; col++) {
                perimeter += perimeter(grid, row, col);
            }
        }
        return perimeter;
    }

    private int perimeter(int[][] grid, int row, int col) {
        if (grid[row][col] == 0) {
            return 0;
        }
        int perimeter = 0;
        // top border
        if (row == 0 || grid[row - 1][col] == 0) {
            perimeter++;
        }
        // bottom border
        if (row == grid.length - 1 || grid[row + 1][col] == 0) {
            perimeter++;
        }
        // left border
        if (col == 0 || grid[row][col - 1] == 0) {
            perimeter++;
        }
        // right border
        if (col == grid[0].length - 1 || grid[row][col + 1] == 0) {
            perimeter++;
        }
        return perimeter;
    }

}
```

156. `Solution00476.java`

```
package leetcode;

/**
 * 476. Number Complement
 * <a href="https://leetcode.com/problems/number-complement">Web link</a>
 * Easy
 */

public class Solution00476 {

    public int findComplement(int num) {
        int mask = 1;
        while (mask < num) {
            mask = (mask << 1) + 1;
        }
        return num ^ mask;
    }

    public int findComplementBinaryStr(int num) {
        if (num == 0) return 1;
        int bitLength = Integer.toBinaryString(num).length();
        int mask = (1 << bitLength) - 1;
        return num ^ mask;
    }

    public int findComplementBitByBit(int num) {
        int complement = 0;
        int power = 0;
        while (num > 0) {
            int rightMostBit = num & 1;
            if (rightMostBit == 0) { // we should place 1 in this position, as complement
to 0
                complement += 1 << power;
            } // else we already have 0 at position as complement to 1, because 0 is
default bit in integer
            num >>= 1;
            power++;
        }
        return complement;
    }

}
```

157. `Solution00494.java`

```
/**
 * 494. Target Sum
 * <a href="https://leetcode.com/problems/target-sum">Web link</a> |
 * Medium
 */

package leetcode;

public class Solution00494 {

    public int findTargetSumWays(int[] nums, int target) {
        return findSumRecursively(nums, 0, target);
    }

    private int findSumRecursively(int[] nums, int currentIndex, int target) {
        if (currentIndex == nums.length) {
            return target == 0 ? 1 : 0;
        }
        int sum1 = findSumRecursively(nums, currentIndex + 1, target -
nums[currentIndex]);
        int sum2 = findSumRecursively(nums, currentIndex + 1, target +
nums[currentIndex]);
        return sum1 + sum2;
    }

    public int findTargetSumWaysDp(int[] nums, int target) {
        int totalSum = 0;
        for (int num : nums) {
            totalSum += num;
        }

        if (Math.abs(target) > totalSum) {
            return 0;
        }

        int[] dp = new int[2 * totalSum + 1];
        dp[nums[0] + totalSum] = 1;
        dp[-nums[0] + totalSum] += 1;

        for (int index = 1; index < nums.length; index++) {
            int[] next = new int[2 * totalSum + 1];
            for (int sum = -totalSum; sum <= totalSum; sum++) {
                if (dp[sum + totalSum] > 0) {
                    next[sum + nums[index] + totalSum] += dp[sum + totalSum];
                    next[sum - nums[index] + totalSum] += dp[sum + totalSum];
                }
            }
            dp = next;
        }

        return dp[target + totalSum];
```

```
    }

}
```

158. `Solution00502.java`

```
package leetcode;

import java.util.Arrays;
import java.util.Comparator;
import java.util.PriorityQueue;

/**
 * 502. IPO
 * <a href="https://leetcode.com/problems/ipo">Web link</a>
 * Hard
 */

public class Solution00502 {

    public int findMaximizedCapitalArrayPlusHeap(int k, int w, int[] profits, int[]
capital) {
        int n = profits.length;
        int currentCapital = w;
        int[][] projects = new int[n][2];
        for (int i = 0; i < n; i++) {
            projects[i][0] = capital[i];
            projects[i][1] = profits[i];
        }
        Arrays.sort(projects, Comparator.comparingInt(a -> a[0]));
        int positionIncluded = 0;
        PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());
        for (int i = 0; i < k; i++) {
            while (positionIncluded < n && projects[positionIncluded][0] <=
currentCapital) {
                maxHeap.add(projects[positionIncluded][1]);
                positionIncluded++;
            }
            if (maxHeap.isEmpty()) {
                break;
            }
            currentCapital += maxHeap.poll();
        }
        return currentCapital;
    }

    public int findMaximizedCapital2Heaps(int k, int w, int[] profits, int[] capital) {
        int n = profits.length;
        int currentCapital = w;
        PriorityQueue<int[]> minHeap = new PriorityQueue<>(Comparator.comparingInt(a ->
a[0]));
        PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());
        for (int i = 0; i < n; i++) {
            if (capital[i] > currentCapital) {
                minHeap.add(new int[]{capital[i], profits[i]});
            } else {
                maxHeap.add(profits[i]);
```

```
            }
        }
        for (int i = 0; i < k; i++) {
            while (!minHeap.isEmpty() && minHeap.peek()[0] <= currentCapital) {
                maxHeap.add(minHeap.poll()[1]);
            }
            if (maxHeap.isEmpty()) {
                break;
            }
            currentCapital += maxHeap.poll();
        }
        return currentCapital;
    }

}
```

159. `Solution00506.java`

```
package leetcode;

import java.util.*;

/**
 * 506. Relative Ranks
 * https://leetcode.com/problems/relative-ranks/
 * Easy
 */

public class Solution00506 {

    public String[] findRelativeRanks(int[] score) {
        final int n = score.length;
        int[] copySorted = Arrays.stream(score).sorted().toArray();
        Map<Integer, Integer> scoreValueToPosition = new HashMap<>();
        for (int i = 0; i < n; i++) {
            final int position = n - i; // if n=5, i0=5-0 = 5th, i4=5-4 = 1st(gold)
            scoreValueToPosition.put(copySorted[i], position);
        }

        String[] result = new String[n];
        for (int i = 0; i < n; i++) {
            result[i] = placeToRank(scoreValueToPosition.get(score[i]));
        }
        return result;
    }

    private String placeToRank(int place) {
        return switch (place) {
            case 1 -> "Gold Medal";
            case 2 -> "Silver Medal";
            case 3 -> "Bronze Medal";
            default -> String.valueOf(place);
        };
    }

    public String[] findRelativeRanksPriorityQueue(int[] score) {
        PriorityQueue<Integer> pq = new PriorityQueue<>(Comparator.reverseOrder());
        int n = score.length;
        for (int individualScore : score) {
            pq.offer(individualScore);
        }
        String[] ranksByScore = new String[pq.peek() + 1];
        String[] ranksInOriginalOrder = new String[n];
        for (int i = 0; i < n; i++) {
            ranksByScore[pq.poll()] = placeToRank(i + 1);
        }
        for (int i = 0; i < n; i++) {
            ranksInOriginalOrder[i] = ranksByScore[score[i]];
        }
```

```
        return ranksInOriginalOrder;
    }

}
```

160. `Solution00509.java`

```
package leetcode;

/**
 * 509. Fibonacci Number
 * <a href="https://leetcode.com/problems/fibonacci-number/">Web link</a> |
 * Easy
 */

public class Solution00509 {

    public int fib(int n) {
        if (n == 0) {
            return 0;
        }
        int[] dp = new int[n + 1];
        dp[0] = 0;
        dp[1] = 1;
        for (int i = 2; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }
        return dp[n];
    }

}
```

161. `Solution00513.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.ArrayDeque;
import java.util.Queue;

/**
 * https://leetcode.com/problems/find-bottom-left-tree-value/
 * 513. Find Bottom Left Tree Value
 * Medium
 */

public class Solution00513 {

    /**
     * Definition for a binary tree node.
     * public class TreeNode {
     * int val;
     * TreeNode left;
     * TreeNode right;
     * TreeNode() {}
     * TreeNode(int val) { this.val = val; }
     * TreeNode(int val, TreeNode left, TreeNode right) {
     * this.val = val;
     * this.left = left;
     * this.right = right;
     * }
     * }
     */

    // Time: O(n)
    // Space: O(n)
    public int findBottomLeftValue(TreeNode root) {
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.add(root);
        int result = 0;
        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                if (node.right != null) {
                    queue.add(node.right);
                }
                if (node.left != null) {
                    queue.add(node.left);
                }
                result = node.val;
            }
        }
        return result;
```

```
    }

}
```

162. `Solution00514.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 514. Freedom Trail
 * <a href="https://leetcode.com/problems/freedom-trail">Web link</a>
 * Hard
 */

public class Solution00514 {

    public int findRotateSteps(String ring, String key) {
        int ringLen = ring.length();
        int keyLen = key.length();
        int[] dp = new int[ringLen];
        for (int i = keyLen - 1; i >= 0; i--) {
            int[] dpNew = new int[ringLen];
            Arrays.fill(dpNew, Integer.MAX_VALUE);
            for (int j = 0; j < ringLen; j++) {
                for (int k = 0; k < ringLen; k++) {
                    if (ring.charAt(k) == key.charAt(i)) {
                        int diff = Math.abs(j - k);
                        int steps = Math.min(diff, ringLen - diff);
                        dpNew[j] = Math.min(dpNew[j], steps + dp[k]);
                    }
                }
            }
            dp = dpNew;
        }
        return dp[0] + keyLen;
    }

}
```

163. `Solution00515.java`

```
/**
 * 515. Find Largest Value in Each Tree Row
 * <a href="https://leetcode.com/problems/find-largest-value-in-each-tree-row">Web
link</a>
 * Medium
 */

package leetcode;

import leetcode.common.TreeNode;

import java.util.ArrayList;
import java.util.List;

public class Solution00515 {

    public List<Integer> largestValues(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        if (root == null) {
            return result;
        }
        levelTraversal(List.of(root), result);
        return result;
    }

    private void levelTraversal(List<TreeNode> levelNodes, List<Integer> result) {
        if (levelNodes.isEmpty()) {
            return;
        }
        int max = Integer.MIN_VALUE;
        List<TreeNode> nextLevelNodes = new ArrayList<>();
        for (TreeNode node : levelNodes) {
            max = Math.max(max, node.val);
            if (node.left != null) {
                nextLevelNodes.add(node.left);
            }
            if (node.right != null) {
                nextLevelNodes.add(node.right);
            }
        }
        result.add(max);
        levelTraversal(nextLevelNodes, result);
    }

}
```

164. `Solution00523.java`

```
package leetcode;

import java.util.HashMap;
import java.util.Map;

/**
 * 523. Continuous Subarray Sum
 * <a href="https://leetcode.com/problems/continuous-subarray-sum">Web link</a> |
 * Medium
 */

public class Solution00523 {

    public boolean checkSubarraySumBruteforce(int[] nums, int k) {
        final int len = nums.length;
        if (len < 2) {
            return false;
        }
        int[] prefixSums = new int[len];
        prefixSums[0] = nums[0];
        for (int i = 1; i < len; i++) {
            prefixSums[i] = prefixSums[i - 1] + nums[i];
            if (prefixSums[i] == 0) {
                return true;
            }
            if (prefixSums[i] % k == 0) {
                return true;
            }
        }
        int start = 1;
        while (start < len) {
            int valueToSubtract = prefixSums[start - 1];
            prefixSums[start] -= valueToSubtract;
            for (int i = start + 1; i < len; i++) {
                prefixSums[i] -= valueToSubtract;
                if (prefixSums[i] == 0) {
                    return true;
                }
                if (prefixSums[i] % k == 0) {
                    return true;
                }
            }
            start++;
        }
        return false;
    }

    public boolean checkSubarraySum(int[] nums, int k) {
        final int len = nums.length;
        if (len < 2) {
            return false;
```

```
        }

        Map<Integer, Integer> reminderMap = new HashMap<>();
        reminderMap.put(0, -1);
        int sum = 0;

        for (int i = 0; i < len; i++) {
            sum += nums[i];
            int reminder = sum % k;

            final Integer existingIndex = reminderMap.get(reminder);
            if (existingIndex == null) {
                reminderMap.put(reminder, i);
            } else {
                if (i - existingIndex > 1) {
                    return true;
                }
            }
        }

        return false;
    }


}
```

165. `Solution00525.java`

```
package leetcode;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

/**
 * 525. Contiguous Array
 * <a href="https://leetcode.com/problems/contiguous-array">Web link</a> |
 * Medium
 */
public class Solution00525 {

    // Time: O(n)
    // Space: O(n)
    public int findMaxLength(int[] nums) {
        int maxLength = 0;

        int zeroCount = 0;
        int oneCount = 0;

        int n = nums.length;

        // Initialize an array to store the first index where the difference between the
count of ones and zeros is a certain value
        int[] firstIndex = new int[2 * n + 1];

        // Fill the array with -1 to indicate that the difference has not been
encountered yet
        Arrays.fill(firstIndex, -1);

        // The difference is zero at the start of the array
        firstIndex[n] = 0;

        for (int i = 0; i < n; i++) {
            if (nums[i] == 0) {
                zeroCount++;
            } else {
                oneCount++;
            }

            int diff = oneCount - zeroCount;

            // If the difference has not been encountered before, store the current index
as the first index where this difference is encountered
            if (firstIndex[diff + n] == -1) {
                firstIndex[diff + n] = i + 1;
            } else {
                // If the difference has been encountered before, calculate the length of
the contiguous array from the first occurrence of this difference to the current index
                final int currentLength = i - firstIndex[diff + n] + 1;
```

```
                // Update the maximum length if the current length is greater
                maxLength = Math.max(maxLength, currentLength);
            }
        }

        // Return the maximum length of the contiguous array with an equal number of 0
and 1
        return maxLength;
    }

    public int findMaxLengthMap(int[] nums) {
        Map<Integer, Integer> hashmap = new HashMap<>();
        int zeros = 0;
        int ones = 0;
        int maxLen = 0;

        hashmap.put(0, -1);

        for (int i = 0; i < nums.length; i++) {
            if (nums[i] == 0) {
                zeros++;
            } else {
                ones++;
            }
            int diff = zeros - ones;

            if (hashmap.containsKey(diff)) {
                maxLen = Math.max(maxLen, i - hashmap.get(diff));
            } else {
                hashmap.put(diff, i);
            }
        }

        return maxLen;
    }

}
```

166. `Solution00538.java`

```
package leetcode;

import leetcode.common.TreeNode;

/**
 * 538. Convert BST to Greater Tree
 * <a href="https://leetcode.com/problems/convert-bst-to-greater-tree">Web link</a>
 * Medium
 */

public class Solution00538 {

    public TreeNode convertBST(TreeNode root) {
        convertBST(root, 0);
        return root;
    }

    private int convertBST(TreeNode root, int sum) {
        if (root == null) {
            return sum;
        }
        int rightSum = convertBST(root.right, sum);
        root.val += rightSum;
        return convertBST(root.left, root.val);
    }

}
```

167. `Solution00539.java`

```
package leetcode;

import java.util.Arrays;
import java.util.List;

/**
 * 539. Minimum Time Difference
 * <a href="https://leetcode.com/problems/minimum-time-difference">Web link</a>
 * Medium
 */

public class Solution00539 {

    public int findMinDifference(List<String> timePoints) {
        int n = timePoints.size();
        int[] minutes = new int[n];
        int i = 0;
        for (String time : timePoints) {
            String[] parts = time.split(":");
            minutes[i++] = Integer.parseInt(parts[0]) * 60 + Integer.parseInt(parts[1]);
        }
        Arrays.sort(minutes);
        int minDiff = minDistance(minutes[0], minutes[n - 1]);
        for (i = 1; i < n; i++) {
            if (minutes[i] == minutes[i - 1]) {
                return 0;
            } else {
                minDiff = Math.min(minDiff, minDistance(minutes[i], minutes[i - 1]));
            }
        }
        return minDiff;
    }

    private int minDistance(int minute1, int minute2) {
        if (minute1 < minute2) {
            return minDistance(minute2, minute1);
        }
        return Math.min(minute1 - minute2, 1440 - (minute1 - minute2));
    }

}
```

168. `Solution00543.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.Deque;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.Map;

/**
 * https://leetcode.com/problems/diameter-of-binary-tree/
 * 543. Diameter of Binary Tree
 * <p>
 * Given the root of a binary tree, return the length of the diameter of the tree.
 * <p>
 * The diameter of a binary tree is the length of the longest path between any two nodes
in a tree.
 * This path may or may not pass through the root.
 * <p>
 * The length of a path between two nodes is represented by the number of edges between
them.
 * <p>
 * Example 1:
 * <p>
 * Input: root = [1,2,3,4,5]
 * Output: 3
 * Explanation: 3 is the length of the path [4,2,1,3] or [5,2,1,3].
 * <p>
 * Example 2:
 * <p>
 * Input: root = [1,2]
 * Output: 1
 * <p>
 * Constraints:
 * <p>
 * The number of nodes in the tree is in the range [1, 104].
 * -100 <= Node.val <= 100
 */
public class Solution00543 {

    // Time: O(n)
    // Space: O(n)
    public int diameterOfBinaryTreeRecursive(TreeNode root) {
        int[] max = new int[1];
        height(root, max);
        return max[0];
    }

    private int height(TreeNode node, int[] max) {
        if (node == null) {
            return 0;
```

```
        }
        int left = height(node.left, max);
        int right = height(node.right, max);
        max[0] = Math.max(max[0], left + right);
        return Math.max(left, right) + 1;
    }

    // Time: O(n)
    // Space: O(n)
    public int diameterOfBinaryTreeIterative(TreeNode root) {
        int max = 0;
        Deque<TreeNode> stack = new LinkedList<>();
        Map<TreeNode, Integer> map = new HashMap<>();
        stack.push(root);
        while (!stack.isEmpty()) {
            TreeNode node = stack.peek();
            if (node.left != null && !map.containsKey(node.left)) {
                stack.push(node.left);
            } else if (node.right != null && !map.containsKey(node.right)) {
                stack.push(node.right);
            } else {
                stack.pop();
                int left = map.getOrDefault(node.left, 0);
                int right = map.getOrDefault(node.right, 0);
                map.put(node, Math.max(left, right) + 1);
                max = Math.max(max, left + right);
            }
        }
        return max;
    }

}
```

169. `Solution00552.java`

```
package leetcode;

/**
 * 552. Student Attendance Record II
 * https://leetcode.com/problems/student-attendance-record-ii/
 * Hard
 */

public class Solution00552 {

    public int checkRecord(int n) {
        int modValue = 1000000007;

        if (n == 1) {
            return 3;
        }

        long[][] attendanceRecord = new long[2][3];
        attendanceRecord[0][0] = 1;
        attendanceRecord[0][1] = 1;
        attendanceRecord[0][2] = 0;
        attendanceRecord[1][0] = 1;
        attendanceRecord[1][1] = 0;
        attendanceRecord[1][2] = 0;

        for (int i = 1; i < n; i++) {
            long[][] nextRecord = new long[2][3];

            // choose P
            long sumOfFirstRow = (attendanceRecord[0][0] + attendanceRecord[0][1] +
attendanceRecord[0][2]) % modValue;
            long sumOfSecondRow = (attendanceRecord[1][0] + attendanceRecord[1][1] +
attendanceRecord[1][2]) % modValue;
            nextRecord[0][0] = (int) sumOfFirstRow;
            nextRecord[1][0] = (int) sumOfSecondRow;

            // choose L
            nextRecord[0][1] = attendanceRecord[0][0];
            nextRecord[1][1] = attendanceRecord[1][0];
            nextRecord[0][2] = attendanceRecord[0][1];
            nextRecord[1][2] = attendanceRecord[1][1];

            // choose A
            nextRecord[1][0] = (int) ((nextRecord[1][0] + sumOfFirstRow) % modValue);

            attendanceRecord = nextRecord;
        }

        long totalSum = 0;
        for (long[] row : attendanceRecord) {
            for (long i : row) {
```

```
                totalSum = (totalSum + i) % modValue;
            }
        }
        return (int) totalSum;
    }

    public int checkRecordAlternative(int n) {
        int modValue = 1000000007;
        long[] PorL = new long[n + 1]; // ending with P or L, no A
        long[] P = new long[n + 1]; // ending with P, no A
        PorL[0] = P[0] = 1;
        PorL[1] = 2;
        P[1] = 1;

        for (int i = 2; i <= n; i++) {
            P[i] = PorL[i - 1];
            PorL[i] = (P[i] + P[i - 1] + P[i - 2]) % modValue;
        }

        long res = PorL[n];
        for (int i = 0; i < n; i++) { // inserting A into (n-1)-length strings
            long s = (PorL[i] * PorL[n - i - 1]) % modValue;
            res = (res + s) % modValue;
        }

        return (int) res;
    }

}
```

170. `Solution00560.java`

```
package leetcode;

import java.util.HashMap;

/**
 * 560. Subarray Sum Equals K
 * <a href="https://leetcode.com/problems/subarray-sum-equals-k/">Web link</a> |
 * Medium
 */

public class Solution00560 {

    public int subarraySum(int[] nums, int k) {
        // Initialize the count of subarrays and the cumulative sum
        int subarrayCount = 0;
        int cumulativeSum = 0;

        // Create a map to store the cumulative sum frequencies
        var cumulativeSumFrequencies = new HashMap<Integer, Integer>();

        // Add the initial condition to the map
        cumulativeSumFrequencies.put(0, 1);

        // Iterate over the array
        for (int num : nums) {
            // Update the cumulative sum
            cumulativeSum += num;

            // Check if there is a subarray sum that when subtracted from the current
cumulative sum results in k
            // If it exists, add the frequency of this sum to the count
            subarrayCount += cumulativeSumFrequencies.getOrDefault(cumulativeSum - k, 0);

            // Update the frequency of the current cumulative sum
            cumulativeSumFrequencies.put(cumulativeSum,
cumulativeSumFrequencies.getOrDefault(cumulativeSum, 0) + 1);
        }

        // Return the count of subarrays
        return subarrayCount;
    }

}
```

171. `Solution00564.java`

```
package leetcode;

/**
 * 564. Find the Closest Palindrome
 * <a href="https://leetcode.com/problems/find-the-closest-palindrome">Web link</a>
 * Hard
 */

public class Solution00564 {

    public String nearestPalindromic(String n) {
        long number = Long.parseLong(n);
        if (number <= 10) return String.valueOf(number - 1);
        if (number == 11) return "9";

        int length = n.length();
        long leftHalf = Long.parseLong(n.substring(0, (length + 1) / 2));

        long[] palindromeCandidates = new long[5];
        palindromeCandidates[0] = generatePalindromeFromLeft(leftHalf - 1, length % 2 ==
0);
        palindromeCandidates[1] = generatePalindromeFromLeft(leftHalf, length % 2 == 0);
        palindromeCandidates[2] = generatePalindromeFromLeft(leftHalf + 1, length % 2 ==
0);
        palindromeCandidates[3] = (long) Math.pow(10, length - 1) - 1;
        palindromeCandidates[4] = (long) Math.pow(10, length) + 1;

        long nearestPalindrome = 0;
        long minDifference = Long.MAX_VALUE;

        for (long candidate : palindromeCandidates) {
            if (candidate == number) continue;
            long difference = Math.abs(candidate - number);
            if (difference < minDifference || (difference == minDifference && candidate <
nearestPalindrome)) {
                minDifference = difference;
                nearestPalindrome = candidate;
            }
        }

        return String.valueOf(nearestPalindrome);
    }

    private long generatePalindromeFromLeft(long leftHalf, boolean isEvenLength) {
        long palindrome = leftHalf;
        if (!isEvenLength) leftHalf /= 10;
        while (leftHalf > 0) {
            palindrome = palindrome * 10 + leftHalf % 10;
            leftHalf /= 10;
        }
        return palindrome;
```

```
    }

    public String nearestPalindromicLargerSmaller(String n) {
        long ip = Long.parseLong(n);
        long small = smallest(String.valueOf(ip - 1));
        long large = larger(String.valueOf(ip + 1));
        if (large - ip < ip - small) return String.valueOf(large);
        else return String.valueOf(small);
    }

    private long larger(String n) {
        char[] data = n.toCharArray();
        int start = 0;
        int end = data.length - 1;
        while (start < end) {
            while (data[start] != data[end]) {
                incrementNumber(data, end);
            }
            start++;
            end--;
        }
        return Long.parseLong(String.valueOf(data));
    }

    private long smallest(String n) {
        char[] data = n.toCharArray();
        int start = 0;
        int end = data.length - 1;
        while (start < end) {
            while (data[start] != data[end]) {
                decrementNumber(data, end);
                if (data[0] == '0') return Long.parseLong(String.valueOf(data));
            }
            start++;
            end--;
        }
        return Long.parseLong(String.valueOf(data));
    }

    private void decrementNumber(char[] data, int i) {
        while (data[i] == '0') {
            data[i] = '9';
            i--;
        }
        data[i]--;
    }

    private void incrementNumber(char[] data, int i) {
        while (data[i] == '9') {
            data[i] = '0';
            i--;
        }
        data[i]++;
    }
```

```
}
```

172. `Solution00567.java`

```
package leetcode;

import java.util.Arrays;

/**
 * https://leetcode.com/problems/permutation-in-string/
 * 567. Permutation in String
 * <p>
 * Given two strings s1 and s2, return true if s2 contains a permutation of s1, or false
otherwise.
 * <p>
 * In other words, return true if one of s1's permutations is the substring of s2.
 * <p>
 * Example 1:
 * <p>
 * Input: s1 = "ab", s2 = "eidbaooo"
 * Output: true
 * Explanation: s2 contains one permutation of s1 ("ba").
 * <p>
 * Example 2:
 * <p>
 * Input: s1 = "ab", s2 = "eidboaoo"
 * Output: false
 * <p>
 * Constraints:
 * <p>
 * 1 <= s1.length, s2.length <= 104
 * s1 and s2 consist of lowercase English letters.
 */
public class Solution00567 {

    // Time: O(n)
    // Space: O(1)
    public boolean checkInclusionArrayEquals(String s1, String s2) {
        final int s1Length = s1.length();
        final int s2Length = s2.length();

        if (s1Length > s2Length) {
            return false;
        }

        int[] s1Counts = new int[26];
        int[] s2Counts = new int[26];

        for (int i = 0; i < s1Length; i++) {
            s1Counts[s1.charAt(i) - 'a']++;
            s2Counts[s2.charAt(i) - 'a']++;
        }

        for (int i = s1Length; i < s2Length; i++) {
            if (Arrays.equals(s1Counts, s2Counts)) {
```

```
                return true;
            }

            final char leftChar = s2.charAt(i - s1Length);
            final char rightChar = s2.charAt(i);
            if (leftChar != rightChar) {
                s2Counts[leftChar - 'a']--;
                s2Counts[rightChar - 'a']++;
            }
        }

        return Arrays.equals(s1Counts, s2Counts);
    }

    // Time: O(n)
    // Space: O(1)
    public boolean checkInclusion(String s1, String s2) {
        final int s1Length = s1.length();
        final int s2Length = s2.length();

        if (s1Length > s2Length) {
            return false;
        }

        int[] s1Counts = new int[26];
        int[] s2Counts = new int[26];
        boolean[] matchedArr = new boolean[26];

        for (int i = 0; i < s1Length; i++) {
            s1Counts[s1.charAt(i) - 'a']++;
            s2Counts[s2.charAt(i) - 'a']++;
        }

        int matched = 0; // Number of matched characters
        for (int i = 0; i < 26; i++) {
            if (s1Counts[i] == s2Counts[i]) {
                matched++;
                matchedArr[i] = true;
            }
        }

        for (int i = s1Length; i < s2Length; i++) {
            if (matched == 26) {
                return true;
            }

            final char leftChar = s2.charAt(i - s1Length);
            final char rightChar = s2.charAt(i);
            if (leftChar != rightChar) {
                final int indexLeft = leftChar - 'a';
                final int indexRight = rightChar - 'a';

                s2Counts[indexLeft]--; // Decrement the count for the character at the
left pointer
```

```
                s2Counts[indexRight]++; // Increment the count for the current character

                if (s1Counts[indexLeft] == s2Counts[indexLeft]) { // if left character
count matches
                    if (!matchedArr[indexLeft]) {
                        matched++;
                        matchedArr[indexLeft] = true;
                    }
                } else {
                    if (matchedArr[indexLeft]) {
                        matched--;
                        matchedArr[indexLeft] = false;
                    }
                }

                if (s1Counts[indexRight] == s2Counts[indexRight]) { // if right character
count matches
                    if (!matchedArr[indexRight]) {
                        matched++;
                        matchedArr[indexRight] = true;
                    }
                } else {
                    if (matchedArr[indexRight]) {
                        matched--;
                        matchedArr[indexRight] = false;
                    }
                }
            }
        }

        return matched == 26;
    }

}
```

173. `Solution00572.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.LinkedList;
import java.util.Queue;

/**
 * https://leetcode.com/problems/subtree-of-another-tree/
 * 572. Subtree of Another Tree
 * <p>
 * Given the roots of two binary trees root and subRoot,
 * return true if there is a subtree of root with the same structure and node values of
subRoot and false otherwise.
 * <p>
 * A subtree of a binary tree tree is a tree that consists of a node in tree and all of
this node's descendants.
 * The tree tree could also be considered as a subtree of itself.
 * <p>
 * Example 1:
 * <p>
 * Input: root = [3,4,5,1,2], subRoot = [4,1,2]
 * Output: true
 * <p>
 * Example 2:
 * <p>
 * Input: root = [3,4,5,1,2,null,null,null,null,0], subRoot = [4,1,2]
 * Output: false
 * <p>
 * Constraints:
 * <p>
 * The number of nodes in the root tree is in the range [1, 2000].
 * The number of nodes in the subRoot tree is in the range [1, 1000].
 * -104 <= root.val <= 104
 * -104 <= subRoot.val <= 104
 */
public class Solution00572 {

    // Time: O(n * m)
    // Space: O(n + m)
    public boolean isSubtreeDfsRecursive(TreeNode root, TreeNode subRoot) {
        // If the subRoot is null, return true.
        if (subRoot == null) {
            return true;
        }
        // If the root is null, return false.
        if (root == null) {
            return false;
        }

        // Return whether the subRoot is a subtree of the left subtree or the right
```

```
subtree.
        return isSameTreeDfsRecursive(root, subRoot) ||
               isSubtreeDfsRecursive(root.left, subRoot) ||
               isSubtreeDfsRecursive(root.right, subRoot);
    }

    // Time: O(n)
    // Space: O(n)
    private boolean isSameTreeDfsRecursive(TreeNode rootFragment, TreeNode subRoot) {
        if (rootFragment == null || subRoot == null) {
            return rootFragment == subRoot;
        }
        if (rootFragment.val != subRoot.val) {
            return false;
        }
        return isSameTreeDfsRecursive(rootFragment.left, subRoot.left) &&
               isSameTreeDfsRecursive(rootFragment.right, subRoot.right);
    }

    // Time: O(n * m)
    // Space: O(n + m)
    public boolean isSubtreeBfsIterative(TreeNode root, TreeNode subRoot) {
        // If the subRoot is null, return true.
        if (subRoot == null) {
            return true;
        }
        // If the root is null, return false.
        if (root == null) {
            return false;
        }

        Queue<TreeNode> queue = new LinkedList<>();
        queue.add(root);

        while (!queue.isEmpty()) {
            TreeNode current = queue.poll();
            if (isSameTreeBfsIterative(current, subRoot)) {
                return true;
            }
            if (current.left != null) {
                queue.add(current.left);
            }
            if (current.right != null) {
                queue.add(current.right);
            }
        }

        return false;
    }

    private boolean isSameTreeBfsIterative(TreeNode rootFragment, TreeNode subRoot) {
        Queue<TreeNode> queue1 = new LinkedList<>();
        Queue<TreeNode> queue2 = new LinkedList<>();
        queue1.add(rootFragment);
```

```
        queue2.add(subRoot);

        while (!queue1.isEmpty() && !queue2.isEmpty()) {
            TreeNode node1 = queue1.poll();
            TreeNode node2 = queue2.poll();

            if (node1 == null && node2 == null) {
                continue;
            }
            if (node1 == null || node2 == null) {
                return false;
            }
            if (node1.val != node2.val) {
                return false;
            }

            queue1.add(node1.left);
            queue1.add(node1.right);
            queue2.add(node2.left);
            queue2.add(node2.right);
        }

        return queue1.isEmpty() && queue2.isEmpty();
    }

}
```

174. `Solution00576.java`

```
package leetcode;

/**
 * 576. Out of Boundary Paths
 * https://leetcode.com/problems/out-of-boundary-paths/description/?envType=daily-question&envId=2024-01-26
 * Medium
 * <p>
 * There is an m x n grid with a ball. The ball is initially at the position [startRow, startColumn]. You are allowed to move the ball to one of the four adjacent cells in the grid (possibly out of the grid crossing the grid boundary). You can apply at most maxMove moves to the ball.
 * <p>
 * Given the five integers m, n, maxMove, startRow, startColumn, return the number of paths to move the ball out of the grid boundary. Since the answer can be very large, return it modulo 109 + 7.
 * <p>
 * Example 1:
 * <p>
 * Input: m = 2, n = 2, maxMove = 2, startRow = 0, startColumn = 0
 * Output: 6
 * Example 2:
 * <p>
 * Input: m = 1, n = 3, maxMove = 3, startRow = 0, startColumn = 1
 * Output: 12
 * <p>
 * Constraints:
 * <p>
 * 1 <= m, n <= 50
 * 0 <= maxMove <= 50
 * 0 <= startRow < m
 * 0 <= startColumn < n
 */
public class Solution00576 {

    // Time O(numRows*numColumns*maxMove)
    // Space O(numRows*numColumns*maxMove)
    public int findPaths(int m, int n, int maxMove, int startRow, int startColumn) {
        // Initialize a 3D array to store the number of paths for each cell at each move
        int numRows = m;
        int numColumns = n;
        int[][][] dp = new int[numRows][numColumns][maxMove + 1];

        // Iterate over the number of moves
        for (int currentMove = 1; currentMove <= maxMove; currentMove++) {

            // Iterate over the rows of the grid
            for (int currentRow = 0; currentRow < numRows; currentRow++) {

                // Iterate over the columns of the grid
                for (int currentColumn = 0; currentColumn < numColumns; currentColumn++)
```

```
{

                    // If the ball is at the top edge of the grid, add 1 to the number of
paths
                    long topEdgePaths = (currentRow == 0 ? 1 : dp[currentRow - 1]
[currentColumn][currentMove - 1]);

                    // If the ball is at the bottom edge of the grid, add 1 to the number
of paths
                    long bottomEdgePaths = (currentRow == numRows - 1 ? 1 : dp[currentRow
+ 1][currentColumn][currentMove - 1]);

                    // If the ball is at the left edge of the grid, add 1 to the number
of paths
                    long leftEdgePaths = (currentColumn == 0 ? 1 : dp[currentRow]
[currentColumn - 1][currentMove - 1]);

                    // If the ball is at the right edge of the grid, add 1 to the number
of paths
                    long rightEdgePaths = (currentColumn == numColumns - 1 ? 1 :
dp[currentRow][currentColumn + 1][currentMove - 1]);

                    // Calculate the total number of paths for the current cell at the
current move
                    final long allEdgePaths = (topEdgePaths + bottomEdgePaths +
leftEdgePaths + rightEdgePaths) % 1000000007;
                    dp[currentRow][currentColumn][currentMove] = (int) allEdgePaths;
                }
            }
        }

        // Return the number of paths for the starting cell after the maximum number of
moves
        return dp[startRow][startColumn][maxMove];
    }

}
```

175. `Solution00590.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.List;

/**
 * 590. N-ary Tree Postorder Traversal
 * <a href="https://leetcode.com/problems/n-ary-tree-postorder-traversal">Web link</a>
 * Easy
 */

public class Solution00590 {

    public class Node {
        public int val;
        public List<Node> children;

        public Node() {
        }

        public Node(int _val) {
            val = _val;
        }

        public Node(int _val, List<Node> _children) {
            val = _val;
            children = _children;
        }
    }

    public List<Integer> postorder(Node root) {
        List<Integer> result = new ArrayList<>();
        postorder(root, result);
        return result;
    }

    private void postorder(Node root, List<Integer> result) {
        if (root == null) {
            return;
        }

        if (root.children != null) {
            for (Node child : root.children) {
                postorder(child, result);
            }
        }

        result.add(root.val);
    }

}
```

176. `Solution00592.java`

```
package leetcode;

import java.math.BigInteger;

/**
 * 592. Fraction Addition and Subtraction
 * <a href="https://leetcode.com/problems/fraction-addition-and-subtraction">Web link</a>
 * Medium
 */

public class Solution00592 {

    public String fractionAdditionBigDecimal(String expression) {
        BigInteger numeratorSum = BigInteger.ZERO;
        BigInteger commonDenominator = BigInteger.ONE;
        int index = 0;
        int length = expression.length();

        while (index < length) {
            int sign = 1;

            // Determine the sign of the current fraction
            if (expression.charAt(index) == '-') {
                sign = -1;
                index++;
            } else if (expression.charAt(index) == '+') {
                index++;
            }

            // Extract the numerator
            int start = index;
            while (index < length && Character.isDigit(expression.charAt(index))) {
                index++;
            }
            BigInteger numerator = new BigInteger(expression.substring(start,
index)).multiply(BigInteger.valueOf(sign));

            // Move past the '/' character
            index++;

            // Extract the denominator
            start = index;
            while (index < length && Character.isDigit(expression.charAt(index))) {
                index++;
            }
            BigInteger denominator = new BigInteger(expression.substring(start, index));

            // Calculate the new common denominator (LCM of current and new denominator)
            BigInteger lcm =
commonDenominator.multiply(denominator).divide(commonDenominator.gcd(denominator));
```

```
            // Adjust the numerators to the new common denominator and add them
            numeratorSum =
numeratorSum.multiply(lcm.divide(commonDenominator)).add(numerator.multiply(lcm.divide(de
nominator)));
            commonDenominator = lcm;
        }

        // Simplify the fraction
        BigInteger gcd = numeratorSum.gcd(commonDenominator);
        numeratorSum = numeratorSum.divide(gcd);
        commonDenominator = commonDenominator.divide(gcd);

        return numeratorSum + "/" + commonDenominator;
    }

    public String fractionAddition(String expression) {
        int numeratorSum = 0;
        int commonDenominator = 1;
        int index = 0;

        while (index < expression.length()) {
            int currentNumerator = 0;
            int currentDenominator = 0;
            boolean isNegative = false;

            // Determine the sign of the current fraction
            if (expression.charAt(index) == '-' || expression.charAt(index) == '+') {
                isNegative = expression.charAt(index) == '-';
                index++;
            }

            // Extract the numerator
            while (index < expression.length() &&
Character.isDigit(expression.charAt(index))) {
                currentNumerator = currentNumerator * 10 + (expression.charAt(index) -
'0');
                index++;
            }
            if (isNegative) currentNumerator *= -1;

            // Skip the '/' character
            index++;

            // Extract the denominator
            while (index < expression.length() &&
Character.isDigit(expression.charAt(index))) {
                currentDenominator = currentDenominator * 10 + (expression.charAt(index)
- '0');
                index++;
            }

            // Add fractions together using common denominator
            numeratorSum = numeratorSum * currentDenominator + currentNumerator *
commonDenominator;
```

```
            commonDenominator *= currentDenominator;
        }

        // Reduce the fraction
        BigInteger gcd =
BigInteger.valueOf(numeratorSum).gcd(BigInteger.valueOf(commonDenominator));
        numeratorSum /= gcd.intValue();
        commonDenominator /= gcd.intValue();

        return numeratorSum + "/" + commonDenominator;
    }

}
```

177. `Solution00621.java`

```
package leetcode;

/**
 * 621. Task Scheduler
 * https://leetcode.com/problems/task-scheduler/
 * Medium
 */

public class Solution00621 {

    // time complexity: O(n),
    // space complexity: O(1)
    public int leastInterval(char[] tasks, int n) {
        if (n == 0) {
            return tasks.length;
        }

        int[] taskCounts = new int[26];
        int maxTaskCount = 0;
        int maxTaskCountFrequency = 0;

        for (char task : tasks) {
            int taskIndex = task - 'A';
            taskCounts[taskIndex]++;

            if (taskCounts[taskIndex] > maxTaskCount) {
                maxTaskCount = taskCounts[taskIndex];
                maxTaskCountFrequency = 1;
            } else if (taskCounts[taskIndex] == maxTaskCount) {
                maxTaskCountFrequency++;
            }
        }

        int idleSlots = (maxTaskCount - 1) * // -1 because the last task doesn't need
idle slots
                        (n + 1); // +1 because we need slot for itself as well

        int totalSlots = idleSlots + maxTaskCountFrequency;

        return Math.max(tasks.length, totalSlots);
    }

}
```

178. `Solution00623.java`

```
package leetcode;

import leetcode.common.TreeNode;

/**
 * 623. Add One Row to Tree
 * <a href="https://leetcode.com/problems/add-one-row-to-tree">Web link</a>
 * Medium
 */

public class Solution00623 {

    public TreeNode addOneRow(TreeNode root, int val, int depth) {
        if (depth == 1) {
            return new TreeNode(val, root, null);
        }
        addOneRow(root, val, depth, 1);
        return root;
    }

    private void addOneRow(TreeNode root, int val, int depth, int currentDepth) {
        if (root == null) {
            return;
        }
        if (currentDepth == depth - 1) {
            root.left = new TreeNode(val, root.left, null);
            root.right = new TreeNode(val, null, root.right);
            return;
        }
        addOneRow(root.left, val, depth, currentDepth + 1);
        addOneRow(root.right, val, depth, currentDepth + 1);
    }

}
```

179. `Solution00624.java`

```
package leetcode;

import java.util.List;

/**
 * 624. Maximum Distance in Arrays
 * <a href="https://leetcode.com/problems/maximum-distance-in-arrays">Web link</a>
 * Medium
 */

public class Solution00624 {

    public int maxDistance(List<List<Integer>> arrays) {
        int n = arrays.size();
        final List<Integer> firstArr = arrays.getFirst();
        int minSoFar = firstArr.getFirst();
        int maxSoFar = firstArr.getLast();
        int result = 0;
        for (int i = 1; i < n; i++) {
            final List<Integer> currentArr = arrays.get(i);
            result = Math.max(result, currentArr.getLast() - minSoFar);
            result = Math.max(result, maxSoFar - currentArr.getFirst());
            minSoFar = Math.min(minSoFar, currentArr.getFirst());
            maxSoFar = Math.max(maxSoFar, currentArr.getLast());
        }
        return result;
    }

}
```

180. `Solution00629.java`

```
package leetcode;

/**
 * 629. K Inverse Pairs Array
 * https://leetcode.com/problems/k-inverse-pairs-array/description/
 * Medium
 * <p>
 * Given two integers n and k, find how many different arrays consist of numbers from 1
to n such that there are exactly k inverse pairs.
 * <p>
 * We define an inverse pair as following: For ith and jth element in the array, if i < j
and a[i] > a[j] then it's an inverse pair; Otherwise, it's not.
 * <p>
 * Since the answer may be very large, the answer should be modulo 109 + 7.
 * <p>
 * Example 1:
 * <p>
 * Input: n = 3, k = 0
 * Output: 1
 * Explanation:
 * Only the array [1,2,3] which consists of numbers from 1 to 3 has exactly 0 inverse
pair.
 * <p>
 * Example 2:
 * <p>
 * Input: n = 3, k = 1
 * Output: 2
 * Explanation:
 * The array [1,3,2] and [2,1,3] have exactly 1 inverse pair.
 * <p>
 * Note:
 * <p>
 * The integer n is in the range [1, 1000] and k is in the range [0, 1000].
 */
public class Solution00629 {

    // Time O(n*k^2)
    // Space O(k)
    public int kInversePairs(int n, int k) {
        // Initialize an array to store the number of permutations with a certain number
of inverse pairs
        int[] dp = new int[k + 1];
        dp[0] = 1; // There is one permutation of 0 numbers with 0 inverse pairs

        // Iterate over the array size
        for (int currentNum = 1; currentNum <= n; currentNum++) {
            // Initialize a new array to store the number of permutations for the current
size and inverse pairs
            int[] newDp = new int[k + 1];

            // Iterate over the number of inverse pairs
```

```
            for (int currentInversePairs = 0; currentInversePairs <= k;
currentInversePairs++) {
                // Calculate the range for the inner loop
                final int innerLoopRange = Math.min(currentInversePairs, currentNum - 1);
                long totalPermutations = 0; // Initialize the total permutations for the
current size and inverse pairs

                // Iterate over the range and calculate the total permutations
                for (int x = 0; x <= innerLoopRange; x++) {
                    if (currentInversePairs - x >= 0) {
                        totalPermutations += dp[currentInversePairs - x];
                        totalPermutations %= 1000000007; // Modulo operation to prevent
overflow
                    }
                }
                newDp[currentInversePairs] = (int) totalPermutations; // Store the total
permutations
            }
            dp = newDp; // Update the dp array
        }

        // Return the number of permutations of the given array size with the given
number of inverse pairs
        return dp[k];
    }

}
```

181. `Solution00632.java`

```
package leetcode;

import java.util.Comparator;
import java.util.List;
import java.util.PriorityQueue;

/**
 * 632. Smallest Range Covering Elements from K Lists
 * <a href="https://leetcode.com/problems/smallest-range-covering-elements-from-k-
lists">Web link</a>
 * Hard
 */

public class Solution00632 {

    public int[] smallestRange(List<List<Integer>> nums) {
        // Min-Heap: stores (value, list index, element index)
        PriorityQueue<int[]> minHeap = new PriorityQueue<>(Comparator.comparingInt(a ->
a[0]));
        int curMax = Integer.MIN_VALUE;

        // Initialize the heap with the first element of each list
        for (int i = 0; i < nums.size(); i++) {
            int val = nums.get(i).getFirst();
            minHeap.offer(new int[]{val, i, 0});
            curMax = Math.max(curMax, val);
        }

        // Track the smallest range
        int[] smallRange = {0, Integer.MAX_VALUE};

        while (!minHeap.isEmpty()) {
            // Get the minimum element from the heap
            int[] curr = minHeap.poll();
            int curMin = curr[0], listIdx = curr[1], elemIdx = curr[2];

            // Update the smallest range if a better one is found
            if (curMax - curMin < smallRange[1] - smallRange[0] ||
                    (curMax - curMin == smallRange[1] - smallRange[0] && curMin <
smallRange[0])) {
                smallRange[0] = curMin;
                smallRange[1] = curMax;
            }

            // Move to the next element in the same list
            if (elemIdx + 1 < nums.get(listIdx).size()) {
                int nextVal = nums.get(listIdx).get(elemIdx + 1);
                minHeap.offer(new int[]{nextVal, listIdx, elemIdx + 1});
                curMax = Math.max(curMax, nextVal);
            } else {
                // If any list is exhausted, stop
```

```
                break;
            }
        }
        return smallRange;
    }

}
```

182. `Solution00633.java`

```
package leetcode;

import java.util.HashSet;
import java.util.Set;

/**
 * 633. Sum of Square Numbers
 * https://leetcode.com/problems/sum-of-square-numbers/description/
 * Medium
 */

public class Solution00633 {

    public boolean judgeSquareSum(int c) {
        int max = (int) Math.sqrt(c);
        Set<Integer> set = new HashSet<>();
        for (int i = 0; i <= max; i++) {
            final int power = i * i;
            set.add(power);
            int remainingPower = c - power;
            if (set.contains(remainingPower)) {
                return true;
            }
        }
        return false;
    }

    public boolean judgeSquareSumOptimized(int c) {
        int max = (int) Math.sqrt(c);
        int start = 0;
        long powerStart = 0;
        int end = max;
        long powerEnd = (long) end * end;
        while (start <= end) {
            long sum = powerStart + powerEnd;
            if (sum < c) {
                start++;
                powerStart = (long) start * start;
            } else if (sum > c) {
                end--;
                powerEnd = (long) end * end;
            } else {
                return true;
            }
        }
        return false;
    }

}
```

183. `Solution00641.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 641. Design Circular Deque
 * <a href="https://leetcode.com/problems/design-circular-deque">Web link</a>
 * Medium
 */

public class Solution00641 {

    static class MyCircularDeque {

        private final int[] data;
        private int front;
        private int rear;
        private int size;

        public MyCircularDeque(int k) {
            data = new int[k];
            Arrays.fill(data, -1);
            front = 0;
            rear = 0;
            size = 0;
        }

        public boolean insertFront(int value) {
            if (isFull()) {
                return false;
            }
            front = (front - 1 + data.length) % data.length;
            data[front] = value;
            size++;
            return true;
        }

        public boolean insertLast(int value) {
            if (isFull()) {
                return false;
            }
            data[rear] = value;
            rear = (rear + 1) % data.length;
            size++;
            return true;
        }

        public boolean deleteFront() {
            if (isEmpty()) {
                return false;
            }
```

```
            front = (front + 1) % data.length;
            size--;
            return true;
        }

        public boolean deleteLast() {
            if (isEmpty()) {
                return false;
            }
            rear = (rear - 1 + data.length) % data.length;
            size--;
            return true;
        }

        public int getFront() {
            if (isEmpty()) {
                return -1;
            }
            return data[front];
        }

        public int getRear() {
            if (isEmpty()) {
                return -1;
            }
            return data[(rear - 1 + data.length) % data.length];
        }

        public boolean isEmpty() {
            return size == 0;
        }

        public boolean isFull() {
            return size == data.length;
        }
    }

/**
 * Your MyCircularDeque object will be instantiated and called as such:
 * MyCircularDeque obj = new MyCircularDeque(k);
 * boolean param_1 = obj.insertFront(value);
 * boolean param_2 = obj.insertLast(value);
 * boolean param_3 = obj.deleteFront();
 * boolean param_4 = obj.deleteLast();
 * int param_5 = obj.getFront();
 * int param_6 = obj.getRear();
 * boolean param_7 = obj.isEmpty();
 * boolean param_8 = obj.isFull();
 */

}
```

184. `Solution00645.java`

```
package leetcode;

/**
 * 645. Set Mismatch
 * https://leetcode.com/problems/set-mismatch
 * <p>
 * You have a set of integers s, which originally contains all the numbers from 1 to n.
Unfortunately, due to some error,
 * one of the numbers in s got duplicated to another number in the set, which results in
repetition of one number and
 * loss of another number.
 * <p>
 * You are given an integer array nums representing the data status of this set after the
error.
 * <p>
 * Find the number that occurs twice and the number that is missing and return them in
the form of an array.
 * <p>
 * Constraints:
 * <p>
 * 2 <= nums.length <= 10^4
 * 1 <= nums[i] <= 10^4
 */
public class Solution00645 {

    // Time: O(n)
    // Space: O(n)
    public int[] findErrorNums(int[] nums) {
        boolean[] seen = new boolean[nums.length + 1];

        int[] result = new int[2];
        for (int num : nums) {
            if (seen[num]) {
                result[0] = num;
            }
            seen[num] = true;
        }

        for (int i = 1; i <= nums.length; i++) {
            if (!seen[i]) {
                result[1] = i;
                break;
            }
        }

        return result;
    }

}
```

185. `Solution00647.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/palindromic-substrings/
 * 647. Palindromic Substrings
 * Medium
 */
public class Solution00647 {

    // Time complexity: O(n^2)
    // Space complexity: O(n^2)
    public int countSubstrings(String s) {
        // Get the length of the input string
        int n = s.length();

        // Initialize a 2D boolean array for dynamic programming
        // dp[i][j] will be 'true' if the string from index i to j is a palindrome
        boolean[][] dp = new boolean[n][n];

        // Initialize a counter for the number of palindromic substrings
        int palindromeCount = 0;

        // Iterate over all possible lengths for substrings
        for (int substringLength = 1; substringLength <= n; substringLength++) {
            // Iterate over all possible starting indices for substrings of the current
length
            for (int startIndex = 0; startIndex <= n - substringLength; startIndex++) {
                // Calculate the ending index for the current substring
                int endIndex = startIndex + substringLength - 1;

                // Check if the characters at the start and end of the substring are the
same
                if (s.charAt(startIndex) == s.charAt(endIndex)) {
                    // If the substring length is 2 or less, or the substring excluding
the first and last characters is a palindrome,
                    // then the current substring is a palindrome
                    if (substringLength <= 2 || dp[startIndex + 1][endIndex - 1]) {
                        dp[startIndex][endIndex] = true;
                        palindromeCount++;
                    }
                }
            }
        }

// Return the total count of palindromic substrings
        return palindromeCount;
    }

}
```

186. `Solution00648.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.List;
import java.util.StringTokenizer;

/**
 * 648. Replace Words
 * <a href="https://leetcode.com/problems/replace-words">Web link</a>
 * Medium
 */

public class Solution00648 {

    public String replaceWords(List<String> dictionary, String sentence) {
        StringTokenizer tokenizer = new StringTokenizer(sentence);
        List<String> result = new ArrayList<>();
        while (tokenizer.hasMoreTokens()) {
            String word = tokenizer.nextToken();
            int minLen = Integer.MAX_VALUE;
            for (String root : dictionary) {
                if (root.length() < minLen &&
                    word.startsWith(root)) {
                    minLen = root.length();
                    word = root;
                }
            }
            result.add(word);
        }
        return String.join(" ", result);
    }

}
```

187. `Solution00650.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 650. 2 Keys Keyboard
 * <a href="https://leetcode.com/problems/2-keys-keyboard">Web link</a>
 * Medium
 */

public class Solution00650 {

    public int minSteps(int n) {
        int[] dp = new int[n + 1];
        Arrays.fill(dp, Integer.MAX_VALUE);
        dp[1] = 0;

        for (int i = 2; i < n + 1; i++) {
            for (int j = 1; j < 1 + i / 2; j++) {
                if (i % j == 0) {
                    dp[i] = Math.min(dp[i], dp[j] + i / j);
                }
            }
        }
        return dp[n];
    }

}
```

188. `Solution00664.java`

```
package leetcode;

/**
 * 664. Strange Printer
 * <a href="https://leetcode.com/problems/strange-printer">Web link</a>
 * Hard
 */

public class Solution00664 {

    public int strangePrinter(String s) {
        int n = s.length();
        int[][] dp = new int[n][n];
        return count(s, dp, 0, n - 1);
    }

    private int count(String s, int[][] dp, int i, int j) {
        if (i > j) {
            return 0;
        }
        if (dp[i][j] > 0) {
            return dp[i][j];
        }
        int res = count(s, dp, i, j - 1) + 1;
        for (int k = i; k < j; k++) {
            if (s.charAt(k) == s.charAt(j)) {
                res = Math.min(res, count(s, dp, i, k) + count(s, dp, k + 1, j - 1));
            }
        }
        dp[i][j] = res;
        return res;
    }

}
```

189. `Solution00670.java`

```
/**
 * 670. Maximum Swap
 * <a href="https://leetcode.com/problems/maximum-swap">Web link</a> |
 * Medium
 */

package leetcode;

public class Solution00670 {

    public int maximumSwap(int num) {
        if (num <= 11) {
            return num;
        }
        int maxIndex = -1;
        int swap1 = -1;
        int swap2 = -1;
        char[] nums = Integer.toString(num).toCharArray();
        int n = nums.length;

        for (int i = n - 1; i >= 0; i--) {
            if (maxIndex == -1 || nums[i] > nums[maxIndex]) {
                maxIndex = i;
            } else if (nums[i] < nums[maxIndex]) {
                swap1 = i;
                swap2 = maxIndex;
            }
        }

        if (swap1 != -1) {
            char temp = nums[swap1];
            nums[swap1] = nums[swap2];
            nums[swap2] = temp;
        }

        return Integer.parseInt(new String(nums));
    }

}
```

190. `Solution00678.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/valid-parenthesis-string/
 * 678. Valid Parenthesis String
 * Medium
 */

public class Solution00678 {

    // Time complexity: O(n)
    // Space complexity: O(1)
    public boolean checkValidString(String s) {
        // Initialize the minimum and maximum number of open brackets
        int maxOpen = 0;
        int minOpen = 0;

        // Iterate over the characters in the string
        for (char c : s.toCharArray()) {
            // Update the minimum and maximum number of open brackets based on the
current character
            if (c == '(') {
                maxOpen++;
                minOpen++;
            } else if (c == ')') {
                maxOpen--;
                minOpen--;
            } else {
                maxOpen++;
                minOpen--;
            }

            // If the maximum number of open brackets becomes negative, return 'false'
            if (maxOpen < 0) {
                return false;
            }
            // If the minimum number of open brackets becomes negative, reset it to 0
            if (minOpen < 0) {
                minOpen = 0;
            }
        }

        // If the minimum number of open brackets is 0, return 'true'; otherwise, return
'false'
        return minOpen == 0;
    }

}
```

191. `Solution00689.java`

```
/**
 * 689. Maximum Sum of 3 Non-Overlapping Subarrays
 * <a href="https://leetcode.com/problems/maximum-sum-of-3-non-overlapping-subarrays">Web
link</a>
 * Hard
 */

package leetcode;

public class Solution00689 {

    public int[] maxSumOfThreeSubarrays(int[] nums, int k) {
        int n = nums.length;
        int[] sum = new int[n + 1];
        for (int i = 0; i < n; i++) {
            sum[i + 1] = sum[i] + nums[i];
        }

        int[] left = new int[n];
        int[] right = new int[n];
        int[] result = new int[3];

        // Calculate left max indices
        int maxLeftIndex = 0;
        for (int i = k - 1; i < n; i++) {
            if (sum[i + 1] - sum[i + 1 - k] > sum[maxLeftIndex + k] - sum[maxLeftIndex])
{
                maxLeftIndex = i + 1 - k;
            }
            left[i] = maxLeftIndex;
        }

        // Calculate right max indices
        int maxRightIndex = n - k;
        for (int i = n - k; i >= 0; i--) {
            if (sum[i + k] - sum[i] >= sum[maxRightIndex + k] - sum[maxRightIndex]) {
                maxRightIndex = i;
            }
            right[i] = maxRightIndex;
        }

        // Find the three intervals with max sum
        int maxSum = 0;
        for (int i = k; i <= n - 2 * k; i++) {
            int l = left[i - 1];
            int r = right[i + k];
            int totalSum = (sum[l + k] - sum[l]) + (sum[i + k] - sum[i]) + (sum[r + k] -
sum[r]);
            if (totalSum > maxSum) {
                maxSum = totalSum;
                result[0] = l;
```

```
                result[1] = i;
                result[2] = r;
            }
        }

        return result;
    }

}
```

192. `Solution00695.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/max-area-of-island/
 * 695. Max Area of Island
 * <p>
 * You are given an m x n binary matrix grid. An island is a group of 1's (representing
land)
 * connected 4-directionally (horizontal or vertical.) You may assume all four edges of
the grid
 * are surrounded by water.
 * <p>
 * The area of an island is the number of cells with a value 1 in the island.
 * <p>
 * Return the maximum area of an island in grid. If there is no island, return 0.
 * <p>
 * Example 1:
 * <p>
 * Input: grid = [
 * [0,0,1,0,0,0,0,1,0,0,0,0,0],
 * [0,0,0,0,0,0,0,1,1,1,0,0,0],
 * [0,1,1,0,1,0,0,0,0,0,0,0,0],
 * [0,1,0,0,1,1,0,0,1,0,1,0,0],
 * [0,1,0,0,1,1,0,0,1,1,1,0,0],
 * [0,0,0,0,0,0,0,0,0,0,1,0,0],
 * [0,0,0,0,0,0,0,1,1,1,0,0,0],
 * [0,0,0,0,0,0,0,1,1,0,0,0,0]
 * ]
 * Output: 6
 * <p>
 * Explanation: The answer is not 11, because the island must be connected 4-
directionally.
 * <p>
 * Example 2:
 * <p>
 * Input: grid = [[0,0,0,0,0,0,0,0]]
 * Output: 0
 * <p>
 * Constraints:
 * <p>
 * m == grid.length
 * n == grid[i].length
 * 1 <= m, n <= 50
 * grid[i][j] is either 0 or 1.
 */
public class Solution00695 {

    public int maxAreaOfIsland(int[][] grid) {
        int maxArea = 0;
        int len1 = grid.length;
        int len2 = grid[0].length;
```

```
        boolean[][] visited = new boolean[len1][len2];
        for (int row = 0; row < len1; row++) {
            for (int col = 0; col < len2; col++) {
                if (grid[row][col] == 1 && !visited[row][col]) {
                    int currentArea = calcAreaDfs(grid, visited, row, col);
                    maxArea = Math.max(maxArea, currentArea);
                }
            }
        }
        return maxArea;
    }

    private int calcAreaDfs(int[][] grid, boolean[][] visited, int row, int col) {
        if (row < 0 || row >= grid.length ||
            col < 0 || col >= grid[row].length ||
            visited[row][col] ||
            grid[row][col] != 1) {
            return 0;
        }
        visited[row][col] = true;

        return 1 +
               calcAreaDfs(grid, visited, row - 1, col) +
               calcAreaDfs(grid, visited, row + 1, col) +
               calcAreaDfs(grid, visited, row, col - 1) +
               calcAreaDfs(grid, visited, row, col + 1);
    }

}
```

193. `Solution00703.java`

```
package leetcode;

import java.util.PriorityQueue;

/**
 * 703. Kth Largest Element in a Stream
 * https://leetcode.com/problems/kth-largest-element-in-a-stream/
 * <p>
 * Easy
 * <p>
 * Design a class to find the kth largest element in a stream. Note that it is the kth
largest element in the sorted
 * order, not the kth distinct element.
 * <p>
 * Implement KthLargest class:
 * <p>
 * KthLargest(int k, int[] nums) Initializes the object with the integer k and the stream
of integers nums. int
 * add(int val) Returns the element representing the kth largest element in the stream.
 * <p>
 * Example 1:
 * <p>
 * Input ["KthLargest", "add", "add", "add", "add", "add"] [[3, [4, 5, 8, 2]], [3], [5],
[10], [9], [4]] Output
 * [null, 4, 5, 5, 8, 8]
 * <p>
 * Explanation KthLargest kthLargest = new KthLargest(3, [4, 5, 8, 2]);
kthLargest.add(3); // return 4
 * kthLargest.add(5); // return 5 kthLargest.add(10); // return 5 kthLargest.add(9); //
return 8 kthLargest.add(4); //
 * return 8
 * <p>
 * Constraints:
 * <p>
 * 1 <= k <= 104 0 <= nums.length <= 104 -104 <= nums[i] <= 104 -104 <= val <= 104 At
most 104 calls will be made to
 * add. It is guaranteed that there will be at least k elements in the array when you
search for the kth element.
 */
public class Solution00703 {

    // Time: O(n * log(k))
    // Space: O(k)
    public static class KthLargest {

        private final int k;
        private final PriorityQueue<Integer> minHeap;

        public KthLargest(int k, int[] nums) {
            this.k = k;
            this.minHeap = new PriorityQueue<>(k);
```

```
            for (int num : nums) {
                add(num);
            }
        }

        public int add(int val) {
            if (minHeap.size() < k) {
                minHeap.offer(val);
            } else if (val > minHeap.peek()) {
                minHeap.poll();
                minHeap.offer(val);
            }
            return minHeap.peek();
        }
    }

}
```

194. `Solution00704.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/binary-search/
 * 704. Binary Search
 * <p>
 * Given a sorted (in ascending order) integer array nums of n elements and a target
value, write a function to search target in nums.
 * If target exists, then return its index, otherwise return -1.
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [-1,0,3,5,9,12], target = 9 Output: 4 Explanation: 9 exists in nums and
its index is 4 Example 2:
 * <p>
 * Input: nums = [-1,0,3,5,9,12], target = 2 Output: -1 Explanation: 2 does not exist in
nums so return -1
 * <p>
 * Note:
 * <p>
 * You may assume that all elements in nums are unique. n will be in the range [1,
10000]. The value of each element in nums will be in the range [-9999, 9999].
 */
public class Solution00704 {

    // Time: O(logn)
    // Space: O(1)
    public int search(int[] nums, int target) {
        // Create a variable to keep track of the left index.
        int left = 0;

        // Create a variable to keep track of the right index.
        int right = nums.length - 1;

        // Loop while the left index is less than or equal to the right index.
        while (left <= right) {
            // Create a variable to keep track of the middle index.
            int mid = left + (right - left) / 2;

            final int middleValue = nums[mid];

            // If the middle element is less than the target.
            if (middleValue < target) {
                // Update the left index.
                left = mid + 1;
            } else if (middleValue > target) {
                // Update the right index.
                right = mid - 1;
            } else {
                // Return the middle index.
                return mid;
```

```
            }
        }

        // Return -1.
        return -1;
    }

}
```

195. `Solution00713.java`

```
package leetcode;

/**
 * 713. Subarray Product Less Than K
 * https://leetcode.com/problems/subarray-product-less-than-k/
 * Medium
 */

public class Solution00713 {

    public int numSubarrayProductLessThanK(int[] nums, int k) {
        if (k <= 1) {
            return 0;
        }

        int n = nums.length;
        int result = 0;
        int product = 1;
        int left = 0;

        for (int right = 0; right < n; right++) {
            product *= nums[right];
            while (product >= k) {
                product /= nums[left];
                left++;
            }
            result += right - left + 1;
        }

        return result;
    }

}
```

196. `Solution00719.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 719. Find K-th Smallest Pair Distance
 * <a href="https://leetcode.com/problems/find-k-th-smallest-pair-distance">Web link</a>
 * Hard
 */
public class Solution00719 {

    public int smallestDistancePair(int[] nums, int k) {
        Arrays.sort(nums);
        int left = 0;
        int right = nums[nums.length - 1] - nums[0];
        while (left < right) {
            int mid = left + (right - left) / 2;
            if (totalPairsWithDiffLessThanCandidate(mid, nums) >= k) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return right;
    }

    private int totalPairsWithDiffLessThanCandidate(int candidate, int[] nums) {
        int n = nums.length;
        int total = 0;
        int left = 0;
        int right = 0;
        while (right < n) {
            while (nums[right] - nums[left] > candidate) {
                ++left;
            }
            total += (right - left);
            right++;
        }
        return total;
    }

}
```

197. `Solution00725.java`

```
package leetcode;

import leetcode.common.ListNode;

/**
 * 725. Split Linked List in Parts
 * <a href="https://leetcode.com/problems/split-linked-list-in-parts">Web link</a>
 * Medium
 */

public class Solution00725 {

    public ListNode[] splitListToParts(ListNode head, int k) {
        int len = 0;
        ListNode curr = head;
        while (curr != null) {
            len++;
            curr = curr.next;
        }
        int partSize = len / k;
        int extra = len % k;
        ListNode[] parts = new ListNode[k];
        curr = head;
        for (int i = 0; i < k; i++) {
            ListNode dummy = new ListNode(-1, curr);
            ListNode prev = dummy;
            final int subSize;
            if (i < extra) {
                subSize = partSize + 1;
            } else {
                subSize = partSize;
            }
            for (int j = 0; j < subSize; j++) {
                prev = curr;
                curr = curr.next;
            }
            prev.next = null;
            parts[i] = dummy.next;
        }
        return parts;
    }

}
```

198. `Solution00726.java`

```
package leetcode;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.HashMap;
import java.util.Map;

/**
 * 726. Number of Atoms
 * <a href="https://leetcode.com/problems/number-of-atoms">Web link</a>
 * Hard
 */

public class Solution00726 {

    public String countOfAtoms(String formula) {
        int n = formula.length();
        Deque<Map<String, Integer>> stack = new ArrayDeque<>();
        Map<String, Integer> rootMap = new HashMap<>();
        stack.push(rootMap);
        for (int index = 0; index < n; ++index) {
            char ch = formula.charAt(index);
            if (ch == '(') {
                stack.push(new HashMap<>());
            } else if (ch == ')') {
                int count = 0;
                if (index + 1 < n && Character.isDigit(formula.charAt(index + 1))) {
                    int nextIndex = index + 1;
                    while (nextIndex < n && Character.isDigit(formula.charAt(nextIndex)))
{
                        count = count * 10 + (formula.charAt(nextIndex) - '0');
                        nextIndex++;
                    }
                    index = nextIndex - 1;
                } else {
                    count = 1;
                }
                Map<String, Integer> currentMap = stack.pop();
                Map<String, Integer> topMap = stack.peek();
                for (var entry : currentMap.entrySet()) {
                    topMap.put(entry.getKey(), topMap.getOrDefault(entry.getKey(), 0) +
entry.getValue() * count);
                }
            } else if (Character.isUpperCase(ch)) {
                int j = index + 1;
                while (j < n && Character.isLowerCase(formula.charAt(j))) {
                    j++;
                }
                String atom = formula.substring(index, j);
                index = j - 1;
                int count = 0;
```

```
                if (index + 1 < n && Character.isDigit(formula.charAt(index + 1))) {
                    int nextIndex = index + 1;
                    while (nextIndex < n && Character.isDigit(formula.charAt(nextIndex)))
{
                        count = count * 10 + (formula.charAt(nextIndex) - '0');
                        nextIndex++;
                    }
                    index = nextIndex - 1;
                } else {
                    count = 1;
                }
                Map<String, Integer> currentMap = stack.peek();
                currentMap.put(atom, currentMap.getOrDefault(atom, 0) + count);
            }
        }
        StringBuilder sb = new StringBuilder();
        rootMap.entrySet()
                .stream()
                .sorted(Map.Entry.comparingByKey())
                .forEach(entry -> {
                    if (entry.getValue() > 1) {
                        sb.append(entry.getKey()).append(entry.getValue());
                    } else {
                        sb.append(entry.getKey());
                    }
                });
        return sb.toString();
    }

}
```

199. `Solution00729.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.List;

/**
 * 729. My Calendar I
 * <a href="https://leetcode.com/problems/my-calendar-i">Web link</a>
 * Medium
 */

public class Solution00729 {

    static class MyCalendar {

        private final List<int[]> bookings;

        public MyCalendar() {
            bookings = new ArrayList<>();
        }

        public boolean book(int start, int end) {
            for (int[] booking : bookings) {
                if (booking[0] < end && start < booking[1]) {
                    return false;
                }
            }
            bookings.add(new int[]{start, end});
            return true;
        }

    }

}
```

200. `Solution00731.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.List;

/**
 * 731. My Calendar II
 * <a href="https://leetcode.com/problems/my-calendar-ii">Web link</a>
 * Medium
 */

public class Solution00731 {

    static class MyCalendarTwo {

        private final List<int[]> nonOverlappingBookings;
        private final List<int[]> overlappingBookings;

        public MyCalendarTwo() {
            nonOverlappingBookings = new ArrayList<>();
            overlappingBookings = new ArrayList<>();
        }

        public boolean book(int start, int end) {
            for (int[] booking : overlappingBookings) {
                if (booking[0] < end && start < booking[1]) {
                    return false;
                }
            }

            for (int[] booking : nonOverlappingBookings) {
                if (booking[0] < end && start < booking[1]) {
                    overlappingBookings.add(new int[]{
                            Math.max(start, booking[0]),
                            Math.min(end, booking[1])
                    });
                }
            }

            nonOverlappingBookings.add(new int[]{start, end});
            return true;
        }

    }

}
```

201. `Solution00739.java`

```
package leetcode;

import java.util.Deque;
import java.util.LinkedList;

/**
 * https://leetcode.com/problems/daily-temperatures/
 * 739. Daily Temperatures
 * <p>
 * Given an array of integers temperatures represents the daily temperatures,
 * return an array answer such that answer[i] is the number of days you have to wait
after the ith day to get a warmer temperature.
 * If there is no future day for which this is possible, keep answer[i] == 0 instead.
 * <p>
 * Example 1:
 * <p>
 * Input: temperatures = [73,74,75,71,69,72,76,73]
 * Output: [1,1,4,2,1,1,0,0]
 * Example 2:
 * <p>
 * Input: temperatures = [30,40,50,60]
 * Output: [1,1,1,0]
 * Example 3:
 * <p>
 * Input: temperatures = [30,60,90]
 * Output: [1,1,0]
 * <p>
 * <p>
 * Constraints:
 * <p>
 * 1 <= temperatures.length <= 105
 * 30 <= temperatures[i] <= 100
 */
public class Solution00739 {

    // Time: O(n^2)
    // Space: O(n)
    public int[] dailyTemperaturesBruteforce(int[] temperatures) {
        int[] result = new int[temperatures.length];
        for (int i = 0; i < temperatures.length; i++) {
            for (int j = i + 1; j < temperatures.length; j++) {
                if (temperatures[j] > temperatures[i]) {
                    result[i] = j - i;
                    break;
                }
            }
        }
        return result;
    }

    // Time: O(n)
```

```
    // Space: O(n)
    public int[] dailyTemperatures(int[] temperatures) {
        final int size = temperatures.length;
        int[] result = new int[size];
        // stack to store the index of the temperatures
        Deque<Integer> stack = new LinkedList<>();
        // push the first index onto the stack
        stack.push(0);

        // Iterate over the temperatures, starting from the second one.
        for (int i = 1; i < size; i++) {
            // While the stack is not empty and the current temperature is greater than
the temperature at the top of the stack.
            while (!stack.isEmpty() && temperatures[i] > temperatures[stack.peek()]) {
                int index = stack.pop();
                result[index] = i - index;
            }

            // Push the current temperature onto the stack
            stack.push(i);
        }
        return result;
    }

}
```

202. `Solution00746.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/min-cost-climbing-stairs/
 * 746. Min Cost Climbing Stairs
 * <p>
 * You are given an integer array cost where cost[i] is the cost of ith step on a
staircase. Once you pay the cost,
 * you can either climb one or two steps.
 * <p>
 * You can either start from the step with index 0, or the step with index 1.
 * <p>
 * Return the minimum cost to reach the top of the floor.
 * <p>
 * Example 1:
 * <p>
 * Input: cost = [10,15,20]
 * Output: 15
 * Explanation: Cheapest is: start on cost[1], pay that cost, and go to the top.
 * Example 2:
 * <p>
 * Input: cost = [1,100,1,1,1,100,1,1,100,1]
 * Output: 6
 * Explanation: Cheapest is: start on cost[0], and only step on 1s, skipping cost[3].
 * <p>
 * Constraints:
 * <p>
 * 2 <= cost.length <= 1000
 * 0 <= cost[i] <= 999
 */
public class Solution00746 {

    // Time: O(n)
    // Space: O(1)
    public int minCostClimbingStairs(int[] cost) {
        int first = cost[0];
        int second = cost[1];

        for (int i = 2; i < cost.length; i++) {
            int tmp = first;
            first = second;
            second = Math.min(tmp, second) + cost[i];
        }

        return Math.min(first, second);
    }

    // Time: O(n)
    // Space: O(n)
    public int minCostClimbingStairsFirstApproach(int[] cost) {
        int[] dp = new int[cost.length + 1];
```

```
        for (int i = 2; i <= cost.length; i++) {
            dp[i] = Math.min(dp[i - 1] + cost[i - 1], dp[i - 2] + cost[i - 2]);
        }

        return dp[cost.length];
    }

}
```

203. `Solution00752.java`

```
package leetcode;

import java.util.*;

/**
 * 752. Open the Lock
 * https://leetcode.com/problems/open-the-lock/
 * Medium
 */

public class Solution00752 {

    public int openLock(String[] deadends, String target) {
        Set<String> deadSet = new HashSet<>();
        Collections.addAll(deadSet, deadends);
        if (deadSet.contains("0000")) {
            return -1;
        }
        if ("0000".equals(target)) {
            return 0;
        }
        Set<String> visited = new HashSet<>();
        Queue<String> queue = new ArrayDeque<>();
        queue.offer("0000");
        visited.add("0000");
        int steps = 0;
        while (!queue.isEmpty()) {
            int size = queue.size();
            steps++;
            for (int i = 0; i < size; i++) {
                String current = queue.poll();
                for (String next : getNextStates(current)) {
                    if (next.equals(target)) {
                        return steps;
                    }
                    if (!visited.contains(next) && !deadSet.contains(next)) {
                        queue.offer(next);
                        visited.add(next);
                    }
                }
            }
        }
        return -1;
    }

    private List<String> getNextStates(String current) {
        List<String> nextStates = new ArrayList<>();
        char[] chars = current.toCharArray();
        for (int i = 0; i < 4; i++) {
            char original = chars[i];
            chars[i] = (char) ((chars[i] - '0' + 1) % 10 + '0');
```

```
            nextStates.add(new String(chars));
            chars[i] = (char) ((original - '0' + 9) % 10 + '0');
            nextStates.add(new String(chars));
            chars[i] = original;
        }
        return nextStates;
    }

}
```

204. `Solution00769.java`

```
/**
 * 769. Max Chunks To Make Sorted
 * <a href="https://leetcode.com/problems/max-chunks-to-make-sorted">Web link</a>
 * Medium
 */

package leetcode;

public class Solution00769 {

    public int maxChunksToSorted(int[] arr) {
        int count = 0;
        int max = Integer.MIN_VALUE;

        for (int i = 0; i < arr.length; i++) {
            max = Math.max(max, arr[i]);
            if (max == i)
                count++;
        }

        return count;
    }

}
```

205. `Solution00773.java`

```
/**
 * 773. Sliding Puzzle
 * <a href="https://leetcode.com/problems/sliding-puzzle">Web link</a>
 * Hard
 */

package leetcode;

import java.util.ArrayDeque;
import java.util.HashSet;
import java.util.Queue;
import java.util.Set;

public class Solution00773 {

    public int slidingPuzzle(int[][] board) {
        // Directions for possible swaps based on '0' position
        int[][] dir = {{1, 3}, {0, 2, 4}, {1, 5}, {0, 4}, {1, 3, 5}, {2, 4}};
        String target = "123450";
        Set<String> vis = new HashSet<>(); // Track visited configurations
        Queue<String> q = new ArrayDeque<>();
        String start = "";

        // Convert 2D board to a single string
        for (int[] row : board) {
            for (int col : row) {
                start += col;
            }
        }

        q.offer(start);
        vis.add(start);
        int step = 0;

        // Perform BFS
        while (!q.isEmpty()) {
            int size = q.size();
            while (size-- > 0) {
                String current = q.poll();

                // Check if target is reached
                if (current.equals(target)) return step;

                int zero = current.indexOf('0'); // Find position of '0'

                // Generate next moves
                for (int move : dir[zero]) {
                    StringBuilder next = new StringBuilder(current);
                    char temp = next.charAt(zero);
                    next.setCharAt(zero, next.charAt(move));
                    next.setCharAt(move, temp);
```

```
                    if (!vis.contains(next.toString())) { // Add unvisited states to the
queue
                        vis.add(next.toString());
                        q.offer(next.toString());
                    }
                }
            }
            step++;
        }
        return -1; // Return -1 if target is unreachable
    }
}
```

206. `Solution00778.java`

```
package leetcode;

import java.util.Comparator;
import java.util.PriorityQueue;
import java.util.Queue;

/**
 * 778. Swim in Rising Water
 * <a href="https://leetcode.com/problems/swim-in-rising-water">Web link</a>
 * Hard
 */

public class Solution00778 {

    public int swimInWater(int[][] grid) {
        int n = grid.length;
        boolean[][] visited = new boolean[n][n];
        Queue<Cell> deque = new PriorityQueue<>(Comparator.comparingInt(cell ->
cell.time));
        deque.offer(new Cell(0, 0, grid[0][0]));
        visited[0][0] = true;

        while (!deque.isEmpty()) {
            Cell cell = deque.poll();
            int row = cell.row;
            int col = cell.col;
            int time = cell.time;

            if (row == n - 1 && col == n - 1) {
                return cell.time;
            }

            visitDirection(grid, time, row, col + 1, visited, deque);
            visitDirection(grid, time, row + 1, col, visited, deque);
            visitDirection(grid, time, row - 1, col, visited, deque);
            visitDirection(grid, time, row, col - 1, visited, deque);
        }

        return 0;
    }

    private static void visitDirection(int[][] grid, int time, int row, int col,
boolean[][] visited, Queue<Cell> deque) {
        int n = grid.length;
        if (row < 0 || row >= n ||
            col < 0 || col >= n ||
            visited[row][col]) {
            return;
        }
        deque.offer(new Cell(row, col, Math.max(time, grid[row][col])));
        visited[row][col] = true;
```

```
    }

    private static class Cell {
        private final int row;
        private final int col;
        private final int time;

        public Cell(int row, int col, int time) {
            this.row = row;
            this.col = col;
            this.time = time;
        }

        @Override
        public String toString() {
            return "Cell{" +
                   "row=" + row +
                   ", col=" + col +
                   ", time=" + time +
                   '}';
        }
    }

}
```

207. `Solution00786.java`

```
package leetcode;

import java.util.Comparator;
import java.util.PriorityQueue;

/**
 * 786. K-th Smallest Prime Fraction
 * https://leetcode.com/problems/k-th-smallest-prime-fraction/description/
 * Medium
 */

public class Solution00786 {
    public int[] kthSmallestPrimeFraction(int[] arr, int k) {
        int n = arr.length;
        PriorityQueue<Data> pq = new PriorityQueue<>(Comparator.comparingDouble(a ->
a.val));
        for (int i = 0; i < n - 1; i++) {
            for (int j = i + 1; j < n; j++) {
                Data data = new Data();
                data.first = arr[i];
                data.second = arr[j];
                data.val = (double) arr[i] / arr[j];
                pq.add(data);
            }
        }
        Data current = null;
        for (int i = 0; i < k; i++) {
            current = pq.poll();
        }
        return new int[]{current.first, current.second};
    }

    private class Data {
        int first;
        int second;
        double val;

        @Override
        public String toString() {
            return "Data{" +
                   "first=" + first +
                   ", second=" + second +
                   ", val=" + val +
                   '}';
        }
    }

    public int[] kthSmallestPrimeFractionOptimized(int[] arr, int k) {
        // Create a priority queue to store pairs of fractions
        PriorityQueue<double[]> pq = new PriorityQueue<>((a, b) -> Double.compare(b[0],
a[0]));
```

```
        // Push the fractions formed by dividing each element by
        // the largest element into the priority queue
        for (int i = 0; i < arr.length; i++) {
            pq.offer(new double[]{
                    -1.0 * arr[i] / arr[arr.length - 1],
                    i,
                    arr.length - 1
            });
        }

        // Iteratively remove the top element (smallest fraction)
        // and replace it with the next smallest fraction
        while (--k > 0) {
            double[] cur = pq.poll();
            int numeratorIndex = (int) cur[1];
            int denominatorIndex = (int) cur[2] - 1;
            if (denominatorIndex > numeratorIndex) {
                pq.offer(new double[]{
                        -1.0 * arr[numeratorIndex] / arr[denominatorIndex],
                        numeratorIndex,
                        denominatorIndex
                });
            }
        }

        // Retrieve the kth smallest fraction from
        // the top of the priority queue
        double[] result = pq.poll();
        return new int[]{arr[(int) result[1]], arr[(int) result[2]]};
    }

}
```

208. `Solution00787.java`

```
package leetcode;

import java.util.Arrays;

/**
 * https://leetcode.com/problems/cheapest-flights-within-k-stops/description/
 * 787. Cheapest Flights Within K Stops
 * Medium
 */

public class Solution00787 {

    // Time: O(n * k)
    // Space: O(n)
    public int findCheapestPrice(int n, int[][] flights, int src, int dst, int k) {
        int[] prices = new int[n];
        Arrays.fill(prices, Integer.MAX_VALUE);
        prices[src] = 0;
        for (int i = 0; i <= k; i++) {
            int[] temp = Arrays.copyOf(prices, n);
            for (int[] flight : flights) {
                int from = flight[0];
                if (prices[from] == Integer.MAX_VALUE) {
                    continue;
                }
                int to = flight[1];
                int price = flight[2];
                temp[to] = Math.min(temp[to], prices[from] + price);
            }
            prices = temp;
        }
        return prices[dst] == Integer.MAX_VALUE ? -1 : prices[dst];
    }

}
```

209. `Solution00791.java`

```
package leetcode;

/**
 * Problem: 791. Custom Sort String
 * <a href="https://leetcode.com/problems/custom-sort-string">Web link</a>
 * Medium
 */

public class Solution00791 {

    // Time: O(n + m)
    // Space: O(n + m)
    public String customSortString(String order, String s) {
        int[] count = new int[26];
        for (char c : s.toCharArray()) {
            count[c - 'a']++;
        }
        StringBuilder sb = new StringBuilder();
        for (char c : order.toCharArray()) {
            final int orderIndex = c - 'a';
            final int currentCount = count[orderIndex];
            if (currentCount > 0) {
                sb.repeat(c, currentCount);
                count[orderIndex] = 0;
            }
        }
        for (char c = 'a'; c <= 'z'; c++) {
            final int orderIndex = c - 'a';
            final int currentCount = count[orderIndex];
            if (currentCount > 0) {
                sb.repeat(c, currentCount);
            }
        }
        return sb.toString();
    }

}
```

210. `Solution00796.java`

```
/**
 * 796. Rotate String
 * <a href="https://leetcode.com/problems/rotate-string">Web link</a> |
 * Easy
 */
package leetcode;

public class Solution00796 {

    public boolean rotateString(String s, String goal) {
        if (s.length() != goal.length()) {
            return false;
        }
        return (s + s).contains(goal);
    }

}
```

211. `Solution00826.java`

```
package leetcode;

import javafx.util.Pair;

import java.util.*;
import java.util.stream.IntStream;

/**
 * 826. Most Profit Assigning Work
 * https://leetcode.com/problems/most-profit-assigning-work/description/
 * Medium
 */

public class Solution00826 {

    public int maxProfitAssignment(int[] difficulty, int[] profit, int[] worker) {
        NavigableMap<Integer, Integer> profitByDifficulty = new TreeMap<>();
        final int n = difficulty.length;
        List<Pair<Integer, Integer>> projects = IntStream.range(0, n)
                .mapToObj(i -> new Pair<>(difficulty[i], profit[i]))
                .sorted(Comparator.comparingInt(Pair::getKey))
                .toList();

        for (Pair<Integer, Integer> project : projects) {
            final int currentDifficulty = project.getKey();
            final int currentProfit = project.getValue();
            final Map.Entry<Integer, Integer> floorEntry =
profitByDifficulty.floorEntry(currentDifficulty);
            if (floorEntry == null) {
                profitByDifficulty.put(currentDifficulty, currentProfit);
            } else {
                profitByDifficulty.put(currentDifficulty, Math.max(floorEntry.getValue(),
currentProfit));
            }
        }

        int totalProfit = 0;
        for (int workerAbility : worker) {
            final Map.Entry<Integer, Integer> floorEntry =
profitByDifficulty.floorEntry(workerAbility);
            if (floorEntry != null) {
                final int currentProfit = floorEntry.getValue();
                totalProfit += currentProfit;
            }
        }
        return totalProfit;
    }

}
```

212. `Solution00834.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 834. Sum of Distances in Tree
 * <a href="https://leetcode.com/problems/sum-of-distances-in-tree/">Web link</a> |
 * Hard
 */

public class Solution00834 {

    public int[] sumOfDistancesInTree(int n, int[][] edges) {
        List<Integer>[] adj = new List[n];
        for (int i = 0; i < n; i++) {
            adj[i] = new ArrayList<>();
        }

        for (int[] edge : edges) {
            adj[edge[0]].add(edge[1]);
            adj[edge[1]].add(edge[0]);
        }

        int[] count = new int[n];
        int[] res = new int[n];
        Arrays.fill(count, 1);

        postOrder(0, -1, adj, count, res);
        preOrder(0, -1, adj, count, res, n);

        return res;
    }

    private void postOrder(int node, int parent, List<Integer>[] adj, int[] count, int[]
res) {
        for (int neighbor : adj[node]) {
            if (neighbor == parent) {
                continue;
            }
            postOrder(neighbor, node, adj, count, res);
            count[node] += count[neighbor];
            res[node] += res[neighbor] + count[neighbor];
        }
    }

    private void preOrder(int node, int parent, List<Integer>[] adj, int[] count, int[]
res, int n) {
        for (int neighbor : adj[node]) {
            if (neighbor == parent) {
```

```
                continue;
            }
            res[neighbor] = res[node] - count[neighbor] + n - count[neighbor];
            preOrder(neighbor, node, adj, count, res, n);
        }
    }

    public int[] sumOfDistancesInTree2(int n, int[][] edges) {
        int[] count = new int[n];
        int[] res = new int[n];
        Arrays.fill(count, 1);
        List<List<Integer>> arr = new ArrayList<>();
        for (int i = 0; i < n; i++) arr.add(new ArrayList<>());
        for (int[] edge : edges) {
            arr.get(edge[0]).add(edge[1]);
            arr.get(edge[1]).add(edge[0]);
        }
        getCounts(arr, 0, -1, count, res);
        buildSum(arr, 0, -1, count, res);
        return res;
    }

    //This is used to get counts of nodes in subtrees and the correct result of root
node(0)
    void getCounts(List<List<Integer>> adj, int cur, int par, int[] count, int[] res) {
        for (int ch : adj.get(cur)) {
            if (ch == par) continue;
            getCounts(adj, ch, cur, count, res);
            count[cur] += count[ch];
            //Here we are formulating res array, but res for only parent is calculated
correct
            res[cur] += res[ch] + count[ch];
        }
    }

    //This is used to get correct res for all other nodes after knowing res of root
    // res[X] = res[Y(parent in this case)] + N - 2*count[X]
    void buildSum(List<List<Integer>> adj, int cur, int par, int[] count, int[] res) {
        for (int ch : adj.get(cur)) {
            if (ch == par) continue;
            res[ch] = res[cur] + adj.size() - 2 * count[ch];
            buildSum(adj, ch, cur, count, res);
        }
    }
}
```

213. `Solution00840.java`

```
package leetcode;

/**
 * 840. Magic Squares In Grid
 * <a href="https://leetcode.com/problems/magic-squares-in-grid">Web link</a>
 * Medium
 */

public class Solution00840 {

    public int numMagicSquaresInside(int[][] grid) {
        int rows = grid.length;
        int cols = grid[0].length;

        int result = 0;
        for (int i = 2; i < rows; i++) {
            for (int j = 2; j < cols; j++) {
                if (isMagicSquare(grid, i, j)) {
                    result++;
                }
            }
        }

        return result;
    }

    private boolean isMagicSquare(int[][] grid, int i, int j) {
        int[] count = new int[15];
        for (int k = 0; k < 3; k++) {
            for (int l = 0; l < 3; l++) {
                int num = grid[i - 2 + k][j - 2 + l];
                if (num < 1 || num > 9 || count[num] > 0) {
                    return false;
                }
                count[num]++;
            }
        }

        int diagonal0 = grid[i - 2][j - 2] + grid[i - 1][j - 1] + grid[i][j];
        int diagonal1 = grid[i][j - 2] + grid[i - 1][j - 1] + grid[i - 2][j];
        if (diagonal0 != diagonal1) {
            return false;
        }
        for (int k = -2; k <= 0; k++) {
            int row = i + k;
            int rowSum = grid[row][j - 2] + grid[row][j - 1] + grid[row][j];
            if (rowSum != diagonal0) {
                return false;
            }
            int col = j + k;
            int colSum = grid[i - 2][col] + grid[i - 1][col] + grid[i][col];
```

```
            if (colSum != diagonal0) {
                return false;
            }
        }
        return true;
    }

}
```

214. `Solution00846.java`

```
package leetcode;

import java.util.*;

/**
 * 846. Hand of Straights
 * <a href="https://leetcode.com/problems/hand-of-straights">Web link</a> |
 * Medium
 */

public class Solution00846 {

    public boolean isNStraightHand(int[] hand, int groupSize) {
        if (groupSize == 1) {
            return true;
        }
        final int len = hand.length;
        if (len % groupSize != 0) {
            return false;
        }
        Map<Integer, Integer> countMap = new TreeMap<>();
        for (int card : hand) {
            countMap.put(card, countMap.getOrDefault(card, 0) + 1);
        }
        for (int i = 0; i < len / groupSize; i++) {
            final Iterator<Integer> iterator = countMap.keySet().iterator();
            if (!iterator.hasNext()) {
                break;
            }
            int firstCard = iterator.next();
            int countFirst = countMap.get(firstCard);
            for (int j = 0; j < groupSize; j++) {
                int card = firstCard + j;
                if (countMap.getOrDefault(card, 0) < countFirst) {
                    return false;
                }
                countMap.put(card, countMap.get(card) - countFirst);
                if (countMap.get(card) == 0) {
                    countMap.remove(card);
                }
            }
        }
        return countMap.isEmpty();
    }

    public boolean isNStraightHandOptimized(int[] hand, int groupSize) {
        if (groupSize == 1) {
            return true;
        }
        final int len = hand.length;
        if (len % groupSize != 0) {
```

```
            return false;
        }
        Arrays.sort(hand);
        Map<Integer, Integer> countMap = new HashMap<>();
        for (int card : hand) {
            countMap.put(card, countMap.getOrDefault(card, 0) + 1);
        }
        for (int i = 0; i < len; ) {
            int firstCard = hand[i];
            int countFirst = countMap.getOrDefault(firstCard, 0);
            if (countFirst == 0) {
                i++;
                continue;
            }
            for (int j = 0; j < groupSize; j++) {
                int card = firstCard + j;
                if (countMap.getOrDefault(card, 0) < countFirst) {
                    return false;
                }
                countMap.put(card, countMap.get(card) - countFirst);
                if (countMap.get(card) == 0) {
                    countMap.remove(card);
                }
            }
            i += countFirst;
        }
        return countMap.isEmpty();
    }

}
```

215. `Solution00853.java`

```
package leetcode;

import java.util.Comparator;
import java.util.Map;
import java.util.TreeMap;

/**
 * https://leetcode.com/problems/car-fleet/
 * 853. Car Fleet
 * <p>
 * N cars are going to the same destination along a one lane road.  The destination is
target miles away.
 * <p>
 * Each car i has a constant speed speed[i] (in miles per hour), and initial position
position[i] miles towards the target along the road.
 * <p>
 * A car can never pass another car ahead of it, but it can catch up to it, and drive
bumper to bumper at the same speed.
 * <p>
 * The distance between these two cars is ignored - they are assumed to have the same
position.
 * <p>
 * A car fleet is some non-empty set of cars driving at the same position and same speed.
Note that a single car is also a car fleet.
 * <p>
 * If a car catches up to a car fleet right at the destination point, it will still be
considered as one car fleet.
 * <p>
 * How many car fleets will arrive at the destination?
 * <p>
 * Example 1:
 * <p>
 * Input: target = 12, position = [10,8,0,5,3], speed = [2,4,1,1,3] Output: 3
Explanation: The cars starting at 10 and 8 become a fleet, meeting each other at 12.
 * The car starting at 0 doesn't catch up to any other car, so it is a fleet by itself.
The cars starting at 5 and 3 become a fleet, meeting each other at 6.
 * Note that no other cars meet these fleets before the destination, so the answer is 3.
Example 2:
 * <p>
 * Input: target = 10, position = [3], speed = [3] Output: 1
 * <p>
 * Note:
 * <p>
 * 0 <= N <= 10 ^ 4 0 < target <= 10 ^ 6 0 < speed[i] <= 10 ^ 6 0 <= position[i] < target
All initial positions are different.
 */
public class Solution00853 {

    // Time: O(nlogn)
    // Space: O(n)
    public int carFleet(int target, int[] position, int[] speed) {
```

```
        // Create a map to keep track of the position and speed.
        Map<Integer, Integer> map = new TreeMap<>(Comparator.reverseOrder());

        // Loop through the position array.
        for (int i = 0; i < position.length; i++) {
            // Put the position and speed in the map.
            map.put(position[i], speed[i]);
        }

        // Create a variable to keep track of the number of fleets.
        int fleet = 0;

        // Create a variable to keep track of the time.
        double time = 0;

        // Loop through the position array.
        for (var entry : map.entrySet()) {
            // Calculate the time.
            final int currentSpeed = entry.getValue();
            double currentTime = (double) (target - entry.getKey()) / currentSpeed;

            // If the current time is greater than the time.
            if (currentTime > time) {
                // Update the time.
                time = currentTime;

                // Increment the result.
                fleet++;
            }
        }

        return fleet;
    }

}
```

216. `Solution00857.java`

```
package leetcode;

import java.util.Arrays;
import java.util.Comparator;
import java.util.PriorityQueue;
import java.util.Queue;

/**
 * 857. Minimum Cost to Hire K Workers
 * https://leetcode.com/problems/minimum-cost-to-hire-k-workers/
 * Hard
 */

public class Solution00857 {

    public double mincostToHireWorkers(int[] quality, int[] wage, int k) {
        int n = quality.length;
        Worker[] workers = new Worker[n];
        for (int i = 0; i < n; i++) {
            workers[i] = new Worker(wage[i], quality[i]);
        }
        Arrays.sort(workers, Comparator.comparingDouble(a -> a.ratePerQuality));

        double result = Double.MAX_VALUE;
        int qualitySum = 0;
        Queue<Worker> qualityQueue = new PriorityQueue<>((a, b) ->
Integer.compare(b.quality, a.quality));
        for (Worker worker : workers) {
            qualityQueue.add(worker);
            qualitySum += worker.quality;
            if (qualityQueue.size() == k) {
                final double biggestRateByNow = worker.ratePerQuality;
                final double resultCandidate = qualitySum * biggestRateByNow;
                result = Math.min(result, resultCandidate);
                Worker biggestQualityWorker = qualityQueue.poll();
                qualitySum -= biggestQualityWorker.quality;
            }
        }
        return result;
    }

    private static class Worker {
        double ratePerQuality;
        int quality;

        public Worker(int wage, int quality) {
            this.ratePerQuality = (double) wage / quality;
            this.quality = quality;
        }

        @Override
```

```
        public String toString() {
            return "Data{" +
                   "ratePerQuality=" + ratePerQuality +
                   ", quality=" + quality +
                   '}';
        }
    }

}
```

217. `Solution00860.java`

```
package leetcode;

/**
 * 860. Lemonade Change
 * <a href="https://leetcode.com/problems/lemonade-change">Web link</a>
 * Easy
 */

public class Solution00860 {

    public boolean lemonadeChange(int[] bills) {
        int five = 0;
        int ten = 0;
        for (int bill : bills) {
            if (bill == 5) {
                five++;
            } else if (bill == 10) {
                if (five == 0) {
                    return false;
                }
                five--;
                ten++;
            } else if (bill == 20) {
                if (ten >= 1 && five >= 1) {
                    ten--;
                    five--;
                } else if (five >= 3) {
                    five -= 3;
                } else {
                    return false;
                }
            }
        }
        return true;
    }

}
```

218. `Solution00861.java`

```
package leetcode;

/**
 * 861. Score After Flipping Matrix
 * https://leetcode.com/problems/score-after-flipping-matrix/
 * Medium
 */

public class Solution00861 {

    public int matrixScore(int[][] grid) {
        int rowCount = grid.length;
        int columnCount = grid[0].length;

        int score = (1 << (columnCount - 1)) * rowCount;

        for (int j = 1; j < columnCount; j++) {
            int countSameBitsAsFirstInRow = 0;
            for (int i = 0; i < rowCount; i++) {
                if (grid[i][j] == grid[i][0]) {
                    countSameBitsAsFirstInRow++;
                }
            }

            int maxSameBitsInColumn = Math.max(countSameBitsAsFirstInRow, rowCount -
countSameBitsAsFirstInRow);
            int columnScore = (1 << (columnCount - j - 1)) * maxSameBitsInColumn;

            score += columnScore;
        }

        return score;
    }

}
```

219. `Solution00862.java`

```
/**
 * 862. Shortest Subarray with Sum at Least K
 * <a href="https://leetcode.com/problems/shortest-subarray-with-sum-at-least-k">Web
link</a> |
 * Hard
 */

package leetcode;

import java.util.ArrayDeque;
import java.util.Deque;

public class Solution00862 {

    public int shortestSubarray(int[] nums, int k) {
        int n = nums.length;
        long[] prefix = new long[n + 1];

        // Step 1: Compute prefix sums
        for (int i = 0; i < n; i++) {
            prefix[i + 1] = prefix[i] + nums[i];
        }

        Deque<Integer> deque = new ArrayDeque<>();
        int minLength = Integer.MAX_VALUE;

        // Step 2: Process prefix sums
        for (int i = 0; i <= n; i++) {
            while (!deque.isEmpty() && prefix[i] - prefix[deque.peekFirst()] >= k) {
                minLength = Math.min(minLength, i - deque.pollFirst());
            }

            while (!deque.isEmpty() && prefix[i] <= prefix[deque.peekLast()]) {
                deque.pollLast();
            }

            deque.addLast(i);
        }

        return minLength == Integer.MAX_VALUE ? -1 : minLength;
    }

}
```

220. `Solution00872.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.ArrayList;
import java.util.List;

/**
 * https://leetcode.com/problems/leaf-similar-trees/
 * 872. Leaf-Similar Trees
 * Easy
 * Consider all the leaves of a binary tree, from left to right order, the values of
those leaves form a leaf value sequence.
 * For example, in the given tree above, the leaf value sequence is (6, 7, 4, 9, 8).
 * Two binary trees are considered leaf-similar if their leaf value sequence is the same.
 * Return true if and only if the two given trees with head nodes root1 and root2 are
leaf-similar.
 * Example 1:
 * Input: root1 = [3,5,1,6,2,9,8,null,null,7,4], root2 =
[3,5,1,6,7,4,2,null,null,null,null,null,null,9,8]
 * Output: true
 * Example 2:
 * Input: root1 = [1], root2 = [1]
 * Output: true
 * Example 3:
 * Input: root1 = [1], root2 = [2]
 * Output: false
 * Example 4:
 * Input: root1 = [1,2], root2 = [2,2]
 * Output: true
 * Example 5:
 * Input: root1 = [1,2,3], root2 = [1,3,2]
 * Output: false
 * Constraints:
 * The number of nodes in each tree will be in the range [1, 200].
 * Both of the given trees will have values in the range [0, 200].
 */
public class Solution00872 {

    // Time O(n)
    // Space O(n)
    public boolean leafSimilarDfs(TreeNode root1, TreeNode root2) {
        List<Integer> leafValues1 = new ArrayList<>();
        List<Integer> leafValues2 = new ArrayList<>();
        dfs(root1, leafValues1);
        dfs(root2, leafValues2);
        return leafValues1.equals(leafValues2);
    }

    private void dfs(TreeNode root, List<Integer> leafValues) {
        if (root == null) {
```

```
            return;
        }

        if (root.left == null && root.right == null) {
            leafValues.add(root.val);
        }

        dfs(root.left, leafValues);
        dfs(root.right, leafValues);
    }

}
```

221. `Solution00874.java`

```
package leetcode;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 874. Walking Robot Simulation
 * <a href="https://leetcode.com/problems/walking-robot-simulation">Web link</a>
 * Medium
 */

public class Solution00874 {

    private static final int TURN_LEFT = -1;
    private static final int TURN_RIGHT = -2;

    public int robotSim(int[] commands, int[][] obstacles) {
        int[][] directions = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        int[] curPos = {0, 0};
        int ans = 0;
        int curDir = 0;
        Map<Integer, Set<Integer>> obs = new HashMap<>();
        for (int[] obstacle : obstacles) {
            Set<Integer> setY = obs.computeIfAbsent(obstacle[0], k -> new HashSet<>());
            setY.add(obstacle[1]);
        }

        for (int command : commands) {
            switch (command) {
                case TURN_LEFT -> curDir = (curDir + 1) % 4;
                case TURN_RIGHT -> curDir = (curDir - 1 + 4) % 4;
                default -> {
                    int[] dir = directions[curDir];
                    for (int i = 0; i < command; i++) {
                        int nextX = curPos[0] + dir[0];
                        int nextY = curPos[1] + dir[1];
                        final Set<Integer> hasObstaclesX = obs.get(nextX);
                        if (hasObstaclesX != null && hasObstaclesX.contains(nextY)) {
                            break;
                        }
                        curPos[0] = nextX;
                        curPos[1] = nextY;
                    }
                    ans = Math.max(ans, curPos[0] * curPos[0] + curPos[1] * curPos[1]);
                }
            }
        }
        return ans;
    }
```

```
}
```

222. `Solution00875.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/koko-eating-bananas/
 * 875. Koko Eating Bananas
 * <p>
 * Koko loves to eat bananas. There are n piles of bananas, the ith pile has piles[i]
bananas.
 * The guards have gone and will come back in h hours.
 * <p>
 * Koko can decide her bananas-per-hour eating speed of k.
 * Each hour, she chooses some pile of bananas and eats k bananas from that pile.
 * If the pile has less than k bananas, she eats all of them instead and will not eat any
more bananas during this hour.
 * <p>
 * Koko likes to eat slowly but still wants to finish eating all the bananas before the
guards return.
 * <p>
 * Return the minimum integer k such that she can eat all the bananas within h hours.
 * <p>
 * Example 1:
 * <p>
 * Input: piles = [3,6,7,11], h = 8 Output: 4 Example 2:
 * <p>
 * Input: piles = [30,11,23,4,20], h = 5 Output: 30 Example 3:
 * <p>
 * Input: piles = [30,11,23,4,20], h = 6 Output: 23
 * <p>
 * <p>
 * Constraints:
 * <p>
 * 1 <= piles.length <= 104 piles.length <= h <= 109 1 <= piles[i] <= 109
 */
public class Solution00875 {

    // Time: O(nlogn)
    // Space: O(1)
    public int minEatingSpeed(int[] piles, int h) {
        // Create a variable to keep track of the left index.
        int left = 1;

        // Create a variable to keep track of the right index.
        int right = 0;

        // Loop through the piles array.
        for (int pile : piles) {
            // Update the right index.
            right = Math.max(right, pile);
        }

        // Loop while the left index is less than or equal to the right index.
```

```
        while (left <= right) {
            // Create a variable to keep track of the middle index.
            int mid = left + (right - left) / 2;

            // If the middle element is less than the target.
            if (canEatAll(piles, h, mid)) {
                // Update the right index.
                right = mid - 1;
            } else {
                // Update the left index.
                left = mid + 1;
            }
        }

        // Return the left index.
        return left;
    }

    private boolean canEatAll(int[] piles, int h, int k) {
        // Create a variable to keep track of the hours.
        long hours = 0;

        // Loop through the piles array.
        for (int pile : piles) {
            // Calculate the hours.
            // hours += (int) Math.ceil((double) pile / k); // runtime 21 ms (slowest)
            hours += (pile + k - 1) / k; // runtime 6 ms (faster)
            //hours += pile / k; // runtime 12 ms (medium)
            //if (pile % k != 0) {
            //    hours++;
            //}
        }

        // Return if the hours is less than or equal to the target.
        return hours <= h;
    }
}
```

223. `Solution00876.java`

```
package leetcode;

import leetcode.common.ListNode;

/**
 * 876. Middle of the Linked List
 * <a href="https://leetcode.com/problems/middle-of-the-linked-list/">...</a>
 * Easy
 */
public class Solution00876 {

    // Time: O(n)
    // Space: O(1)
    public ListNode middleNode(ListNode head) {
        ListNode slow = head;
        ListNode fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }

}
```

224. `Solution00881.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 881. Boats to Save People
 * <a href="https://leetcode.com/problems/boats-to-save-people">Web link</a>
 * Medium
 */

public class Solution00881 {

    // Time complexity: O(n log n)
    // Space complexity: O(1)
    public int numRescueBoats(int[] people, int limit) {
        Arrays.sort(people);
        int boats = 0;
        int left = 0;
        int right = people.length - 1;
        while (left <= right) {
            if (people[left] + people[right] <= limit) {
                left++;
            }
            right--;
            boats++;
        }
        return boats;
    }

    // Time complexity: O(n)
    // Space complexity: O(n)
    public int numRescueBoatsBucketSort(int[] people, int limit) {
        int[] buckets = new int[limit + 1];
        for (int p : people) {
            buckets[p]++;
        }

        int left = 0;
        int right = limit;
        int boats = 0;

        while (left <= right) {
            while (buckets[left] == 0 && left <= right) {
                left++;
            }
            while (buckets[right] == 0 && left <= right) {
                right--;
            }
            if (left > right) {
                break;
            }
```

```
            if (left == right) {
                if (left * 2 > limit) {
                    boats += buckets[left];
                    break;
                } else {
                    boats += buckets[left] / 2 + buckets[left] % 2;
                    break;
                }
            }
            boolean hasPairs = false;
            while (left + right <= limit && buckets[left] > 0 && buckets[right] > 0) {
                int min = Math.min(buckets[left], buckets[right]);
                buckets[left] -= min;
                buckets[right] -= min;
                boats += min;
                hasPairs = true;
            }
            if (!hasPairs) {
                boats += buckets[right]; // if cannot be paired with smallest left,
cannot be paired with any other
                buckets[right] = 0;
            }
        }

        return boats;
    }

}
```

225. `Solution00884.java`

```
package leetcode;

import java.util.HashMap;
import java.util.Map;

/**
 * 884. Uncommon Words from Two Sentences
 * <a href="https://leetcode.com/problems/uncommon-words-from-two-sentences">Web link</a>
 * Easy
 */

public class Solution00884 {

    public String[] uncommonFromSentences(String s1, String s2) {
        Map<String, Integer> map = new HashMap<>();
        for (String word : s1.split(" ")) {
            map.merge(word, 1, Integer::sum);
        }
        for (String word : s2.split(" ")) {
            map.merge(word, 1, Integer::sum);
        }
        return map.entrySet().stream()
                .filter(entry -> entry.getValue() == 1)
                .map(Map.Entry::getKey)
                .toArray(String[]::new);
    }

}
```

226. `Solution00885.java`

```
package leetcode;

/**
 * 885. Spiral Matrix III
 * <a href="https://leetcode.com/problems/spiral-matrix-iii">Web link</a>
 * Medium
 */

public class Solution00885 {

    public int[][] spiralMatrixIII(int rows, int cols, int rStart, int cStart) {
        int len = rows * cols;
        int[][] result = new int[len][2];
        int[][] direction = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

        int r = rStart;
        int c = cStart;
        int steps = 1;
        int i = 0;
        int added = 0;

        while (added < len) {
            for (int j = 0; j < 2; j++) {
                for (int k = 0; k < steps; k++) {
                    if (r >= 0 && r < rows && c >= 0 && c < cols) {
                        result[added][0] = r;
                        result[added][1] = c;
                        added++;
                    }
                    r += direction[i][0];
                    c += direction[i][1];
                }
                i = (i + 1) % 4;
            }
            steps++;
        }
        return result;
    }

}
```

227. `Solution00907.java`

```
package leetcode;

import java.util.Arrays;
import java.util.Deque;
import java.util.LinkedList;

/**
 * 907. Sum of Subarray Minimums
 * https://leetcode.com/problems/sum-of-subarray-minimums/
 * #medium
 * Given an array of integers arr, find the sum of min(b), where b ranges over every
(contiguous) subarray of arr. Since the answer may be large, return the answer modulo 109
+ 7.
 * <p>
 * Example 1:
 * <p>
 * Input: arr = [3,1,2,4]
 * Output: 17
 * Explanation:
 * Subarrays are [3], [1], [2], [4], [3,1], [1,2], [2,4], [3,1,2], [1,2,4], [3,1,2,4].
 * Minimums are 3, 1, 2, 4, 1, 1, 2, 1, 1, 1.
 * Sum is 17.
 * Example 2:
 * <p>
 * Input: arr = [11,81,94,43,3]
 * Output: 444
 * <p>
 * Constraints:
 * <p>
 * 1 <= arr.length <= 3 * 104
 * 1 <= arr[i] <= 3 * 104
 */
public class Solution00907 {

    public int sumSubarrayMins(int[] arr) {
        int length = arr.length;

        long[] left = new long[length];
        long[] right = new long[length];
        Arrays.fill(left, -1);
        Arrays.fill(right, length);

        Deque<Integer> stack = new LinkedList<>();
        for (int i = 0; i < length; i++) {
            while (!stack.isEmpty() && arr[stack.peek()] > arr[i]) {
                stack.pop();
            }
            if (!stack.isEmpty()) {
                left[i] = stack.peek();
            }
            stack.push(i);
```

```
        }

        stack.clear();

        for (int i = length - 1; i >= 0; i--) {
            while (!stack.isEmpty() && arr[stack.peek()] >= arr[i]) {
                stack.pop();
            }
            if (!stack.isEmpty()) {
                right[i] = stack.peek();
            }
            stack.push(i);
        }

        long result = 0;
        for (int i = 0; i < length; i++) {
            result += arr[i] * (i - left[i]) * (right[i] - i);
            result %= 1000000007;
        }
        return (int) result;
    }

}
```

228. `Solution00912.java`

```
package leetcode;

/**
 * 912. Sort an Array
 * <a href="https://leetcode.com/problems/sort-an-array">Web link</a>
 * Medium
 */

public class Solution00912 {

    public int[] sortArray(int[] nums) {
        quickSort(nums, 0, nums.length - 1);
        return nums;
    }

    private void quickSort(int[] nums, int start, int end) {
        if (start >= end) {
            return;
        }
        int pivot = partition(nums, start, end);
        quickSort(nums, start, pivot - 1);
        quickSort(nums, pivot + 1, end);
    }

    private int partition(int[] nums, int start, int end) {
        int pivot = nums[end];
        int i = start - 1;
        for (int j = start; j < end; j++) {
            if (nums[j] < pivot) {
                i++;
                swap(nums, i, j);
            }
        }
        swap(nums, i + 1, end);
        return i + 1;
    }

    private void swap(int[] nums, int i, int end) {
        int temp = nums[i];
        nums[i] = nums[end];
        nums[end] = temp;
    }

    public int[] sortArrayCountingSort(int[] nums) {
        if (nums == null || nums.length <= 1) return nums;

        int max = nums[0];
        int min = nums[0];
        for (int num : nums) {
            if (num > max) max = num;
            if (num < min) min = num;
```

```
        }

        int range = max - min + 1;
        int[] count = new int[range];

        for (int num : nums) {
            count[num - min]++;
        }

        int index = 0;
        for (int i = 0; i < range; i++) {
            int freq = count[i];
            if (freq > 0) {
                int value = i + min;
                while (freq-- > 0) {
                    nums[index++] = value;
                }
            }
        }

        return nums;
    }

}
```

229. `Solution00916.java`

```
/**
 * 916. Word Subsets
 * <a href="https://leetcode.com/problems/word-subsets">Web link</a>
 * Medium
 */

package leetcode;

import java.util.ArrayList;
import java.util.List;

public class Solution00916 {

    public List<String> wordSubsetsTle(String[] words1, String[] words2) {
        int[][] countWords2 = new int[words1.length][26];
        for (int i = 0; i < words2.length; i++) {
            final String word2 = words2[i];
            for (int j = 0; j < word2.length(); j++) {
                countWords2[i][word2.charAt(j) - 'a']++;
            }
        }
        List<String> result = new ArrayList<>();
        for (final String word1 : words1) {
            if (isUniversal(word1, countWords2)) {
                result.add(word1);
            }
        }
        return result;
    }

    private boolean isUniversal(String word1, int[][] countWords2) {
        int[] countWord1 = new int[26];
        for (int j = 0; j < word1.length(); j++) {
            countWord1[word1.charAt(j) - 'a']++;
        }
        for (int[] countWord2 : countWords2) {
            for (int k = 0; k < 26; k++) {
                if (countWord2[k] > countWord1[k]) {
                    return false;
                }
            }
        }
        return true;
    }

    public List<String> wordSubsets(String[] words1, String[] words2) {
        int[] countWords2Max = new int[26];
        for (String word2 : words2) {
            int[] countWords2 = new int[26];
            for (int j = 0; j < word2.length(); j++) {
                int idx = word2.charAt(j) - 'a';
```

```
                countWords2[idx]++;
                countWords2Max[idx] = Math.max(countWords2Max[idx], countWords2[idx]);
            }
        }
        List<String> result = new ArrayList<>();
        for (final String word1 : words1) {
            if (isUniversal(word1, countWords2Max)) {
                result.add(word1);
            }
        }
        return result;
    }

    private boolean isUniversal(String word1, int[] countWords2Max) {
        int[] countWord1 = new int[26];
        for (int j = 0; j < word1.length(); j++) {
            countWord1[word1.charAt(j) - 'a']++;
        }
        for (int k = 0; k < 26; k++) {
            if (countWords2Max[k] > countWord1[k]) {
                return false;
            }
        }
        return true;
    }

}
```

230. `Solution00921.java`

```
package leetcode;

/**
 * 921. Minimum Add to Make Parentheses Valid
 * <a href="https://leetcode.com/problems/minimum-add-to-make-parentheses-valid">Web
link</a>
 * Medium
 */

public class Solution00921 {

    public int minAddToMakeValid(String s) {
        int o = 0, c = 0;
        for (char ch : s.toCharArray()) {
            if (ch == '(') {
                o++;
            } else if (o > 0) {
                o--;
            } else {
                c++;
            }
        }
        return o + c;
    }

}
```

231. `Solution00930.java`

```
package leetcode;

/**
 * 930. Binary Subarrays With Sum
 * <a href="https://leetcode.com/problems/binary-subarrays-with-sum">Web link</a>
 * Medium
 */

public class Solution00930 {

    // Time: O(n)
    // Space: O(n)
    public int numSubarraysWithSum(int[] nums, int goal) {
        int n = nums.length;
        int[] prefixSum = new int[n + 1];
        prefixSum[0] = 1;
        int sum = 0;
        int result = 0;
        for (int num : nums) {
            sum += num;
            if (sum >= goal) {
                result += prefixSum[sum - goal];
            }
            prefixSum[sum]++;
        }
        return result;
    }

    public int numSubarraysWithSumAlt(int[] nums, int goal) {
        int x = solve(nums, goal);
        int y = solve(nums, goal - 1);
        return x - y;
    }

    public int solve(int[] nums, int k) {
        if (k < 0) {
            return 0;
        }
        int s = 0;
        int e = 0;
        int sum = 0;
        int ans = 0;
        while (e < nums.length) {
            sum += nums[e];
            while (sum > k && s < nums.length) {
                sum -= nums[s];
                s++;
            }
            if (sum <= k) {
                ans += e - s + 1;
            }
```

```
            e++;
        }
        return ans;
    }
}
```

232. `Solution00931.java`

```
package leetcode;

import java.util.Arrays;

/**
 * https://leetcode.com/problems/minimum-falling-path-sum/
 * 931. Minimum Falling Path Sum
 * Medium
 * Given an n x n array of integers matrix, return the minimum sum of any falling path
through matrix.
 * <p>
 * A falling path starts at any element in the first row and chooses the element in the
next row that is either directly below or diagonally left/right. Specifically, the next
element from position (row, col) will be (row + 1, col - 1), (row + 1, col), or (row + 1,
col + 1).
 * <p>
 * Example 1:
 * Input: matrix = [[2,1,3],[6,5,4],[7,8,9]]
 * Output: 13
 * <p>
 * Example 2:
 * Input: matrix = [[-19,57],[-40,-5]]
 * Output: -59
 * <p>
 * Example 3:
 * Input: matrix = [[-48]]
 * Output: -48
 * <p>
 * Constraints:
 * n == matrix.length
 * n == matrix[i].length
 * 1 <= n <= 100
 * -100 <= matrix[i][j] <= 100
 */
public class Solution00931 {

    // Time: O(n^2)
    // Space: O(n^2)
    public static int minFallingPathSum(int[][] matrix) {
        int size = matrix.length;
        int lastIndex = size - 1;
        int[][] dp = new int[size][size];

        // copy first row
        dp[0] = Arrays.copyOf(matrix[0], size);

        // fill dp, row by row starting from second row
        for (int row = 1; row < size; row++) {
            for (int column = 0; column < size; column++) {
                final int min;
                final int[] previousDpRow = dp[row - 1];
```

```
                if (column == 0) { // first column
                    min = Math.min(previousDpRow[0], previousDpRow[1]);
                } else if (column == lastIndex) { // last column
                    min = Math.min(previousDpRow[lastIndex - 1],
previousDpRow[lastIndex]);
                } else {
                    min = Math.min(Math.min(previousDpRow[column - 1],
previousDpRow[column]), previousDpRow[column + 1]);
                }
                dp[row][column] = min + matrix[row][column];
            }
        }

        // find min in last row
        int min = dp[lastIndex][0];
        for (int i = 1; i < size; ++i) {
            min = Math.min(min, dp[lastIndex][i]);
        }

        return min;
    }

}
```

233. `Solution00938.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.LinkedList;
import java.util.Queue;

/**
 * https://leetcode.com/problems/range-sum-of-bst/
 * 938. Range Sum of BST
 * Easy
 * Given the root node of a binary search tree, return the sum of values of all nodes
with a value in the range [low, high].
 * Example 1:
 * Input: root = [10,5,15,3,7,null,18], low = 7, high = 15
 * Output: 32
 * Example 2:
 * Input: root = [10,5,15,3,7,13,18,1,null,6], low = 6, high = 10
 * Output: 23
 * Constraints:
 * The number of nodes in the tree is in the range [1, 2 * 104].
 * 1 <= Node.val <= 105
 * 1 <= low <= high <= 105
 */
public class Solution00938 {

    // Time O(n)
    // Space O(n)
    public int rangeSumBstBfs(TreeNode root, int low, int high) {
        int sum = 0;

        Queue<TreeNode> queue = new LinkedList<>();
        queue.add(root);

        while (!queue.isEmpty()) {
            TreeNode curr = queue.poll();
            if (curr.val >= low && curr.val <= high) {
                sum += curr.val;
            }

            if (curr.left != null && curr.val > low) {
                queue.add(curr.left);
            }

            if (curr.right != null && curr.val < high) {
                queue.add(curr.right);
            }
        }

        return sum;
    }
```

```
    // Time O(n)
    // Space O(n)
    public int rangeSumBstDfs(TreeNode root, int low, int high) {
        if (root == null) {
            return 0;
        }

        int sum = 0;
        if (root.val >= low && root.val <= high) {
            sum += root.val;
        }

        if (root.val > low) {
            sum += rangeSumBstDfs(root.left, low, high);
        }

        if (root.val < high) {
            sum += rangeSumBstDfs(root.right, low, high);
        }

        return sum;
    }

}
```

234. `Solution00945.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 945. Minimum Increment to Make Array Unique
 * <a href="https://leetcode.com/problems/minimum-increment-to-make-array-unique/">Web
link</a> |
 * Medium
 */

public class Solution00945 {

    public int minIncrementForUnique(int[] nums) {
        int n = nums.length;
        Arrays.sort(nums);
        int moves = 0;
        int prev = nums[0];
        for (int i = 1; i < n; i++) {
            int curr = nums[i];
            if (curr <= prev) {
                moves += prev - curr + 1;
                prev++;
            } else {
                prev = curr;
            }
        }
        return moves;
    }

}
```

235. `Solution00947.java`

```
package leetcode;

/**
 * 947. Most Stones Removed with Same Row or Column
 * <a href="https://leetcode.com/problems/most-stones-removed-with-same-row-or-
column">Web link</a>
 * Medium
 */

public class Solution00947 {

    public int removeStones(int[][] stones) {
        int[] numOfIslands = {0};
        int[] parent = new int[20003];
        for (int[] stone : stones) {
            unionSets(stone[0] + 1, stone[1] + 10002, parent, numOfIslands);
        }
        return stones.length - numOfIslands[0];
    }

    public void unionSets(int a, int b, int[] parent, int[] numOfIslands) {
        int parA = findParent(a, parent, numOfIslands);
        int parB = findParent(b, parent, numOfIslands);
        if (parA != parB) {
            parent[parB] = parA;
            numOfIslands[0]--;
        }
    }

    public int findParent(int node, int[] parent, int[] numOfIslands) {
        if (parent[node] == 0) {
            parent[node] = node;
            numOfIslands[0]++;
        }
        if (parent[node] == node) {
            return node;
        }
        int par = findParent(parent[node], parent, numOfIslands);
        parent[node] = par;
        return par;
    }

}
```

236. `Solution00948.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 948. Bag of Tokens
 * https://leetcode.com/problems/bag-of-tokens/
 * Medium
 */
public class Solution00948 {

    // Time: O(nlogn)
    // Space: O(1)
    public int bagOfTokensScore(int[] tokens, int power) {
        Arrays.sort(tokens);
        int left = 0;
        int right = tokens.length - 1;
        int score = 0;
        int maxScore = 0;
        while (left <= right) {
            if (power >= tokens[left]) {
                power -= tokens[left];
                score++;
                maxScore = Math.max(maxScore, score);
                left++;
            } else if (score > 0) {
                power += tokens[right];
                score--;
                right--;
            } else {
                break;
            }
        }
        return maxScore;
    }

}
```

237. `Solution00950.java`

```
package leetcode;

import java.util.ArrayDeque;
import java.util.Arrays;
import java.util.Queue;

/**
 * https://leetcode.com/problems/reveal-cards-in-increasing-order/
 * 950. Reveal Cards In Increasing Order
 * Medium
 */

public class Solution00950 {
    public int[] deckRevealedIncreasing(int[] deck) {
        int n = deck.length;
        int[] res = new int[n];
        Queue<Integer> queue = new ArrayDeque<>(n);
        for (int i = 0; i < n; i++) {
            queue.add(i);
        }
        Arrays.sort(deck);
        for (int deckElement : deck) {
            final int idx = queue.poll();
            // put first element into the result
            res[idx] = deckElement;
            // put second element to the end of the queue
            if (!queue.isEmpty()) {
                queue.add(queue.poll());
            }
        }
        return res;
    }
}
```

238. `Solution00951.java`

```
/**
 * 951. Flip Equivalent Binary Trees
 * <a href="https://leetcode.com/problems/flip-equivalent-binary-trees">Web link</a>
 * Medium
 */

package leetcode;

import leetcode.common.TreeNode;

public class Solution00951 {

    public boolean flipEquiv(TreeNode root1, TreeNode root2) {
        if (root1 == null && root2 == null) {
            return true;
        }
        if (root1 == null || root2 == null) {
            return false;
        }
        if (root1.val != root2.val) {
            return false;
        }
        return (flipEquiv(root1.left, root2.left) && flipEquiv(root1.right, root2.right))
|| // without flip
                (flipEquiv(root1.left, root2.right) && flipEquiv(root1.right,
root2.left)); // or with flip
    }

}
```

239. `Solution00959.java`

```
package leetcode;

/**
 * 959. Regions Cut By Slashes
 * <a href="https://leetcode.com/problems/regions-cut-by-slashes">Web link</a>
 * Medium
 */

public class Solution00959 {

    public int regionsBySlashes(String[] grid) {
        int n = grid.length;
        int[][] graph = new int[3 * n][3 * n];

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                int r = i * 3;
                int c = j * 3;
                if (grid[i].charAt(j) == '\\') {
                    graph[r][c] = 1;
                    graph[r + 1][c + 1] = 1;
                    graph[r + 2][c + 2] = 1;
                } else if (grid[i].charAt(j) == '/') {
                    graph[r][c + 2] = 1;
                    graph[r + 1][c + 1] = 1;
                    graph[r + 2][c] = 1;
                }
            }
        }

        return numsOfIslands(graph);
    }

    private int numsOfIslands(int[][] graph) {
        int count = 0;
        int m = graph.length;
        int n = graph[0].length;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (graph[i][j] == 0) {
                    count++;
                    dfs(graph, i, j);
                }
            }
        }
        return count;
    }

    private void dfs(int[][] graph, int r, int c) {
        if (r < 0 || r >= graph.length ||
            c < 0 || c >= graph[0].length ||
```

```
            graph[r][c] == 1) {
            return;
        }
        graph[r][c] = 1;
        dfs(graph, r - 1, c);
        dfs(graph, r + 1, c);
        dfs(graph, r, c - 1);
        dfs(graph, r, c + 1);
    }

}
```

240. `Solution00962.java`

```
package leetcode;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * 962. Maximum Width Ramp
 * <a href="https://leetcode.com/problems/maximum-width-ramp">Web link</a>
 * Medium
 */

public class Solution00962 {

    public int maxWidthRamp(int[] nums) {
        int n = nums.length;
        Deque<Integer> stack = new ArrayDeque<>();
        for (int i = 0; i < n; i++) {
            if (stack.isEmpty() || nums[i] < nums[stack.peek()]) {
                stack.push(i);
            }
        }
        int maxWidth = 0;
        for (int j = n - 1; j >= 0; j--) {
            while (!stack.isEmpty() && nums[j] >= nums[stack.peek()]) {
                maxWidth = Math.max(maxWidth, j - stack.pop());
            }
        }
        return maxWidth;
    }
}
```

241. `Solution00974.java`

```
package leetcode;

import java.util.HashMap;

/**
 * 974. Subarray Sums Divisible by K
 * <a href="https://leetcode.com/problems/subarray-sums-divisible-by-k/">Web link</a> |
 * Medium
 */

public class Solution00974 {

    public int subarraysDivByK(int[] nums, int k) {
        // Initialize the cumulative sum of the array elements
        int cumulativeSum = 0;

        // Initialize the count of subarrays divisible by K
        int subarrayCount = 0;

        // Create a map to store the frequency of remainders
        var remainderFrequencyMap = new HashMap<Integer, Integer>();

        // Add the initial condition to the map
        remainderFrequencyMap.put(0, 1);

        // Iterate over the array
        for (int num : nums) {
            // Update the cumulative sum
            cumulativeSum += num;

            // Calculate the remainder of the cumulative sum divided by K
            int remainder = cumulativeSum % k;
            if (remainder < 0) {
                remainder += k; // Handle negative remainders
            }

            // Get the frequency of the current remainder from the map
            int remainderFrequency = remainderFrequencyMap.getOrDefault(remainder, 0);

            // Add the frequency of the current remainder to the count of subarrays
            subarrayCount += remainderFrequency;

            // Update the frequency of the current remainder in the map
            remainderFrequencyMap.put(remainder, remainderFrequency + 1);
        }

        // Return the count of subarrays divisible by K
        return subarrayCount;
    }

}
```

242. `Solution00977.java`

```
package leetcode;

/**
 * 977. Squares of a Sorted Array
 * https://leetcode.com/problems/squares-of-a-sorted-array/
 * Easy
 */

public class Solution00977 {

    // Time: O(n)
    // Space: O(n)
    public int[] sortedSquares(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];
        int left = 0;
        int right = n - 1;
        int leftSquare = nums[left] * nums[left];
        int rightSquare = nums[right] * nums[right];

        for (int i = n - 1; i > 0; i--) {
            if (leftSquare > rightSquare) {
                result[i] = leftSquare;
                left++;
                leftSquare = nums[left] * nums[left];
            } else {
                result[i] = rightSquare;
                right--;
                rightSquare = nums[right] * nums[right];
            }
        }
        result[0] = leftSquare;

        return result;
    }

}
```

243. `Solution00979.java`

```
package leetcode;

import leetcode.common.TreeNode;

/**
 * 979. Distribute Coins in Binary Tree
 * <a href="https://leetcode.com/problems/distribute-coins-in-binary-tree">Web link</a>
 * Medium
 */

public class Solution00979 {

    public int distributeCoins(TreeNode root) {
        int[] moves = {0};
        dfs(root, moves);
        return moves[0];
    }

    private int dfs(TreeNode node, int[] moves) {
        if (node == null) {
            return 0;
        }
        int left = dfs(node.left, moves);
        int right = dfs(node.right, moves);
        moves[0] += Math.abs(left) + Math.abs(right);
        return node.val + left + right - 1;
    }

}
```

244. `Solution00981.java`

```
package leetcode;

import java.util.*;

/**
 * https://leetcode.com/problems/time-based-key-value-store/
 * 981. Time Based Key-Value Store
 * Medium
 * <p>
 * Share Create a timebased key-value store class TimeMap, that supports two
 * operations.
 * <p>
 * 1. set(string key, string value, int timestamp)
 * <p>
 * Stores the key and value, along with the given timestamp. 2. get(string
 * key, int timestamp)
 * <p>
 * Returns a value such that set(key, value, timestamp_prev) was called
 * previously, with timestamp_prev <= timestamp. If there are multiple such
 * values, it returns the one with the largest timestamp_prev. If there are no
 * values, it returns the empty string ("").
 * <p>
 * <p>
 * Example 1:
 * <p>
 * Input: inputs = ["TimeMap","set","get","get","set","get","get"], inputs =
 * [[],["foo","bar",1],["foo",1],["foo",3],["foo","bar2",4],["foo",4],["foo",5]]
 * Output: [null,null,"bar","bar",null,"bar2","bar2"] Explanation: TimeMap
 * kv; kv.set("foo", "bar", 1); // store the key "foo" and value "bar" along
 * with timestamp = 1 kv.get("foo", 1); // output "bar" kv.get("foo", 3); //
 * output "bar" since there is no value corresponding to foo at timestamp 3 and
 * timestamp 2, then the only value is at timestamp 1 ie "bar" kv.set("foo",
 * "bar2", 4); kv.get("foo", 4); // output "bar2" kv.get("foo", 5); //output
 * "bar2"
 * <p>
 * Example 2:
 * <p>
 * Input: inputs = ["TimeMap","set","set","get","get","get","get","get"], inputs
 * =
 * [[],["love","high",10],["love","low",20],["love",5],["love",10],["love",15],
 * ["love",20],["love",25]] Output:
 * [null,null,null,"","high","high","low","low"]
 * <p>
 * Constraints:
 * <p>
 * 1 <= key.length, value.length <= 100
 * key and value consist of lowercase English letters and digits.
 * 1 <= timestamp <= 107
 * All the timestamps timestamp of set are strictly increasing.
 * At most 2 * 105 calls will be made to set and get.
 */
```

```
public class Solution00981 {

    static class TimeMap {

        private final Map<String, TreeMap<Integer, String>> map;

        public TimeMap() {
            map = new TreeMap<>();
        }

        // Time: O(logn)
        // Space: O(1)
        public void set(String key, String value, int timestamp) {
            TreeMap<Integer, String> treeMap = map.getOrDefault(key, new TreeMap<>());
            treeMap.put(timestamp, value);
            map.put(key, treeMap);
        }

        // Time: O(logn)
        // Space: O(1)
        public String get(String key, int timestamp) {
            TreeMap<Integer, String> treeMap = map.get(key);
            if (treeMap == null) {
                return "";
            }
            Map.Entry<Integer, String> entry = treeMap.floorEntry(timestamp);
            if (entry != null) {
                return entry.getValue();
            }
            return "";
        }

    }

    /**
     * Your TimeMap object will be instantiated and called as such:
     * TimeMap obj = new TimeMap();
     * obj.set(key,value,timestamp);
     * String param_2 = obj.get(key,timestamp);
     */

    static class TimeMapListBased {
        private Map<String, List<Data>> map;

        public TimeMapListBased() {
            map = new HashMap<>();
        }

        public void set(String key, String value, int timestamp) {
            List<Data> list = map.getOrDefault(key, new ArrayList<>());
            list.add(new Data(timestamp, value));
            map.put(key, list);
        }
```

```
        public String get(String key, int timestamp) {
            List<Data> list = map.get(key);
            if (list == null) {
                return "";
            }
            int left = 0;
            int right = list.size() - 1;
            while (left <= right) {
                int mid = left + (right - left) / 2;

                final int middleTimestamp = list.get(mid).timestamp;

                if (middleTimestamp == timestamp) {
                    return list.get(mid).value;
                }

                if (middleTimestamp < timestamp) {
                    left = mid + 1;
                } else {
                    right = mid - 1;
                }
            }
            if (right < 0) {
                return "";
            }
            return list.get(right).value;
        }

        private static class Data {
            int timestamp;
            String value;

            public Data(int timestamp, String value) {
                this.timestamp = timestamp;
                this.value = value;
            }
        }

    }

}
```

245. `Solution00983.java`

```
/**
 * 983. Minimum Cost For Tickets
 * <a href="https://leetcode.com/problems/minimum-cost-for-tickets">Web link</a>
 * Medium
 */

package leetcode;

public class Solution00983 {

    public int mincostTickets(int[] days, int[] costs) {
        int n = days.length;
        int left7 = 0;
        int left30 = 0;
        int[] dp = new int[n];

        for (int right = 0; right < n; ++right) {
            while (days[right] - days[left7] >= 7) left7++;
            while (days[right] - days[left30] >= 30) left30++;

            int cost1 = (right > 0 ? dp[right - 1] : 0) + costs[0];
            int cost7 = (left7 > 0 ? dp[left7 - 1] : 0) + costs[1];
            int cost30 = (left30 > 0 ? dp[left30 - 1] : 0) + costs[2];

            dp[right] = Math.min(Math.min(cost1, cost7), cost30);
        }

        return dp[n - 1];
    }

}
```

246. `Solution00988.java`

```
package leetcode;

import leetcode.common.TreeNode;

/**
 * 988. Smallest String Starting From Leaf
 * https://leetcode.com/problems/smallest-string-starting-from-leaf/
 * Medium
 */

public class Solution00988 {

    public String smallestFromLeaf(TreeNode root) {
        return smallestFromLeaf(root, new StringBuilder()).toString();
    }

    private StringBuilder smallestFromLeaf(TreeNode root, StringBuilder suffix) {
        if (root == null) {
            return suffix;
        }
        StringBuilder newSuffix = new StringBuilder(suffix.length() + 1)
                .append((char) ('a' + root.val))
                .append(suffix);
        if (root.left == null && root.right == null) {
            return newSuffix;
        }
        if (root.left == null) {
            return smallestFromLeaf(root.right, newSuffix);
        }
        if (root.right == null) {
            return smallestFromLeaf(root.left, newSuffix);
        }
        StringBuilder left = smallestFromLeaf(root.left, newSuffix);
        StringBuilder right = smallestFromLeaf(root.right, newSuffix);
        return left.compareTo(right) <= 0 ? left : right;
    }

}
```

247. `Solution00992.java`

```
package leetcode;

import java.util.HashMap;
import java.util.Map;

/**
 * 992. Subarrays with K Different Integers
 * <a href="https://leetcode.com/problems/subarrays-with-k-different-integers">Web
link</a> |
 * Hard
 */

public class Solution00992 {

    public int subarraysWithKDistinctViaAtMost(int[] nums, int k) {
        return atMostK(nums, k) - atMostK(nums, k - 1);
    }

    private int atMostK(int[] nums, int k) {
        int left = 0;
        int totalSubarrays = 0;
        Map<Integer, Integer> countMap = new HashMap<>();
        for (int right = 0; right < nums.length; right++) {
            int rightValue = countMap.getOrDefault(nums[right], 0);
            if (rightValue == 0) {
                k--;
            }
            countMap.put(nums[right], rightValue + 1);
            while (k < 0) {
                int newValueLeft = countMap.getOrDefault(nums[left], 0) - 1;
                countMap.put(nums[left], newValueLeft);
                if (newValueLeft == 0) {
                    k++;
                }
                left++;
            }
            totalSubarrays += right - left + 1;
        }
        return totalSubarrays;
    }

    public int subarraysWithKDistinctVia3Pointers(int[] nums, int k) {
        int leftFar = 0;
        int leftNear = 0;
        int totalSubarrays = 0;
        Map<Integer, Integer> countMap = new HashMap<>();

        for (int rightValue : nums) {
            countMap.put(rightValue, countMap.getOrDefault(rightValue, 0) + 1);

            while (countMap.size() > k) {
```

```
                final int leftNearCount = countMap.get(nums[leftNear]) - 1;
                if (leftNearCount == 0) {
                    countMap.remove(nums[leftNear]);
                } else {
                    countMap.put(nums[leftNear], leftNearCount);
                }
                leftNear++;
                leftFar = leftNear;
            }

            while (countMap.get(nums[leftNear]) > 1) {
                countMap.put(nums[leftNear], countMap.get(nums[leftNear]) - 1);
                leftNear++;
            }

            if (countMap.size() == k) {
                totalSubarrays += leftNear - leftFar + 1;
            }
        }

        return totalSubarrays;
    }

}
```

248. `Solution00994.java`

```
package leetcode;

import javafx.util.Pair;

import java.util.ArrayDeque;
import java.util.Queue;

/**
 * https://leetcode.com/problems/rotting-oranges/
 * 994. Rotting Oranges
 * Medium
 */

public class Solution00994 {

    // Time complexity: O(m*n)
    // Space complexity: O(m*n)
    public int orangesRotting(int[][] grid) {
        int fresh = 0;
        Queue<Pair<Integer, Integer>> queue = new ArrayDeque<>();
        for (int row = 0; row < grid.length; row++) {
            for (int col = 0; col < grid[0].length; col++) {
                final int orangeState = grid[row][col];
                if (orangeState == 1) {
                    fresh++;
                } else if (orangeState == 2) {
                    queue.add(new Pair<>(row, col));
                }
            }
        }
        if (fresh == 0) {
            return 0;
        }
        int minutes = 0;
        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int i = 0; i < size; i++) {
                Pair<Integer, Integer> pair = queue.poll();
                int row = pair.getKey();
                int col = pair.getValue();
                if (row > 0 && grid[row - 1][col] == 1) {
                    grid[row - 1][col] = 2;
                    queue.add(new Pair<>(row - 1, col));
                    fresh--;
                }
                if (row < grid.length - 1 && grid[row + 1][col] == 1) {
                    grid[row + 1][col] = 2;
                    queue.add(new Pair<>(row + 1, col));
                    fresh--;
                }
                if (col > 0 && grid[row][col - 1] == 1) {
```

```
                    grid[row][col - 1] = 2;
                    queue.add(new Pair<>(row, col - 1));
                    fresh--;
                }
                if (col < grid[0].length - 1 && grid[row][col + 1] == 1) {
                    grid[row][col + 1] = 2;
                    queue.add(new Pair<>(row, col + 1));
                    fresh--;
                }
            }
            minutes++;
            if (fresh == 0) {
                return minutes;
            }
        }
        return -1;
    }

}
```

249. `Solution00995.java`

```
package leetcode;

import java.util.ArrayDeque;
import java.util.Queue;

/**
 * 995. Minimum Number of K Consecutive Bit Flips
 * <a href="https://leetcode.com/problems/minimum-number-of-k-consecutive-bit-flips">Web
link</a>
 * Hard
 */

public class Solution00995 {

    public int minKBitFlips(int[] nums, int k) {
        int n = nums.length;
        int[] diff = new int[n + 1];
        int flipCount = 0;
        int result = 0;
        for (int i = 0; i < n; i++) {
            flipCount += diff[i];
            if ((nums[i] + flipCount) % 2 == 0) {
                if (i + k > n) {
                    return -1;
                }
                result++;
                flipCount++;
                diff[i + k]--;
            }
        }
        return result;
    }

    public int minKBitFlipsQueue(int[] nums, int k) {
        int n = nums.length;
        int numberOfFlips = 0;
        Queue<Integer> flipIndices = new ArrayDeque<>();
        for (int i = 0; i < n; i++) {
            if (!flipIndices.isEmpty() && flipIndices.peek() < i) {
                flipIndices.poll();
            }
            if (flipIndices.size() % 2 == nums[i]) {
                final int nextFlipIndex = i + k - 1;
                if (nextFlipIndex >= n) {
                    return -1;
                }
                numberOfFlips++;
                flipIndices.add(nextFlipIndex);
            }
        }
        return numberOfFlips;
```

```
    }

}
```

250. `Solution00997.java`

```
package leetcode;

/**
 * 997. Find the Town Judge
 * https://leetcode.com/problems/find-the-town-judge/
 * Easy
 */

public class Solution00997 {

    // Time: O(n)
    // Space: O(n)
    public int findJudge(int n, int[][] trust) {
        // Create an array to store the number of trusts each person receives
        int[] inDegree = new int[n + 1];

        // Create an array to store the number of trusts each person gives
        int[] outDegree = new int[n + 1];

        // Iterate over the trust array
        for (int[] t : trust) {
            // Increment the number of trusts the person gives
            outDegree[t[0]]++;
            // Increment the number of trusts the person receives
            inDegree[t[1]]++;
        }

        // Iterate over all the people
        for (int i = 1; i <= n; i++) {
            // If a person does not trust anyone (outDegree is 0) and is trusted by
everyone else (inDegree is n - 1)
            if (outDegree[i] == 0 && inDegree[i] == n - 1) {
                // Return the person as the town judge
                return i;
            }
        }

        // If no town judge is found, return -1
        return -1;
    }

}
```

251. `Solution01002.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 1002. Find Common Characters
 * <a href="https://leetcode.com/problems/find-common-characters">Web link</a>
 * Easy
 */

public class Solution01002 {

    public List<String> commonChars(String[] words) {
        Map<Character, Integer> countMap = new HashMap<>();
        for (char c : words[0].toCharArray()) {
            countMap.put(c, countMap.getOrDefault(c, 0) + 1);
        }
        for (int i = 1; i < words.length; i++) {
            Map<Character, Integer> tempMap = new HashMap<>();
            for (char c : words[i].toCharArray()) {
                tempMap.put(c, tempMap.getOrDefault(c, 0) + 1);
            }
            countMap.replaceAll((k, v) -> Math.min(v, tempMap.getOrDefault(k, 0)));
        }
        List<String> result = new ArrayList<>();
        for (var entry : countMap.entrySet()) {
            for (int i = 0; i < entry.getValue(); i++) {
                result.add(String.valueOf(entry.getKey()));
            }
        }
        return result;
    }

}
```

252. `Solution01014.java`

```
/**
 * 1014. Best Sightseeing Pair
 * <a href="https://leetcode.com/problems/best-sightseeing-pair">Web link</a>
 * Medium
 */

package leetcode;

public class Solution01014 {

    public int maxScoreSightseeingPair(int[] values) {
        int result = 0;
        int maxPair = values[0] - 1;
        for (int i = 1; i < values.length; i++) {
            result = Math.max(result, maxPair + values[i]);
            maxPair = Math.max(maxPair - 1, values[i] - 1);
        }
        return result;
    }

    public int maxScoreSightseeingPairDp(int[] values) {
        int result = 0;
        int[] dpLeftMax = new int[values.length];
        dpLeftMax[0] = values[0] - 1;
        for (int i = 1; i < values.length; i++) {
            result = Math.max(result, dpLeftMax[i - 1] + values[i]);
            dpLeftMax[i] = Math.max(dpLeftMax[i - 1] - 1, values[i] - 1);
        }
        return result;
    }
}
```

253. `Solution01026.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 1026. Maximum Difference Between Node and Ancestor
 * https://leetcode.com/problems/maximum-difference-between-node-and-ancestor/
 * #medium
 * Given the root of a binary tree, find the maximum value v for which there exist
different nodes a and b where v = |a.val - b.val| and a is an ancestor of b.
 * <p>
 * A node a is an ancestor of b if either: any child of a is equal to b or any child of a
is an ancestor of b.
 * <p>
 * Example 1:
 * <p>
 * Input: root = [8,3,10,1,6,null,14,null,null,4,7,13]
 * Output: 7
 * Explanation: We have various ancestor-node differences, some of which are given below
:
 * |8 - 3| = 5
 * |3 - 7| = 4
 * |8 - 1| = 7
 * |10 - 13| = 3
 * Among all possible differences, the maximum value of 7 is obtained by |8 - 1| = 7.
 * Example 2:
 * <p>
 * Input: root = [1,null,2,null,0,3]
 * Output: 3
 * <p>
 * Constraints:
 * <p>
 * The number of nodes in the tree is in the range [2, 5000].
 * 0 <= Node.val <= 105
 */
public class Solution01026 {

    public static void main(String[] args) {
        Solution01026 s = new Solution01026();
        System.out.println(s.maxAncestorDiffDfs(new TreeNode(8, new TreeNode(3, new
TreeNode(1), new TreeNode(6, new TreeNode(4), new TreeNode(7))), new TreeNode(10, null,
new TreeNode(14, new TreeNode(13), null)))));
        System.out.println(s.maxAncestorDiffDfs(new TreeNode(1, null, new TreeNode(2,
null, new TreeNode(0, new TreeNode(3), null))))); // 3
    }

    // Time O(n)
    // Space O(n)
```

```
    public int maxAncestorDiffDfs(TreeNode root) {
        return dfs(root, root.val, root.val);
    }

    private int dfs(TreeNode root, int min, int max) {
        if (root == null) {
            return max - min;
        }

        min = Math.min(min, root.val);
        max = Math.max(max, root.val);

        return Math.max(dfs(root.left, min, max), dfs(root.right, min, max));
    }

    // Time O(n)
    // Space O(n)
    public int maxAncestorDiffBfs(TreeNode root) {
        if (root == null) {
            return 0;
        }

        int max = Integer.MIN_VALUE;
        Queue<QueueData> queue = new LinkedList<>();
        queue.add(new QueueData(root, root.val, root.val));

        while (!queue.isEmpty()) {
            QueueData current = queue.poll();
            max = Math.max(max, current.max - current.min);

            if (current.node.left != null) {
                queue.add(new QueueData(current.node.left, Math.min(current.min,
current.node.left.val), Math.max(current.max, current.node.left.val)));
            }

            if (current.node.right != null) {
                queue.add(new QueueData(current.node.right, Math.min(current.min,
current.node.right.val), Math.max(current.max, current.node.right.val)));
            }
        }

        return max;
    }

    private static class QueueData {
        TreeNode node;
        int min;
        int max;

        public QueueData(TreeNode node, int min, int max) {
            this.node = node;
            this.min = min;
            this.max = max;
        }
```

```
    }

}
```

254. `Solution01038.java`

```
package leetcode;

import leetcode.common.TreeNode;

/**
 * 1038. Binary Search Tree to Greater Sum Tree
 * <a href="https://leetcode.com/problems/binary-search-tree-to-greater-sum-tree/">Web
link</a>
 * Medium
 */

public class Solution01038 {

    public TreeNode bstToGst(TreeNode root) {
        bstToGst(root, 0);
        return root;
    }

    private int bstToGst(TreeNode root, int sum) {
        if (root == null) {
            return sum;
        }
        int rightSum = bstToGst(root.right, sum);
        root.val += rightSum;
        return bstToGst(root.left, root.val);
    }

}
```

255. `Solution01043.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/partition-array-for-maximum-sum
 * 1043. Partition Array for Maximum Sum
 * Medium
 * <p>
 * Given an integer array arr, partition the array into (contiguous) subarrays of length
at most k. After partitioning, each subarray has their values changed to become the
maximum value of that subarray.
 * <p>
 * Return the largest sum of the given array after partitioning. Test cases are generated
so that the answer fits in a 32-bit integer.
 * <p>
 * Example 1:
 * <p>
 * Input: arr = [1,15,7,9,2,5,10], k = 3
 * Output: 84
 * Explanation: arr becomes [15,15,15,9,10,10,10]
 * Example 2:
 * <p>
 * Input: arr = [1,4,1,5,7,3,6,1,9,9,3], k = 4
 * Output: 83
 * Example 3:
 * <p>
 * Input: arr = [1], k = 1
 * Output: 1
 * <p>
 * Constraints:
 * <p>
 * 1 <= arr.length <= 500
 * 0 <= arr[i] <= 109
 * 1 <= k <= arr.length
 */
public class Solution01043 {

    // Time complexity: O(n * k)
    // Space complexity: O(n)
    public int maxSumAfterPartitioning(int[] arr, int k) {
        // Get the length of the input array
        int n = arr.length;

        // Initialize a dynamic programming array with size n+1
        int[] dp = new int[n + 1];

        // Iterate over the array
        for (int i = 1; i <= n; i++) {
            // Initialize the maximum value for the current subarray
            int max = 0;

            // Iterate over the subarray of length at most k
```

```
            for (int j = 1; j <= k && i - j >= 0; j++) {
                // Update the maximum value if the current value is greater
                max = Math.max(max, arr[i - j]);

                // Update the dp value for the current position with the maximum sum of
partition
                dp[i] = Math.max(dp[i], dp[i - j] + max * j);
            }
        }

        // Return the maximum sum after partitioning
        return dp[n];
    }

}
```

256. `Solution01046.java`

```
package leetcode;

import java.util.Arrays;
import java.util.Comparator;
import java.util.PriorityQueue;

/**
 * 1046. Last Stone Weight
 * https://leetcode.com/problems/last-stone-weight/
 * Easy
 * <p>
 * We have a collection of stones, each stone has a positive integer weight.
 * <p>
 * Each turn, we choose the two heaviest stones and smash them together.
 * Suppose the stones have weights x and y with x <= y.
 * <p>
 * The result of this smash is:
 * <p>
 * If x == y, both stones are totally destroyed; If x != y, the stone of weight x is
totally destroyed,
 * and the stone of weight y has new weight y-x. At the end, there is at most 1 stone
left.
 * Return the weight of this stone (or 0 if there are no stones left.)
 * <p>
 * Example 1:
 * <p>
 * Input: [2,7,4,1,8,1] Output: 1 Explanation: We combine 7 and 8 to get 1 so the array
converts to [2,4,1,1,1] then,
 * we combine 2 and 4 to get 2 so the array converts to [2,1,1,1] then, we combine 2 and
1 to get 1 so the array
 * converts to [1,1,1] then, we combine 1 and 1 to get 0 so the array converts to [1]
then that's the value of last
 * stone.
 * <p>
 * Note:
 * <p>
 * 1 <= stones.length <= 30 1 <= stones[i] <= 1000
 */
public class Solution01046 {

    // Time: O(n * log(n))
    // Space: O(n)
    public int lastStoneWeight(int[] stones) {
        PriorityQueue<Integer> maxHeap = new PriorityQueue<>(stones.length,
Comparator.reverseOrder());
        for (int stone : stones) {
            maxHeap.offer(stone);
        }

        while (maxHeap.size() > 1) {
            int first = maxHeap.poll();
```

```
            int second = maxHeap.poll();
            if (first != second) {
                maxHeap.offer(first - second);
            }
        }

        return maxHeap.isEmpty() ? 0 : maxHeap.poll();
    }

    // Time: O(n * log(n))
    // Space: O(1)
    public int lastStoneWeightFirstApproach(int[] stones) {
        int last = stones.length - 1;
        while (last > 0) {
            Arrays.sort(stones, 0, last + 1);
            stones[last - 1] = stones[last] - stones[last - 1];
            last--;
        }
        return stones[0];
    }

}
```

257. `Solution01051.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 1051. Height Checker
 * https://leetcode.com/problems/height-checker/
 * Easy
 */

public class Solution01051 {

    public int heightChecker(int[] heights) {
        final int n = heights.length;
        int[] heightsCopy = Arrays.copyOf(heights, n);
        Arrays.sort(heightsCopy);
        int mismatch = 0;
        for (int i = 0; i < n; i++) {
            if (heights[i] != heightsCopy[i]) {
                mismatch++;
            }
        }
        return mismatch;
    }

    public int heightCheckerCountingSort(int[] heights) {
        int min = Integer.MAX_VALUE;
        int max = Integer.MIN_VALUE;
        for (int height : heights) {
            min = Math.min(min, height);
            max = Math.max(max, height);
        }
        int[] countArr = new int[max - min + 1];
        for (int height : heights) {
            countArr[height - min]++;
        }
        int pos = 0;
        int mismatch = 0;
        for (int i = 0; i < countArr.length; i++) {
            for (int j = 0; j < countArr[i]; j++) {
                int expectedValueAtPosition = i + min;
                if (heights[pos] != expectedValueAtPosition) {
                    mismatch++;
                }
                pos++;
            }
        }
        return mismatch;
    }

}
```

258. `Solution01052.java`

```
package leetcode;

/**
 * 1052. Grumpy Bookstore Owner
 * https://leetcode.com/problems/grumpy-bookstore-owner/description/
 * Medium
 */

public class Solution01052 {

    public int maxSatisfied(int[] customers, int[] grumpy, int minutes) {
        int n = grumpy.length;
        int i = 0;
        int curWin = 0;
        int maxWin = 0;

        for (int j = 0; j < n; j++) {
            curWin += customers[j] * grumpy[j];
            maxWin = Math.max(curWin, maxWin);
            if (j >= minutes - 1) {
                curWin -= customers[i] * grumpy[i++];
            }
        }
        int res = maxWin;
        for (i = 0; i < grumpy.length; i++) {
            res += customers[i] * (1 - grumpy[i]);
        }
        return res;
    }

    public int maxSatisfiedAlternative(int[] customers, int[] grumpy, int minutes) {
        final int n = customers.length;
        int guaranteedSatisfied = 0;
        int maxConditional = 0;
        int currentConditional = 0;
        int low = 0;
        for (int high = 0; high < n; high++) {
            if (grumpy[high] == 0) {
                guaranteedSatisfied += customers[high];
            } else {
                currentConditional += customers[high];
                while (high - low + 1 > minutes) {
                    if (grumpy[low] == 1) {
                        currentConditional -= customers[low];
                    }
                    low++;
                }
                maxConditional = Math.max(maxConditional, currentConditional);
            }
        }
        return guaranteedSatisfied + maxConditional;
```

```
    }

}
```

259. `Solution01072.java`

```
/**
 * 1072. Flip Columns For Maximum Number of Equal Rows
 * <a href="https://leetcode.com/problems/flip-columns-for-maximum-number-of-equal-
rows">Web link</a>
 * Medium
 */

package leetcode;

import java.util.Arrays;

public class Solution01072 {

    public int maxEqualRowsAfterFlips(int[][] matrix) {
        int n = matrix[0].length;
        int ans = 0;

        for (int[] bin : matrix) {
            // Create inverse of the current row
            int[] invBin = new int[n];
            for (int j = 0; j < n; j++) {
                invBin[j] = bin[j] == 0 ? 1 : 0;
            }

            // Compare with all rows
            int tmp = 0;
            for (int[] cur : matrix) {
                if (Arrays.equals(cur, bin) || Arrays.equals(cur, invBin)) {
                    tmp++;
                }
            }

            ans = Math.max(tmp, ans);
        }

        return ans;
    }

}
```

260. `Solution01074.java`

```
package leetcode;

import java.util.HashMap;
import java.util.Map;

/**
 * <a href="https://leetcode-cn.com/problems/number-of-submatrices-that-sum-to-target/">
 * 1074. Number of Submatrices That Sum to Target
 * Hard
 * <p>
 * Given a matrix and a target, return the number of non-empty submatrices that sum to
target.
 * <p>
 * A submatrix x1, y1, x2, y2 is the set of all cells matrix[x][y] with x1 <= x <= x2 and
y1 <= y <= y2.
 * <p>
 * Two submatrices (x1, y1, x2, y2) and (x1', y1', x2', y2') are different if they have
some coordinate that is different: for example, if x1 != x1'.
 * <p>
 * Example 1:
 * <p>
 * Input: matrix = [[0,1,0],[1,1,1],[0,1,0]], target = 0
 * Output: 4
 * Explanation: The four 1x1 submatrices that only contain 0.
 * Example 2:
 * <p>
 * Input: matrix = [[1,-1],[-1,1]], target = 0
 * Output: 5
 * Explanation: The two 1x2 submatrices, plus the two 2x1 submatrices, plus the 2x2
submatrix.
 * Example 3:
 * <p>
 * Input: matrix = [[904]], target = 0
 * Output: 0
 * <p>
 * Constraints:
 * <p>
 * 1 <= matrix.length <= 100
 * 1 <= matrix[0].length <= 100
 * -1000 <= matrix[i] <= 1000
 * -10^8 <= target <= 10^8
 */
public class Solution01074 {

    // Time O(m^2 * n^2)
    // Space O(m * n)
    public int numSubmatrixSumTarget(int[][] matrix, int target) {
        // Get the number of rows and columns in the matrix
        int m = matrix.length;
        int n = matrix[0].length;
```

```
        // Initialize a prefix sum matrix with an extra row and column for easier
calculation
        int[][] prefixSum = new int[m + 1][n + 1];

        // Calculate the prefix sum for each cell in the matrix
        for (int row = 1; row <= m; row++) {
            int rowSum = 0;
            for (int column = 1; column <= n; column++) {
                // Add the current cell value to the row sum
                rowSum += matrix[row - 1][column - 1];

                // The prefix sum at the current cell is the sum of the previous row's
prefix sum and the current row sum
                prefixSum[row][column] = prefixSum[row - 1][column] + rowSum;
            }
        }

        // Initialize a counter for the number of submatrices that sum to the target
        int count = 0;

        // Iterate over all possible submatrices in the matrix
        for (int rowEnd = 1; rowEnd <= m; rowEnd++) {
            for (int rowStart = 1; rowStart <= rowEnd; rowStart++) {
                for (int columnEnd = 1; columnEnd <= n; columnEnd++) {
                    for (int columnStart = 1; columnStart <= columnEnd; columnStart++) {
                        // Calculate the sum of the current submatrix using the prefix
sum matrix
                        int sum = prefixSum[rowEnd][columnEnd] -
                                  prefixSum[rowEnd - rowStart][columnEnd] -
                                  prefixSum[rowEnd][columnEnd - columnStart] +
                                  prefixSum[rowEnd - rowStart][columnEnd - columnStart];

                        // If the sum of the current submatrix equals the target,
increment the counter
                        if (sum == target) {
                            count++;
                        }
                    }
                }
            }
        }

        // Return the number of submatrices that sum to the target
        return count;
    }

    // Time O(m^2 * n)
    // Space O(m * n)
    public int numSubmatrixSumTargetOptimized(int[][] matrix, int target) {
        // Get the number of rows and columns in the matrix
        int m = matrix.length;
        int n = matrix[0].length;

        // Initialize a prefix sum matrix with an extra row and column for easier
```

```
calculation
        int[][] prefixSum = new int[m + 1][n + 1];

        // Calculate the prefix sum for each cell in the matrix
        for (int row = 1; row <= m; row++) {
            int rowSum = 0;
            for (int column = 1; column <= n; column++) {
                // Add the current cell value to the row sum
                rowSum += matrix[row - 1][column - 1];

                // The prefix sum at the current cell is the sum of the previous row's
prefix sum and the current row sum
                prefixSum[row][column] = prefixSum[row - 1][column] + rowSum;
            }
        }

        // Initialize a counter for the number of submatrices that sum to the target
        int count = 0;

        // Iterate over all possible submatrices in the matrix
        for (int rowEnd = 1; rowEnd <= m; rowEnd++) {
            for (int rowStart = 1; rowStart <= rowEnd; rowStart++) {
                Map<Integer, Integer> map = new HashMap<>();
                map.put(0, 1);
                for (int column = 1; column <= n; column++) {
                    int sum = prefixSum[rowEnd][column] - prefixSum[rowStart - 1]
[column];
                    count += map.getOrDefault(sum - target, 0);
                    map.put(sum, map.getOrDefault(sum, 0) + 1);
                }
            }
        }

        // Return the number of submatrices that sum to the target
        return count;
    }

}
```

261. `Solution01105.java`

```
package leetcode;

/**
 * 1105. Filling Bookcase Shelves
 * <a href="https://leetcode.com/problems/filling-bookcase-shelves">Web link</a>
 * Medium
 */

public class Solution01105 {

    public int minHeightShelves(int[][] books, int shelfWidth) {
        final int n = books.length;
        int[] dp = new int[n + 1];
        dp[0] = 0;

        for (int i = 1; i <= n; i++) {
            int currentShelfWidth = 0;
            int currentShelfHeight = 0;
            dp[i] = Integer.MAX_VALUE;
            for (int j = i - 1; j >= 0; j--) {
                final int[] currentBook = books[j];
                final int currentBookWidth = currentBook[0];
                currentShelfWidth += currentBookWidth;
                if (currentShelfWidth > shelfWidth) {
                    break;
                }
                final int currentBookHeight = currentBook[1];
                currentShelfHeight = Math.max(currentShelfHeight, currentBookHeight);
                dp[i] = Math.min(dp[i], dp[j] + currentShelfHeight);
            }
        }
        return dp[n];
    }

}
```

262. `Solution01106.java`

```
/**
 * 1106. Parsing A Boolean Expression
 * <a href="https://leetcode.com/problems/parsing-a-boolean-expression">Web link</a>
 * Hard
 */

package leetcode;

import java.util.ArrayDeque;
import java.util.Deque;

public class Solution01106 {

    public boolean parseBoolExpr(String expression) {
        Deque<Character> stack = new ArrayDeque<>();
        for (char currChar : expression.toCharArray()) {
            if (currChar == ',' || currChar == '(') {
                continue;
            }
            if (currChar == 't' ||
                    currChar == 'f' ||
                    currChar == '!' ||
                    currChar == '&' ||
                    currChar == '|') {
                stack.push(currChar);
            } else if (currChar == ')') {
                boolean hasTrue = false;
                boolean hasFalse = false;
                while (stack.peek() != '!' &&
                        stack.peek() != '&' &&
                        stack.peek() != '|') {
                    char topValue = stack.pop();
                    if (topValue == 't') hasTrue = true;
                    if (topValue == 'f') hasFalse = true;
                }
                char op = stack.pop();
                if (op == '!') {
                    stack.push(hasTrue ? 'f' : 't');
                } else if (op == '&') {
                    stack.push(hasFalse ? 'f' : 't');
                } else {
                    stack.push(hasTrue ? 't' : 'f');
                }
            }
        }
        return stack.peek() == 't';
    }

}
```

263. `Solution01110.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;

/**
 * 1110. Delete Nodes And Return Forest
 * <a href="https://leetcode.com/problems/delete-nodes-and-return-forest">Web link</a>
 * Medium
 */

public class Solution01110 {

    public List<TreeNode> delNodes(TreeNode root, int[] to_delete) {
        Set<Integer> toDeleteSet =
Arrays.stream(to_delete).boxed().collect(Collectors.toSet());
        List<TreeNode> forest = new ArrayList<>();
        if (!toDeleteSet.contains(root.val)) {
            forest.add(root);
        }
        deleteNodes(root, toDeleteSet, forest);
        return forest;
    }

    private TreeNode deleteNodes(TreeNode node, Set<Integer> toDeleteSet, List<TreeNode>
forest) {
        if (node == null) return null;
        node.left = deleteNodes(node.left, toDeleteSet, forest);
        node.right = deleteNodes(node.right, toDeleteSet, forest);
        if (toDeleteSet.contains(node.val)) {
            if (node.left != null) forest.add(node.left);
            if (node.right != null) forest.add(node.right);
            return null;
        }
        return node;
    }

}
```

264. `Solution01122.java`

```
package leetcode;

import java.util.Comparator;
import java.util.HashMap;
import java.util.Map;

/**
 * 1122. Relative Sort Array
 * <a href="https://leetcode.com/problems/relative-sort-array">Web link</a>
 * Easy
 */

public class Solution01122 {

    public int[] relativeSortArray(int[] arr1, int[] arr2) {
        Map<Integer, Integer> countMap = new HashMap<>();
        for (int num : arr1) {
            countMap.put(num, countMap.getOrDefault(num, 0) + 1);
        }
        int pos = 0;
        int[] result = new int[arr1.length];
        for (int num : arr2) {
            int count = countMap.getOrDefault(num, 0);
            for (int i = 0; i < count; i++) {
                result[pos++] = num;
            }
            countMap.remove(num);
        }
        var sorted = countMap.entrySet()
                .stream()
                .sorted(Comparator.comparingInt(Map.Entry::getKey))
                .toList();
        for (Map.Entry<Integer, Integer> entry : sorted) {
            int count = entry.getValue();
            for (int i = 0; i < count; i++) {
                result[pos++] = entry.getKey();
            }
        }
        return result;
    }

}
```

265. `Solution01137.java`

```
package leetcode;

/**
 * 1137. N-th Tribonacci Number
 * <a href="https://leetcode.com/problems/n-th-tribonacci-number">Web link</a>
 * Easy
 */

public class Solution01137 {

    public int tribonacci(int n) {
        if (n == 0) {
            return 0;
        }
        if (n == 1 || n == 2) {
            return 1;
        }

        int[] dp = new int[n + 1];
        dp[0] = 0;
        dp[1] = dp[2] = 1;

        for (int i = 3; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2] + dp[i - 3];
        }

        return dp[n];
    }

}
```

266. `Solution01140.java`

```
package leetcode;

/**
 * 1140. Stone Game II
 * <a href="https://leetcode.com/problems/stone-game-ii">Web link</a>
 * Medium
 */

public class Solution01140 {

    public int stoneGameII(int[] piles) {
        int n = piles.length;

        int[][] dp = new int[n][n + 1];
        int[] suffixSum = new int[n];
        suffixSum[n - 1] = piles[n - 1];

        for (int i = n - 2; i >= 0; i--) {
            suffixSum[i] = suffixSum[i + 1] + piles[i];
        }

        for (int i = n - 1; i >= 0; i--) {
            for (int m = 1; m <= n; m++) {
                final int maxSteps = 2 * m;
                if (i + maxSteps >= n) {
                    dp[i][m] = suffixSum[i];
                } else {
                    for (int x = 1; x <= maxSteps; x++) {
                        dp[i][m] = Math.max(dp[i][m], suffixSum[i] - dp[i + x]
[Math.max(m, x)]);
                    }
                }
            }
        }
        return dp[0][1];
    }

}
```

267. `Solution01143.java`

```
package leetcode;

/**
 * 1143. Longest Common Subsequence
 * https://leetcode.com/problems/longest-common-subsequence/
 * Medium
 * <p>
 * Given two strings text1 and text2, return the length of their longest common
subsequence. If there is no common subsequence, return 0.
 * <p>
 * A subsequence of a string is a new string generated from the original string with some
characters (can be none) deleted without changing the relative order of the remaining
characters.
 * <p>
 * For example, "ace" is a subsequence of "abcde".
 * A common subsequence of two strings is a subsequence that is common to both strings.
 * <p>
 * <p>
 * <p>
 * Example 1:
 * <p>
 * Input: text1 = "abcde", text2 = "ace"
 * Output: 3
 * Explanation: The longest common subsequence is "ace" and its length is 3.
 * Example 2:
 * <p>
 * Input: text1 = "abc", text2 = "abc"
 * Output: 3
 * Explanation: The longest common subsequence is "abc" and its length is 3.
 * Example 3:
 * <p>
 * Input: text1 = "abc", text2 = "def"
 * Output: 0
 * Explanation: There is no such common subsequence, so the result is 0.
 * <p>
 * <p>
 * Constraints:
 * <p>
 * 1 <= text1.length, text2.length <= 1000
 * text1 and text2 consist of only lowercase English characters.
 */
public class Solution01143 {

    public int longestCommonSubsequence(String text1, String text2) {
        final int length1 = text1.length();
        final int length2 = text2.length();

        int[][] dp = new int[length1 + 1][length2 + 1];

        for (int i = 0; i <= length1; ++i) {
            dp[i][0] = 0;
```

```
        }

        for (int j = 0; j <= length2; ++j) {
            dp[0][j] = 0;
        }

        for (int i = 1; i <= length1; ++i) {
            final char c1 = text1.charAt(i - 1);
            for (int j = 1; j <= length2; ++j) {
                final char c2 = text2.charAt(j - 1);
                if (c1 == c2) {
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                    continue;
                }

                dp[i][j] = Math.max(dp[i][j - 1], dp[i - 1][j]);
            }
        }

        return dp[length1][length2];
    }

}
```

268. `Solution01171.java`

```
package leetcode;

import leetcode.common.ListNode;

import java.util.HashMap;
import java.util.Map;

/**
 * 1171. Remove Zero Sum Consecutive Nodes from Linked List
 * <a href="https://leetcode.com/problems/remove-zero-sum-consecutive-nodes-from-linked-
list">Web link</a>
 * Medium
 */

public class Solution01171 {

    // Time: O(n^2)
    // Space: O(1)
    public ListNode removeZeroSumSublists(ListNode head) {
        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode current = dummy;
        while (current != null) {
            int sum = 0;
            ListNode next = current.next;
            while (next != null) {
                sum += next.val;
                if (sum == 0) {
                    current.next = next.next;
                }
                next = next.next;
            }
            current = current.next;
        }
        return dummy.next;
    }

    // Time: O(n)
    // Space: O(n)
    public ListNode removeZeroSumSublistsMap(ListNode head) {
        ListNode dummy = new ListNode();
        dummy.next = head;

        int prefixSum = 0;
        Map<Integer, ListNode> map = new HashMap<>();
        map.put(0, dummy);

        ListNode current = head;
        while (current != null) {
            prefixSum += current.val;
            map.put(prefixSum, current);
```

```
            current = current.next;
        }

        current = dummy;
        prefixSum = 0;
        while (current != null) {
            prefixSum += current.val;
            current.next = map.get(prefixSum).next;
            current = current.next;
        }

        return dummy.next;
    }

}
```

269. `Solution01190.java`

```
package leetcode;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * 1190. Reverse Substrings Between Each Pair of Parentheses
 * <a href="https://leetcode.com/problems/reverse-substrings-between-each-pair-of-
parentheses/">Web link</a>
 * Medium
 */

public class Solution01190 {

    public String reverseParentheses(String s) {
        int n = s.length();
        State state = new State(0, n - 1);
        return reverseParentheses(s, state).toString();
    }

    StringBuilder reverseParentheses(String s, State state) {
        StringBuilder stringBuilder = new StringBuilder();
        int i = state.start;
        while (i <= state.end) {
            char c = s.charAt(i);
            if (c == '(') {
                state.start = i;
                State newState = new State(i + 1, state.end);
                StringBuilder stringBuilderBody = reverseParentheses(s, newState);
                stringBuilder.append(stringBuilderBody.reverse());
                i = newState.end + 1;
            } else if (c == ')') {
                state.end = i;
                return stringBuilder;
            } else {
                stringBuilder.append(c);
                ++i;
            }
        }
        return stringBuilder;
    }

    static class State {
        int start;
        int end;

        State(int start, int end) {
            this.start = start;
            this.end = end;
        }
    }
```

```
    public String reverseParenthesesStackBased(String s) {
        Deque<Integer> stack = new ArrayDeque<>();
        char[] result = s.toCharArray();
        for (int i = 0; i < s.length(); i++) {
            if (result[i] == '(') {
                stack.push(i);
            } else if (result[i] == ')') {
                reverse(result, stack.pop() + 1, i - 1);
            }
        }
        StringBuilder stringBuilder = new StringBuilder();
        for (char c : result) {
            if (c != '(' && c != ')') {
                stringBuilder.append(c);
            }
        }
        return stringBuilder.toString();
    }

    private void reverse(char[] result, int start, int end) {
        while (start < end) {
            char temp = result[start];
            result[start] = result[end];
            result[end] = temp;
            start++;
            end--;
        }
    }

}
```

270. `Solution01207.java`

```
package leetcode;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * https://leetcode.com/problems/unique-number-of-occurrences/
 * 1207. Unique Number of Occurrences
 * Easy
 * <p>
 * Given an array of integers arr, write a function that returns true if and only if the
number of occurrences of each value in the array is unique.
 * Example 1:
 * Input: arr = [1,2,2,1,1,3]
 * Output: true
 * Explanation: The value 1 has 3 occurrences, 2 has 2 and 3 has 1. No two values have
the same number of occurrences.
 * <p>
 * Example 2:
 * Input: arr = [1,2]
 * Output: false
 * <p>
 * Example 3:
 * Input: arr = [-3,0,1,-3,1,1,1,-3,10,0]
 * Output: true
 * <p>
 * Constraints:
 * 1 <= arr.length <= 1000
 * -1000 <= arr[i] <= 1000
 */
public class Solution01207 {

    // O(n) time,
    // O(n) space
    public boolean uniqueOccurrences(int[] arr) {
        Map<Integer, Integer> freqCount = new HashMap<>();
        for (int num : arr) {
            freqCount.merge(num, 1, Integer::sum);
        }
        Set<Integer> uniq = new HashSet<>(freqCount.size());
        for (Integer freq : freqCount.values()) {
            if (uniq.contains(freq)) {
                return false;
            }
            uniq.add(freq);
        }
        return true;
    }
```

```
}
```

271. `Solution01208.java`

```
package leetcode;

/**
 * 1208. Get Equal Substrings Within Budget
 * <a href="https://leetcode.com/problems/get-equal-substrings-within-budget">Web
link</a>
 * Medium
 */

public class Solution01208 {
    public int equalSubstring(String s, String t, int maxCost) {
        int n = s.length();
        int maxLength = 0;
        int currentCost = 0;
        int start = 0;

        for (int end = 0; end < n; end++) {
            currentCost += Math.abs(s.charAt(end) - t.charAt(end));
            while (currentCost > maxCost) {
                currentCost -= Math.abs(s.charAt(start) - t.charAt(start));
                start++;
            }
            maxLength = Math.max(maxLength, end - start + 1);
        }

        return maxLength;
    }
}
```

272. `Solution01219.java`

```
package leetcode;

/**
 * 1219. Path with Maximum Gold
 * <a href="https://leetcode.com/problems/path-with-maximum-gold">Web link</a>
 * Medium
 */

public class Solution01219 {

    public int getMaximumGold(int[][] grid) {
        final int rowCount = grid.length;
        final int colCount = grid[0].length;
        boolean[][] visited = new boolean[rowCount][colCount];
        int result = Integer.MIN_VALUE;
        for (int row = 0; row < rowCount; row++) {
            for (int col = 0; col < colCount; col++) {
                result = Math.max(result, dfsAndBacktrack(grid, row, col, visited));
            }
        }
        return result;
    }

    private int dfsAndBacktrack(int[][] grid, int row, int col, boolean[][] visited) {
        if (row < 0 || row >= grid.length ||
            col < 0 || col >= grid[0].length) {
            return 0;
        }
        if (grid[row][col] == 0 || visited[row][col]) {
            return 0;
        }

        visited[row][col] = true;

        int result = grid[row][col];

        // try left
        result = Math.max(result, grid[row][col] + dfsAndBacktrack(grid, row, col - 1,
visited));
        // try top
        result = Math.max(result, grid[row][col] + dfsAndBacktrack(grid, row - 1, col,
visited));
        // try right
        result = Math.max(result, grid[row][col] + dfsAndBacktrack(grid, row, col + 1,
visited));
        // try bottom
        result = Math.max(result, grid[row][col] + dfsAndBacktrack(grid, row + 1, col,
visited));

        visited[row][col] = false;
```

```
        return result;
    }

}

/*

        int rowCount = grid.length;
```

```
        int columnCount = grid[0].length;

        int maxGold = 0;
        boolean[][] visited = new boolean[rowCount][columnCount];

        for (int i = 0; i < rowCount; i++) {
            for (int j = 0; j < columnCount; j++) {
                if (grid[i][j] != 0) {
                    maxGold = Math.max(maxGold, dfs(grid, i, j, visited));
                }
            }
        }

        return maxGold;
    }

    private int dfs(int[][] grid, int i, int j, boolean[][] visited) {
        int rowCount = grid.length;
        int columnCount = grid[0].length;

        if (i < 0 || i >= rowCount || j < 0 || j >= columnCount || grid[i][j] == 0 ||
visited[i][j]) {
            return 0;
        }

        visited[i][j] = true;
        int gold = grid[i][j];
        int maxGold = 0;

        maxGold = Math.max(maxGold, gold + dfs(grid, i - 1, j, visited));
        maxGold = Math.max(maxGold, gold + dfs(grid, i + 1, j, visited));
        maxGold = Math.max(maxGold, gold + dfs(grid, i, j - 1, visited));
        maxGold = Math.max(maxGold, gold + dfs(grid, i, j + 1, visited));

        visited[i][j] = false;
        return maxGold;
    }

 */
```

273. `Solution01233.java`

```
/**
 * 1233. Remove Sub-Folders from the Filesystem
 * <a href="https://leetcode.com/problems/remove-sub-folders-from-the-filesystem">Web
link</a> |
 * Medium
 */

package leetcode;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Solution01233 {

    public List<String> removeSubfolders(String[] folder) {
        Arrays.sort(folder);
        List<String> result = new ArrayList<>();
        result.add(folder[0]);
        String currentValidFolder = folder[0] + "/";
        for (int i = 1; i < folder.length; i++) {
            if (!folder[i].startsWith(currentValidFolder)) {
                result.add(folder[i]);
                currentValidFolder = folder[i] + "/";
            }
        }
        return result;
    }

}
```

274. `Solution01235.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.TreeMap;

/**
 * https://leetcode.com/problems/maximum-profit-in-job-scheduling/
 * 1235. Maximum Profit in Job Scheduling
 * Hard
 * We have n jobs, where every job is scheduled to be done from startTime[i] to
endTime[i], obtaining a profit of profit[i].
 * <p>
 * You're given the startTime , endTime and profit arrays, you need to output the maximum
profit you can take such that there are no 2 jobs in the subset with overlapping time
range.
 * <p>
 * If you choose a job that ends at time X you will be able to start another job that
starts at time X.
 * <p>
 * Example 1:
 * <p>
 * Input: startTime = [1,2,3,3], endTime = [3,4,5,6], profit = [50,10,40,70]
 * Output: 120
 * Explanation: The subset chosen is the first and fourth job.
 * Time range [1-3]+[3-6] , we get profit of 120 = 50 + 70.
 * Example 2:
 * <p>
 * Input: startTime = [1,2,3,4,6], endTime = [3,5,10,6,9], profit = [20,20,100,70,60]
 * Output: 150
 * Explanation: The subset chosen is the first, fourth and fifth job.
 * Profit obtained 150 = 20 + 70 + 60.
 * Example 3:
 * <p>
 * Input: startTime = [1,1,1], endTime = [2,3,4], profit = [5,6,4]
 * Output: 6
 * <p>
 * <p>
 * Constraints:
 * <p>
 * 1 <= startTime.length == endTime.length == profit.length <= 5 * 104
 * 1 <= startTime[i] < endTime[i] <= 109
 * 1 <= profit[i] <= 104
 */
public class Solution01235 {

    // Time: O(nlogn)
    // Space: O(n)
    public int jobScheduling(int[] startTime, int[] endTime, int[] profit) {
        List<Job> jobs = new ArrayList<>();
```

```
        for (int i = 0; i < startTime.length; i++) {
            jobs.add(new Job(startTime[i], endTime[i], profit[i]));
        }

        jobs.sort(Comparator.comparingInt(a -> a.end));
        TreeMap<Integer, Integer> map = new TreeMap<>(); // key - time, value - profit

        int result = 0;

        for (Job job : jobs) {
            var floorEntry = map.floorEntry(job.start);
            int maxProfitTillStart = floorEntry == null ? 0 : floorEntry.getValue();
            result = Math.max(result, maxProfitTillStart + job.profit);
            map.put(job.end, result);
        }
        return result;
    }

    static class Job {
        int start;
        int end;
        int profit;

        public Job(int start, int end, int profit) {
            this.start = start;
            this.end = end;
            this.profit = profit;
        }
    }

}
```

275. `Solution01239.java`

```
package leetcode;

import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 1239. Maximum Length of a Concatenated String with Unique Characters
 * https://leetcode.com/problems/maximum-length-of-a-concatenated-string-with-unique-
characters
 * <p>
 * Given an array of strings arr. String s is a concatenation of a sub-sequence of arr
which have unique characters.
 * <p>
 * Return the maximum possible length of s.
 * <p>
 * Example 1:
 * <p>
 * Input: arr = ["un","iq","ue"]
 * Output: 4
 * Explanation: All possible concatenations are "","un","iq","ue","uniq" and "ique".
 * Maximum length is 4.
 * Example 2:
 * <p>
 * Input: arr = ["cha","r","act","ers"]
 * Output: 6
 * Explanation: Possible solutions are "chaers" and "acters".
 * Example 3:
 * <p>
 * Input: arr = ["abcdefghijklmnopqrstuvwxyz"]
 * Output: 26
 * <p>
 * Constraints:
 * <p>
 * 1 <= arr.length <= 16
 * 1 <= arr[i].length <= 26
 * arr[i] contains only lower case English letters.
 */
public class Solution01239 {

    // Time complexity: O(n * 2^n)
    // Space complexity: O(n)
    public int maxLength(List<String> arr) {
        ResultWrapper resultWrapper = new ResultWrapper();
        backtrace(arr, 0, new HashSet<>(), resultWrapper);
        return resultWrapper.maxLen;
    }

    private void backtrace(List<String> arr, int index, Set<Character> set, ResultWrapper
resultWrapper) {
        if (index == arr.size()) {
```

```
            return;
        }

        String s = arr.get(index);
        Set<Character> newSet = getSet(s, set);
        if (newSet == null) {
            // If the current string contains duplicate characters or characters already
in the set,
            // skip it and move on to the next string.
            backtrace(arr, index + 1, set, resultWrapper);
        } else {
            // If the current string contains unique characters, add them to the set and
update the maximum length.
            set.addAll(newSet);
            resultWrapper.maxLen = Math.max(resultWrapper.maxLen, set.size());
            backtrace(arr, index + 1, set, resultWrapper);

            // After exploring the current string, remove its characters from the set and
explore the remaining strings.
            set.removeAll(newSet);
            backtrace(arr, index + 1, set, resultWrapper);
        }
    }

    private Set<Character> getSet(String s, Set<Character> set) {
        Set<Character> newSet = new HashSet<>();
        for (char c : s.toCharArray()) {
            if (set.contains(c) || newSet.contains(c)) {
                return null;
            }
            newSet.add(c);
        }
        return newSet;
    }

    static class ResultWrapper {
        int maxLen = 0;
    }
}
```

276. `Solution01248.java`

```
package leetcode;

/**
 * 1248. Count Number of Nice Subarrays
 * <a href="https://leetcode.com/problems/count-number-of-nice-subarrays">Web link</a> |
 * Medium
 */

public class Solution01248 {

    public int numberOfSubarrays(int[] nums, int k) {
        int n = nums.length;
        int left = 0;
        int medium = 0;
        int oddCount = 0;
        int niceCount = 0;
        for (int right = 0; right < n; right++) {
            if (nums[right] % 2 == 1) {
                oddCount++;
            }

            while (oddCount > k) {
                if (nums[left] % 2 == 1) {
                    oddCount--;
                }
                left++;
                medium = left;
            }

            if (oddCount == k) {
                while (medium <= right && nums[medium] % 2 == 0) {
                    medium++;
                }
                niceCount += (medium - left + 1);
            }
        }
        return niceCount;
    }

}
```

277. `Solution01249.java`

```
package leetcode;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * 1249. Minimum Remove to Make Valid Parentheses
 * https://leetcode.com/problems/minimum-remove-to-make-valid-parentheses/
 * Medium
 */

public class Solution01249 {
    // Time: O(n)
    // Space: O(n)
    public String minRemoveToMakeValid(String s) {
        StringBuilder sb = new StringBuilder();
        int openedParentesis = 0;

        for (char c : s.toCharArray()) {
            if (c == '(') {
                openedParentesis++;
            } else if (c == ')' && openedParentesis == 0) {
                continue;
            } else if (c == ')') {
                openedParentesis--;
            }
            sb.append(c);
        }

        StringBuilder finalResult = new StringBuilder();
        int length = sb.length();

        for (int i = length - 1; i >= 0; i--) {
            if (sb.charAt(i) == '(' && openedParentesis-- > 0) {
                continue;
            }
            finalResult.append(sb.charAt(i));
        }

        return finalResult.reverse().toString();
    }

    public String minRemoveToMakeValidStack(String s) {
        StringBuilder res = new StringBuilder();
        Deque<Integer> stack = new ArrayDeque<>();
        boolean[] arr = new boolean[s.length()];
        for (int i = 0; i < s.length(); i++) {
            char cur = s.charAt(i);
            if (cur == ')') {
                if (stack.isEmpty()) {
                    arr[i] = true;
```

```
                } else {
                    arr[stack.pop()] = false;
                }
            } else if (cur == '(') {
                stack.add(i);
            }
        }

        while (!stack.isEmpty()) {
            arr[stack.pop()] = true;
        }

        for (int i = 0; i < s.length(); i++) {
            char cur = s.charAt(i);
            if ((cur == '(' || cur == ')') && arr[i]) continue;
            res.append(cur);
        }
        return res.toString();
    }

}
```

278. `Solution01255.java`

```
package leetcode;

/**
 * 1255. Maximum Score Words Formed by Letters
 * https://leetcode.com/problems/maximum-score-words-formed-by-letters/
 * Hard
 */

public class Solution01255 {
    public int maxScoreWords(String[] words, char[] letters, int[] score) {
        int[] letterCountsInWords = new int[26];
        int[] letterCountsInLetters = new int[26];

        for (char letter : letters) {
            letterCountsInLetters[letter - 'a']++;
        }

        int maxScore = 0;
        maxScore = calculateMaxScore(words, score, letterCountsInLetters,
letterCountsInWords, 0, 0, maxScore);
        return maxScore;
    }

    private int calculateMaxScore(String[] words, int[] score, int[]
letterCountsInLetters, int[] letterCountsInWords, int wordIndex, int currentScore, int
maxScore) {
        for (int i = 0; i < 26; i++) {
            if (letterCountsInWords[i] > letterCountsInLetters[i]) return maxScore;
        }

        maxScore = Math.max(maxScore, currentScore);

        for (int i = wordIndex; i < words.length; i++) {
            for (char c : words[i].toCharArray()) {
                letterCountsInWords[c - 'a']++;
                currentScore += score[c - 'a'];
            }

            maxScore = calculateMaxScore(words, score, letterCountsInLetters,
letterCountsInWords, i + 1, currentScore, maxScore);

            for (char c : words[i].toCharArray()) {
                letterCountsInWords[c - 'a']--;
                currentScore -= score[c - 'a'];
            }
        }

        return maxScore;
    }

}
```

279. `Solution01277.java`

```
/**
 * 1277. Count Square Submatrices with All Ones
 * <a href="https://leetcode.com/problems/count-square-submatrices-with-all-ones">Web
link</a>
 * Medium
 */

package leetcode;

public class Solution01277 {

    public int countSquares(int[][] matrix) {
        int rowSize = matrix.length;
        int colSize = matrix[0].length;

        for (int i = 1; i < rowSize; i++) {
            for (int j = 1; j < colSize; j++) {
                if (matrix[i][j] == 1) {
                    matrix[i][j] += Math.min(matrix[i - 1][j],
                            Math.min(matrix[i - 1][j - 1], matrix[i][j - 1]));
                }
            }
        }

        int total = 0;
        for (int[] rows : matrix) {
            for (int rowValue : rows) {
                total += rowValue;
            }
        }

        return total;
    }

}
```

280. `Solution01289.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 1289. Minimum Falling Path Sum II
 * <a href="https://leetcode.com/problems/minimum-falling-path-sum-ii/">Web link</a> |
 * Hard
 */

public class Solution01289 {

    public int minFallingPathSum(int[][] grid) {
        int n = grid.length;

        int[] dp = grid[0];

        for (int i = 1; i < n; i++) {
            int min1 = Integer.MAX_VALUE;
            int min2 = Integer.MAX_VALUE;
            int idx1 = -1;

            for (int j = 0; j < n; j++) {
                if (dp[j] < min1) {
                    min2 = min1;
                    min1 = dp[j];
                    idx1 = j;
                } else if (dp[j] < min2) {
                    min2 = dp[j];
                }
            }

            int[] dpNew = new int[n];
            final int[] currentGridLine = grid[i];
            for (int j = 0; j < n; j++) {
                dpNew[j] = currentGridLine[j] + (j == idx1 ? min2 : min1);
            }

            dp = dpNew;
        }

        return Arrays.stream(dp).min().orElse(0);
    }

}
```

281. `Solution01291.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.List;

/**
 * https://leetcode.com/problems/sequential-digits
 * 1291. Sequential Digits
 * Medium
 * <p>
 * An integer has sequential digits if and only if each digit in the number is one more
than the previous digit.
 * <p>
 * Return a sorted list of all the integers in the range [low, high] inclusive that have
sequential digits.
 * <p>
 * Example 1:
 * <p>
 * Input: low = 100, high = 300
 * Output: [123,234]
 * Example 2:
 * <p>
 * Input: low = 1000, high = 13000
 * Output: [1234,2345,3456,4567,5678,6789,12345]
 * <p>
 * Constraints:
 * <p>
 * 10 <= low <= high <= 10^9
 */
public class Solution01291 {

    // Time complexity: O(1)
    // Space complexity: O(1)
    public List<Integer> sequentialDigits(int low, int high) {
        List<Integer> result = new ArrayList<>();

        int minDigits = getNumberOfDigits(low);
        int maxDigits = getNumberOfDigits(high);

        for (int length = minDigits; length <= maxDigits; length++) {
            for (int start = 1; start <= 9 - length + 1; start++) {
                int num = 0;
                for (int i = 0; i < length; i++) {
                    num = num * 10 + (start + i);
                }
                if (num >= low && num <= high) {
                    result.add(num);
                }
            }
        }
```

```
        return result;
    }

    private int getNumberOfDigits(int number) {
        return (int) (Math.log10(number) + 1);
    }

    // Time complexity: O(1)
    // Space complexity: O(1)
    public List<Integer> sequentialDigitsWithSorting(int low, int high) {
        List<Integer> result = new ArrayList<>();

        for (int i = 1; i <= 9; i++) {
            int num = i;
            for (int j = i + 1; j <= 9; j++) {
                num = num * 10 + j;
                if (num >= low && num <= high) {
                    result.add(num);
                }
            }
        }
        result.sort(null);

        return result;
    }

}
```

282. `Solution01310.java`

```
package leetcode;

/**
 * 1310. XOR Queries of a Subarray
 * <a href="https://leetcode.com/problems/xor-queries-of-a-subarray">Web link</a>
 * Medium
 */

public class Solution01310 {

    public int[] xorQueries(int[] arr, int[][] queries) {
        int[] xorPrefixes = new int[arr.length + 1];
        for (int i = 0; i < arr.length; i++) {
            xorPrefixes[i + 1] = xorPrefixes[i] ^ arr[i];
        }
        int[] result = new int[queries.length];
        for (int i = 0; i < queries.length; i++) {
            int[] query = queries[i];
            final int left = query[0];
            final int right = query[1];
            result[i] = xorPrefixes[left] ^ xorPrefixes[right + 1];
        }
        return result;
    }

}
```

283. `Solution01325.java`

```
package leetcode;

import leetcode.common.TreeNode;

/**
 * 1325. Delete Leaves With a Given Value
 * <a href="https://leetcode.com/problems/delete-leaves-with-a-given-value">Web link</a>
 * Medium
 */

public class Solution01325 {

    public TreeNode removeLeafNodes(TreeNode root, int target) {
        if (root == null) {
            return null;
        }

        root.left = removeLeafNodes(root.left, target);
        root.right = removeLeafNodes(root.right, target);

        if (root.val == target &&
            root.left == null &&
            root.right == null) {
            return null;
        }

        return root;
    }

}
```

284. `Solution01331.java`

```
/**
 * 1331. Rank Transform of an Array
 * <a href="https://leetcode.com/problems/rank-transform-of-an-array">Web link</a>
 * Easy
 */

package leetcode;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

public class Solution01331 {

    public int[] arrayRankTransform(int[] arr) {
        int[] arrCopy = Arrays.copyOf(arr, arr.length);
        Arrays.sort(arrCopy);
        int index = 1;
        int prev = Integer.MIN_VALUE;
        Map<Integer, Integer> map = new HashMap<>();
        for (int j : arrCopy) {
            if (j == prev)
                continue;
            map.put(j, index++);
            prev = j;
        }
        for (int i = 0; i < arr.length; i++) { // n
            arr[i] = map.get(arr[i]);
        }
        return arr;
    }

}
```

285. `Solution01334.java`

```
package leetcode;

import java.util.*;

/**
 * 1334. Find the City With the Smallest Number of Neighbors at a Threshold Distance
 * <a href="https://leetcode.com/problems/find-the-city-with-the-smallest-number-of-
neighbors-at-a-threshold-distance">Web link</a>
 * Medium
 */

public class Solution01334 {

    public int findTheCity(int n, int[][] edges, int distanceThreshold) {
        List<List<Connection>> graph = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
        }
        for (int[] edge : edges) {
            int node1 = edge[0];
            int node2 = edge[1];
            int distance = edge[2];
            graph.get(node1).add(new Connection(node2, distance));
            graph.get(node2).add(new Connection(node1, distance));
        }

        int minNumber = n;
        int minCity = -1;

        for (int source = 0; source < n; source++) {
            int neighbors = dijkstra(graph, source, distanceThreshold);
            if (neighbors <= minNumber) {
                minNumber = neighbors;
                minCity = source;
            }
        }

        return minCity;
    }

    private int dijkstra(List<List<Connection>> graph, int source, int distanceThreshold)
{
        PriorityQueue<Connection> minHeap = new PriorityQueue<>
(Comparator.comparingInt(Connection::distance));
        minHeap.add(new Connection(source, 0));
        Set<Integer> visited = new HashSet<>();

        while (!minHeap.isEmpty()) {
            Connection current = minHeap.poll();
            final int currentCity = current.city();
            if (visited.contains(currentCity)) {
```

```
                continue;
            }
            visited.add(currentCity);
            for (Connection neighbor : graph.get(currentCity)) {
                final int newDistance = current.distance() + neighbor.distance();
                if (newDistance <= distanceThreshold) {
                    minHeap.add(new Connection(neighbor.city(), newDistance));
                }
            }
        }
        return visited.size() - 1;
    }

    private record Connection(int city, int distance) {
    }

}
```

286. `Solution01346.java`

```
/**
 * 1346. Check If N and Its Double Exist
 * <a href="https://leetcode.com/problems/check-if-n-and-its-double-exist">Web link</a>
 * Easy
 */

package leetcode;

import java.util.HashSet;
import java.util.Set;

public class Solution01346 {

    public boolean checkIfExist(int[] arr) {
        Set<Integer> set = new HashSet<>();
        for (int elem : arr) {
            if (set.contains(elem * 2)) {
                return true;
            }
            if (elem % 2 == 0 && set.contains(elem / 2)) {
                return true;
            }
            set.add(elem);
        }
        return false;
    }

}
```

287. `Solution01347.java`

```
package leetcode;

import java.util.HashMap;
import java.util.Map;

/**
 * https://leetcode.com/problems/minimum-number-of-steps-to-make-two-strings-anagram/
 * 1347. Minimum Number of Steps to Make Two Strings Anagram
 * Medium
 * ---
 * Given two equal-size strings s and t. In one step you can choose any character of t
and replace it with another
 * character.
 * Return the minimum number of steps to make t an anagram of s.
 * An Anagram of a string is a string that contains the same characters with a different
(or the same) ordering.
 * Example 1:
 * Input: s = "bab", t = "aba"
 * Output: 1
 * Explanation: Replace the first 'a' in t with b, t = "bba" which is anagram of s.
 * Example 2:
 * Input: s = "leetcode", t = "practice"
 * Output: 5
 * Explanation: Replace 'p', 'r', 'a', 'i' and 'c' from t with proper characters to make
t anagram of s.
 * Example 3:
 * Input: s = "anagram", t = "mangaar"
 * Output: 0
 * Explanation: "anagram" and "mangaar" are anagrams.
 * <p>
 * Constraints:
 * 1 <= s.length <= 50000
 * s.length == t.length
 * s and t contain lower-case English letters only.
 */
public class Solution01347 {

    // Time: O(n)
    // Space: O(n)
    public int minSteps(String s, String t) {
        int len = s.length();
        Map<Character, Integer> freqMap = new HashMap<>();
        for (int i = 0; i < len; i++) {
            freqMap.merge(s.charAt(i), 1, Integer::sum);
            freqMap.merge(t.charAt(i), -1, Integer::sum);
        }
        return freqMap.values()
                .stream()
                .mapToInt(Integer::intValue)
                .filter(i -> i > 0)
                .sum();
```

```
    }

}
```

288. `Solution01367.java`

```
package leetcode;

import leetcode.common.ListNode;
import leetcode.common.TreeNode;

/**
 * 1367. Linked List in Binary Tree
 * <a href="https://leetcode.com/problems/linked-list-in-binary-tree">Web link</a>
 * Medium
 */

public class Solution01367 {

    public boolean isSubPath(ListNode head, TreeNode root) {
        if (checkSubTree(head, root)) {
            return true;
        } else if (root == null) {
            return false;
        } else {
            return isSubPath(head, root.left) || isSubPath(head, root.right);
        }
    }

    private boolean checkSubTree(ListNode head, TreeNode root) {
        if (head == null) {
            return true;
        }
        if (root == null) {
            return false;
        }
        if (head.val != root.val) {
            return false;
        }
        return checkSubTree(head.next, root.left) || checkSubTree(head.next, root.right);
    }

}
```

289. `Solution01368.java`

```
/**
 * 1368. Minimum Cost to Make at Least One Valid Path in a Grid
 * <a href="https://leetcode.com/problems/minimum-cost-to-make-at-least-one-valid-path-
in-a-grid">Web link</a>
 * Hard
 */

package leetcode;

import java.util.ArrayDeque;
import java.util.Arrays;
import java.util.Deque;

public class Solution01368 {

    public int minCost(int[][] grid) {
        // Direction: right, left, down, up
        int[][] dirs = new int[][]{{0, 1}, {0, -1}, {1, 0}, {-1, 0}};

        int rows = grid.length;
        int cols = grid[0].length;
        int[][] minCost = new int[rows][cols];
        for (int row = 0; row < rows; row++) {
            Arrays.fill(minCost[row], Integer.MAX_VALUE);
        }
        //element [row, col]
        Deque<int[]> deque = new ArrayDeque<>();
        //insert first element
        deque.offer(new int[]{0, 0});
        minCost[0][0] = 0; //cost of source to source

        while (!deque.isEmpty()) {
            int[] curr = deque.poll(); //pollFirst
            int row = curr[0];
            int col = curr[1];

            for (int dir = 0; dir < 4; dir++) {
                int nextRow = row + dirs[dir][0];
                int nextCol = col + dirs[dir][1];

                // Check if new position is valid
                if (
                        nextRow >= 0 &&
                                nextRow < rows &&
                                nextCol >= 0 &&
                                nextCol < cols
                ) {
                    // if direction is changed then cost will be 1
                    // grid[row][col] -> [1,4]
                    // dir -> [0,3] -> [1,4]
                    int cost = minCost[row][col];
```

```
                    int nextCellCost = (dir + 1 == grid[row][col]) ? 0 : 1;
                    int newCost = cost + nextCellCost;

                    if (newCost < minCost[nextRow][nextCol]) {
                        minCost[nextRow][nextCol] = newCost;
                        if (nextCellCost == 0) {
                            deque.offerFirst(new int[]{nextRow, nextCol});
                        } else {
                            deque.offerLast(new int[]{nextRow, nextCol});
                        }

                    }
                }
            }
        }

        return minCost[rows - 1][cols - 1];
    }

}
```

290. `Solution01371.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 1371. Find the Longest Substring Containing Vowels in Even Counts
 * <a href="https://leetcode.com/problems/find-the-longest-substring-containing-vowels-
in-even-counts">Web link</a>
 * Medium
 */

public class Solution01371 {

    public int findTheLongestSubstring(String s) {
        int length = s.length();
        int[] vowelCounts = new int[5]; // a, e, i, o, u
        int[] firstOccurrence = new int[32];
        Arrays.fill(firstOccurrence, -1);
        firstOccurrence[0] = 0;
        int result = 0;

        for (int i = 0; i < length; i++) {
            char c = s.charAt(i);
            if (c == 'a') {
                vowelCounts[0]++;
            } else if (c == 'e') {
                vowelCounts[1]++;
            } else if (c == 'i') {
                vowelCounts[2]++;
            } else if (c == 'o') {
                vowelCounts[3]++;
            } else if (c == 'u') {
                vowelCounts[4]++;
            }

            int state = 0;
            for (int j = 0; j < 5; j++) {
                if (vowelCounts[j] % 2 != 0) {
                    state |= (1 << j);
                }
            }

            if (firstOccurrence[state] == -1) {
                firstOccurrence[state] = i + 1;
            } else {
                result = Math.max(result, i - firstOccurrence[state] + 1);
            }
        }

        return result;
    }
```

```
}
```

291. `Solution01380.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 1380. Lucky Numbers in a Matrix
 * <a href="https://leetcode.com/problems/lucky-numbers-in-a-matrix">Web link</a>
 * Easy
 */

public class Solution01380 {

    public List<Integer> luckyNumbers(int[][] matrix) {
        Map<Integer, Integer> resultCandidates = new HashMap<>(); // <columnIndex,
minValue>
        int columns = matrix[0].length;
        for (int[] row : matrix) {
            int minIndex = 0;
            for (int i = 1; i < columns; i++) {
                if (row[i] < row[minIndex]) {
                    minIndex = i;
                }
            }
            int previousColumnCandidate = resultCandidates.getOrDefault(minIndex, 0);
            if (row[minIndex] > previousColumnCandidate) {
                resultCandidates.put(minIndex, row[minIndex]);
            }
        }
        List<Integer> result = new ArrayList<>();
        for (var entry : resultCandidates.entrySet()) {
            int columnIndex = entry.getKey();
            int minValue = entry.getValue();
            boolean isLucky = true;
            for (int[] row : matrix) {
                if (row[columnIndex] > minValue) {
                    isLucky = false;
                    break;
                }
            }
            if (isLucky) {
                result.add(minValue);
            }
        }
        return result;
    }

    public List<Integer> luckyNumbersOptimized(int[][] matrix) {
        int currentCandidate = 0;
```

```
        int indexOfCandidate = 0;
        int columns = matrix[0].length;
        for (int[] row : matrix) {
            int minIndex = 0;
            for (int i = 1; i < columns; i++) {
                if (row[i] < row[minIndex]) {
                    minIndex = i;
                }
            }
            if (row[minIndex] > currentCandidate) {
                currentCandidate = row[minIndex];
                indexOfCandidate = minIndex;
            }
        }
        for (int[] row : matrix) {
            if (row[indexOfCandidate] > currentCandidate) {
                return List.of();
            }
        }
        return List.of(currentCandidate);
    }

}
```

292. `Solution01381.java`

```
package leetcode;

/**
 * 1381. Design a Stack With Increment Operation
 * <a href="https://leetcode.com/problems/design-a-stack-with-increment-operation">Web
link</a>
 * Medium
 */

public class Solution01381 {

    static class CustomStack {

        private final int[] data;
        private int top;

        public CustomStack(int maxSize) {
            data = new int[maxSize];
            top = -1;
        }

        public void push(int x) {
            if (top < data.length - 1) {
                data[++top] = x;
            }
        }

        public int pop() {
            if (top == -1) {
                return -1;
            }
            return data[top--];
        }

        public void increment(int k, int val) {
            final int limit = Math.min(k, top + 1);
            for (int i = 0; i < limit; i++) {
                data[i] += val;
            }
        }

    }

}

/**
 * Your CustomStack object will be instantiated and called as such:
 * CustomStack obj = new CustomStack(maxSize);
 * obj.push(x);
 * int param_2 = obj.pop();
```

```
 * obj.increment(k,val);
 */
```

293. `Solution01382.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.ArrayList;
import java.util.List;

/**
 * 1382. Balance a Binary Search Tree
 * <a href="https://leetcode.com/problems/balance-a-binary-search-tree/">Web link</a>
 * Medium
 */

public class Solution01382 {

    public TreeNode balanceBST(TreeNode root) {
        List<Integer> inorderList = new ArrayList<>();
        inorderTraversal(root, inorderList);
        return buildBalancedBST(inorderList, 0, inorderList.size() - 1);
    }

    private void inorderTraversal(TreeNode root, List<Integer> inorderList) {
        if (root == null) {
            return;
        }
        inorderTraversal(root.left, inorderList);
        inorderList.add(root.val);
        inorderTraversal(root.right, inorderList);
    }

    private TreeNode buildBalancedBST(List<Integer> inorderList, int left, int right) {
        if (left > right) {
            return null;
        }
        int mid = left + (right - left) / 2;
        TreeNode root = new TreeNode(inorderList.get(mid));
        root.left = buildBalancedBST(inorderList, left, mid - 1);
        root.right = buildBalancedBST(inorderList, mid + 1, right);
        return root;
    }

}
```

294. `Solution01395.java`

```
package leetcode;

/**
 * 1395. Count Number of Teams
 * <a href="https://leetcode.com/problems/count-number-of-teams">Web link</a>
 * Medium
 */

public class Solution01395 {

    public int numTeams(int[] rating) {
        int n = rating.length;
        int count = 0;
        for (int i = 1; i < n - 1; i++) {
            int leftLess = 0;
            int leftGreater = 0;
            int rightLess = 0;
            int rightGreater = 0;
            for (int j = 0; j < i; j++) {
                if (rating[j] < rating[i]) {
                    leftLess++;
                } else if (rating[j] > rating[i]) {
                    leftGreater++;
                }
            }
            for (int j = i + 1; j < n; j++) {
                if (rating[j] < rating[i]) {
                    rightLess++;
                } else if (rating[j] > rating[i]) {
                    rightGreater++;
                }
            }
            count += leftLess * rightGreater + leftGreater * rightLess;
        }
        return count;
    }

}
```

295. `Solution01400.java`

```
/**
 * 1400. Construct K Palindrome Strings
 * <a href="https://leetcode.com/problems/construct-k-palindrome-strings">Web link</a>
 * Medium
 */

package leetcode;

public class Solution01400 {

    public boolean canConstruct(String s, int k) {
        if (s.length() < k)
            return false;

        int[] freq = new int[26];
        for (char c : s.toCharArray()) {
            freq[c - 'a']++;
        }

        int odds = 0;
        for (int count : freq) {
            if (count % 2 != 0)
                odds++;
        }

        return odds <= k;
    }

}
```

296. `Solution01404.java`

```
package leetcode;

/**
 * 1404. Number of Steps to Reduce a Number in Binary Representation to One
 * <a href="https://leetcode.com/problems/number-of-steps-to-reduce-a-number-in-binary-representation-to-one">Web link</a> |
 * Medium
 */

public class Solution01404 {

    public int numSteps(String s) {
        int steps = 0;
        char[] binary = s.toCharArray();
        int n = binary.length;
        int carry = 0;

        for (int i = n - 1; i > 0; i--) {
            if (binary[i] - '0' + carry == 1) {
                steps += 2;
                carry = 1;
            } else {
                steps++;
            }
        }

        return steps + carry;
    }

}
```

297. `Solution01405.java`

```
package leetcode;

/**
 * 1405. Longest Happy String
 * <a href="https://leetcode.com/problems/longest-happy-string">Web link</a>
 * Medium
 */

public class Solution01405 {

    public String longestDiverseString(int a, int b, int c) {
        int currentA = 0;
        int currentB = 0;
        int currentC = 0;

        int totalIterations = a + b + c;
        StringBuilder ans = new StringBuilder();

        for (int i = 0; i < totalIterations; i++) {
            if (shouldAppend(a, b, c, currentA, currentB, currentC)) {
                ans.append('a');
                a--;
                currentA++;
                currentB = 0;
                currentC = 0;
            } else if (shouldAppend(b, a, c, currentB, currentA, currentC)) {
                ans.append('b');
                b--;
                currentB++;
                currentA = 0;
                currentC = 0;
            } else if (shouldAppend(c, a, b, currentC, currentA, currentB)) {
                ans.append('c');
                c--;
                currentC++;
                currentA = 0;
                currentB = 0;
            }
        }
        return ans.toString();
    }

    private boolean shouldAppend(int allowedFirst, int allowedSecond, int allowedThird,
int currentFirst, int currentSecond, int currentThird) {
        return (allowedFirst >= allowedSecond && allowedFirst >= allowedThird &&
currentFirst != 2) || (allowedFirst > 0 && (currentSecond == 2 || currentThird == 2));
    }


}
```

298. `Solution01408.java`

```
/**
 * 1408. String Matching in an Array
 * <a href="https://leetcode.com/problems/string-matching-in-an-array">Web link</a>
 * Easy
 */

package leetcode;

import java.util.ArrayList;
import java.util.List;

public class Solution01408 {

    public List<String> stringMatching(String[] words) {
        List<String> result = new ArrayList<>();
        int len = words.length;
        for (int i = 0; i < len; i++) {
            checkIfSubstring(words, i, result);
        }
        return result;
    }

    private void checkIfSubstring(String[] words, int i, List<String> result) {
        int len = words.length;
        String wordToCheck = words[i];
        for (int j = 0; j < len; j++) {
            final String word = words[j];
            if (i != j &&
                    word.length() >= wordToCheck.length() &&
                    word.contains(wordToCheck)) {
                result.add(wordToCheck);
                break;
            }
        }
    }

}
```

299. `Solution01422.java`

```
/**
 * 1422. Maximum Score After Splitting a String
 * <a href="https://leetcode.com/problems/maximum-score-after-splitting-a-string">Web
link</a>
 * Easy
 */

package leetcode;

public class Solution01422 {

    public int maxScore(String s) {
        char[] chArr = s.toCharArray();
        int count1 = 0;
        for (char ch : chArr) {
            if (ch == '1') {
                count1++;
            }
        }
        int maxResult = 0;
        int count0 = 0;
        int lastIndex = chArr.length - 1;
        for (int i = 0; i <= lastIndex; i++) {
            char ch = chArr[i];
            if (ch == '0') {
                if (i < lastIndex) {
                    count0++;
                }
            } else if (ch == '1') {
                count1--;
            }
            maxResult = Math.max(maxResult, count0 + count1);
        }
        return maxResult;
    }

}
```

300. `Solution01438.java`

```
package leetcode;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.NavigableMap;
import java.util.TreeMap;

/**
 * 1438. Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit
 * <a href="https://leetcode.com/problems/longest-continuous-subarray-with-absolute-diff-less-than-or-equal-to-limit">Web link</a>
 * Medium
 */

public class Solution01438 {

    public int longestSubarray(int[] nums, int limit) {
        int leftIndex = 0;
        int max = Integer.MIN_VALUE;
        int min = Integer.MAX_VALUE;
        NavigableMap<Integer, Integer> map = new TreeMap<>();
        int maxDiff = 0;
        int n = nums.length;
        for (int rightIndex = 0; rightIndex < n; rightIndex++) {
            int right = nums[rightIndex];
            map.put(right, map.getOrDefault(right, 0) + 1);
            max = Math.max(max, right);
            min = Math.min(min, right);
            while (max - min > limit) {
                int left = nums[leftIndex];
                final int newLeftValue = map.get(left) - 1;
                if (newLeftValue == 0) {
                    map.remove(left);
                    if (map.isEmpty()) {
                        max = 0;
                        min = 0;
                    } else {
                        max = map.lastKey();
                        min = map.firstKey();
                    }
                } else {
                    map.put(left, newLeftValue);
                }
                leftIndex++;
            }
            maxDiff = Math.max(maxDiff, rightIndex - leftIndex + 1);
        }
        return maxDiff;
    }

    public int longestSubarrayOptimized(int[] nums, int limit) {
```

```
        Deque<Integer> maxQueue = new ArrayDeque<>();
        Deque<Integer> minQueue = new ArrayDeque<>();
        int leftIndex = 0;
        int maxDiff = 0;
        int n = nums.length;
        for (int rightIndex = 0; rightIndex < n; rightIndex++) {
            int right = nums[rightIndex];
            while (!maxQueue.isEmpty() && maxQueue.peekLast() < right) {
                maxQueue.pollLast();
            }
            while (!minQueue.isEmpty() && minQueue.peekLast() > right) {
                minQueue.pollLast();
            }
            maxQueue.add(right);
            int currentMax = maxQueue.peekFirst();
            minQueue.add(right);
            int currentMin = minQueue.peekFirst();
            if (currentMax - currentMin > limit) {
                final int left = nums[leftIndex];
                if (currentMax == left) {
                    maxQueue.pollFirst();
                }
                if (currentMin == left) {
                    minQueue.pollFirst();
                }
                leftIndex++;
            }
            maxDiff = Math.max(maxDiff, rightIndex - leftIndex + 1);
        }
        return maxDiff;
    }
}
```

301. `Solution01442.java`

```
package leetcode;

/**
 * 1442. Count Triplets That Can Form Two Arrays of Equal XOR
 * <a href="https://leetcode.com/problems/count-triplets-that-can-form-two-arrays-of-
equal-xor">Web link</a>
 * Medium
 */

public class Solution01442 {
    public int countTriplets(int[] arr) {
        int n = arr.length;
        int[] prefix = new int[n + 1];
        for (int i = 0; i < n; i++) {
            prefix[i + 1] = prefix[i] ^ arr[i];
        }
        int count = 0;
        for (int i = 0; i < n; i++) {
            for (int k = i + 1; k < n; k++) {
                if (prefix[i] == prefix[k + 1]) {
                    count += k - i;
                }
            }
        }
        return count;
    }

    public int countTripletsOptimized(int[] arr) {
        int res = 0;
        for (int i = 0; i < arr.length; i++) {
            int val = arr[i];
            for (int k = i + 1; k < arr.length; k++) {
                val = val ^ arr[k];
                if (val == 0) {
                    res += (k - i);
                }
            }

        }
        return res;
    }
}
```

302. `Solution01448.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.LinkedList;
import java.util.Queue;

/**
 * https://leetcode.com/problems/count-good-nodes-in-binary-tree/
 * 1448. Count Good Nodes in Binary Tree
 * Medium
 */
public class Solution01448 {

    // Time O(n)
    // Space O(n)
    public int goodNodesDfs(TreeNode root) {
        return goodNodesDfs(root, Integer.MIN_VALUE);
    }

    private int goodNodesDfs(TreeNode root, int max) {
        if (root == null) {
            return 0;
        }

        int count = 0;
        if (root.val >= max) {
            count++;
        }

        count += goodNodesDfs(root.left, Math.max(max, root.val));
        count += goodNodesDfs(root.right, Math.max(max, root.val));

        return count;
    }

    public int goodNodesBfs(TreeNode root) {
        if (root == null) {
            return 0;
        }

        int count = 0;
        Queue<Pair> queue = new LinkedList<>();
        queue.add(new Pair(root, root.val));

        while (!queue.isEmpty()) {
            Pair pair = queue.poll();
            TreeNode node = pair.node;
            int max = pair.max;

            if (node.val >= max) {
```

```
                count++;
            }

            max = Math.max(max, node.val);

            if (node.left != null) {
                queue.add(new Pair(node.left, max));
            }

            if (node.right != null) {
                queue.add(new Pair(node.right, max));
            }
        }

        return count;
    }

    // Pair class to store a node and the maximum value in its path from the root
    private static class Pair {
        TreeNode node;
        int max;

        Pair(TreeNode node, int max) {
            this.node = node;
            this.max = max;
        }
    }

}
```

303. `Solution01455.java`

```
/**
 * 1455. Check If a Word Occurs As a Prefix of Any Word in a Sentence
 * <a href="https://leetcode.com/problems/check-if-a-word-occurs-as-a-prefix-of-any-word-in-a-sentence">Web link</a>
 * Easy
 */

package leetcode;

public class Solution01455 {

    public int isPrefixOfWord(String sentence, String searchWord) {
        String[] splitted = sentence.split(" ");
        for (int i = 0; i < splitted.length; i++) {
            if (splitted[i].startsWith(searchWord)) {
                return i + 1;
            }
        }
        return -1;
    }

}
```

304. `Solution01457.java`

```
package leetcode;

import leetcode.common.TreeNode;

/**
 * 1457. Pseudo-Palindromic Paths in a Binary Tree
 * https://leetcode.com/problems/pseudo-palindromic-paths-in-a-binary-tree/description/
 * Medium
 * <p>
 * Given a binary tree where node values are digits from 1 to 9. A path in the binary tree is said to be pseudo-palindromic if at least one permutation of the node values in the path is a palindrome.
 * <p>
 * Return the number of pseudo-palindromic paths going from the root node to leaf nodes.
 * <p>
 * Example 1:
 * <p>
 * Input: root = [2,3,1,3,1,null,1]
 * Output: 2
 * Explanation: The figure above represents the given binary tree. There are three paths going from the root node to leaf nodes: the red path [2,3,3], the green path [2,1,1], and the path [2,3,1]. Among these paths only red path and green path are pseudo-palindromic paths since the red path [2,3,3] can be rearranged in [3,2,3] (palindrome) and the green path [2,1,1] can be rearranged in [1,2,1] (palindrome).
 * Example 2:
 * <p>
 * Input: root = [2,1,1,1,3,null,null,null,null,null,1]
 * Output: 1
 * Explanation: The figure above represents the given binary tree. There are three paths going from the root node to leaf nodes: the green path [2,1,1], the path [2,1,3,1], and the path [2,1]. Among these paths only the green path is pseudo-palindromic since [2,1,1] can be rearranged in [1,2,1] (palindrome).
 * Example 3:
 * <p>
 * Input: root = [9]
 * Output: 1
 * <p>
 * Constraints:
 * <p>
 * The number of nodes in the tree is in the range [1, 105].
 * 1 <= Node.val <= 9
 */
public class Solution01457 {

    public int pseudoPalindromicPaths(TreeNode root) {
        int[] freq = new int[10];
        return dfs(root, freq);
    }

    private int dfs(TreeNode node, int[] freq) {
        if (node == null) {
```

```
            return 0;
        }

        freq[node.val]++;
        int count = 0;
        if (node.left == null && node.right == null) {
            if (isPseudoPalindromic(freq)) {
                count++;
            }
        } else {
            count += dfs(node.left, freq);
            count += dfs(node.right, freq);
        }
        freq[node.val]--;

        return count;
    }

    private boolean isPseudoPalindromic(int[] freq) {
        int oddCount = 0;
        for (int i = 1; i < 10; i++) {
            if (freq[i] % 2 != 0) {
                oddCount++;
            }
            if (oddCount > 1) {
                return false;
            }
        }
        return true;
    }

}
```

305. `Solution01460.java`

```
package leetcode;

import java.util.HashMap;
import java.util.Map;

/**
 * 1460. Make Two Arrays Equal by Reversing Subarrays
 * <a href="https://leetcode.com/problems/make-two-arrays-equal-by-reversing-
subarrays">Web link</a>
 * Easy
 */

public class Solution01460 {

    public boolean canBeEqual(int[] target, int[] arr) {
        Map<Integer, Integer> freqMapArr = new HashMap<>();
        for (int num : arr) {
            freqMapArr.put(num, freqMapArr.getOrDefault(num, 0) + 1);
        }
        for (int num : target) {
            int freq = freqMapArr.getOrDefault(num, 0);
            if (freq == 0) {
                return false;
            } else {
                freqMapArr.put(num, freq - 1);
            }
        }
        return freqMapArr.values()
                .stream()
                .noneMatch(freq -> freq != 0);
    }

}
```

306. `Solution01463.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/cherry-pickup-ii/
 * 1463. Cherry Pickup II
 * Hard
 */
public class Solution01463 {

    // Time: O(m*n^2)
    // Space: O(m*n^2)
    public int cherryPickupDp3d(int[][] grid) {
        // Get the number of rows and columns in the grid
        int m = grid.length;
        int n = grid[0].length;

        // Initialize a 3D dynamic programming array
        int[][][] dp = new int[m][n][n];

        // Iterate over the rows from bottom to top
        for (int row = m - 1; row >= 0; row--) {
            // Iterate over all possible positions for the first robot
            for (int col1 = 0; col1 < n; col1++) {
                // Iterate over all possible positions for the second robot
                for (int col2 = 0; col2 < n; col2++) {
                    int result = 0;
                    // Add the cherries picked by the first robot
                    result += grid[row][col1];
                    // If the two robots are not at the same position, add the cherries
picked by the second robot
                    if (col1 != col2) {
                        result += grid[row][col2];
                    }
                    // If we are not at the last row, add the maximum cherries that can
be picked from the next row
                    if (row != m - 1) {
                        int max = 0;
                        // Iterate over all possible positions for the first robot in the
next row
                        for (int newCol1 = col1 - 1; newCol1 <= col1 + 1; newCol1++) {
                            // Iterate over all possible positions for the second robot
in the next row
                            for (int newCol2 = col2 - 1; newCol2 <= col2 + 1; newCol2++)
{
                                // If the new positions are valid, update the maximum
cherries that can be picked
                                if (newCol1 >= 0 && newCol1 < n && newCol2 >= 0 &&
newCol2 < n) {
                                    max = Math.max(max, dp[row + 1][newCol1][newCol2]);
                                }
                            }
```

```
                    }
                    // Add the maximum cherries that can be picked from the next row
to the current result
                    result += max;
                }
                // Update the dynamic programming array
                dp[row][col1][col2] = result;
            }
        }
    }

    // Return the maximum cherries that can be picked
    return dp[0][0][n - 1];
}

// Time: O(m*n^2)
// Space: O(n^2)
public int cherryPickupDp3dOptimized(int[][] grid) {
    int m = grid.length;
    int n = grid[0].length;

    // Initialize two 2D dynamic programming arrays
    int[][][] dp = new int[2][n][n];
    // Variables to keep track of current and next rows
    int curr = 0;
    int next = 1;

    for (int row = m - 1; row >= 0; row--) {
        for (int col1 = 0; col1 < n; col1++) {
            for (int col2 = 0; col2 < n; col2++) {
                int result = grid[row][col1]; // Cherries picked by the first robot
                if (col1 != col2) {
                    result += grid[row][col2]; // Add cherries picked by the second
robot if they are not at the same position
                }
                if (row != m - 1) {
                    int max = 0;
                    for (int newCol1 = col1 - 1; newCol1 <= col1 + 1; newCol1++) {
                        for (int newCol2 = col2 - 1; newCol2 <= col2 + 1; newCol2++)
{
                            // If the new positions are valid, update the maximum
cherries that can be picked
                            if (newCol1 >= 0 && newCol1 < n && newCol2 >= 0 &&
newCol2 < n) {
                                max = Math.max(max, dp[next][newCol1][newCol2]);
                            }
                        }
                    }
                    result += max; // Add the maximum cherries that can be picked
from the next row to the current result
                }
                dp[curr][col1][col2] = result; // Update the dynamic programming
array
            }
```

```
            }
            // Swap the current and next rows
            int temp = curr;
            curr = next;
            next = temp;
        }

        return dp[next][0][n - 1]; // Return the maximum cherries that can be picked
    }

}
```

307. `Solution01475.java`

```
/**
 * 1475. Final Prices With a Special Discount in a Shop
 * <a href="https://leetcode.com/problems/final-prices-with-a-special-discount-in-a-
shop">Web link</a>
 * Easy
 */

package leetcode;

public class Solution01475 {

    public int[] finalPrices(int[] prices) {
        int n = prices.length;
        int[] result = new int[n];
        for (int i = 0; i < n; i++) {
            int originalPrice = prices[i];
            int discount = 0;
            for (int j = i + 1; j < n; j++) {
                if (prices[j] <= originalPrice) {
                    discount = prices[j];
                    break;
                }
            }
            result[i] = originalPrice - discount;
        }
        return result;
    }

}
```

308. `Solution01481.java`

```
package leetcode;

import java.util.HashMap;
import java.util.Map;

/**
 * https://leetcode.com/problems/least-number-of-unique-integers-after-k-removals/
 * 1481. Least Number of Unique Integers after K Removals
 * Medium
 */

public class Solution01481 {

    // Time: O(n)
    // Space: O(n)
    public int findLeastNumOfUniqueInts(int[] arr, int k) {
        // Create a frequency map to store the frequency of each number in the array
        Map<Integer, Integer> frequencyMap = new HashMap<>();
        // Iterate over the array and update the frequency map
        for (int number : arr) {
            frequencyMap.merge(number, 1, Integer::sum);
        }

        // Create an array to store the count of frequencies
        int[] frequencyCountArray = new int[arr.length + 1];
        // Iterate over the frequency map and update the frequency count array
        for (int frequency : frequencyMap.values()) {
            frequencyCountArray[frequency]++;
        }

        // Initialize the result with the total number of unique numbers in the array
        int uniqueNumbers = frequencyMap.size();

        // Iterate over the frequency count array
        for (int currentFrequency = 1; currentFrequency < frequencyCountArray.length;
currentFrequency++) {
            // Calculate the total count of numbers with the current frequency
            final int totalCountCurrentFrequency = currentFrequency *
frequencyCountArray[currentFrequency];
            // If k is greater than or equal to the total count, subtract the total count
from k and the count of numbers with the current frequency from the result
            if (k >= totalCountCurrentFrequency) {
                k -= totalCountCurrentFrequency;
                uniqueNumbers -= frequencyCountArray[currentFrequency];
            } else {
                // If k is less than the total count, subtract the number of numbers that
can be removed from the result and break the loop
                uniqueNumbers -= k / currentFrequency;
                break;
            }
        }
```

```
        // Return the result
        return uniqueNumbers;
    }

}
```

309. `Solution01482.java`

```
package leetcode;

/**
 * 1482. Minimum Number of Days to Make m Bouquets
 * <a href="https://leetcode.com/problems/minimum-number-of-days-to-make-m-bouquets">Web
link</a> |
 * Medium
 */

public class Solution01482 {

    public int minDays(int[] bloomDay, int m, int k) {
        final int n = bloomDay.length;
        if (n < (long) m * k) {
            return -1;
        }
        int low = Integer.MAX_VALUE;
        int high = Integer.MIN_VALUE;
        for (int day : bloomDay) {
            low = Math.min(low, day);
            high = Math.max(high, day);
        }
        while (low < high) {
            int mid = low + (high - low) / 2;
            if (canMakeBouquets(bloomDay, m, k, mid)) {
                high = mid;
            } else {
                low = mid + 1;
            }
        }
        return low;
    }

    private boolean canMakeBouquets(int[] bloomDay, int m, int k, int mid) {
        int bouquets = 0;
        int flowers = 0;
        for (int day : bloomDay) {
            if (day <= mid) {
                flowers++;
                if (flowers == k) {
                    bouquets++;
                    if (bouquets >= m) {
                        return true;
                    }
                    flowers = 0;
                }
            } else {
                flowers = 0;
            }
        }
        return false;
```

```
    }

}
```

310. `Solution01497.java`

```
package leetcode;

/**
 * 1497. Check If Array Pairs Are Divisible by k
 * <a href="https://leetcode.com/problems/check-if-array-pairs-are-divisible-by-k">Web
link</a>
 * Medium
 */

public class Solution01497 {

    public boolean canArrange(int[] arr, int k) {
        int[] freq = new int[k];
        for (int num : arr) {
            int mod = num % k;
            if (mod < 0) {
                mod += k;
            }
            freq[mod]++;
        }
        if (freq[0] % 2 != 0) {
            return false;
        }
        for (int i = 1; i <= k / 2; i++) {
            if (freq[i] != freq[k - i]) {
                return false;
            }
        }
        return true;
    }

}
```

311. `Solution01508.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.PriorityQueue;

/**
 * 1508. Range Sum of Sorted Subarray Sums
 * <a href="https://leetcode.com/problems/range-sum-of-sorted-subarray-sums">Web link</a>
|
 * Medium
 */

public class Solution01508 {

    final static int MOD = 1000000007;

    public int rangeSum(int[] nums, int n, int left, int right) {
        List<Long> subarraySums = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            long sum = 0;
            for (int j = i; j < n; j++) {
                sum += nums[j];
                sum %= MOD;
                subarraySums.add(sum);
            }
        }

        subarraySums.sort(null);

        long result = 0;
        for (int i = left - 1; i < right; i++) {
            result += subarraySums.get(i);
            result %= MOD;
        }
        return (int) result;
    }

    public int rangeSumMinHeap(int[] nums, int n, int left, int right) {
        PriorityQueue<Data> minHeap = new PriorityQueue<>
(Comparator.comparingLong(Data::sum));
        for (int i = 0; i < n; i++) {
            minHeap.offer(new Data(nums[i], i));
        }

        long result = 0;
        for (int i = 1; i <= right; i++) {
            Data data = minHeap.poll();
            if (i >= left) {
```

```
                result += data.sum();
                result %= MOD;
            }
            if (data.index() < n - 1) {
                int newIndex = data.index() + 1;
                long newSum = data.sum() + nums[newIndex];
                newSum %= MOD;
                Data newData = new Data(newSum, newIndex);
                minHeap.offer(newData);
            }
        }
        return (int) result;
    }

    private record Data(long sum, int index) {
    }

}
```

312. `Solution01509.java`

```
package leetcode;

import java.util.Arrays;
import java.util.Comparator;
import java.util.PriorityQueue;

/**
 * 1509. Minimum Difference Between Largest and Smallest Value in Three Moves
 * <a href="https://leetcode.com/problems/minimum-difference-between-largest-and-
smallest-value-in-three-moves">Web link</a>
 * Medium
 */

public class Solution01509 {

    public int minDifferenceSorting(int[] nums) {
        if (nums.length <= 4) {
            return 0;
        }
        Arrays.sort(nums);
        int minDifference = Integer.MAX_VALUE;
        for (int left = 0; left <= 3; left++) {
            int right = nums.length - 4 + left;
            minDifference = Math.min(minDifference, nums[right] - nums[left]);
        }
        return minDifference;
    }

    public int minDifference(int[] nums) {
        int n = nums.length;
        if (n <= 4) {
            return 0;
        }

        PriorityQueue<Integer> minHeap = new PriorityQueue<>(Comparator.reverseOrder());
        PriorityQueue<Integer> maxHeap = new PriorityQueue<>();

        for (int i = 0; i < n; i++) {
            int num = nums[i];
            if (i < 4) {
                minHeap.offer(num);
                maxHeap.offer(num);
                continue;
            }

            if (num < minHeap.peek()) {
                minHeap.poll();
                minHeap.offer(num);
            }

            if (num > maxHeap.peek()) {
```

```
                maxHeap.poll();
                maxHeap.offer(num);
            }
        }

        int[] minValues = new int[4];
        int index = 3;
        while (!minHeap.isEmpty()) {
            minValues[index] = minHeap.poll();
            --index;
        }
        int[] maxValues = new int[4];
        index = 0;
        while (!maxHeap.isEmpty()) {
            maxValues[index] = maxHeap.poll();
            ++index;
        }

        int minDifference = Integer.MAX_VALUE;
        for (int i = 0; i < 4; i++) {
            minDifference = Math.min(minDifference, maxValues[i] - minValues[i]);
        }
        return minDifference;
    }

}
```

313. `Solution01514.java`

```
package leetcode;

import javafx.util.Pair;

import java.util.ArrayList;
import java.util.List;
import java.util.PriorityQueue;

/**
 * 1514. Path with Maximum Probability
 * <a href="https://leetcode.com/problems/path-with-maximum-probability">Web link</a> |
 * Medium
 */

public class Solution01514 {
    public double maxProbabilityTLE(int n, int[][] edges, double[] succProb, int
start_node, int end_node) {
        double[] prob = new double[n];
        prob[start_node] = 1;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < edges.length; j++) {
                int u = edges[j][0], v = edges[j][1];
                double w = succProb[j];
                prob[v] = Math.max(prob[v], prob[u] * w);
                prob[u] = Math.max(prob[u], prob[v] * w);
            }
        }
        return prob[end_node];
    }

    public double maxProbabilityGreedyDijkstra(int n, int[][] edges, double[] succProb,
int start_node, int end_node) {
        List<List<Pair<Integer, Double>>> adjacencyList = new ArrayList<>();

        for (int i = 0; i < n; i++) adjacencyList.add(new ArrayList<>());

        for (int i = 0; i < edges.length; i++) {
            int[] currentEdge = edges[i];
            adjacencyList.get(currentEdge[0]).add(new Pair<>(currentEdge[1],
succProb[i]));
            adjacencyList.get(currentEdge[1]).add(new Pair<>(currentEdge[0],
succProb[i]));
        }

        PriorityQueue<Pair<Integer, Double>> q = new PriorityQueue<>((a, b) ->
Double.compare(b.getValue(), a.getValue()));
        double[] curr_prob = new double[n];
        curr_prob[start_node] = 1;
        q.offer(new Pair<>(start_node, 1.0));
        while (!q.isEmpty()) {
            Pair<Integer, Double> pair = q.poll();
```

```
            int node = pair.getKey();
            double prob = pair.getValue();
            if (node == end_node) return prob;

            for (Pair<Integer, Double> neighbor : adjacencyList.get(node)) {
                int neighborNode = neighbor.getKey();
                double neighborProb = neighbor.getValue();
                if (curr_prob[neighborNode] < neighborProb * prob) {
                    curr_prob[neighborNode] = neighborProb * prob;
                    q.offer(new Pair<>(neighborNode, neighborProb * prob));
                }
            }
        }

        return 0.0;
    }
}
```

314. `Solution01518.java`

```
package leetcode;

/**
 * 1518. Water Bottles
 * <a href="https://leetcode.com/problems/water-bottles">Web link</a> |
 * Easy
 */

public class Solution01518 {

    public int numWaterBottles(int numBottles, int numExchange) {
        int toDrink = numBottles;
        int consumed = toDrink;
        while (toDrink >= numExchange) {
            int newToDrink = toDrink / numExchange;
            int emptyLeft = toDrink % numExchange;
            consumed += newToDrink;
            toDrink = newToDrink + emptyLeft;
        }
        return consumed;
    }
}
```

315. `Solution01530.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.ArrayList;
import java.util.List;

/**
 * 1530. Number of Good Leaf Nodes Pairs
 * <a href="https://leetcode.com/problems/number-of-good-leaf-nodes-pairs">Web link</a> |
 * Medium
 */

public class Solution01530 {

    public int countPairs(TreeNode root, int distance) {
        int[] count = new int[1];
        dfs(root, distance, count);
        return count[0];
    }

    private List<Integer> dfs(TreeNode node, int distance, int[] count) {
        if (node == null) return null;
        if (node.left == null && node.right == null) {
            return List.of(1);
        }
        List<Integer> left = dfs(node.left, distance, count);
        List<Integer> right = dfs(node.right, distance, count);
        if (left == null && right == null) {
            return null;
        }
        if (left != null && right != null) {
            for (int l : left) {
                for (int r : right) {
                    if (l + r <= distance) {
                        count[0]++;
                    }
                }
            }
        }
        List<Integer> current = new ArrayList<>();
        promoteToParent(distance, left, current);
        promoteToParent(distance, right, current);
        return current;
    }

    private static void promoteToParent(int distance, List<Integer> left, List<Integer>
current) {
        if (left == null) {
            return;
        }
```

```
        for (int l : left) {
            if (l + 1 < distance) {
                current.add(l + 1);
            }
        }
    }

}
```

316. `Solution01544.java`

```
package leetcode;

/**
 * 1544. Make The String Great
 * https://leetcode.com/problems/make-the-string-great/
 * Easy
 */

public class Solution01544 {

    public String makeGood(String s) {
        StringBuilder sb = new StringBuilder();
        for (char currentCharacter : s.toCharArray()) {
            if (sb.isEmpty()) {
                sb.append(currentCharacter);
            } else {
                final char lastCharInBuilder = sb.charAt(sb.length() - 1);
                final int asciiDifference = Math.abs(lastCharInBuilder -
currentCharacter);
                if (asciiDifference == 32) {
                    sb.deleteCharAt(sb.length() - 1);
                } else {
                    sb.append(currentCharacter);
                }
            }
        }
        return sb.toString();
    }

}
```

317. `Solution01545.java`

```
/**
 * 1545. Find Kth Bit in Nth Binary String
 * <a href="https://leetcode.com/problems/find-kth-bit-in-nth-binary-string">Web link</a>
|
 * Medium
 */

package leetcode;

public class Solution01545 {

    public char findKthBit(int n, int k) {
        // Base case: When n = 1, the binary string is "0"
        if (n == 1) return '0';

        // Find the length of the current string Sn, which is 2^n - 1
        int length = (1 << n) - 1;

        // Find the middle position
        int mid = length / 2 + 1;

        // If k is the middle position, return '1'
        if (k == mid) return '1';

        // If k is in the first half, find the bit in Sn-1
        if (k < mid) return findKthBit(n - 1, k);

        // If k is in the second half, find the bit in Sn-1 and invert it
        return findKthBit(n - 1, length - k + 1) == '0' ? '1' : '0';
    }

}
```

318. `Solution01550.java`

```
package leetcode;

/**
 * 1550. Three Consecutive Odds
 * <a href="https://leetcode.com/problems/three-consecutive-odds">Web link</a>
 * Easy
 */

public class Solution01550 {

    public boolean threeConsecutiveOdds(int[] arr) {
        int oddCount = 0;
        for (int num : arr) {
            if (num % 2 == 1) {
                ++oddCount;
                if (oddCount == 3) {
                    return true;
                }
            } else {
                oddCount = 0;
            }
        }
        return false;
    }

}
```

319. `Solution01552.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 1552. Magnetic Force Between Two Balls
 * <a href="https://leetcode.com/problems/magnetic-force-between-two-balls">Web link</a>
|
 * Medium
 */

public class Solution01552 {

    public int maxDistance(int[] position, int m) {
        int n = position.length;
        Arrays.sort(position);
        int lo = 1;
        int hi = (position[n - 1] - position[0]) / (m - 1);
        int maxForce = 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (canPlaceBalls(position, m, mid)) {
                maxForce = mid;
                lo = mid + 1;
            } else {
                hi = mid - 1;
            }
        }
        return maxForce;
    }

    private boolean canPlaceBalls(int[] position, int m, int mid) {
        int n = position.length;
        int balls = 1;
        int lastPosition = position[0];
        for (int i = 1; i < n; i++) {
            if (position[i] - lastPosition >= mid) {
                balls++;
                lastPosition = position[i];
            }
        }
        return balls >= m;
    }

}
```

320. `Solution01568.java`

```
package leetcode;

/**
 * 1568. Minimum Number of Days to Disconnect Island
 * <a href="https://leetcode.com/problems/minimum-number-of-days-to-disconnect-
island">Web link</a> |
 * Hard
 */

public class Solution01568 {

    public int minDays(int[][] grid) {
        int amountOfIslandsOriginal = getAmountOfIslands(grid);
        if (amountOfIslandsOriginal != 1) {
            return 0;
        }

        int rows = grid.length;
        int cols = grid[0].length;

        for (int row = 0; row < rows; row++) {
            for (int col = 0; col < cols; col++) {
                if (grid[row][col] == 1) {
                    grid[row][col] = 0;
                    int amountOfIslands = getAmountOfIslands(grid);
                    if (amountOfIslands != 1) {
                        return 1;
                    }
                    grid[row][col] = 1;
                }
            }
        }

        return 2;
    }

    private int getAmountOfIslands(int[][] grid) {
        int rows = grid.length;
        int cols = grid[0].length;
        boolean[][] visited = new boolean[rows][cols];

        int count;
        count = 0;
        for (int row = 0; row < rows; row++) {
            for (int col = 0; col < cols; col++) {
                if (grid[row][col] == 1) {
                    if (!visited[row][col]) {
                        dfs(row, col, visited, grid);
                        count += 1;
                        if (count > 1) {
                            return count;
```

```
                    }
                }
            }
        }
    }
    return count;
}

private void dfs(int row, int col, boolean[][] visited, int[][] grid) {
    if (row < 0 ||
        col < 0 ||
        row >= grid.length ||
        col >= grid[0].length ||
        grid[row][col] == 0 ||
        visited[row][col]) {
        return;
    }
    visited[row][col] = true;
    dfs(row - 1, col, visited, grid);
    dfs(row + 1, col, visited, grid);
    dfs(row, col - 1, visited, grid);
    dfs(row, col + 1, visited, grid);
}

}
```

321. `Solution01574.java`

```
/**
 * 1574. Shortest Subarray to be Removed to Make Array Sorted
 * <a href="https://leetcode.com/problems/shortest-subarray-to-be-removed-to-make-array-sorted">Web link</a> |
 * Medium
 */

package leetcode;

public class Solution01574 {

    public int findLengthOfShortestSubarray(int[] arr) {
        int n = arr.length;

        if (n == 1) {
            return 0;
        }

        // Find the longest non-decreasing prefix
        int left = 0;
        while (left + 1 < n && arr[left] <= arr[left + 1]) {
            left++;
        }

        if (left == n - 1) {  // Array is already non-decreasing
            return 0;
        }

        // Find the longest non-decreasing suffix
        int right = n - 1;
        while (right > 0 && arr[right - 1] <= arr[right]) {
            right--;
        }

        // Initial result: remove everything between left and right
        int result = Math.min(n - left - 1, right);

        // Try to merge prefix and suffix
        int i = 0;
        int j = right;
        while (i <= left && j < n) {
            if (arr[i] <= arr[j]) {
                result = Math.min(result, j - i - 1);
                i++;
            } else {
                j++;
            }
        }

        return result;
    }
```

```
}
```

322. `Solution01579.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 1579. Remove Max Number of Edges to Keep Graph Fully Traversable
 * https://leetcode.com/problems/remove-max-number-of-edges-to-keep-graph-fully-
traversable/
 * Hard
 */

public class Solution01579 {

    public int maxNumEdgesToRemove(int n, int[][] edges) {
        // Create UnionFind instances for Alice and Bob
        UnionFind unionFindAlice = new UnionFind(n);
        UnionFind unionFindBob = new UnionFind(n);

        // Initialize the count of connections needed to keep the graph fully traversable
        int requiredConnectionsCount = 0;

        // First loop: Process type 3 edges
        for (int[] edge : edges) {
            int edgeType = edge[0]; // Type of the edge
            int node1 = edge[1]; // First node of the edge
            int node2 = edge[2]; // Second node of the edge

            // If the edge type is 3, it can be traversed by both Alice and Bob
            if (edgeType == 3) {
                // Try to connect the nodes in both UnionFind instances
                final boolean isAliceConnected =
unionFindAlice.unionAndCheckConnectionRequired(node1, node2);
                final boolean isBobConnected =
unionFindBob.unionAndCheckConnectionRequired(node1, node2);

                // If either Alice or Bob can traverse the edge, increment the count of
required connections
                if (isAliceConnected || isBobConnected) {
                    ++requiredConnectionsCount;
                }
            }
        }

        // Second loop: Process type 1 and type 2 edges
        for (int[] edge : edges) {
            int edgeType = edge[0]; // Type of the edge
            int node1 = edge[1]; // First node of the edge
            int node2 = edge[2]; // Second node of the edge

            // If the edge type is 2, it can only be traversed by Bob
            if (edgeType == 2) {
```

```
                // If Bob can traverse the edge, increment the count of required
connections
                if (unionFindBob.unionAndCheckConnectionRequired(node1, node2)) {
                    ++requiredConnectionsCount;
                }
            }
            // If the edge type is 1, it can only be traversed by Alice
            else if (edgeType == 1) {
                // If Alice can traverse the edge, increment the count of required
connections
                if (unionFindAlice.unionAndCheckConnectionRequired(node1, node2)) {
                    ++requiredConnectionsCount;
                }
            }
        }

        // If both Alice and Bob can traverse the entire graph, return the number of
edges that can be removed
        // Else, return -1 indicating that the graph cannot be fully traversed by both
Alice and Bob
        if (unionFindAlice.isConnected() && unionFindBob.isConnected()) {
            return edges.length - requiredConnectionsCount;
        } else {
            return -1;
        }

    }

    static class UnionFind {
        int[] parent;
        int[] size;
        int count;

        public UnionFind(int n) {
            parent = new int[n + 1];
            for (int i = 0; i < n; i++) {
                parent[i] = i;
            }
            size = new int[n + 1];
            Arrays.fill(size, 1);
            count = n;
        }

        public int find(int x) {
            if (parent[x] != x) {
                parent[x] = find(parent[x]);
            }
            return parent[x];
        }

        public boolean unionAndCheckConnectionRequired(int x, int y) {
            int rootX = find(x);
            int rootY = find(y);
            if (rootX == rootY) {
```

```
                return false;
            }
            if (size[rootX] < size[rootY]) {
                parent[rootX] = rootY;
                size[rootY] += size[rootX];
            } else {
                parent[rootY] = rootX;
                size[rootX] += size[rootY];
            }
            count--;
            return true;
        }

        boolean isConnected() {
            return count == 1;
        }
    }

}
```

323. `Solution01590.java`

```
package leetcode;

import java.util.HashMap;
import java.util.Map;

/**
 * 1590. Make Sum Divisible by P
 * <a href="https://leetcode.com/problems/make-sum-divisible-by-p">Web link</a> |
 * Medium
 */

public class Solution01590 {

    public int minSubarray(int[] nums, int p) {
        long totalSum = 0;
        for (int num : nums) {
            totalSum += num;
        }

        int rem = (int) (totalSum % p);
        if (rem == 0) return 0; // If remainder is 0, no subarray needs to be removed

        Map<Integer, Integer> prefixMod = new HashMap<>();
        prefixMod.put(0, -1);  // Initialize to handle full prefix
        long prefixSum = 0;
        int minLength = nums.length;

        for (int i = 0; i < nums.length; ++i) {
            prefixSum += nums[i];
            int currentMod = (int) (prefixSum % p);
            int targetMod = (currentMod - rem + p) % p;

            if (prefixMod.containsKey(targetMod)) {
                minLength = Math.min(minLength, i - prefixMod.get(targetMod));
            }

            prefixMod.put(currentMod, i);
        }

        return minLength == nums.length ? -1 : minLength;
    }

}
```

324. `Solution01593.java`

```
/**
 * 1593. Split a String Into the Max Number of Unique Substrings
 * <a href="https://leetcode.com/problems/split-a-string-into-the-max-number-of-unique-substrings">Web link</a> |
 * Medium
 */

package leetcode;

import java.util.HashSet;
import java.util.Set;

public class Solution01593 {

    public int maxUniqueSplit(String s) {
        Set<String> seen = new HashSet<>();
        int[] maxCount = {0};
        backtrack(s, 0, seen, 0, maxCount);
        return maxCount[0];
    }

    private void backtrack(String s, int start, Set<String> seen,
                           int count, int[] maxCount) {
        if (count + (s.length() - start) <= maxCount[0]) {
            return;
        }

        if (start == s.length()) {
            maxCount[0] = Math.max(maxCount[0], count);
            return;
        }

        for (int end = start + 1; end <= s.length(); ++end) {
            String substring = s.substring(start, end);

            if (!seen.contains(substring)) {
                seen.add(substring);

                backtrack(s, end, seen, count + 1, maxCount);

                seen.remove(substring);
            }
        }
    }

}
```

325. `Solution01598.java`

```
package leetcode;

/**
 * 1598. Crawler Log Folder
 * <a href="https://leetcode.com/problems/crawler-log-folder">Web link</a> |
 * Easy
 */

public class Solution01598 {

    public int minOperations(String[] logs) {
        int depth = 0;
        for (String log : logs) {
            switch (log) {
                case "../" -> {
                    if (depth > 0) {
                        --depth;
                    }
                }
                case "./" -> {
                }
                default -> ++depth;
            }
        }
        return depth;
    }

}
```

326. `Solution01605.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 1605. Find Valid Matrix Given Row and Column Sums
 * <a href="https://leetcode.com/problems/find-valid-matrix-given-row-and-column-
sums">Web link</a> |
 * Medium
 */

public class Solution01605 {

    public int[][] restoreMatrix(int[] rowSum, int[] colSum) {
        int[][] result = new int[rowSum.length][colSum.length];
        for (int rowIdx = 0; rowIdx < rowSum.length; rowIdx++) {
            result[rowIdx][0] = rowSum[rowIdx];
        }
        for (int colIdx = 0; colIdx < colSum.length; colIdx++) {
            int currentColSum = 0;
            for (int rowIdx = 0; rowIdx < rowSum.length; rowIdx++) {
                currentColSum += result[rowIdx][colIdx];
            }

            int rowIdx = 0;
            while (currentColSum > colSum[colIdx]) {
                int diff = currentColSum - colSum[colIdx];
                int maxShift = Math.min(diff, result[rowIdx][colIdx]);
                result[rowIdx][colIdx] -= maxShift;
                result[rowIdx][colIdx + 1] += maxShift;
                currentColSum -= maxShift;
                rowIdx++;
            }
        }

        return result;
    }

    public int[][] restoreMatrixOptimized(int[] rowSum, int[] colSum) {
        int rows = rowSum.length;
        int cols = colSum.length;
        int[][] result = new int[rows][cols];

        int[] rowSumCopy = Arrays.copyOf(rowSum, rows);
        int[] colSumCopy = Arrays.copyOf(colSum, cols);

        for (int rowIdx = 0; rowIdx < rows; rowIdx++) {
            for (int colIdx = 0; colIdx < cols; colIdx++) {
                int minVal = Math.min(rowSumCopy[rowIdx], colSumCopy[colIdx]);
                result[rowIdx][colIdx] = minVal;
                rowSumCopy[rowIdx] -= minVal;
```

```
                colSumCopy[colIdx] -= minVal;
            }
        }
        return result;
    }

}
```

327. `Solution01608.java`

```
package leetcode;

/**
 * 1608. Special Array With X Elements Greater Than or Equal X
 * <a href="https://leetcode.com/problems/special-array-with-x-elements-greater-than-or-
equal-x">Web link</a>
 * Easy
 */

public class Solution01608 {
    public int specialArray(int[] nums) {
        int n = nums.length;
        int[] count = new int[n + 1];
        for (int num : nums) {
            if (num < n) {
                count[num] += 1;
            } else {
                count[n] += 1;
            }
        }
        int totalRight = 0;
        for (int i = n; i > 0; i--) {
            totalRight += count[i];
            if (totalRight == i) {
                return i;
            } else if (totalRight > i) {
                return -1;
            }
        }
        return -1;
    }
}
```

328. `Solution01609.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.ArrayDeque;
import java.util.Queue;

/**
 * 1609. Even Odd Tree
 * https://leetcode.com/problems/even-odd-tree/description/
 * Medium
 */

public class Solution01609 {
    /**
     * Definition for a binary tree node.
     * public class TreeNode {
     * int val;
     * TreeNode left;
     * TreeNode right;
     * TreeNode() {}
     * TreeNode(int val) { this.val = val; }
     * TreeNode(int val, TreeNode left, TreeNode right) {
     * this.val = val;
     * this.left = left;
     * this.right = right;
     * }
     * }
     */
    public boolean isEvenOddTree(TreeNode root) {
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        boolean even = true;
        while (!queue.isEmpty()) {
            int size = queue.size();
            int prev = even ? Integer.MIN_VALUE : Integer.MAX_VALUE;
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                if (even) {
                    if (node.val % 2 == 0 || node.val <= prev) {
                        return false;
                    }
                } else {
                    if (node.val % 2 != 0 || node.val >= prev) {
                        return false;
                    }
                }
                prev = node.val;
                if (node.left != null) {
                    queue.offer(node.left);
                }
```

```
                if (node.right != null) {
                    queue.offer(node.right);
                }
            }
            even = !even;
        }
        return true;
    }

}
```

329. `Solution01614.java`

```
package leetcode;

/**
 * 1614. Maximum Nesting Depth of the Parentheses
 * https://leetcode.com/problems/maximum-nesting-depth-of-the-parentheses/
 * Easy
 */

public class Solution01614 {

    public int maxDepth(String s) {
        int max = 0;
        int count = 0;
        for (char c : s.toCharArray()) {
            if (c == '(') {
                count++;
                max = Math.max(max, count);
            } else if (c == ')') {
                count--;
            }
        }
        return max;
    }

}
```

330. `Solution01624.java`

```
package leetcode;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

/**
 * https://leetcode.com/problems/largest-substring-between-two-equal-characters/
 * 1624. Largest Substring Between Two Equal Characters
 * <p>
 * Given a string s, return the length of the longest substring between two equal
characters,
 * excluding the two characters. If there is no such substring return -1.
 * <p>
 * A substring is a contiguous sequence of characters within a string.
 * <p>
 * Example 1:
 * <p>
 * Input: s = "aa"
 * Output: 0
 * Explanation: The optimal substring here is an empty substring between the two 'a's.
 * <p>
 * Example 2:
 * <p>
 * Input: s = "abca"
 * Output: 2
 * Explanation: The optimal substring here is "bc".
 * <p>
 * Example 3:
 * <p>
 * Input: s = "cbzxy"
 * Output: -1
 * Explanation: There are no characters that appear twice in s.
 * <p>
 * Example 4:
 * <p>
 * Input: s = "cabbac"
 * Output: 4
 * Explanation: The optimal substring here is "abba". Other non-optimal substrings
include
 * "bb" and "".
 * <p>
 * Constraints:
 * <p>
 * 1 <= s.length <= 300
 * s contains only lowercase English letters.
 */
public class Solution01624 {

    // Time: O(n^2)
    // Space: O(1)
```

```
    public int maxLengthBetweenEqualCharactersBruteforce(String s) {
        int result = -1;
        for (int left = 0; left < s.length(); left++) {
            for (int right = left + 1; right < s.length(); right++) {
                if (s.charAt(left) == s.charAt(right)) {
                    result = Math.max(result, right - left - 1);
                }
            }
        }

        return result;
    }

    // Time: O(n)
    // Space: O(n)
    public int maxLengthBetweenEqualCharactersMap(String s) {
        int result = -1;
        Map<Character, Integer> leftMap = new HashMap<>();
        for (int right = 0; right < s.length(); right++) {
            char ch = s.charAt(right);
            Integer left = leftMap.get(ch);
            if (left == null) {
                leftMap.put(ch, right);
            } else {
                result = Math.max(result, right - left - 1);
            }
        }
        return result;
    }

    // Time: O(n)
    // Space: O(1)
    public int maxLengthBetweenEqualCharactersArray(String s) {
        // Initialize the result to -1.
        int result = -1;
        // Initialize the array of the first occurrences of each character to -1.
        int[] leftArray = new int[26];
        Arrays.fill(leftArray, -1);

        // Iterate over the characters of the string.
        for (int i = 0; i < s.length(); i++) {
            // Get the index of the current character.
            int index = s.charAt(i) - 'a';
            // If the first occurrence of the current character is not set, set it.
            if (leftArray[index] == -1) {
                leftArray[index] = i;
            } else {
                // Otherwise, update the result if needed.
                result = Math.max(result, i - leftArray[index] - 1);
            }
        }

        return result;
```

```
    }
}
```

331. `Solution01636.java`

```
package leetcode;

import java.util.*;
import java.util.stream.Collectors;

/**
 * 1636. Sort Array by Increasing Frequency
 * <a href="https://leetcode.com/problems/sort-array-by-increasing-frequency">Web
link</a> |
 * Easy
 */

public class Solution01636 {

    public int[] frequencySort(int[] nums) {
        Map<Integer, Integer> frequencyMap = new TreeMap<>();
        for (int num : nums) {
            frequencyMap.put(num, frequencyMap.getOrDefault(num, 0) + 1);
        }
        TreeMap<Integer, List<Map.Entry<Integer, Integer>>> treeMap =
frequencyMap.entrySet()
                .stream()
                .collect(Collectors.groupingBy(Map.Entry::getValue,
                        TreeMap::new,
                        Collectors.toList()));
        List<Integer> result = new ArrayList<>();
        for (var entry : treeMap.entrySet()) {
            List<Map.Entry<Integer, Integer>> entries = entry.getValue();
            if (entries.size() > 1) {
                entries.stream()
                        .sorted((e1, e2) -> e2.getKey() - e1.getKey())
                        .forEach(entry2 -> {
                            for (int i = 0; i < entry2.getValue(); i++) {
                                result.add(entry2.getKey());
                            }
                        });
            } else {
                final Map.Entry<Integer, Integer> first = entries.getFirst();
                for (int i = 0; i < first.getValue(); i++) {
                    result.add(first.getKey());
                }
            }
        }
        int[] arr = new int[result.size()];
        for (int i = 0; i < result.size(); i++) {
            arr[i] = result.get(i);
        }
        return arr;
    }

    public int[] frequencySortOptimized(int[] nums) {
```

```
        Map<Integer, Integer> frequencyMap = new TreeMap<>();
        for (int num : nums) {
            frequencyMap.put(num, frequencyMap.getOrDefault(num, 0) + 1);
        }
        return Arrays.stream(nums)
                .boxed()
                .sorted((a, b) -> {
                    int freqA = frequencyMap.get(a);
                    int freqB = frequencyMap.get(b);
                    if (freqA != freqB) {
                        return freqA - freqB;
                    }
                    return b - a;
                })
                .mapToInt(Integer::intValue)
                .toArray();
    }

    public int[] frequencySortOptimized2(int[] nums) {
        Map<Integer, Integer> frequencyMap = new HashMap<>();
        for (int num : nums) {
            frequencyMap.put(num, frequencyMap.getOrDefault(num, 0) + 1);
        }
        List<Map.Entry<Integer, Integer>> entries = new ArrayList<>
(frequencyMap.entrySet());
        entries.sort((e1, e2) -> {
            int freqCompare = Integer.compare(e1.getValue(), e2.getValue());
            return freqCompare != 0 ? freqCompare : Integer.compare(e2.getKey(),
e1.getKey());
        });
        List<Integer> result = new ArrayList<>();
        for (Map.Entry<Integer, Integer> entry : entries) {
            for (int i = 0; i < entry.getValue(); i++) {
                result.add(entry.getKey());
            }
        }
        return result.stream().mapToInt(i -> i).toArray();
    }

}
```

332. `Solution01639.java`

```
/**
 * 1639. Number of Ways to Form a Target String Given a Dictionary
 * https://leetcode.com/problems/number-of-ways-to-form-a-target-string-given-a-
dictionary/
 * Hard
 */

package leetcode;

public class Solution01639 {

    public int numWays(String[] words, String target) {
        final long mod = 1_000_000_007L;
        int n = words[0].length();
        int m = target.length();

        long[][] freq = new long[n][26];
        for (String word : words) {
            for (int i = 0; i < n; i++) {
                freq[i][word.charAt(i) - 'a']++;
            }
        }

        char[] targetCharArr = target.toCharArray();

        long[][] dp = new long[n + 1][m + 1];
        for (int i = 0; i <= n; i++) {
            dp[i][0] = 1;
        }

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                dp[i][j] = dp[i - 1][j];
                dp[i][j] = (dp[i][j] + (freq[i - 1][targetCharArr[j - 1] - 'a'] * dp[i -
1][j - 1])) % mod;
            }
        }
        return (int) dp[n][m];
    }

}
```

333. `Solution01642.java`

```
package leetcode;

import java.util.PriorityQueue;

/**
 * https://leetcode.com/problems/furthest-building-you-can-reach/
 * 1642. Furthest Building You Can Reach
 * Medium
 */
public class Solution01642 {

    // Time: O(nlogk)
    // Space: O(k)
    public int furthestBuilding(int[] heights, int bricks, int ladders) {
        // Create a priority queue to store the differences in heights
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();

        // Iterate over the heights array
        for (int i = 1; i < heights.length; i++) {
            // Calculate the difference in height between the current and previous
building
            int diff = heights[i] - heights[i - 1];

            // If the difference is less than or equal to 0, skip the current iteration
            if (diff <= 0) {
                continue;
            }

            // Add the difference to the min heap
            minHeap.add(diff);

            // If the size of the min heap is greater than the number of ladders
            if (minHeap.size() > ladders) {
                // Remove the smallest difference from the min heap and subtract it from
the bricks
                bricks -= minHeap.poll();

                // If there are not enough bricks, return the index of the last reachable
building
                if (bricks < 0) {
                    return i - 1;
                }
            }
        }

        // If all buildings are reachable, return the index of the last building
        return heights.length - 1;
    }

}
```

334. `Solution01652.java`

```
/**
 * 1652. Defuse the Bomb
 * <a href="https://leetcode.com/problems/defuse-the-bomb">Web link</a> |
 * Easy
 */

package leetcode;

public class Solution01652 {

    public int[] decrypt(int[] code, int k) {
        int n = code.length;
        int[] res = new int[n];
        if (k == 0) {
            return res;
        }
        for (int i = 0; i < n; i++) {
            if (k > 0) {
                for (int j = 0; j < k; j++) {
                    res[i] += code[(i + j + 1) % n];
                }
            } else {
                for (int j = 0; j < -k; j++) {
                    res[i] += code[(i - j - 1 + n) % n];
                }
            }
        }
        return res;
    }

}
```

335. `Solution01653.java`

```
package leetcode;

/**
 * 1653. Minimum Deletions to Make String Balanced
 * <a href="https://leetcode.com/problems/minimum-deletions-to-make-string-balanced">Web
link</a> |
 * Medium
 */

public class Solution01653 {

    public int minimumDeletions(String s) {
        int n = s.length();
        int aCountRight = 0;
        for (int i = 0; i < n; i++) {
            char ch = s.charAt(i);
            if (ch == 'a') {
                aCountRight++;
            }
        }

        int bCountLeft = 0;
        int minDeletions = n;

        for (int i = 0; i < n; i++) {
            char ch = s.charAt(i);
            if (ch == 'a') {
                aCountRight--;
            }
            minDeletions = Math.min(minDeletions, bCountLeft + aCountRight);
            if (ch == 'b') {
                bCountLeft++;
            }
        }
        return minDeletions;
    }

    public int minimumDeletionsOptimized(String s) {
        int countB = 0;
        int minDeletions = 0;

        for (char c : s.toCharArray()) {
            if (c == 'a') {
                minDeletions = Math.min(minDeletions + 1, countB);
            } else {
                countB++;
            }
        }

        return minDeletions;
    }
```

```
}
```

336. `Solution01657.java`

```
package leetcode;

import java.util.Arrays;

/**
 * https://leetcode.com/problems/determine-if-two-strings-are-close/
 * 1657. Determine if Two Strings Are Close
 * Medium
 * <p>
 * Share
 * Two strings are considered close if you can attain one from the other using the
following operations:
 * <p>
 * Operation 1: Swap any two existing characters.
 * For example, abcde -> aecdb
 * Operation 2: Transform every occurrence of one existing character into another
existing character,
 * and do the same with the other character.
 * For example, aacabb -> bbcbaa (all a's turn into b's, and all b's turn into a's)
 * You can use the operations on either string as many times as necessary.
 * <p>
 * Given two strings, word1 and word2, return true if word1 and word2 are close, and
false otherwise.
 * <p>
 * Example 1:
 * <p>
 * Input: word1 = "abc", word2 = "bca"
 * Output: true
 * Explanation: You can attain word2 from word1 in 2 operations.
 * Apply Operation 1: "abc" -> "acb"
 * Apply Operation 1: "acb" -> "bca"
 * Example 2:
 * <p>
 * Input: word1 = "a", word2 = "aa"
 * Output: false
 * Explanation: It is impossible to attain word2 from word1, or vice versa, in any number
of operations.
 * Example 3:
 * <p>
 * Input: word1 = "cabbba", word2 = "abbccc"
 * Output: true
 * Explanation: You can attain word2 from word1 in 3 operations.
 * Apply Operation 1: "cabbba" -> "caabbb"
 * Apply Operation 2: "caabbb" -> "baaccc"
 * Apply Operation 2: "baaccc" -> "abbccc"
 * Example 4:
 * <p>
 * Input: word1 = "cabbba", word2 = "aabbss"
 * Output: false
 * Explanation: It is impossible to attain word2 from word1, or vice versa, in any amount
of operations.
```

```
 * <p>
 * Constraints:
 * <p>
 * 1 <= word1.length, word2.length <= 105
 * word1 and word2 contain only lowercase English letters.
 */
public class Solution01657 {

    // Time: O(n)
    // Space: O(1)
    public boolean closeStrings(String word1, String word2) {
        if (word1.length() != word2.length()) {
            return false;
        }

        int[] count1 = new int[26];
        int[] count2 = new int[26];
        for (int i = 0; i < word1.length(); i++) {
            count1[word1.charAt(i) - 'a']++;
            count2[word2.charAt(i) - 'a']++;
        }

        for (int i = 0; i < 26; i++) {
            if (count1[i] == 0 && count2[i] != 0 || count1[i] != 0 && count2[i] == 0) {
                return false;
            }
        }

        int[] freq1 = new int[word1.length() + 1];
        int[] freq2 = new int[word2.length() + 1];
        for (int i = 0; i < 26; i++) {
            freq1[count1[i]]++;
            freq2[count2[i]]++;
        }

        for (int i = 0; i < freq1.length; i++) {
            if (freq1[i] != freq2[i]) {
                return false;
            }
        }

        return true;
    }

    // Time: O(n)
    // Space: O(1)
    public boolean closeStrings2(String word1, String word2) {
        if (word1.length() != word2.length())
            return false;

        int[] f1 = new int[26];
        int[] f2 = new int[26];

        // Count frequency of each character in both words
```

```
        for (int i = 0; i < word1.length(); i++) {
            f1[word1.charAt(i) - 'a']++;
            f2[word2.charAt(i) - 'a']++;
        }

        // Check if the set of characters is the same for both words
        for (int i = 0; i < 26; i++) {
            if ((f1[i] == 0 && f2[i] != 0) || (f1[i] != 0 && f2[i] == 0))
                return false;
        }

        // Sort frequency arrays
        Arrays.sort(f1);
        Arrays.sort(f2);

        // Check if the frequency of each character is the same
        for (int i = 0; i < 26; i++) {
            if (f1[i] != f2[i])
                return false;
        }

        return true;
    }

}
```

337. `Solution01669.java`

```
package leetcode;

import leetcode.common.ListNode;

/**
 * 1669. Merge In Between Linked Lists
 * https://leetcode.com/problems/merge-in-between-linked-lists/
 * Medium
 */

public class Solution01669 {

    public ListNode mergeInBetween(ListNode list1, int a, int b, ListNode list2) {
        ListNode currentNode = list1;
        for (int i = 0; i < a - 1; i++) {
            currentNode = currentNode.next;
        }
        ListNode postNode = currentNode;
        for (int i = a; i <= b + 1; i++) {
            postNode = postNode.next;
        }
        currentNode.next = list2;
        while (currentNode.next != null) {
            currentNode = currentNode.next;
        }
        currentNode.next = postNode;
        return list1;
    }

}
```

338. `Solution01671.java`

```
/**
 * 1671. Minimum Number of Removals to Make Mountain Array
 * <a href="https://leetcode.com/problems/minimum-number-of-removals-to-make-mountain-
array">Web link</a> |
 * Hard
 */

package leetcode;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;

public class Solution01671 {

    public int minimumMountainRemovals(int[] nums) {
        int n = nums.length;

        List<Integer> lis = lisLength(nums);

        int[] reversed = new int[n];
        for (int i = 0; i < n; i++) {
            reversed[i] = nums[n - 1 - i];
        }
        List<Integer> lds = lisLength(reversed);
        Collections.reverse(lds);

        int removals = Integer.MAX_VALUE;

        for (int i = 0; i < n; i++) {
            if (lis.get(i) > 1 && lds.get(i) > 1) {
                removals = Math.min(removals, n + 1 - lis.get(i) - lds.get(i));
            }
        }

        return removals;
    }

    private List<Integer> lisLength(int[] nums) {
        List<Integer> lis = new ArrayList<>();
        lis.add(nums[0]);
        List<Integer> lisLen = new ArrayList<>();
        for (int i = 0; i < nums.length; i++) {
            lisLen.add(1);
        }

        for (int i = 1; i < nums.length; i++) {
            if (nums[i] > lis.getLast()) {
                lis.add(nums[i]);
            } else {
```

```
                int index = Collections.binarySearch(lis, nums[i]);
                if (index < 0) {
                    index = -(index + 1);
                }
                lis.set(index, nums[i]);
            }
            lisLen.set(i, lis.size());
        }
        return lisLen;
    }

    public int minimumMountainRemovalsDp(int[] nums) {
        int[] dp = new int[nums.length];
        Arrays.fill(dp, 1);
        for (int i = 1; i < nums.length; i++) {
            for (int j = 0; j < i; j++) {
                if (nums[i] > nums[j] && 1 + dp[j] > dp[i]) {
                    dp[i] = 1 + dp[j];
                }
            }
        }
        int[] dp2 = new int[nums.length];
        Arrays.fill(dp2, 1);
        for (int i = nums.length - 1; i >= 0; i--) {
            for (int j = nums.length - 1; j > i; j--) {
                if (nums[i] > nums[j] && 1 + dp2[j] > dp2[i]) {
                    dp2[i] = 1 + dp2[j];
                }
            }
        }
        int maxi = 0;
        for (int i = 0; i < nums.length; i++) {
            if (dp[i] > 1 && dp2[i] > 1)
                maxi = Math.max(maxi, dp[i] + dp2[i] - 1);
        }
        return nums.length - maxi;
    }

}
```

339. `Solution01684.java`

```
package leetcode;

/**
 * 1684. Count the Number of Consistent Strings
 * <a href="https://leetcode.com/problems/count-the-number-of-consistent-strings">Web
link</a> |
 * Easy
 */

public class Solution01684 {

    public int countConsistentStrings(String allowed, String[] words) {
        boolean[] allowedChars = new boolean[26];
        for (char ch : allowed.toCharArray()) {
            allowedChars[ch - 'a'] = true;
        }
        int res = 0;
        for (String word : words) {
            if (isWordConsistent(word, allowedChars)) {
                res++;
            }
        }
        return res;
    }

    private static boolean isWordConsistent(String word, boolean[] allowedChars) {
        for (char ch : word.toCharArray()) {
            if (!allowedChars[ch - 'a']) {
                return false;
            }
        }
        return true;
    }
}
```

340. `Solution01700.java`

```
package leetcode;

import java.util.ArrayDeque;
import java.util.Queue;

/**
 * https://leetcode.com/problems/number-of-students-unable-to-eat-lunch/
 * 1700. Number of Students Unable to Eat Lunch
 * Easy
 */

public class Solution01700 {

    // Time complexity: O(n)
    // Space complexity: O(1)
    public int countStudents(int[] students, int[] sandwiches) {
        Queue<Integer> studentQueue = new ArrayDeque<>();
        for (int student : students) {
            studentQueue.add(student);
        }

        for (int sandwich : sandwiches) {
            int queueSize = studentQueue.size();
            int unableToEat = 0;
            for (int i = 0; i < queueSize; i++) {
                int currentStudent = studentQueue.poll();
                if (sandwich == currentStudent) {
                    unableToEat = 0;
                    break;
                } else {
                    unableToEat++;
                    studentQueue.add(currentStudent);
                }
            }
            if (unableToEat == queueSize) {
                return unableToEat;
            }
        }

        return studentQueue.size();
    }

    public int countStudentsNoQueue(int[] students, int[] sandwiches) {
        int count0 = 0;
        int count1 = 0;

        for (int student : students) {
            if (student == 0) {
                count0++;
            } else {
                count1++;
```

```
            }
        }

        for (int sandwich : sandwiches) {
            if (sandwich == 0) {
                if (count0 == 0) {
                    return count0 + count1;
                } else {
                    count0--;
                }
            } else if (sandwich == 1) {
                if (count1 == 0) {
                    return count0 + count1;
                } else {
                    count1--;
                }
            }
        }

        return count0 + count1;
    }

}
```

341. `Solution01701.java`

```
package leetcode;

/**
 * 1701. Average Waiting Time
 * <a href="https://leetcode.com/problems/average-waiting-time">Web link</a> |
 * Medium
 */

public class Solution01701 {

    public double averageWaitingTime(int[][] customers) {
        long totalWaitTime = 0;
        long finishCooking = 0;

        for (int[] customer : customers) {
            int arrivalTime = customer[0];
            int cookingTime = customer[1];

            long startCooking = Math.max(finishCooking, arrivalTime);
            finishCooking = startCooking + cookingTime;

            totalWaitTime += finishCooking - arrivalTime;
        }

        return (double) totalWaitTime / customers.length;
    }

}
```

342. `Solution01704.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/determine-if-string-halves-are-alike/
 * 1704. Determine if String Halves Are Alike
 * Easy
 * You are given a string s of even length. Split this string into two halves of equal
lengths, and let a be the first
 * half and b be the second half.
 * Two strings are alike if they have the same number of vowels ('a', 'e', 'i', 'o', 'u',
'A', 'E', 'I', 'O',
 * 'U'). Notice that s contains uppercase and lowercase letters.
 * Return true if a and b are alike. Otherwise, return false.
 * Example 1:
 * Input: s = "book"
 * Output: true
 * Explanation: a = "bo" and b = "ok". a has 1 vowel and b has 1 vowel. Therefore, they
are alike.
 * Example 2:
 * Input: s = "textbook"
 * Output: false
 * Explanation: a = "text" and b = "book". a has 1 vowel whereas b has 2. Therefore, they
are not alike.
 * Notice that the vowel o is counted twice.
 * Example 3:
 * Input: s = "MerryChristmas"
 * Output: false
 * Example 4:
 * Input: s = "AbCdEfGh"
 * Output: true
 * Constraints:
 * 2 <= s.length <= 1000
 * s.length is even.
 * s consists of uppercase and lowercase letters.
 */
public class Solution01704 {

    // Time O(n)
    // Space O(1)
    public boolean halvesAreAlike(String s) {
        int strLen = s.length();
        int midLen = strLen / 2;
        int vowels = 0;
        for (int i = 0; i < midLen; i++) {
            if (isVowel(s.charAt(i))) {
                vowels++;
            }
        }
        for (int i = midLen; i < strLen; i++) {
            if (isVowel(s.charAt(i))) {
                vowels--;
```

```
            }
        }
        return vowels == 0;
    }

    private boolean isVowel(char ch) {
        return switch (Character.toLowerCase(ch)) {
            case 'a', 'e', 'i', 'o', 'u' -> true;
            default -> false;
        };
    }

}
```

343. `Solution01717.java`

```
package leetcode;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Iterator;

/**
 * 1717. Maximum Score From Removing Substrings
 * <a href="https://leetcode.com/problems/maximum-score-from-removing-substrings">Web
link</a> |
 * Medium
 */

public class Solution01717 {

    public int maximumGain(String s, int x, int y) {
        if (x > y) {
            return maximumGain(s, x, y, 'a', 'b');
        } else {
            return maximumGain(s, y, x, 'b', 'a');
        }
    }

    public int maximumGain(String s, int x, int y, char prefferedChar, char otherChar) {
        int n = s.length();
        int result = 0;
        Deque<Character> firstStack = new ArrayDeque<>();
        for (int i = 0; i < n; ++i) {
            char ch = s.charAt(i);
            result += addPairToResult(ch, x, prefferedChar, otherChar, firstStack);
        }
        Deque<Character> secondStack = new ArrayDeque<>();
        Iterator<Character> it = firstStack.descendingIterator();
        while (it.hasNext()) {
            char ch = it.next();
            result += addPairToResult(ch, y, otherChar, prefferedChar, secondStack);
        }
        return result;
    }

    private static int addPairToResult(char ch, int cost, char firstChar, char
secondChar, Deque<Character> stack) {
        if (ch == secondChar
            && !stack.isEmpty()
            && stack.peek() == firstChar) {
            stack.pop();
            return cost;
        } else {
            stack.push(ch);
            return 0;
        }
```

```
    }

    public int maximumGainOptimal(String s, int x, int y) {
        // Ensure "ab" always has higher points than "ba"
        if (x < y) {
            // Swap points
            int temp = x;
            x = y;
            y = temp;
            // Reverse the string to maintain logic
            s = new StringBuilder(s).reverse().toString();
        }

        int aCount = 0, bCount = 0, totalPoints = 0;

        for (int i = 0; i < s.length(); i++) {
            char currentChar = s.charAt(i);

            if (currentChar == 'a') {
                aCount++;
            } else if (currentChar == 'b') {
                if (aCount > 0) {
                    // Can form "ab", remove it and add points
                    aCount--;
                    totalPoints += x;
                } else {
                    // Can't form "ab", keep 'b' for potential future 'ba'
                    bCount++;
                }
            } else {
                // Non 'a' or 'b' character encountered
                // Calculate points for any remaining 'ba' pairs
                totalPoints += Math.min(bCount, aCount) * y;
                // Reset counters for next segment
                aCount = bCount = 0;
            }
        }

        // Calculate points for any remaining "ba" pairs at the end
        totalPoints += Math.min(bCount, aCount) * y;

        return totalPoints;
    }

}
```

344. `Solution01750.java`

```
package leetcode;

/**
 * 1750. Minimum Length of String After Deleting Similar Ends
 * https://leetcode.com/problems/minimum-length-of-string-after-deleting-similar-ends/
 * Medium
 */

public class Solution01750 {

    // Time: O(n)
    // Space: O(1)
    public int minimumLength(String s) {
        int left = 0;
        int right = s.length() - 1;
        while (left < right &&
               s.charAt(left) == s.charAt(right)) {
            char current = s.charAt(left);
            while (left <= right && s.charAt(left) == current) {
                left++;
            }
            while (left <= right && s.charAt(right) == current) {
                right--;
            }
        }
        return right - left + 1;
    }

}
```

345. `Solution01760.java`

```
/**
 * 1760. Minimum Limit of Balls in a Bag
 * <a href="https://leetcode.com/problems/minimum-limit-of-balls-in-a-bag">Web link</a> |
 * Medium
 */

package leetcode;

import java.util.Arrays;

public class Solution01760 {

    public int minimumSize(int[] nums, int maxOperations) {
        int left = 1;
        int right = Arrays.stream(nums).max().getAsInt();
        int ans = right;

        while (left <= right) {
            int mid = (left + right) / 2;
            int ops = 0;

            for (int n : nums) {
                ops += (n - 1) / mid;
                if (ops > maxOperations) {
                    break;
                }
            }

            if (ops <= maxOperations) {
                ans = mid;
                right = mid - 1;
            } else {
                left = mid + 1;
            }
        }

        return ans;
    }

}
```

346. `Solution01769.java`

```
/**
 * 1769. Minimum Number of Operations to Move All Balls to Each Box
 * <a href="https://leetcode.com/problems/minimum-number-of-operations-to-move-all-balls-to-each-box">Web link</a> |
 * Medium
 */

package leetcode;

public class Solution01769 {

    public int[] minOperations(String boxes) {
        int strLen = boxes.length();
        int[] result = new int[strLen];
        int ballCount = 0;
        int moveCount = 0;
        for (int i = 0; i < strLen; i++) {
            char ch = boxes.charAt(i);
            result[i] = ballCount + moveCount;
            moveCount += ballCount;
            if (ch == '1') {
                ballCount++;
            }
        }
        ballCount = 0;
        moveCount = 0;
        for (int i = strLen - 1; i >= 0; i--) {
            char ch = boxes.charAt(i);
            result[i] += ballCount + moveCount;
            moveCount += ballCount;
            if (ch == '1') {
                ballCount++;
            }
        }
        return result;
    }

    public int[] minOperationsOnePass(String boxes) {
        int strLen = boxes.length();

        int[] result = new int[strLen];

        int ballCountLeft = 0;
        int moveCountLeft = 0;
        int ballCountRight = 0;
        int moveCountRight = 0;

        for (int i = 0; i < strLen; i++) {
            // left
            char charLeft = boxes.charAt(i);
            result[i] += moveCountLeft;
```

```
            if (charLeft == '1') {
                ballCountLeft++;
            }
            moveCountLeft += ballCountLeft;
            // right
            int j = strLen - 1 - i;
            char charRight = boxes.charAt(j);
            result[j] += moveCountRight;
            if (charRight == '1') {
                ballCountRight++;
            }
            moveCountRight += ballCountRight;
        }
        return result;
    }

}
```

347. `Solution01791.java`

```
package leetcode;

/**
 * 1791. Find Center of Star Graph
 * <a href="https://leetcode.com/problems/find-center-of-star-graph">Web link</a> |
 * Easy
 */

public class Solution01791 {

    public int findCenter(int[][] edges) {
        int first = edges[0][0];
        int second = edges[0][1];
        int third = edges[1][0];
        if (third == first || third == second) {
            return third;
        }
        return edges[1][1]; // fourth
    }
}
```

348. `Solution01792.java`

```
/**
 * 1792. Maximum Average Pass Ratio
 * <a href="https://leetcode.com/problems/maximum-average-pass-ratio">Web link</a> |
 * Medium
 */

package leetcode;

import java.util.PriorityQueue;
import java.util.Queue;

public class Solution01792 {

    public double maxAverageRatio(int[][] classes, int extraStudents) {
        int n = classes.length;
        double[] ratios = new double[n];
        for (int i = 0; i < n; i++) {
            int[] c = classes[i];
            ratios[i] = (double) (c[0] + 1) / (c[1] + 1) - (double) c[0] / c[1];
        }

        Queue<Integer> pq = new PriorityQueue<>((a, b) -> Double.compare(ratios[b],
ratios[a]));
        for (int i = 0; i < n; i++) {
            pq.offer(i);
        }

        while (extraStudents-- > 0) {
            int i = pq.poll();
            classes[i][0]++;
            classes[i][1]++;
            ratios[i] = (double) (classes[i][0] + 1) / (classes[i][1] + 1) - (double)
classes[i][0] / classes[i][1];
            pq.offer(i);
        }

        double ans = 0;
        for (int[] aClass : classes) {
            ans += (double) aClass[0] / aClass[1];
        }

        return ans / n;
    }
}
```

349. `Solution01813.java`

```
package leetcode;

/**
 * 1813. Sentence Similarity III
 * <a href="https://leetcode.com/problems/sentence-similarity-iii">Web link</a> |
 * Medium
 */

public class Solution01813 {

    public boolean areSentencesSimilar(String sentence1, String sentence2) {
        final String smaller;
        final String bigger;
        int sentence1Length = sentence1.length();
        int sentence2Length = sentence2.length();
        if (sentence1Length == sentence2Length) {
            return sentence1.equals(sentence2);
        } else if (sentence1Length < sentence2Length) {
            smaller = sentence1;
            bigger = sentence2;
        } else {
            smaller = sentence2;
            bigger = sentence1;
        }

        if (bigger.startsWith(smaller)) {
            if (bigger.charAt(smaller.length()) == ' ') {
                return true;
            }
        } else if (bigger.endsWith(smaller)) {
            final char charBeforeSmaller = bigger.charAt(Math.abs(sentence1Length -
sentence2Length) - 1);
            if (charBeforeSmaller == ' ') {
                return true;
            }
        }

        String[] smallerWords = smaller.split(" ");
        String[] biggerWords = bigger.split(" ");
        int smallerWordsLength = smallerWords.length;
        int biggerWordsLength = biggerWords.length;

        int left = 0;
        while (left < smallerWordsLength &&
                smallerWords[left].equals(biggerWords[left])) {
            left++;
        }

        int rightSmaller = smallerWordsLength - 1;
        int rightBigger = biggerWordsLength - 1;
        while (rightSmaller >= left &&
```

```
                rightBigger >= 0 &&
                smallerWords[rightSmaller].equals(biggerWords[rightBigger])) {
            rightSmaller--;
            rightBigger--;
        }


        return left > rightSmaller;
    }

}
```

350. `Solution01823.java`

```
package leetcode;

/**
 * 1823. Find the Winner of the Circular Game
 * <a href="https://leetcode.com/problems/find-the-winner-of-the-circular-game">Web
link</a> |
 * Medium
 */

public class Solution01823 {

    public int findTheWinner(int n, int k) {
        int winner = 0; // Starting from the first player
        for (int totalPlayers = 2; totalPlayers <= n; totalPlayers++) {
            winner = (winner + k) % totalPlayers;
        }
        return winner + 1; // Adjusting to 1-based indexing
    }

    public int findTheWinnerRecursively(int n, int k) {
        return findWinnerRecursivelyHelper(n, k) + 1;
    }

    private int findWinnerRecursivelyHelper(int n, int eliminationStep) {
        if (n == 1) {
            return 0;
        }
        int nextEliminationPosition = findWinnerRecursivelyHelper(n - 1, eliminationStep)
+ eliminationStep;
        return nextEliminationPosition % n;
    }

}
```

351. `Solution01829.java`

```
/**
 * 1829. Maximum XOR for Each Query
 * <a href="https://leetcode.com/problems/maximum-xor-for-each-query">Web link</a> |
 * Medium
 */

package leetcode;

public class Solution01829 {

    public int[] getMaximumXor(int[] nums, int maximumBit) {
        int n = nums.length;
        int[] res = new int[n];
        int max = (1 << maximumBit) - 1;
        int xor = 0;
        for (int i = 0; i < n; i++) {
            xor ^= nums[i];
            res[n - i - 1] = xor ^ max;
        }
        return res;
    }

}
```

352. `Solution01861.java`

```
/**
 * 1861. Rotating the Box
 * <a href="https://leetcode.com/problems/rotating-the-box">Web link</a> |
 * Medium
 */

package leetcode;

public class Solution01861 {

    public char[][] rotateTheBox(char[][] box) {
        int totalOldRows = box.length;
        int totalOldColumns = box[0].length;
        char[][] rotated = new char[totalOldColumns][totalOldRows];

        for (int oldRow = 0; oldRow < totalOldRows; oldRow++) {
            int last = totalOldColumns - 1;
            final int newColumn = totalOldRows - 1 - oldRow;
            for (int oldColumn = totalOldColumns - 1; oldColumn >= 0; oldColumn--) {
                int newRow = oldColumn;
                if (box[oldRow][oldColumn] == '#') {
                    if (newRow != last) {
                        rotated[newRow][newColumn] = '.';
                        newRow = last;
                    }
                    rotated[newRow][newColumn] = '#';
                    --last;
                } else if (box[oldRow][oldColumn] == '*') {
                    rotated[newRow][newColumn] = '*';
                    last = newRow - 1;
                } else {
                    rotated[newRow][newColumn] = '.';
                }
            }
        }

        return rotated;
    }

}
```

353. `Solution01863.java`

```
package leetcode;

/**
 * 1863. Sum of All Subset XOR Totals
 * <a href="https://leetcode.com/problems/sum-of-all-subset-xor-totals/">Web link</a>
 * Easy
 */

public class Solution01863 {

    public int subsetXORSum(int[] nums) {
        int n = nums.length;
        int sum = 0;
        for (int num : nums) {
            sum |= num;
        }
        return sum << (n - 1);
    }

    public int subsetXORSumRecursion(int[] nums) {
        return dfs(nums, 0, 0);
    }

    private int dfs(int[] nums, int index, int total) {
        if (index == nums.length) {
            return total;
        }
        return dfs(nums, index + 1, total) + dfs(nums, index + 1, total ^ nums[index]);
    }

}
```

354. `Solution01894.java`

```
package leetcode;

/**
 * 1894. Find the Student that Will Replace the Chalk
 * <a href="https://leetcode.com/problems/find-the-student-that-will-replace-the-
chalk">Web link</a> |
 * Medium
 */

public class Solution01894 {

    public int chalkReplacer(int[] chalk, int k) {
        long totalChalk = 0;
        final int numStudents = chalk.length;

        // Calculate total chalk and reduce k in one loop
        for (int i = 0; i < numStudents; i++) {
            if (chalk[i] > k) return i;
            totalChalk += chalk[i];
            k -= chalk[i];
        }

        // Find the remainder of k when divided by the total chalk
        int remainingChalk = (int) (k % totalChalk);

        // Iterate through the chalk array to find the student
        for (int i = 0; i < numStudents; i++) {
            if (chalk[i] > remainingChalk) return i;
            remainingChalk -= chalk[i];
        }

        return 0;
    }

}
```

355. `Solution01897.java`

```
package leetcode;

import java.util.HashMap;
import java.util.Map;

/**
 * https://leetcode.com/problems/redistribute-characters-to-make-all-strings-equal/
 * 1897. Redistribute Characters to Make All Strings Equal
 * <p>
 * You are given an array of strings words (0-indexed).
 * <p>
 * In one operation, pick two distinct indices i and j, where words[i] is a non-empty
string,
 * and move any character from words[i] to any position in words[j].
 * <p>
 * Return true if you can make every string in words equal using any number of
operations, and false otherwise.
 * <p>
 * Example 1:
 * <p>
 * Input: words = ["abc","aabc","bc"]
 * Output: true
 * Explanation: Move the first 'a' in words[1] to the front of words[2],
 * to make words[1] = "abc" and words[2] = "abc".
 * All the strings are now equal to "abc", so return true.
 * <p>
 * Example 2:
 * <p>
 * Input: words = ["ab","a"]
 * Output: false
 * Explanation: It is impossible to make all the strings equal using the operation.
 * <p>
 * Constraints:
 * <p>
 * 1 <= words.length <= 100
 * 1 <= words[i].length <= 100
 * words[i] consists of lowercase English letters.
 */
public class Solution01897 {

    // Time: O(n * m)
    // Space: O(1)
    public boolean makeEqualMap(String[] words) {
        // Count the number of occurrences of each character.
        Map<Character, Integer> charCounts = new HashMap<>();
        for (String word : words) {
            word.chars().forEach(c -> charCounts.merge((char) c, 1, Integer::sum));
        }

        // Return whether the number of occurrences of each character is divisible by the
number of words.
```

```
        for (int count : charCounts.values()) {
            if (count % words.length != 0) {
                return false;
            }
        }
        return true;
    }

    // Time: O(n * m)
    // Space: O(1)
    public boolean makeEqualArray(String[] words) {
        // Count the number of occurrences of each character.
        int[] charCounts = new int[26];
        for (String word : words) {
            word.chars().forEach(c -> charCounts[(char) c - 'a']++);
        }

        // Return whether the number of occurrences of each character is divisible by the
number of words.
        for (int count : charCounts) {
            if (count % words.length != 0) {
                return false;
            }
        }
        return true;
    }

}
```

356. `Solution01905.java`

```
package leetcode;

/**
 * 1905. Count Sub Islands
 * <a href="https://leetcode.com/problems/count-sub-islands">Web link</a> |
 * Medium
 */

public class Solution01905 {

    public int countSubIslands(int[][] grid1, int[][] grid2) {
        int m = grid1.length;    // Number of rows in the grid
        int n = grid1[0].length; // Number of columns in the grid

        int subIslandCount = 0;  // To store the count of sub-islands

        // Iterate through each cell in grid2
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                // If the cell is land in grid2, it could be part of a sub-island
                if (grid2[i][j] == 1) {
                    boolean[] isSubIsland = {true};  // Start assuming this is a sub-
island
                    dfs(grid1, grid2, i, j, isSubIsland);  // Perform DFS to check the
whole island

                    // If the flag remains true after DFS, it means this island is a sub-
island
                    if (isSubIsland[0]) {
                        subIslandCount++;  // Increment the count of sub-islands
                    }
                }
            }
        }

        return subIslandCount;  // Return the total count of sub-islands
    }

    public void dfs(int[][] grid1, int[][] grid2, int i, int j, boolean[] isSubIsland) {
        int m = grid1.length;    // Number of rows in the grid
        int n = grid1[0].length; // Number of columns in the grid

        // If the current position is out of bounds or already visited (grid2[i][j] ==
0), return
        if (i < 0 || i == m || j < 0 || j == n || grid2[i][j] == 0) return;

        // If the current cell in grid2 is land but not in grid1, mark this island as not
a sub-island
        if (grid1[i][j] != grid2[i][j]) {
            isSubIsland[0] = false;
        }
```

```
        // Mark the current cell as visited by setting grid2[i][j] to 0
        grid2[i][j] = 0;

        // Continue the DFS in all four directions
        dfs(grid1, grid2, i + 1, j, isSubIsland);  // Down
        dfs(grid1, grid2, i - 1, j, isSubIsland);  // Up
        dfs(grid1, grid2, i, j + 1, isSubIsland);  // Right
        dfs(grid1, grid2, i, j - 1, isSubIsland);  // Left
    }
}
```

357. `Solution01915.java`

```
package leetcode;

/**
 * 1915. Number of Wonderful Substrings
 * <a href="https://leetcode.com/problems/number-of-wonderful-substrings">Web link</a> |
 * Medium
 */

public class Solution01915 {

    public long wonderfulSubstrings(String word) {
        long result = 0;
        int n = word.length();
        int[] count = new int[1024];
        count[0] = 1;
        int mask = 0;
        for (int i = 0; i < n; i++) {
            mask ^= 1 << (word.charAt(i) - 'a');
            result += count[mask];
            for (int j = 0; j < 10; j++) {
                result += count[mask ^ (1 << j)];
            }
            count[mask]++;
        }
        return result;
    }

}
```

358. `Solution01930.java`

```
/**
 * 1930. Unique Length-3 Palindromic Subsequences
 * <a href="https://leetcode.com/problems/unique-length-3-palindromic-subsequences">Web
link</a> |
 * Medium
 */

package leetcode;

import javafx.util.Pair;

import java.util.HashSet;
import java.util.Set;

public class Solution01930 {

    public int countPalindromicSubsequence(String s) {
        final int LETTERS_COUNT = 26;
        Pair<Integer, Integer>[] pairs = new Pair[LETTERS_COUNT];
        for (int i = 0; i < LETTERS_COUNT; i++) {
            pairs[i] = new Pair<>(-1, -1);
        }
        for (int i = 0; i < s.length(); i++) {
            int index = s.charAt(i) - 'a';
            if (pairs[index].getKey() == -1) {
                pairs[index] = new Pair<>(i, -1);
            } else {
                pairs[index] = new Pair<>(pairs[index].getKey(), i);
            }
        }
        int result = 0;
        for (int i = 0; i < LETTERS_COUNT; i++) {
            if (pairs[i].getValue() != -1) {
                Set<Character> set = new HashSet<>();
                for (int j = pairs[i].getKey() + 1; j < pairs[i].getValue(); j++) {
                    set.add(s.charAt(j));
                }
                result += set.size();
            }
        }
        return result;
    }

}
```

359. `Solution01937.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 1937. Maximum Number of Points with Cost
 * <a href="https://leetcode.com/problems/maximum-number-of-points-with-cost">Web
link</a>
 * Medium
 */
public class Solution01937 {

    public long maxPoints(int[][] points) {
        int m = points.length;
        int n = points[0].length;
        long[] dpPrev = new long[n];
        for (int j = 0; j < n; j++) {
            dpPrev[j] = points[0][j];
        }
        long[] dpCurr = dpPrev;
        for (int i = 1; i < m; i++) {
            long[] leftMax = new long[n];
            leftMax[0] = dpPrev[0];
            for (int j = 1; j < n; j++) {
                leftMax[j] = Math.max(dpPrev[j], leftMax[j - 1] - 1);
            }

            long[] rightMax = new long[n];
            rightMax[n - 1] = dpPrev[n - 1];
            for (int j = n - 2; j >= 0; j--) {
                rightMax[j] = Math.max(dpPrev[j], rightMax[j + 1] - 1);
            }

            dpCurr = new long[n];
            for (int j = 0; j < n; j++) {
                dpCurr[j] = points[i][j] + Math.max(leftMax[j], rightMax[j]);
            }

            dpPrev = dpCurr;
        }
        return Arrays.stream(dpCurr).max().getAsLong();
    }

    public long maxPoints_TLE(int[][] points) {
        int m = points.length;
        int n = points[0].length;
        long[] dpPrev = new long[n];
        for (int j = 0; j < n; j++) {
            dpPrev[j] = points[0][j];
        }
        long[] dpCurr = dpPrev;
```

```
        for (int i = 1; i < m; i++) {
            int[] pointLine = points[i];
            dpCurr = new long[n];
            for (int j = 0; j < n; j++) {
                long currPoint = pointLine[j];
                for (int k = 0; k < n; k++) {
                    dpCurr[j] = Math.max(dpCurr[j], currPoint + dpPrev[k] - Math.abs(j -
k));
                }
            }
            dpPrev = dpCurr;
        }
        return Arrays.stream(dpCurr).max().getAsLong();
    }

}
```

360. `Solution01942.java`

```
package leetcode;

import java.util.Arrays;
import java.util.Comparator;
import java.util.PriorityQueue;

/**
 * 1942. The Number of the Smallest Unoccupied Chair
 * <a href="https://leetcode.com/problems/the-number-of-the-smallest-unoccupied-
chair">Web link</a> |
 * Medium
 */

public class Solution01942 {

    public int smallestChair(int[][] times, int targetFriend) {
        int n = times.length;
        int targetArrival = times[targetFriend][0];

        Arrays.sort(times, Comparator.comparingInt(a -> a[0]));

        PriorityQueue<Integer> availableChairs = new PriorityQueue<>();
        for (int i = 0; i < n; i++) {
            availableChairs.add(i);
        }

        PriorityQueue<int[]> leavingQueue = new PriorityQueue<>(Comparator.comparingInt(a
-> a[0]));

        int nextChair = 0;
        for (int[] time : times) {
            int arrival = time[0];
            int leave = time[1];

            while (!leavingQueue.isEmpty() && leavingQueue.peek()[0] <= arrival) {
                availableChairs.add(leavingQueue.poll()[1]);
            }

            int chair;
            if (availableChairs.isEmpty()) {
                chair = nextChair++;
            } else {
                chair = availableChairs.poll();
            }

            leavingQueue.offer(new int[]{leave, chair});

            if (arrival == targetArrival) {
                return chair;
            }
        }
```

```
        return 0;
    }

}
```

361. `Solution01945.java`

```
package leetcode;

import java.math.BigInteger;

/**
 * 1945. Sum of Digits of String After Convert
 * <a href="https://leetcode.com/problems/sum-of-digits-of-string-after-convert">Web
link</a> |
 * Easy
 */

public class Solution01945 {

    public int getLucky(String s, int k) {
        BigInteger convert = convert(s);
        for (int i = 0; i < k; i++) {
            convert = transform(convert);
        }
        return convert.intValue();
    }

    private BigInteger transform(BigInteger convert) {
        BigInteger sum = BigInteger.ZERO;
        while (convert.compareTo(BigInteger.valueOf(9)) > 0) {
            sum = sum.add(convert.mod(BigInteger.TEN));
            convert = convert.divide(BigInteger.TEN);
        }
        return sum.add(convert);
    }

    private BigInteger convert(String s) {
        int n = s.length();
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < n; i++) {
            sb.append(s.charAt(i) - 'a' + 1);
        }
        return new BigInteger(sb.toString());
    }

}
```

362. `Solution01957.java`

```
/**
 * 1957. Delete Characters to Make Fancy String
 * <a href="https://leetcode.com/problems/delete-characters-to-make-fancy-string">Web
link</a> |
 * Easy
 */

package leetcode;

public class Solution01957 {

    public String makeFancyString(String s) {
        if (s.length() < 3) {
            return s;
        }

        char[] chars = s.toCharArray();

        StringBuilder sb = new StringBuilder();
        sb.append(chars[0])
                .append(chars[1]);

        int strLen = chars.length;
        for (int pos = 2; pos < strLen; pos++) {
            char currentCharacter = s.charAt(pos);
            if (currentCharacter != chars[pos - 1] || currentCharacter != chars[pos - 2])
{
                sb.append(currentCharacter);
            }
        }

        return sb.toString();
    }

}
```

363. `Solution01963.java`

```
package leetcode;

/**
 * 1963. Minimum Number of Swaps to Make the String Balanced
 * <a href="https://leetcode.com/problems/minimum-number-of-swaps-to-make-the-string-
balanced">Web link</a> |
 * Medium
 */

public class Solution01963 {

    public int minSwaps(String s) {
        int size = 0;
        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);
            if (ch == '[') {
                size++;
            } else if (size > 0) {
                size--;
            }
        }
        return (size + 1) / 2;
    }

}
```

364. `Solution01971.java`

```
package leetcode;

import java.util.*;

/**
 * 1971. Find if Path Exists in Graph
 * Source: https://leetcode.com/problems/find-if-path-exists-in-graph/
 * Easy
 */

public class Solution01971 {

    int[] parent;

    public boolean validPath(int n, int[][] edges, int source, int destination) {
        Map<Integer, List<Integer>> graph = new HashMap<>();
        for (int[] edge : edges) {
            int u = edge[0];
            int v = edge[1];
            graph.computeIfAbsent(u, k -> new ArrayList<>()).add(v);
            graph.computeIfAbsent(v, k -> new ArrayList<>()).add(u);
        }
        Set<Integer> visited = new HashSet<>();
        Queue<Integer> queue = new LinkedList<>();
        queue.offer(source);
        visited.add(source);
        while (!queue.isEmpty()) {
            int node = queue.poll();
            if (node == destination) {
                return true;
            }
            for (int neighbor : graph.getOrDefault(node, Collections.emptyList())) {
                if (visited.add(neighbor)) {
                    queue.offer(neighbor);
                }
            }
        }
        return false;
    }

    public boolean validPathUf(int n, int[][] edges, int source, int destination) {
        parent = new int[n];
        for (int i = 0; i < n; i++) {
            parent[i] = i;
        }

        for (int[] edge : edges) {
            union(edge[0], edge[1]);
        }

        return find(source) == find(destination);
```

```
    }

    private void union(int x, int y) {
        parent[find(x)] = find(y);
    }

    private int find(int x) {
        if (parent[x] != x) {
            parent[x] = find(parent[x]);
        }
        return parent[x];
    }

    public boolean validPathSpread(int n, int[][] edges, int source, int destination) {
        boolean[] isNodeVisited = new boolean[n];
        isNodeVisited[source] = true;

        boolean isStateChangedInCurrentIteration = true;

        while (isStateChangedInCurrentIteration) {
            isStateChangedInCurrentIteration = false;

            for (int[] edge : edges) {
                if (isNodeVisited[edge[0]] != isNodeVisited[edge[1]]) {
                    isNodeVisited[edge[0]] = true;
                    isNodeVisited[edge[1]] = true;

                    isStateChangedInCurrentIteration = true;
                }

                if (isNodeVisited[destination]) {
                    return true;
                }
            }
        }

        return isNodeVisited[destination];
    }

}
```

365. `Solution01975.java`

```
/**
 * 1975. Maximum Matrix Sum
 * <a href="https://leetcode.com/problems/maximum-matrix-sum">Web link</a> |
 * Medium
 */

package leetcode;

public class Solution01975 {

    public long maxMatrixSum(int[][] matrix) {
        long sum = 0;
        int min = Integer.MAX_VALUE;
        int negativeCount = 0;

        for (int[] row : matrix) {
            for (int num : row) {
                int absNum;
                if (num < 0) {
                    absNum = Math.abs(num);
                    negativeCount++;
                } else {
                    absNum = num;
                }
                sum += absNum;
                min = Math.min(min, absNum);
            }
        }

        if (negativeCount % 2 == 0) {
            return sum;
        }
        return sum - 2L * min;
    }

}
```

366. `Solution01992.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.List;

/**
 * 1992. Find All Groups of Farmland
 * Source: https://leetcode.com/problems/find-all-groups-of-farmland/
 * Medium
 */

public class Solution01992 {

    public int[][] findFarmland(int[][] land) {
        List<int[]> farmlands = new ArrayList<>();
        int rows = land.length;
        int cols = land[0].length;
        for (int row = 0; row < rows; row++) {
            for (int col = 0; col < cols; col++) {
                // If the current cell is 1, then it is the top-left corner of a farmland
                if (land[row][col] == 1) {
                    int farmlandLastRow = row;
                    int farmlandLastCol = col;
                    // Find the bottom-right corner of the farmland
                    while (farmlandLastRow < rows && land[farmlandLastRow][col] == 1) {
                        farmlandLastRow++;
                    }

                    while (farmlandLastCol < cols && land[row][farmlandLastCol] == 1) {
                        farmlandLastCol++;
                    }

                    farmlands.add(new int[]{row, col, farmlandLastRow - 1,
farmlandLastCol - 1});
                    // Mark the farmland as visited
                    for (int k = row; k < farmlandLastRow; k++) {
                        for (int l = col; l < farmlandLastCol; l++) {
                            land[k][l] = 0;
                        }
                    }
                }
            }
        }
        return farmlands.toArray(new int[0][]);
    }

}
```

367. `Solution02000.java`

```
package leetcode;

/**
 * 2000. Reverse Prefix of Word
 * <a href="https://leetcode.com/problems/reverse-prefix-of-word">Web link</a> |
 * Easy
 */

public class Solution02000 {

    public String reversePrefix(String word, char ch) {
        int i = word.indexOf(ch);
        if (i == -1) {
            return word;
        }
        return new StringBuilder(word.substring(0, i + 1))
                .reverse()
                .append(word.substring(i + 1))
                .toString();
    }

}
```

368. `Solution02022.java`

```
package leetcode;

/**
 * 2022. Convert 1D Array Into 2D Array
 * <a href="https://leetcode.com/problems/convert-1d-array-into-2d-array">Web link</a> |
 * Easy
 */

public class Solution02022 {

    public int[][] construct2DArray(int[] original, int m, int n) {
        int len = original.length;
        if (m * n != len) return new int[][]{};
        int[][] result = new int[m][n];
        for (int i = 0; i < len; i++) {
            result[i / n][i % n] = original[i];
        }
        return result;
    }

}
```

369. `Solution02028.java`

```
package leetcode;

/**
 * 2028. Find Missing Observations
 * <a href="https://leetcode.com/problems/find-missing-observations">Web link</a> |
 * Medium
 */

public class Solution02028 {

    public int[] missingRolls(int[] rolls, int mean, int n) {
        int numRolls = rolls.length;
        int totalSum = mean * (n + numRolls);
        int observedSum = 0;

        // Calculate observed sum in a single loop
        for (int roll : rolls) {
            observedSum += roll;
        }

        int missingSum = totalSum - observedSum;

        // Check if the missing sum is within the valid range
        if (missingSum < n || missingSum > 6 * n) {
            return new int[0];
        }

        int baseValue = missingSum / n;
        int extra = missingSum % n;

        int[] result = new int[n];
        for (int i = 0; i < n; i++) {
            result[i] = baseValue + (i < extra ? 1 : 0);
        }

        return result;
    }

}
```

370. `Solution02037.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 2037. Minimum Number of Moves to Seat Everyone
 * <a href="https://leetcode.com/problems/minimum-number-of-moves-to-seat-everyone">Web
link</a> |
 * Easy
 */

public class Solution02037 {

    public int minMovesToSeat(int[] seats, int[] students) {
        int n = seats.length;
        int moves = 0;
        Arrays.sort(seats);
        Arrays.sort(students);
        for (int i = 0; i < n; i++) {
            moves += Math.abs(seats[i] - students[i]);
        }
        return moves;
    }

}
```

371. `Solution02044.java`

```
/**
 * 2044. Count Number of Maximum Bitwise-OR Subsets
 * <a href="https://leetcode.com/problems/count-number-of-maximum-bitwise-or-subsets">Web
link</a> |
 * Medium
 */

package leetcode;

public class Solution02044 {

    public int countMaxOrSubsets(int[] nums) {
        int maxOr = 0;

        for (int num : nums) {
            maxOr |= num;
        }

        int[] count = new int[1];
        backtrack(nums, 0, 0, maxOr, count);

        return count[0];
    }

    private void backtrack(int[] nums, int index, int currentOr, int maxOr, int[] count)
{
        if (currentOr == maxOr) {
            count[0]++;
        }

        for (int i = index; i < nums.length; i++) {
            backtrack(nums, i + 1, currentOr | nums[i], maxOr, count);
        }
    }

}
```

372. `Solution02045.java`

```
package leetcode;

import java.util.*;

/**
 * 2045. Second Minimum Time to Reach Destination
 * <a href="https://leetcode.com/problems/second-minimum-time-to-reach-destination">Web
link</a> |
 * Hard
 */

public class Solution02045 {

    public int secondMinimum(int n, int[][] edges, int time, int change) {
        List<List<Integer>> adj = new ArrayList<>(n + 1);

        for (int i = 0; i <= n; i++) {
            adj.add(new ArrayList<>());
        }

        for (int[] edge : edges) {
            int node1 = edge[0];
            int node2 = edge[1];
            adj.get(node1).add(node2);
            adj.get(node2).add(node1);
        }

        Queue<Integer> queue = new ArrayDeque<>();
        queue.offer(1);
        int currentTime = 0;
        int result = -1;
        Map<Integer, List<Integer>> visited = new HashMap<>();

        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int i = 0; i < size; i++) {
                int current = queue.poll();
                if (current == n) {
                    if (result != -1) {
                        return currentTime;
                    }
                    result = currentTime;
                }
                for (int neighbor : adj.get(current)) {
                    var list = visited.computeIfAbsent(neighbor, key -> new ArrayList<>
());
                    if (list.isEmpty() || (list.size() == 1 && list.getFirst() !=
currentTime)) {
                        list.add(currentTime);
                        queue.offer(neighbor);
                    }
```

```
                }
            }
            if ((currentTime / change) % 2 == 1) {
                currentTime += change - (currentTime % change);
            }
            currentTime += time;
        }

        return 0;
    }

}
```

373. `Solution02053.java`

```
package leetcode;

import java.util.LinkedHashMap;
import java.util.Map;

/**
 * 2053. Kth Distinct String in an Array
 * <a href="https://leetcode.com/problems/kth-distinct-string-in-an-array">Web link</a> |
 * Medium
 */

public class Solution02053 {

    public String kthDistinct(String[] arr, int k) {
        Map<String, Integer> sequencedMap = new LinkedHashMap<>();
        for (String s : arr) {
            sequencedMap.put(s, sequencedMap.getOrDefault(s, 0) + 1);
        }
        return sequencedMap.entrySet()
                .stream()
                .filter(entry -> entry.getValue() == 1)
                .skip(k - 1)
                .findFirst()
                .map(Map.Entry::getKey)
                .orElse("");
    }

}
```

374. `Solution02054.java`

```
/**
 * 2054. Two Best Non-Overlapping Events
 * <a href="https://leetcode.com/problems/two-best-non-overlapping-events">Web link</a> |
 * Medium
 */

package leetcode;

import java.util.Arrays;
import java.util.Comparator;

public class Solution02054 {

    public int maxTwoEvents(int[][] events) {
        Arrays.sort(events, Comparator.comparingInt(a -> a[1]));

        int n = events.length;
        int[] maxValues = new int[n];
        maxValues[0] = events[0][2];

        for (int i = 1; i < n; i++) {
            maxValues[i] = Math.max(maxValues[i - 1], events[i][2]);
        }

        int maxSum = 0;

        for (int[] event : events) {
            int start = event[0];
            int value = event[2];
            int idx = binarySearch(events, start - 1);
            int currentSum = value + (idx != -1 ? maxValues[idx] : 0);
            maxSum = Math.max(maxSum, currentSum);
        }

        return maxSum;
    }

    private int binarySearch(int[][] events, int targetEnd) {
        int start = 0;
        int end = events.length - 1;
        int res = -1;

        while (start <= end) {
            int mid = start + (end - start) / 2;
            if (events[mid][1] <= targetEnd) {
                res = mid;
                start = mid + 1;
            } else {
                end = mid - 1;
            }
        }
```

```
        return res;
    }

}
```

375. `Solution02058.java`

```
package leetcode;

import leetcode.common.ListNode;

/**
 * 2058. Find the Minimum and Maximum Number of Nodes Between Critical Points
 * <a href="https://leetcode.com/problems/find-the-minimum-and-maximum-number-of-nodes-
between-critical-points">Web link</a>
 * Medium
 */

public class Solution02058 {

    public int[] nodesBetweenCriticalPoints(ListNode head) {
        int firstCriticalPoint = -1;
        int lastCriticalPoint = -1;

        ListNode cursor = head;

        int previousValue = cursor.val;
        cursor = cursor.next;
        int currentValue = cursor.val;
        cursor = cursor.next;
        int nextValue;
        int distancePos = 0;
        int minDistance = Integer.MAX_VALUE;
        while (cursor != null) {
            ++distancePos;
            nextValue = cursor.val;
            if ((currentValue > previousValue && currentValue > nextValue) ||
                (currentValue < previousValue && currentValue < nextValue)) {
                if (firstCriticalPoint == -1) {
                    firstCriticalPoint = distancePos;
                } else {
                    minDistance = Math.min(minDistance, distancePos - lastCriticalPoint);
                }
                lastCriticalPoint = distancePos;
            }
            previousValue = currentValue;
            currentValue = nextValue;
            cursor = cursor.next;
        }

        int[] result = new int[2];
        if (minDistance == Integer.MAX_VALUE) {
            result[0] = -1;
            result[1] = -1;
        } else {
            result[0] = minDistance;
            result[1] = lastCriticalPoint - firstCriticalPoint;
        }
```

```
        return result;
    }

}
```

376. `Solution02064.java`

```
/**
 * 2064. Minimized Maximum of Products Distributed to Any Store
 * <a href="https://leetcode.com/problems/minimized-maximum-of-products-distributed-to-
any-store">Web link</a> |
 * Medium
 */

package leetcode;

import java.util.Arrays;

public class Solution02064 {

    public int minimizedMaximum(int n, int[] quantities) {
        int low = 1;
        int high = Arrays.stream(quantities).max().getAsInt();
        int ans = -1;

        while (low <= high) {
            int mid = (low + high) / 2;
            if (solve(n, quantities, mid)) {
                ans = mid;
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        }

        return ans;
    }

    private boolean solve(int n, int[] quantities, int item) {
        if (item == 0) return false;
        int store = 0;
        for (int product : quantities) {
            store += (product - 1) / item + 1;
            if (store > n) return false;
        }
        return true;
    }

}
```

377. `Solution02070.java`

```
/**
 * 2070. Most Beautiful Item for Each Query
 * <a href="https://leetcode.com/problems/most-beautiful-item-for-each-query">Web
link</a> |
 * Medium
 */

package leetcode;

import java.util.Arrays;
import java.util.Comparator;

public class Solution02070 {

    public int[] maximumBeauty(int[][] items, int[] queries) {
        int[] ans = new int[queries.length];

        // Sort and store max beauty
        Arrays.sort(items, Comparator.comparingInt(a -> a[0]));
        int max = items[0][1];
        for (int i = 0; i < items.length; i++) {
            max = Math.max(max, items[i][1]);
            items[i][1] = max;
        }

        for (int i = 0; i < queries.length; i++) {
            // answer i-th query
            ans[i] = binarySearch(items, queries[i]);
        }

        return ans;
    }

    private int binarySearch(int[][] items, int targetPrice) {
        int l = 0;
        int r = items.length - 1;
        int maxBeauty = 0;
        while (l <= r) {
            int mid = (l + r) / 2;
            if (items[mid][0] > targetPrice) {
                r = mid - 1;
            } else {
                // Found viable price. Keep moving to right
                maxBeauty = Math.max(maxBeauty, items[mid][1]);
                l = mid + 1;
            }
        }
        return maxBeauty;
    }

}
```

378. `Solution02073.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/time-needed-to-buy-tickets/
 * 2073. Time Needed to Buy Tickets
 * Easy
 */

public class Solution02073 {

    // Time complexity: O(n)
    // Space complexity: O(1)
    public int timeRequiredToBuy(int[] tickets, int k) {
        int n = tickets.length;
        int seconds = 0;
        for (int i = 0; i < n; i++, i %= n) {
            if (tickets[i] > 1) {
                seconds++;
                tickets[i]--;
            } else if (tickets[i] == 1) {
                seconds++;
                if (i == k) {
                    return seconds;
                } else {
                    tickets[i]--;
                }
            }
        }
        return seconds;
    }

    public int timeRequiredToBuyOptimized(int[] tickets, int k) {
        int n = tickets.length;
        int seconds = 0;

        int timesKOrBefore = tickets[k];
        int timesAfterK = timesKOrBefore - 1;

        for (int i = 0; i < n; i++) {
            if (i <= k) {
                seconds += Math.min(tickets[i], timesKOrBefore);
            } else {
                seconds += Math.min(tickets[i], timesAfterK);
            }
        }
        return seconds;
    }
}
```

379. `Solution02092.java`

```
package leetcode;

import java.util.*;

/**
 * https://leetcode.com/problems/find-all-people-with-secret/description/?envType=daily-question&envId=2024-02-24
 * 2092. Find All People With Secret
 * Hard
 */

public class Solution02092 {

    // Time: O(n * log(n))
    // Space: O(n)
    public List<Integer> findAllPeople(int n, int[][] meetings, int firstPerson) {
        Set<Integer> secrets = new LinkedHashSet<>();
        secrets.add(0);
        secrets.add(firstPerson);

        Map<Integer, Map<Integer, Set<Integer>>> timeToPerson = new HashMap<>();
        for (int[] meeting : meetings) {
            int time = meeting[2];
            int first = meeting[0];
            int second = meeting[1];
            Map<Integer, Set<Integer>> participantsAtTime = timeToPerson.computeIfAbsent(time, k -> new HashMap<>());
            Set<Integer> adjToFirst = participantsAtTime.computeIfAbsent(first, k -> new HashSet<>());
            adjToFirst.add(second);
            Set<Integer> adjToSecond = participantsAtTime.computeIfAbsent(second, k -> new HashSet<>());
            adjToSecond.add(first);
        }

        timeToPerson.entrySet()
                .stream()
                .sorted(Map.Entry.comparingByKey())
                .forEach(entry -> {
                    Set<Integer> visited = new HashSet<>();
                    Map<Integer, Set<Integer>> participantsAtTime = entry.getValue();
                    for (var participantEntry : participantsAtTime.entrySet()) {
                        var sharingUser = participantEntry.getKey();
                        if (secrets.contains(sharingUser)) {
                            dfs(sharingUser, participantsAtTime, secrets, visited);
                        }
                    }
                });

        return new ArrayList<>(secrets);
    }
```

```
    private void dfs(Integer sharingUser, Map<Integer, Set<Integer>> participantsAtTime,
                     Set<Integer> secrets, Set<Integer> visited) {
        if (visited.contains(sharingUser)) {
            return;
        }
        visited.add(sharingUser);
        secrets.add(sharingUser);
        Set<Integer> adjParticipants = participantsAtTime.get(sharingUser);
        if (adjParticipants == null) {
            return;
        }
        for (Integer adj : adjParticipants) {
            if (adj.equals(sharingUser)) {
                continue;
            }
            dfs(adj, participantsAtTime, secrets, visited);
        }
    }

}
```

380. `Solution02096.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.ArrayList;
import java.util.List;

/**
 * 2096. Step-By-Step Directions From a Binary Tree Node to Another
 * <a href="https://leetcode.com/problems/step-by-step-directions-from-a-binary-tree-
node-to-another">Web link</a> |
 * Medium
 */

public class Solution02096 {

    public String getDirections(TreeNode root, int startValue, int destValue) {
        List<Character> startPath = new ArrayList<>();
        List<Character> destPath = new ArrayList<>();
        findPath(root, startValue, startPath);
        findPath(root, destValue, destPath);
        int commonAncestorIndex = findCommonAncestorIndex(startPath, destPath);
        return buildDirection(startPath.size() - commonAncestorIndex, destPath,
commonAncestorIndex);
    }

    private boolean findPath(TreeNode node, int value, List<Character> path) {
        if (node == null) return false;
        if (node.val == value) return true;
        path.add('L');
        if (findPath(node.left, value, path)) return true;
        path.removeLast();
        path.add('R');
        if (findPath(node.right, value, path)) return true;
        path.removeLast();
        return false;
    }

    private int findCommonAncestorIndex(List<Character> startPath, List<Character>
destPath) {
        int index = 0;
        while (index < startPath.size() && index < destPath.size() &&
startPath.get(index).equals(destPath.get(index))) {
            index++;
        }
        return index;
    }

    private String buildDirection(int upMoves, List<Character> destPath, int
commonAncestorIndex) {
        StringBuilder direction = new StringBuilder();
```

```
        direction.append("U".repeat(upMoves));
        for (int i = commonAncestorIndex; i < destPath.size(); i++) {
            direction.append(destPath.get(i));
        }
        return direction.toString();
    }

}
```

381. `Solution02097.java`

```
/**
 * 2097. Valid Arrangement of Pairs
 * <a href="https://leetcode.com/problems/valid-arrangement-of-pairs">Web link</a> |
 * Hard
 */

package leetcode;

import java.util.*;

public class Solution02097 {

    public int[][] validArrangement(int[][] pairs) {
        Map<Integer, List<Integer>> adjacencyList = new HashMap<>();
        Map<Integer, Integer> inOutDegree = new HashMap<>();

        // Build graph and count in/out degrees
        for (int[] pair : pairs) {
            adjacencyList.computeIfAbsent(pair[0], k -> new ArrayList<>()).add(pair[1]);
            inOutDegree.merge(pair[0], 1, Integer::sum);  // out-degree
            inOutDegree.merge(pair[1], -1, Integer::sum);  // in-degree
        }

        // Find starting node
        int startNode = pairs[0][0];
        for (Map.Entry<Integer, Integer> entry : inOutDegree.entrySet()) {
            if (entry.getValue() == 1) {
                startNode = entry.getKey();
                break;
            }
        }

        List<Integer> path = new ArrayList<>();
        Deque<Integer> nodeStack = new ArrayDeque<>();
        nodeStack.push(startNode);

        while (!nodeStack.isEmpty()) {
            List<Integer> neighbors = adjacencyList.getOrDefault(nodeStack.peek(), new
ArrayList<>());
            if (neighbors.isEmpty()) {
                path.add(nodeStack.pop());
            } else {
                int nextNode = neighbors.getLast();
                nodeStack.push(nextNode);
                neighbors.removeLast();
            }
        }

        int pathSize = path.size();
        int[][] arrangement = new int[pathSize - 1][2];
```

```
        for (int i = pathSize - 1; i > 0; --i) {
            arrangement[pathSize - 1 - i] = new int[]{path.get(i), path.get(i - 1)};
        }

        return arrangement;
    }

}
```

382. `Solution02108.java`

```
package leetcode;

/**
 * 2108. Find First Palindromic String in the Array
 * https://leetcode.com/problems/find-first-palindromic-string-in-the-array
 * Easy
 */
public class Solution02108 {

    // Time: O(n^2)
    // Space: O(1)
    public String firstPalindrome(String[] words) {
        for (String word : words) {
            if (isPalindrome(word)) {
                return word;
            }
        }
        return "";
    }

    private boolean isPalindrome(String s) {
        int start = 0;
        int end = s.length() - 1;

        while (start < end) {
            char startChar = s.charAt(start);
            char endChar = s.charAt(end);
            if (startChar != endChar) {
                return false;
            }
            start++;
            end--;
        }
        return true;
    }

}
```

383. `Solution02109.java`

```
/**
 * 2109. Adding Spaces to a String
 * <a href="https://leetcode.com/problems/adding-spaces-to-a-string">Web link</a> |
 * Medium
 */

package leetcode;

public class Solution02109 {

    public String addSpaces(String s, int[] spaces) {
        StringBuilder sb = new StringBuilder();
        if (spaces[0] > 0) {
            sb.append(s.substring(0, spaces[0]));
        }
        int currentPosition = spaces[0];
        for (int i = 1; i < spaces.length; i++) {
            int nextPosition = spaces[i];
            sb.append(" ")
                    .append(s.substring(currentPosition, nextPosition));
            currentPosition = nextPosition;
        }
        sb.append(" ")
                .append(s.substring(currentPosition));
        return sb.toString();
    }

}
```

384. `Solution02116.java`

```
/**
 * 2116. Check if a Parentheses String Can Be Valid
 * <a href="https://leetcode.com/problems/check-if-a-parentheses-string-can-be-valid">Web
link</a> |
 * Medium
 */

package leetcode;

import java.util.ArrayDeque;
import java.util.Deque;

public class Solution02116 {

    public boolean canBeValid(String s, String locked) {
        int strLen = s.length();
        if (strLen % 2 != 0) {
            return false;
        }

        Deque<Integer> indicesForOpened = new ArrayDeque<>();
        Deque<Integer> indicesForUnlocked = new ArrayDeque<>();

        for (int i = 0; i < strLen; i++) {
            if (locked.charAt(i) == '0') {
                indicesForUnlocked.push(i);
            } else {
                if (s.charAt(i) == '(') {
                    indicesForOpened.push(i);
                } else {
                    if (indicesForOpened.isEmpty()) {
                        if (indicesForUnlocked.isEmpty()) {
                            return false;
                        }
                        indicesForUnlocked.pop();
                    } else {
                        indicesForOpened.pop();
                    }
                }
            }
        }

        while (!indicesForOpened.isEmpty() && !indicesForUnlocked.isEmpty()) {
            if (indicesForOpened.peek() < indicesForUnlocked.peek()) {
                indicesForOpened.pop();
                indicesForUnlocked.pop();
            } else {
                return false;
            }
        }
        return indicesForOpened.isEmpty();
```

```
    }

}
```

385. `Solution02125.java`

```
package leetcode;

/**
 * <a href="https://leetcode.com/problems/number-of-laser-beams-in-a-bank/">
 * 2125. Number of Laser Beams in a Bank
 * Anti-theft security devices are activated inside a bank. You are given a 0-indexed
binary string array bank representing the floor plan of the bank, which is an m x n 2D
matrix. bank[i] represents the ith row, consisting of '0's and '1's. '0' means the cell
is empty, while'1' means the cell has a security device.
 * <p>
 * There is one laser beam between any two security devices if both conditions are met:
 * <p>
 * The two devices are located on two different rows: r1 and r2, where r1 < r2.
 * For each row i where r1 < i < r2, there are no security devices in the ith row.
 * Laser beams are independent, i.e., one beam does not interfere nor join with another.
 * <p>
 * Return the total number of laser beams in the bank.
 * </p>
 * <p><strong>Example 1:</strong></p>
 * Input: bank = ["011001","000000","010100","001000"]
 * Output: 8
 * Explanation: Between each of the following device pairs, there is one beam. In total,
there are 8 beams:
 * bank[0][1] -- bank[2][1]
 * bank[0][1] -- bank[2][3]
 * bank[0][2] -- bank[2][1]
 * bank[0][2] -- bank[2][3]
 * bank[0][5] -- bank[2][1]
 * bank[0][5] -- bank[2][3]
 * bank[2][1] -- bank[3][2]
 * bank[2][3] -- bank[3][2]
 * Note that there is no beam between any device on the 0th row with any on the 3rd row.
 * This is because the 2nd row contains security devices, which breaks the second
condition.
 *     <p><strong>Example 2:</strong></p>
 * Input: bank = ["000","111","000"]
 * Output: 0
 * Explanation: There does not exist two devices located on two different rows.
 */
public class Solution02125 {

    // Time: O(n)
    // Space: O(1)
    public int numberOfLaserBeamsInABank(String[] bank) {
        long previous = 0;
        long result = 0;

        for (String row : bank) {
            long current = row.chars().filter(charInt -> (char) charInt == '1').count();
            if (current > 0) {
                result += (previous * current);
```

```
                previous = current;
            }
        }

        return (int) result;
    }
}
```

386. `Solution02134.java`

```
package leetcode;

/**
 * 2134. Minimum Swaps to Group All 1's Together II
 * <a href="https://leetcode.com/problems/minimum-swaps-to-group-all-1s-together-ii">Web
link</a> |
 * Medium
 */

public class Solution02134 {

    public int minSwaps(int[] nums) {
        int n = nums.length;
        int ones = 0;
        for (int num : nums) {
            if (num == 1) {
                ones++;
            }
        }
        int windowOnes = 0;
        int maxWindowOnes = 0;
        int left = 0;
        int checkRange = n + ones;
        for (int right = 0; right < checkRange; right++) {
            if (nums[right % n] == 1) {
                windowOnes++;
            }
            if (right - left + 1 > ones) {
                windowOnes -= nums[left % n];
                left++;
            }
            maxWindowOnes = Math.max(maxWindowOnes, windowOnes);
        }
        return ones - maxWindowOnes;
    }

}
```

387. `Solution02149.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/rearrange-array-elements-by-sign/
 * 2149. Rearrange Array Elements by Sign
 * Medium
 */
public class Solution02149 {

    // Time complexity: O(n)
    // Space complexity: O(n)
    public int[] rearrangeArray(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];

        int positiveIndex = 0;
        int negativeIndex = 1;
        for (int num : nums) {
            if (num >= 0) {
                result[positiveIndex] = num;
                positiveIndex += 2;
            } else {
                result[negativeIndex] = num;
                negativeIndex += 2;
            }
        }
        return result;
    }

}
```

388. `Solution02181.java`

```
package leetcode;

import leetcode.common.ListNode;

/**
 * 2181. Merge Nodes in Between Zeros
 * <a href="https://leetcode.com/problems/merge-nodes-in-between-zeros/">Web link</a>
 * Medium
 */

public class Solution02181 {

    public ListNode mergeNodes(ListNode head) {
        ListNode dummy = new ListNode();
        ListNode sourceCursor = head;
        ListNode targetCursor = dummy;

        int sum = 0;
        while (sourceCursor != null) {
            if (sourceCursor.val != 0) {
                sum += sourceCursor.val;
            } else if (sum > 0) {
                targetCursor.next = new ListNode(sum);
                targetCursor = targetCursor.next;
                sum = 0;
            }
            sourceCursor = sourceCursor.next;
        }

        return dummy.next;
    }

}
```

389. `Solution02182.java`

```
/**
 * 2182. Construct String With Repeat Limit
 * <a href="https://leetcode.com/problems/construct-string-with-repeat-limit">Web
link</a> |
 * Medium
 */

package leetcode;

import java.util.PriorityQueue;

public class Solution02182 {

    public String repeatLimitedString(String s, int repeatLimit) {
        int[] freq = new int[26];
        for (char c : s.toCharArray()) {
            freq[c - 'a']++;
        }
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> b[0] - a[0]);
        for (int i = 0; i < 26; i++) {
            if (freq[i] > 0) {
                pq.add(new int[]{i, freq[i]});
            }
        }
        StringBuilder result = new StringBuilder();
        while (!pq.isEmpty()) {
            int[] curr = pq.poll();
            int use = Math.min(repeatLimit, curr[1]);
            result.append(String.valueOf((char) ('a' + curr[0])).repeat(use));
            curr[1] -= use;
            if (curr[1] > 0 && !pq.isEmpty()) {
                int[] next = pq.poll();
                result.append((char) ('a' + next[0]));
                next[1]--;
                if (next[1] > 0) {
                    pq.add(next);
                }
                pq.add(curr);
            }
        }
        return result.toString();
    }
}
```

390. `Solution02185.java`

```
/**
 * 2185. Counting Words With a Given Prefix
 * <a href="https://leetcode.com/problems/counting-words-with-a-given-prefix">Web
link</a> |
 * Easy
 */

package leetcode;

public class Solution02185 {

    public int prefixCount(String[] words, String pref) {
        int count = 0;
        for (String word : words) {
            if (word.startsWith(pref)) {
                count++;
            }
        }
        return count;
    }

}
```

391. `Solution02191.java`

```
package leetcode;

import java.util.Arrays;
import java.util.Comparator;

/**
 * 2191. Sort the Jumbled Numbers
 * <a href="https://leetcode.com/problems/sort-the-jumbled-numbers">Web link</a> |
 * Medium
 */

public class Solution02191 {

    public int[] sortJumbled(int[] mapping, int[] nums) {
        return Arrays.stream(nums)
                .mapToObj(num -> new Data(num, getValueForSort(num, mapping)))
                .sorted(Comparator.comparingInt(Data::forSort))
                .mapToInt(Data::original)
                .toArray();
    }

    private static int getValueForSort(int num, int[] mapping) {
        int newNum = 0;
        int multiplier = 1;
        do {
            int digit = num % 10;
            newNum += mapping[digit] * multiplier;
            multiplier *= 10;
            num /= 10;
        } while (num > 0);
        return newNum;
    }

    private record Data(int original, int forSort) {
    }

}
```

392. `Solution02192.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.List;

/**
 * 2192. All Ancestors of a Node in a Directed Acyclic Graph
 * https://leetcode.com/problems/all-ancestors-of-a-node-in-a-directed-acyclic-graph/
 * Medium
 */

public class Solution02192 {

    public List<List<Integer>> getAncestors(int n, int[][] edges) {
        List<Integer>[] adjacencyList = new List[n];
        List<List<Integer>> result = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            adjacencyList[i] = new ArrayList<>();
            result.add(new ArrayList<>());
        }

        for (int[] edge : edges) {
            int from = edge[0];
            int to = edge[1];
            adjacencyList[from].add(to);
        }

        for (int currentIndex = 0; currentIndex < n; currentIndex++) {
            addRecursivelyAncestors(currentIndex, adjacencyList, currentIndex, result);
        }
        return result;
    }

    private static void addRecursivelyAncestors(int parentIndex, List<Integer>[]
adjacencyList, int currentIndex, List<List<Integer>> result) {
        List<Integer> currentResultLine = adjacencyList[currentIndex];
        for (int child : currentResultLine) {
            // add parent to the child's ancestors if it's not already there
            final List<Integer> childResultLine = result.get(child);
            if (childResultLine.isEmpty() ||
                childResultLine.getLast() != parentIndex) {
                childResultLine.add(parentIndex);
                addRecursivelyAncestors(parentIndex, adjacencyList, child, result);
            }
        }
    }

}
```

393. `Solution02196.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 2196. Create Binary Tree From Descriptions
 * <a href="https://leetcode.com/problems/create-binary-tree-from-descriptions">Web
link</a> |
 * Medium
 */

public class Solution02196 {

    public TreeNode createBinaryTree(int[][] descriptions) {
        Map<Integer, TreeNode> nodes = new HashMap<>();
        Set<Integer> children = new HashSet<>();

        for (int[] description : descriptions) {
            int parent = description[0];
            int child = description[1];
            children.add(child);
            boolean isLeft = description[2] == 1;

            if (!nodes.containsKey(parent)) {
                nodes.put(parent, new TreeNode(parent));
            }
            if (!nodes.containsKey(child)) {
                nodes.put(child, new TreeNode(child));
            }

            if (isLeft) {
                nodes.get(parent).left = nodes.get(child);
            } else {
                nodes.get(parent).right = nodes.get(child);
            }
        }

        for (int[] description : descriptions) {
            int parent = description[0];
            if (!children.contains(parent)) {
                return nodes.get(parent);
            }
        }

        return null;
    }
```

```
}
```

394. `Solution02220.java`

```
package leetcode;

/**
 * 2220. Minimum Bit Flips to Convert Number
 * <a href="https://leetcode.com/problems/minimum-bit-flips-to-convert-number">Web
link</a> |
 * Easy
 */

public class Solution02220 {

    public int minBitFlips(int start, int goal) {
        return Integer.bitCount(start ^ goal);
    }

}
```

395. `Solution02225.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * https://leetcode.com/problems/find-players-with-zero-or-one-losses/
 * 2225. Find Players With Zero or One Losses
 * Medium
 * You are given an integer array matches where matches[i] = [winneri, loseri] indicates
that the player winneri defeated player loseri in a match.
 * <p>
 * Return a list answer of size 2 where:
 * <p>
 * answer[0] is a list of all players that have not lost any matches.
 * answer[1] is a list of all players that have lost exactly one match.
 * The values in the two lists should be returned in increasing order.
 * <p>
 * Note:
 * <p>
 * You should only consider the players that have played at least one match.
 * The testcases will be generated such that no two matches will have the same outcome.
 * <p>
 * Example 1:
 * <p>
 * Input: matches = [[1,3],[2,3],[3,6],[5,6],[5,7],[4,5],[4,8],[4,9],[10,4],[10,9]]
 * Output: [[1,2,10],[4,5,7,8]]
 * Explanation:
 * Players 1, 2, and 10 have not lost any matches.
 * Players 4, 5, 7, and 8 each have lost one match.
 * Players 3, 6, and 9 each have lost two matches.
 * Thus, answer[0] = [1,2,10] and answer[1] = [4,5,7,8].
 * Example 2:
 * <p>
 * Input: matches = [[2,3],[1,3],[5,4],[6,4]]
 * Output: [[1,2,5,6],[]]
 * Explanation:
 * Players 1, 2, 5, and 6 have not lost any matches.
 * Players 3 and 4 each have lost two matches.
 * Thus, answer[0] = [1,2,5,6] and answer[1] = [].
 * <p>
 * <p>
 * Constraints:
 * <p>
 * 1 <= matches.length <= 105
 * matches[i].length == 2
 * 1 <= winneri, loseri <= 105
 * winneri != loseri
 * All matches[i] are unique.
```

```
 */
public class Solution02225 {

    public List<List<Integer>> findWinners(int[][] matches) {
        Map<Integer, Player> matchesMap = new HashMap<>();
        for (int[] match : matches) {
            // winner should be added to the map if not present
            matchesMap.computeIfAbsent(match[0], key -> new Player());

            Player loser = matchesMap.computeIfAbsent(match[1], key -> new Player());
            loser.losses++;
        }

        List<Integer> zeroLoss = new ArrayList<>();
        List<Integer> oneLoss = new ArrayList<>();
        for (Map.Entry<Integer, Player> entry : matchesMap.entrySet()) {
            Player player = entry.getValue();
            if (player.losses == 0) {
                zeroLoss.add(entry.getKey());
            } else if (player.losses == 1) {
                oneLoss.add(entry.getKey());
            }
        }

        zeroLoss.sort(null);
        oneLoss.sort(null);

        return List.of(zeroLoss, oneLoss);
    }

    private static class Player {
        int losses;
    }

}
```

396. `Solution02257.java`

```
/**
 * 2257. Count Unguarded Cells in the Grid
 * <a href="https://leetcode.com/problems/count-unguarded-cells-in-the-grid">Web link</a>
|
 * Medium
 */

package leetcode;

public class Solution02257 {

    public int countUnguarded(int m, int n, int[][] guards, int[][] walls) {
        char[][] board = new char[m][n];

        for (int[] wall : walls) {
            board[wall[0]][wall[1]] = 'W';
        }
        for (int[] guard : guards) {
            board[guard[0]][guard[1]] = 'G';
        }

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (board[i][j] == 'G') {
                    //RIGHT
                    for (int k = j + 1; k < n; k++) {
                        if (board[i][k] == 'W' || board[i][k] == 'G') {
                            break;
                        }
                        board[i][k] = 'V';
                    }
                    // LEFT
                    for (int k = j - 1; k >= 0; k--) {
                        if (board[i][k] == 'W' || board[i][k] == 'G') {
                            break;
                        }
                        board[i][k] = 'V';
                    }
                    // DOWN
                    for (int k = i + 1; k < m; k++) {
                        if (board[k][j] == 'W' || board[k][j] == 'G') {
                            break;
                        }
                        board[k][j] = 'V';
                    }
                    // UP
                    for (int k = i - 1; k >= 0; k--) {
                        if (board[k][j] == 'W' || board[k][j] == 'G') {
                            break;
                        }
                        board[k][j] = 'V';
```

```
                    }
                }
            }
        }

        int count = 0;
        for (char[] row : board) {
            for (char cell : row) {
                if (cell == 0) {
                    count++;
                }
            }
        }
        return count;
    }

}
```

397. `Solution02270.java`

```
/**
 * 2270. Number of Ways to Split Array
 * <a href="https://leetcode.com/problems/number-of-ways-to-split-array">Web link</a> |
 * Medium
 */

package leetcode;

public class Solution02270 {

    public int waysToSplitArray(int[] nums) {
        long leftSum = 0;
        long rightSum = 0;

        for (int num : nums) {
            rightSum += num;
        }

        int validSplits = 0;
        for (int i = 0; i < nums.length - 1; i++) {
            leftSum += nums[i];
            rightSum -= nums[i];
            if (leftSum >= rightSum) {
                validSplits++;
            }
        }

        return validSplits;
    }

}
```

398. `Solution02275.java`

```
/**
 * 2275. Largest Combination With Bitwise AND Greater Than Zero
 * <a href="https://leetcode.com/problems/largest-combination-with-bitwise-and-greater-
than-zero">Web link</a> |
 * Medium
 */

package leetcode;

public class Solution02275 {

    public int largestCombination(int[] candidates) {
        int maxFreq = 0;
        for (int i = 0; i < 32; i++) {
            int freq = 0;
            for (int candidate : candidates) {
                if ((candidate & (1 << i)) != 0) {
                    freq++;
                }
            }
            maxFreq = Math.max(maxFreq, freq);
        }
        return maxFreq;
    }

}
```

399. `Solution02285.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 2285. Maximum Total Importance of Roads
 * <a href="https://leetcode.com/problems/maximum-total-importance-of-roads">Web link</a>
|
 * Medium
 */

public class Solution02285 {

    public long maximumImportance(int n, int[][] roads) {
        int[] connectionsCount = new int[n];
        for (int[] road : roads) {
            connectionsCount[road[0]]++;
            connectionsCount[road[1]]++;
        }
        long startImportance = 1;
        Arrays.sort(connectionsCount);
        long totalImportance = 0;
        for (int i = 0; i < n; i++) {
            totalImportance += startImportance * connectionsCount[i];
            startImportance++;
        }
        return totalImportance;
    }

}
```

400. `Solution02290.java`

```
/**
 * 2290. Minimum Obstacle Removal to Reach Corner
 * <a href="https://leetcode.com/problems/minimum-obstacle-removal-to-reach-corner">Web
link</a> |
 * Hard
 */

package leetcode;

import java.util.ArrayDeque;
import java.util.Arrays;
import java.util.Deque;

public class Solution02290 {

    public int minimumObstacles(int[][] grid) {
        int rows = grid.length;
        int cols = grid[0].length;

        // Step 1: Initialize visited array to track min cost to reach each cell
        int[][] visited = new int[rows][cols];
        for (int[] row : visited) {
            Arrays.fill(row, Integer.MAX_VALUE);
        }

        // Step 2: Define directions for moving (Right, Down, Left, Up)
        int[][] directions = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

        // Step 3: Use a Deque for BFS with prioritized traversal
        Deque<int[]> queue = new ArrayDeque<>(); // (Row, Col, Cost)

        // Step 4: Start BFS from the top-left corner (0, 0) with an initial cost of 0
        queue.addFirst(new int[]{0, 0, 0});
        visited[0][0] = 0;

        // Step 5: Process cells in the BFS loop
        while (!queue.isEmpty()) {
            int[] curr = queue.pollFirst(); // Pop the cell with the current smallest
cost
            int x = curr[0];
            int y = curr[1];
            int cost = curr[2];

            // Step 6: Explore all neighbors (up, down, left, right)
            for (int[] dir : directions) {
                int nextX = x + dir[0];
                int nextY = y + dir[1];

                // Step 7: Check if the neighbor is within bounds
                if (nextX >= 0 && nextX < rows && nextY >= 0 && nextY < cols) {
                    int nextCost = cost + grid[nextX][nextY]; // Add cost for obstacle
```

```
(1) or free cell (0)

                    // Step 8: Update the visited array if a cheaper path is found
                    if (nextCost < visited[nextX][nextY]) {
                        visited[nextX][nextY] = nextCost;

                        // Step 9: Prioritize open cells (0) by adding to the front of
the queue
                        if (grid[nextX][nextY] == 0) {
                            queue.addFirst(new int[]{nextX, nextY, nextCost});
                        } else // Process obstacle cells (1) later by adding to the end
of the queue
                        {
                            queue.addLast(new int[]{nextX, nextY, nextCost});
                        }
                    }
                }
            }
        }

        // Step 10: Return the minimum cost to reach the bottom-right corner
        return visited[rows - 1][cols - 1];
    }

}
```

401. `Solution02326.java`

```
package leetcode;

import leetcode.common.ListNode;

import java.util.Arrays;

/**
 * 2326. Spiral Matrix IV
 * <a href="https://leetcode.com/problems/spiral-matrix-iv">Web link</a> |
 * Medium
 */

public class Solution02326 {

    public int[][] spiralMatrix(int m, int n, ListNode head) {
        int[][] result = new int[m][n];
        for (int i = 0; i < m; i++) {
            Arrays.fill(result[i], -1);
        }
        int[][] directions = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        int[] pos = new int[]{0, 0};
        int dirIndex = 0;
        while (head != null) {
            result[pos[0]][pos[1]] = head.val;
            head = head.next;
            int nextRow = pos[0] + directions[dirIndex][0];
            int nextCol = pos[1] + directions[dirIndex][1];
            if (nextRow < 0 || nextRow >= m ||
                nextCol < 0 || nextCol >= n ||
                result[nextRow][nextCol] != -1) {
                dirIndex = (dirIndex + 1) % 4;
                int[] newDirection = directions[dirIndex];
                nextRow = pos[0] + newDirection[0];
                nextCol = pos[1] + newDirection[1];
            }
            pos[0] = nextRow;
            pos[1] = nextCol;
        }
        return result;
    }

}
```

402. `Solution02331.java`

```
package leetcode;

import leetcode.common.TreeNode;

/**
 * 2331. Evaluate Boolean Binary Tree
 * <a href="https://leetcode.com/problems/evaluate-boolean-binary-tree">Web link</a> |
 * Easy
 */

public class Solution02331 {

    public boolean evaluateTree(TreeNode root) {
        if (root == null) {
            return false;
        }
        return switch (root.val) {
            case 1 -> true;
            case 2 -> evaluateTree(root.left) || evaluateTree(root.right);
            case 3 -> evaluateTree(root.left) && evaluateTree(root.right);
            default -> false;
        };
    }

}
```

403. `Solution02337.java`

```
/**
 * 2337. Move Pieces to Obtain a String
 * <a href="https://leetcode.com/problems/move-pieces-to-obtain-a-string">Web link</a> |
 * Medium
 */

package leetcode;

public class Solution02337 {

    public boolean canChange(String start, String target) {
        int i = 0;
        int j = 0;
        int n = start.length();

        while (i < n || j < n) {
            while (i < n && start.charAt(i) == '_')
                i++;
            while (j < n && target.charAt(j) == '_')
                j++;

            if (i == n || j == n)
                break;
            if (start.charAt(i) != target.charAt(j))
                return false;
            if (start.charAt(i) == 'L' && i < j)
                return false;
            if (start.charAt(i) == 'R' && i > j)
                return false;

            i++;
            j++;
        }

        return i == n && j == n;
    }

}
```

404. `Solution02370.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 2370. Longest Ideal Subsequence
 * <a href="https://leetcode.com/problems/longest-ideal-subsequence">Web link</a> |
 * Medium
 */

public class Solution02370 {

    public int longestIdealString(String s, int k) {
        // Get the length of the input string
        int stringLength = s.length();

        // Initialize a dynamic programming array to keep track of the longest ideal
subsequence for each character
        int[] longestIdealSubsequenceForEachChar = new int[26];

        // Iterate over each character in the input string
        for (int i = 0; i < stringLength; i++) {
            // Convert the current character to its corresponding index in the alphabet
(0-25)
            int currentCharIndex = s.charAt(i) - 'a';

            // Initialize a variable to keep track of the maximum length of the ideal
subsequence found so far
            int maxLengthSoFar = 0;

            // Iterate over each possible character in the alphabet
            for (int j = 0; j < 26; j++) {
                // If the absolute difference between the current character and the
character represented by 'j' is less than or equal to 'k'
                if (Math.abs(j - currentCharIndex) <= k) {
                    // Update the maximum length of the ideal subsequence found so far
                    maxLengthSoFar = Math.max(longestIdealSubsequenceForEachChar[j],
maxLengthSoFar);
                }
            }

            // Update the longest ideal subsequence for the current character
            longestIdealSubsequenceForEachChar[currentCharIndex] =
Math.max(maxLengthSoFar + 1, longestIdealSubsequenceForEachChar[currentCharIndex]);
        }

        // Initialize a variable to keep track of the result
        int longestIdealSubsequence = 0;

        // Iterate over the dynamic programming array to find the maximum length of the
ideal subsequence
```

```
        for (int longestSubsequence : longestIdealSubsequenceForEachChar) {
            longestIdealSubsequence = Math.max(longestSubsequence,
longestIdealSubsequence);
        }

        // Return the maximum length of the ideal subsequence
        return longestIdealSubsequence;
    }

    public int longestIdealStringSimple(String s, int k) {
        int[] dp = new int[26];

        for (char c : s.toCharArray()) {
            int max = 0;
            int idx = c - 'a';
            for (int j = 0; j < 26; j++) {
                if (Math.abs(j - idx) <= k) {
                    max = Math.max(dp[j], max);
                }
            }
            dp[idx] = Math.max(max + 1, dp[idx]);
        }

        return Arrays.stream(dp).max().orElse(0);
    }

}
```

405. `Solution02373.java`

```
package leetcode;

/**
 * 2373. Largest Local Values in a Matrix
 * https://leetcode.com/problems/largest-local-values-in-a-matrix/
 * Easy
 */

public class Solution02373 {
    public int[][] largestLocal(int[][] grid) {
        int size = grid.length;
        final int newSize = size - 2;
        int[][] result = new int[newSize][newSize];

        for (int row = 1; row <= newSize; row++) {
            for (int col = 1; col <= newSize; col++) {
                result[row - 1][col - 1] = getMax3x3(grid, row, col);
            }
        }

        return result;
    }

    private int getMax3x3(int[][] grid, int row, int col) {
        int max = Integer.MIN_VALUE;
        for (int i = row - 1; i <= row + 1; i++) {
            for (int j = col - 1; j <= col + 1; j++) {
                max = Math.max(max, grid[i][j]);
            }
        }
        return max;
    }
}
```

406. `Solution02381.java`

```
/**
 * 2381. Shifting Letters II
 * <a href="https://leetcode.com/problems/shifting-letters-ii">Web link</a> |
 * Medium
 */

package leetcode;

public class Solution02381 {

    public String shiftingLettersTle(String s, int[][] shifts) {
        int n = s.length();
        char[] chars = s.toCharArray();
        int[] charIndexes = new int[n];
        for (int i = 0; i < n; i++) {
            charIndexes[i] = chars[i] - 'a';
        }
        for (int[] shift : shifts) {
            int start = shift[0];
            int end = shift[1];
            int shiftDirection = shift[2];
            for (int i = start; i <= end; i++) {
                if (shiftDirection == 0) {
                    charIndexes[i] = (charIndexes[i] + 25) % 26;
                } else {
                    charIndexes[i] = (charIndexes[i] + 1) % 26;
                }
            }
        }
        for (int i = 0; i < n; i++) {
            chars[i] = (char) ('a' + charIndexes[i]);
        }
        return new String(chars);
    }

    public String shiftingLetters(String s, int[][] shifts) {
        int n = s.length();
        int[] shiftsAggregated = new int[n];
        for (int[] shift : shifts) {
            int start = shift[0];
            int end = shift[1];
            int shiftDirection = shift[2];
            if (shiftDirection == 1) {
                shiftsAggregated[start] += 1;
                if (end + 1 < n) {
                    shiftsAggregated[end + 1] -= 1;
                }
            } else {
                shiftsAggregated[start] -= 1;
                if (end + 1 < n) {
                    shiftsAggregated[end + 1] += 1;
```

```
                }
            }
        }
        for (int i = 1; i < n; ++i) {
            shiftsAggregated[i] += shiftsAggregated[i - 1];
        }
        char[] chars = s.toCharArray();
        for (int i = 0; i < n; ++i) {
            int shift = (shiftsAggregated[i] % 26 + 26) % 26;
            int ch = chars[i] - 'a';
            ch = (ch + shift) % 26;
            chars[i] = (char) ('a' + ch);
        }
        return new String(chars);
    }

}
```

407. `Solution02385.java`

```
package leetcode;

import leetcode.common.TreeNode;

import java.util.*;

/**
 * https://leetcode.com/problems/amount-of-time-for-binary-tree-to-be-infected/
 * 2385. Amount of Time for Binary Tree to Be Infected
 * Medium
 * You are given the root of a binary tree with unique values, and an integer start. At
minute 0, an infection starts from the node with value start.
 * <p>
 * Each minute, a node becomes infected if:
 * <p>
 * The node is currently uninfected.
 * The node is adjacent to an infected node.
 * Return the number of minutes needed for the entire tree to be infected.
 * <p>
 * Example 1:
 * Input: root = [1,5,3,null,4,10,6,9,2], start = 3
 * Output: 4
 * Explanation: The following nodes are infected during:
 * - Minute 0: Node 3
 * - Minute 1: Nodes 1, 10 and 6
 * - Minute 2: Node 5
 * - Minute 3: Node 4
 * - Minute 4: Nodes 9 and 2
 * It takes 4 minutes for the whole tree to be infected so we return 4.
 * Example 2:
 * Input: root = [1], start = 1
 * Output: 0
 * Explanation: At minute 0, the only node in the tree is infected so we return 0.
 * <p>
 * <p>
 * Constraints:
 * <p>
 * The number of nodes in the tree is in the range [1, 105].
 * 1 <= Node.val <= 105
 * Each node has a unique value.
 * A node with a value of start exists in the tree.
 */
public class Solution02385 {

    // Time O(n)
    // Space O(n)
    public int amountOfTimeBfs(TreeNode root, int start) {
        Map<Integer, Set<Integer>> map = new HashMap<>();
        convert(root, 0, map);
        Queue<Integer> queue = new LinkedList<>();
        queue.add(start);
```

```
        int minute = 0;
        Set<Integer> visited = new HashSet<>();
        visited.add(start);

        while (!queue.isEmpty()) {
            int levelSize = queue.size();
            while (levelSize > 0) {
                int current = queue.poll();
                for (int num : map.get(current)) {
                    if (!visited.contains(num)) {
                        visited.add(num);
                        queue.add(num);
                    }
                }
                levelSize--;
            }
            minute++;
        }
        return minute - 1;
    }

    private void convert(TreeNode current, int parent, Map<Integer, Set<Integer>> map) {
        if (current == null) {
            return;
        }
        if (!map.containsKey(current.val)) {
            map.put(current.val, new HashSet<>());
        }
        Set<Integer> adjacentList = map.get(current.val);
        if (parent != 0) {
            adjacentList.add(parent);
        }
        if (current.left != null) {
            adjacentList.add(current.left.val);
        }
        if (current.right != null) {
            adjacentList.add(current.right.val);
        }
        convert(current.left, current.val, map);
        convert(current.right, current.val, map);
    }

    public int amountOfTimeDfs(TreeNode root, int start) {
        int[] maxDistance = new int[]{0};
        traverse(root, start, maxDistance);
        return maxDistance[0];
    }

    private int traverse(TreeNode root, int start, int[] maxDistance) {
        int depth = 0;
        if (root == null) {
            return depth;
        }
```

```
        int leftDepth = traverse(root.left, start, maxDistance);
        int rightDepth = traverse(root.right, start, maxDistance);

        if (root.val == start) {
            maxDistance[0] = Math.max(leftDepth, rightDepth);
            depth = -1;
        } else if (leftDepth >= 0 && rightDepth >= 0) {
            depth = Math.max(leftDepth, rightDepth) + 1;
        } else {
            int distance = Math.abs(leftDepth) + Math.abs(rightDepth);
            maxDistance[0] = Math.max(maxDistance[0], distance);
            depth = Math.min(leftDepth, rightDepth) - 1;
        }

        return depth;
    }
}
```

408. `Solution02392.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.List;

/**
 * 2392. Build a Matrix With Conditions
 * <a href="https://leetcode.com/problems/build-a-matrix-with-conditions">Web link</a> |
 * Hard
 */

public class Solution02392 {

    public int[][] buildMatrix(int k, int[][] rowConditions, int[][] colConditions) {
        List<Integer> sortedRows = topoSort(rowConditions, k);
        List<Integer> sortedCols = topoSort(colConditions, k);

        if (sortedRows.isEmpty() || sortedCols.isEmpty()) {
            return new int[0][0];
        }

        int[][] result = new int[k][k];
        for (int row = 0; row < k; row++) {
            for (int col = 0; col < k; col++) {
                if (sortedRows.get(row).equals(sortedCols.get(col))) {
                    result[row][col] = sortedRows.get(row);
                }
            }
        }
        return result;
    }

    private List<Integer> topoSort(int[][] edges, int k) {
        List<List<Integer>> adjacencyList = new ArrayList<>();
        for (int i = 0; i <= k; i++) {
            adjacencyList.add(new ArrayList<>());
        }
        for (int[] edge : edges) {
            adjacencyList.get(edge[0]).add(edge[1]);
        }
        List<Integer> sorted = new ArrayList<>();
        int[] visited = new int[k + 1];
        boolean[] hasCycle = {false};
        for (int i = 1; i <= k; i++) {
            if (visited[i] == 0) {
                dfs(i, adjacencyList, visited, sorted, hasCycle);
                if (hasCycle[0]) {
                    return List.of();
                }
            }
        }
```

```
        return sorted.reversed();
    }

    private void dfs(int node, List<List<Integer>> adjacencyList, int[] visited,
List<Integer> sorted, boolean[] hasCycle) {
        visited[node] = 1;
        for (int neighbor : adjacencyList.get(node)) {
            if (visited[neighbor] == 0) {
                dfs(neighbor, adjacencyList, visited, sorted, hasCycle);
            } else if (visited[neighbor] == 1) {
                hasCycle[0] = true;
                return;
            }
        }
        visited[node] = 2;
        sorted.add(node);
    }

}
```

409. `Solution02402.java`

```
package leetcode;

import java.util.*;

/**
 * 2402. Meeting Rooms III
 * https://leetcode.com/problems/meeting-rooms-iii/
 * Hard
 */

public class Solution02402 {

    // Time: O(n * log(n))
    // Space: O(n)
    public int mostBooked(int n, int[][] meetings) {
        Arrays.sort(meetings, Comparator.comparingInt(meeting -> meeting[0]));

        PriorityQueue<MeetingData> currentMeetingData = new PriorityQueue<>
(Comparator.comparingLong((MeetingData meetingData) ->
meetingData.end).thenComparingInt(meetingData -> meetingData.index));

        PriorityQueue<Integer> availableRooms = new PriorityQueue<>();
        for (int i = 0; i < n; ++i) {
            availableRooms.offer(i);
        }

        Map<Integer, Integer> roomFrequency = new HashMap<>();

        int maxFrequencyRoom = 0;

        for (int[] meeting : meetings) {
            int meetingStart = meeting[0];
            int meetingEnd = meeting[1];

            while (!currentMeetingData.isEmpty() && currentMeetingData.peek().end <=
meetingStart) {
                availableRooms.offer(currentMeetingData.poll().index);
            }

            int roomIndex;
            if (availableRooms.isEmpty()) {
                MeetingData meetingData = currentMeetingData.poll();
                roomIndex = meetingData.index;
                int duration = meetingEnd - meetingStart;
                currentMeetingData.offer(new MeetingData(meetingData.end + duration,
roomIndex));
            } else {
                roomIndex = availableRooms.poll();
                currentMeetingData.offer(new MeetingData(meetingEnd, roomIndex));
            }
```

```
            roomFrequency.merge(roomIndex, 1, Integer::sum);
            int currentFrequency = roomFrequency.get(roomIndex);
            int maxFrequency = roomFrequency.getOrDefault(maxFrequencyRoom,
Integer.MIN_VALUE);
            if (currentFrequency > maxFrequency) {
                maxFrequencyRoom = roomIndex;
            } else if (currentFrequency == maxFrequency) {
                maxFrequencyRoom = Math.min(roomIndex, maxFrequencyRoom);
            }
        }

        return maxFrequencyRoom;
    }

    private static class MeetingData {
        long end;
        int index;

        public MeetingData(long end, int index) {
            this.end = end;
            this.index = index;
        }
    }

    public int mostBooked2(int n, int[][] meetings) {
        // Sort meetings by start times
        Arrays.sort(meetings, Comparator.comparingInt(meeting -> meeting[0]));

        // Queue to keep track of available room numbers
        PriorityQueue<Integer> availableRoomsQueue = new PriorityQueue<>();

        // Queue to store MeetingData objects for each meeting, sorted by end times
        // If meeting's end time is the same, sort by smaller room number
        PriorityQueue<MeetingData> ongoingMeetingsQueue = new PriorityQueue<>
(Comparator.comparingLong((MeetingData meetingData) ->
meetingData.end).thenComparingInt(meetingData -> meetingData.index));

        // Array to track usage count of each room
        int[] roomUsageCount = new int[n];

        // Final result: room number with maximum usage - we will keep it updating
        int roomWithMaxUsage = 0;

        // Initially, all rooms are available
        for (int i = 0; i < n; i++) {
            availableRoomsQueue.offer(i);
        }

        // Processing: Iterate over each meeting interval
        for (int[] currentMeeting : meetings) {
            // Check if any room is available before scheduling next meeting
            while (!ongoingMeetingsQueue.isEmpty() && ongoingMeetingsQueue.peek().end <=
currentMeeting[0]) {
                availableRoomsQueue.offer(ongoingMeetingsQueue.poll().index);
```

```
            }

            // Initialize 'latestPossibleStartTime' with currentMeeting[0]. We will see
if there is a delay
            long latestPossibleStartTime = currentMeeting[0];

            // Check if there is delay due to non-availability of rooms
            if (availableRoomsQueue.isEmpty()) {
                // Pick the first ending meeting (or one of the such meetings with
smaller room number)
                MeetingData firstEndingMeeting = ongoingMeetingsQueue.poll();
                latestPossibleStartTime = firstEndingMeeting.end;

                // Room number released by firstEndingMeeting
                availableRoomsQueue.offer(firstEndingMeeting.index);
            }

            // Now we have a room available - schedule the next meeting
            int assignedRoom = availableRoomsQueue.poll();
            roomUsageCount[assignedRoom]++;
            int meetingDuration = currentMeeting[1] - currentMeeting[0];
            ongoingMeetingsQueue.offer(new MeetingData(meetingDuration +
latestPossibleStartTime, assignedRoom));

            // Update 'roomWithMaxUsage' with room number that holds maximum meetings
            if (roomUsageCount[assignedRoom] > roomUsageCount[roomWithMaxUsage]) {
                roomWithMaxUsage = assignedRoom;
            } else if (roomUsageCount[assignedRoom] == roomUsageCount[roomWithMaxUsage])
{
                roomWithMaxUsage = Math.min(roomWithMaxUsage, assignedRoom);
            }
        }
        return roomWithMaxUsage;
    }

}
```

410. `Solution02406.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 2406. Divide Intervals Into Minimum Number of Groups
 * <a href="https://leetcode.com/problems/divide-intervals-into-minimum-number-of-
groups">Web link</a> |
 * Medium
 */

public class Solution02406 {

    public int minGroups(int[][] intervals) {
        int n = intervals.length;
        int[] startTimes = new int[n];
        int[] endTimes = new int[n];

        for (int i = 0; i < n; i++) {
            startTimes[i] = intervals[i][0];
            endTimes[i] = intervals[i][1];
        }

        Arrays.sort(startTimes);
        Arrays.sort(endTimes);

        int endPtr = 0;
        int groupCount = 0;

        for (int start : startTimes) {
            if (start > endTimes[endPtr]) {
                endPtr++;
            } else {
                groupCount++;
            }
        }

        return groupCount;
    }

}
```

411. `Solution02415.java`

```
/**
 * 2415. Reverse Odd Levels of Binary Tree
 * <a href="https://leetcode.com/problems/reverse-odd-levels-of-binary-tree">Web link</a>
|
 * Medium
 */

package leetcode;

import leetcode.common.TreeNode;

public class Solution02415 {

    public TreeNode reverseOddLevels(TreeNode root) {
        levelTraverse(root.left, root.right, 0);
        return root;
    }

    private void levelTraverse(TreeNode node1, TreeNode node2, int level) {
        if (node1 == null || node2 == null) {
            return;
        }
        if (level % 2 == 0) {
            int val = node1.val;
            node1.val = node2.val;
            node2.val = val;
        }
        final int newLevel = level + 1;
        levelTraverse(node1.left, node2.right, newLevel);
        levelTraverse(node1.right, node2.left, newLevel);
    }

}
```

412. `Solution02416.java`

```
package leetcode;

/**
 * 2416. Sum of Prefix Scores of Strings
 * <a href="https://leetcode.com/problems/sum-of-prefix-scores-of-strings">Web link</a> |
 * Hard
 */

public class Solution02416 {

    public int[] sumPrefixScores(String[] words) {
        TrieNode root = new TrieNode();
        for (var word : words) {
            root.insert(word);
        }
        int size = words.length;
        int[] answer = new int[size];
        for (int indx = 0; indx < size; indx++) {
            answer[indx] = root.findScore(words[indx]);
        }
        return answer;
    }

    private static class TrieNode {
        private final TrieNode[] children = new TrieNode[26];
        private int nodeFreq = 0;

        void insert(String key) {
            TrieNode curr = this;
            for (var chr : key.toCharArray()) {
                int indx = chr - 'a';
                if (curr.children[indx] == null) {
                    curr.children[indx] = new TrieNode();
                }
                curr.children[indx].nodeFreq++;
                curr = curr.children[indx];
            }
        }

        int findScore(String key) {
            TrieNode curr = this;
            int score = 0;
            for (var chr : key.toCharArray()) {
                int indx = chr - 'a';
                score += curr.children[indx].nodeFreq;
                curr = curr.children[indx];
            }
            return score;
        }
    }
```

```
}
```

413. `Solution02418.java`

```
package leetcode;

import javafx.util.Pair;

import java.util.ArrayList;
import java.util.List;

/**
 * 2418. Sort the People
 * <a href="https://leetcode.com/problems/sort-the-people">Web link</a> |
 * Easy
 */

public class Solution02418 {

    public String[] sortPeople(String[] names, int[] heights) {
        int n = names.length;
        List<Pair<String, Integer>> people = new ArrayList<>(n);
        for (int i = 0; i < n; i++) {
            people.add(new Pair<>(names[i], heights[i]));
        }
        return people.stream()
                .sorted((a, b) -> b.getValue() - a.getValue())
                .map(Pair::getKey)
                .toArray(String[]::new);
    }

}
```

414. `Solution02419.java`

```
package leetcode;

/**
 * 2419. Longest Subarray With Maximum Bitwise AND
 * <a href="https://leetcode.com/problems/longest-subarray-with-maximum-bitwise-and">Web
link</a> |
 * Medium
 */

public class Solution02419 {

    public int longestSubarray(int[] nums) {
        int length = nums.length;
        int maxElement = nums[0];
        int currentStreak = 1;
        int longestStreak = 1;

        for (int i = 1; i < length; i++) {
            int value = nums[i];
            if (value == maxElement) {
                currentStreak++;
                longestStreak = Math.max(longestStreak, currentStreak);
            } else if (value > maxElement) {
                maxElement = value;
                currentStreak = 1;
                longestStreak = 1;
            } else {
                currentStreak = 0;
            }
        }

        return longestStreak;
    }

}
```

415. `Solution02425.java`

```
/**
 * 2425. Bitwise XOR of All Pairings
 * <a href="https://leetcode.com/problems/bitwise-xor-of-all-pairings">Web link</a> |
 * Medium
 */

package leetcode;

public class Solution02425 {

    public int xorAllNums(int[] nums1, int[] nums2) {
        int x = 0;
        int y = 0;
        for (int a : nums1) {
            x ^= a;
        }
        for (int b : nums2) {
            y ^= b;
        }
        return (nums1.length % 2 * y) ^ (nums2.length % 2 * x);
    }

}
```

416. `Solution02429.java`

```
/**
 * 2429. Minimize XOR
 * <a href="https://leetcode.com/problems/minimize-xor">Web link</a> |
 * Medium
 */

package leetcode;

public class Solution02429 {

    public int minimizeXor(int num1, int num2) {
        int countBitsNum = Integer.bitCount(num2);
        int result = 0;

        for (int i = 31; i >= 0 && countBitsNum > 0; i--) {
            if ((num1 & (1 << i)) != 0) {
                result |= (1 << i);
                countBitsNum--;
            }
        }

        for (int i = 0; i < 32 && countBitsNum > 0; i++) {
            if ((result & (1 << i)) == 0) {
                result |= (1 << i);
                countBitsNum--;
            }
        }

        return result;
    }

}
```

417. `Solution02441.java`

```
package leetcode;

import java.util.HashSet;
import java.util.Iterator;
import java.util.Set;
import java.util.TreeSet;

/**
 * 2441. Largest Positive Integer That Exists With Its Negative
 * https://leetcode.com/problems/largest-positive-integer-that-exists-with-its-negative/
 * Medium
 */

public class Solution02441 {

    public int findMaxKFirstApproach(int[] nums) {
        TreeSet<Integer> positiveSet = new TreeSet<>();
        TreeSet<Integer> negativeSet = new TreeSet<>();
        for (int num : nums) {
            if (num > 0) {
                positiveSet.add(num);
            } else {
                negativeSet.add(-num);
            }
        }
        if (positiveSet.isEmpty() || negativeSet.isEmpty()) {
            return -1;
        }
        Iterator<Integer> positiveIterator = positiveSet.descendingIterator();
        Iterator<Integer> negativeIterator = negativeSet.descendingIterator();
        int currentNegativeValue = negativeIterator.next();
        while (positiveIterator.hasNext()) {
            int currentPositiveValue = positiveIterator.next();
            while (currentNegativeValue > currentPositiveValue &&
negativeIterator.hasNext()) {
                currentNegativeValue = negativeIterator.next();
            }
            if (currentNegativeValue == currentPositiveValue) {
                return currentPositiveValue;
            }
        }
        return -1;
    }

    public int findMaxK(int[] nums) {
        Set<Integer> negativeSet = new HashSet<>();
        int max = -1;
        for (int num : nums) {
            if (num < 0) {
                negativeSet.add(num);
            }
```

```
        }
        for (int num : nums) {
            if (num > 0 && negativeSet.contains(-num)) {
                max = Math.max(max, num);
            }
        }
        return max;
    }

}
```

418. `Solution02444.java`

```
package leetcode;

/**
 * 2444. Count Subarrays With Fixed Bounds
 * <a href="https://leetcode.com/problems/count-subarrays-with-fixed-bounds">Web link</a>
|
 * Hard
 */

public class Solution02444 {

    public long countSubarrays(int[] nums, int minK, int maxK) {
        long result = 0;
        int invalidIndex = -1;
        int minIndex = -1;
        int maxIndex = -1;

        for (int i = 0; i < nums.length; i++) {
            int currentNum = nums[i];

            if (currentNum < minK || currentNum > maxK) {
                invalidIndex = i;
            }

            if (currentNum == minK) {
                minIndex = i;
            }

            if (currentNum == maxK) {
                maxIndex = i;
            }

            int left = Math.min(minIndex, maxIndex);
            if (left > invalidIndex) {
                result += left - invalidIndex;
            }
        }

        return result;
    }

}
```

419. `Solution02458.java`

```
/**
 * 2458. Height of Binary Tree After Subtree Removal Queries
 * <a href="https://leetcode.com/problems/height-of-binary-tree-after-subtree-removal-
queries">Web link</a> |
 * Hard
 */

package leetcode;

import leetcode.common.TreeNode;

public class Solution02458 {

    public int[] treeQueries(TreeNode root, int[] queries) {
        int[] preOrderArr = new int[100001];
        int[] postOrderArr = new int[100001];
        int[] maxH = {0};

        preOrder(root, 0, preOrderArr, maxH);
        maxH[0] = 0;
        postOrder(root, 0, postOrderArr, maxH);

        int[] res = new int[queries.length];
        for (int i = 0; i < res.length; i++) {
            res[i] = Math.max(preOrderArr[queries[i]], postOrderArr[queries[i]]);
        }
        return res;
    }

    private void preOrder(TreeNode root, int height, int[] preOrderArr, int[] maxH) {
        if (root == null) {
            return;
        }

        preOrderArr[root.val] = maxH[0];
        maxH[0] = Math.max(maxH[0], height);
        preOrder(root.left, height + 1, preOrderArr, maxH);
        preOrder(root.right, height + 1, preOrderArr, maxH);
    }

    private void postOrder(TreeNode root, int height, int[] postOrderArr, int[] maxH) {
        if (root == null) {
            return;
        }

        postOrderArr[root.val] = maxH[0];
        maxH[0] = Math.max(maxH[0], height);
        postOrder(root.right, height + 1, postOrderArr, maxH);
        postOrder(root.left, height + 1, postOrderArr, maxH);
    }
```

```
}
```

420. `Solution02461.java`

```
/**
 * 2461. Maximum Sum of Distinct Subarrays With Length K
 * <a href="https://leetcode.com/problems/maximum-sum-of-distinct-subarrays-with-length-
k">Web link</a> |
 * Medium
 */

package leetcode;

import java.util.HashMap;
import java.util.Map;

public class Solution02461 {

    public long maximumSubarraySum(int[] nums, int k) {
        long result = 0L;
        int n = nums.length;
        if (n < k) {
            return result;
        }
        int left = 0;
        int right = 0;
        Map<Integer, Integer> map = new HashMap<>();
        long currentSum = 0L;
        while (right < n) {
            currentSum += nums[right];
            map.put(nums[right], map.getOrDefault(nums[right], 0) + 1);
            right++;
            final int diff = right - left;
            if (diff == k) {
                if (map.size() == k) {
                    result = Math.max(result, currentSum);
                }
                int newValue = map.getOrDefault(nums[left], 0) - 1;
                if (newValue > 0) {
                    map.put(nums[left], newValue);
                } else {
                    map.remove(nums[left]);
                }
                currentSum -= nums[left];
                left++;
            }
        }
        return result;
    }

}
```

421. `Solution02463.java`

```
/**
 * 2463. Minimum Total Distance Traveled
 * <a href="https://leetcode.com/problems/minimum-total-distance-traveled">Web link</a> |
 * Hard
 */

package leetcode;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;

public class Solution02463 {

    public long minimumTotalDistance(List<Integer> robot, int[][] factory) {
        robot.sort(null);
        Arrays.sort(factory, Comparator.comparingInt(a -> a[0]));

        List<Integer> factoryPos = new ArrayList<>();
        for (int[] position : factory) {
            for (int i = 0; i < position[1]; i++) {
                factoryPos.add(position[0]);
            }
        }

        int m = robot.size();
        int n = factoryPos.size();
        long[][] dp = new long[m + 1][n + 1];

        for (int i = 0; i < m; i++) {
            dp[i][n] = 1_000_000_000_000L;
        }

        for (int i = m - 1; i >= 0; i--) {
            for (int j = n - 1; j >= 0; j--) {
                long pick = Math.abs(robot.get(i) - factoryPos.get(j)) + dp[i + 1][j +
1];
                long notPick = dp[i][j + 1];
                dp[i][j] = Math.min(pick, notPick);
            }
        }
        return dp[0][0];
    }

    public long minimumTotalDistance2(List<Integer> robot, int[][] factory) {
        robot.sort(null);
        Arrays.sort(factory, Comparator.comparingInt(a -> a[0]));

        List<Integer> factoryPos = new ArrayList<>();
        for (int[] position : factory) {
```

```
            for (int i = 0; i < position[1]; i++) {
                factoryPos.add(position[0]);
            }
        }

        int m = robot.size();
        int n = factoryPos.size();
        long[][] dp = new long[m + 1][n + 1];

        for (int i = 0; i <= m; i++) {
            for (int j = 0; j <= n; j++) {
                if (i == 0) {
                    dp[i][j] = 0L;
                } else if (j == 0 || j < i) {
                    dp[i][j] = Long.MAX_VALUE;
                } else {
                    dp[i][j] = Math.min(dp[i][j - 1], dp[i - 1][j - 1] +
Math.abs(robot.get(i - 1) - factoryPos.get(j - 1)));
                }
            }
        }
        return dp[m][n];
    }

}
```

422. `Solution02466.java`

```
/**
 * 2466. Count Ways To Build Good Strings
 * <a href="https://leetcode.com/problems/count-ways-to-build-good-strings">Web link</a>
|
 * Medium
 */

package leetcode;

public class Solution02466 {

    public int countGoodStrings(int low, int high, int zero, int one) {
        final long MOD = 1_000_000_007L;
        long[] dp = new long[high + 1];
        dp[0] = 1;

        for (int length = 1; length <= high; length++) {
            if (length >= zero) dp[length] = (dp[length] + dp[length - zero]) % MOD;
            if (length >= one) dp[length] = (dp[length] + dp[length - one]) % MOD;
        }

        long result = 0;
        for (int length = low; length <= high; length++) {
            result = (result + dp[length]) % MOD;
        }

        return (int) result;
    }

}
```

423. `Solution02471.java`

```
/**
 * 2471. Minimum Number of Operations to Sort a Binary Tree by Level
 * <a href="https://leetcode.com/problems/minimum-number-of-operations-to-sort-a-binary-tree-by-level">Web link</a> |
 * Medium
 */

package leetcode;

import leetcode.common.TreeNode;

import java.util.*;

public class Solution02471 {

    public int minimumOperations(TreeNode root) {
        Queue<TreeNode> q = new ArrayDeque<>();
        q.add(root);
        int count = 0;

        while (!q.isEmpty()) {
            int size = q.size();
            int[] arr = new int[size];
            for (int i = 0; i < size; i++) {
                TreeNode curr = q.poll();
                arr[i] = curr.val;
                if (curr.left != null) q.add(curr.left);
                if (curr.right != null) q.add(curr.right);
            }
            count += minSwaps(arr);
        }
        return count;
    }

    private int minSwaps(int[] arr) {
        int n = arr.length;
        int[] temp = arr.clone();
        Arrays.sort(temp);
        boolean[] visited = new boolean[n];
        int swaps = 0;
        Map<Integer, Integer> indexMap = createIndexMap(arr);

        for (int i = 0; i < n; i++) {
            if (visited[i] || arr[i] == temp[i]) continue;
            int cycleSize = 0;
            int j = i;
            while (!visited[j]) {
                visited[j] = true;
                j = indexMap.get(temp[j]);
                cycleSize++;
            }
```

```
            if (cycleSize > 0) swaps += (cycleSize - 1);
        }
        return swaps;
    }

    private Map<Integer, Integer> createIndexMap(int[] arr) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < arr.length; i++) {
            map.put(arr[i], i);
        }
        return map;
    }

}
```

424. `Solution02485.java`

```
package leetcode;

/**
 * 2485. Find the Pivot Integer
 * <a href="https://leetcode.com/problems/find-the-pivot-integer">Web link</a> |
 * Easy
 */

public class Solution02485 {

    // Time: O(n)
    // Space: O(1)
    public int pivotInteger(int n) {
        int left = 1;
        int right = n;
        int sumLeft = 1;
        int sumRight = n;
        while (left < right) {
            if (sumLeft < sumRight) {
                left++;
                sumLeft += left;
            } else {
                right--;
                sumRight += right;
            }
        }
        if (sumLeft == sumRight) {
            return left;
        } else {
            return -1;
        }
    }

    public int pivotIntegerImproved(int n) {
        int sum = (n * (n + 1)) / 2;
        double x = Math.sqrt(sum);
        if (x - Math.ceil(x) == 0) {
            return (int) x;
        } else {
            return -1;
        }
    }

}
```

425. `Solution02486.java`

```
package leetcode;

/**
 * 2486. Append Characters to String to Make Subsequence
 * <a href="https://leetcode.com/problems/append-characters-to-string-to-make-
subsequence">Web link</a> |
 * Medium
 */

public class Solution02486 {

    public int appendCharacters(String s, String t) {
        int firstMatch = s.indexOf(t.charAt(0));
        final int tLen = t.length();
        if (firstMatch < 0) {
            return tLen;
        }
        int posT = 1;
        int posS = firstMatch + 1;
        int matched = 1;
        final int sLen = s.length();
        while (posS < sLen && posT < tLen) {
            if (s.charAt(posS) == t.charAt(posT)) {
                matched++;
                posT++;
            }
            posS++;
        }
        return tLen - matched;
    }

}
```

426. `Solution02487.java`

```
package leetcode;

import leetcode.common.ListNode;

/**
 * 2487. Remove Nodes From Linked List
 * <a href="https://leetcode.com/problems/remove-nodes-from-linked-list">Web link</a> |
 * Medium
 */

public class Solution02487 {

    public ListNode removeNodes(ListNode head) {
        if (head == null) {
            return null;
        }
        ListNode nextBigger = removeNodes(head.next);
        head.next = nextBigger;
        if (nextBigger == null || head.val >= nextBigger.val) {
            return head;
        }
        return nextBigger;
    }

}
```

427. `Solution02490.java`

```
/**
 * 2490. Circular Sentence
 * <a href="https://leetcode.com/problems/circular-sentence">Web link</a> |
 * Easy
 */

package leetcode;

public class Solution02490 {

    public boolean isCircularSentence(String sentence) {
        int strLen = sentence.length();
        if (sentence.charAt(0) != sentence.charAt(strLen - 1)) {
            return false;
        }
        char endChar = '-';
        boolean previousCharIsSpace = false;
        for (int i = 0; i < strLen; i++) {
            if (sentence.charAt(i) == ' ') {
                previousCharIsSpace = true;
                continue;
            }
            if (previousCharIsSpace) {
                if (endChar != sentence.charAt(i)) {
                    return false;
                }
                previousCharIsSpace = false;
            }
            endChar = sentence.charAt(i);
        }
        return true;
    }

}
```

428. `Solution02491.java`

```
package leetcode;

/**
 * 2491. Divide Players Into Teams of Equal Skill
 * <a href="https://leetcode.com/problems/divide-players-into-teams-of-equal-skill">Web
link</a> |
 * Medium
 */

public class Solution02491 {

    public long dividePlayers(int[] skill) {
        int[] skillFreq = new int[1001];

        int total = 0;
        for (int x : skill) {
            skillFreq[x]++;
            total += x;
        }
        if (total % (skill.length / 2) != 0) {
            return -1;
        }

        int perTeamSkill = total / (skill.length / 2);
        long chemistry = 0;

        for (int s : skill) {
            int partnerVal = perTeamSkill - s;
            if (skillFreq[partnerVal] == 0) {
                return -1;
            }
            chemistry += (long) s * partnerVal;
            skillFreq[partnerVal]--;
        }

        return chemistry / 2;
    }

}
```

429. `Solution02501.java`

```
/**
 * 2501. Longest Square Streak in an Array
 * <a href="https://leetcode.com/problems/longest-square-streak-in-an-array">Web link</a>
|
 * Medium
 */

package leetcode;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

public class Solution02501 {

    public int longestSquareStreak(int[] nums) {
        Arrays.sort(nums);

        Map<Integer, Integer> mp = new HashMap<>();
        int result = -1;

        int previous = -1;
        for (int num : nums) {
            if (num == previous) {
                continue;
            }
            int sqrt = (int) Math.sqrt(num);

            if (sqrt * sqrt == num && mp.containsKey(sqrt)) {
                mp.put(num, mp.get(sqrt) + 1);
                result = Math.max(result, mp.get(num));
            } else {
                mp.put(num, 1);
            }
            previous = num;
        }

        return result;
    }

}
```

430. `Solution02516.java`

```
/**
 * 2516. Take K of Each Character From Left and Right
 * <a href="https://leetcode.com/problems/take-k-of-each-character-from-left-and-
right">Web link</a> |
 * Medium
 */

package leetcode;

public class Solution02516 {

    public int takeCharacters(String s, int k) {
        if (k == 0) {
            return 0;
        }
        int[] count = new int[3];
        for (char ch : s.toCharArray()) {
            count[ch - 'a']++;
        }
        if (invalid(count, k)) {
            return -1;
        }

        int begin = 0;
        int end = 0;
        int max = -1;
        while (end < s.length()) {
            count[s.charAt(end) - 'a']--;
            while (invalid(count, k)) {
                count[s.charAt(begin++) - 'a']++;
            }
            max = Math.max(max, end - begin + 1);
            end++;
        }
        return s.length() - max;
    }

    private boolean invalid(int[] count, int k) {
        for (int i : count) {
            if (i < k) return true;
        }
        return false;
    }

}
```

431. `Solution02530.java`

```
package leetcode;

import java.util.Comparator;
import java.util.PriorityQueue;

/**
 * 2530. Maximal Score After Applying K Operations
 * <a href="https://leetcode.com/problems/maximal-score-after-applying-k-operations">Web
link</a> |
 * Medium
 */

public class Solution02530 {

    public long maxKelements(int[] nums, int k) {
        long result = 0L;
        PriorityQueue<Integer> pq = new PriorityQueue<>(Comparator.reverseOrder());
        for (int num : nums) {
            pq.add(num);
        }
        while (k-- > 0) {
            int max = pq.poll();
            result += max;
            pq.add(Math.ceilDiv(max, 3));
        }
        return result;
    }

}
```

432. `Solution02540.java`

```
package leetcode;

/**
 * 2540. Minimum Common Value
 * https://leetcode.com/problems/minimum-common-value/
 * Easy
 */

public class Solution02540 {

    // Time: O(n)
    // Space: O(1)
    public int getCommon(int[] nums1, int[] nums2) {
        int leftIndex1 = 0;
        int leftIndex2 = 0;
        final int n1 = nums1.length;
        final int n2 = nums2.length;
        while (leftIndex1 < n1 && leftIndex2 < n2) {
            if (nums1[leftIndex1] == nums2[leftIndex2]) {
                return nums1[leftIndex1];
            }
            if (nums1[leftIndex1] < nums2[leftIndex2]) {
                leftIndex1++;
            } else {
                leftIndex2++;
            }
        }
        return -1;
    }

}
```

433. `Solution02554.java`

```
/**
 * 2554. Maximum Number of Integers to Choose From a Range I
 * <a href="https://leetcode.com/problems/maximum-number-of-integers-to-choose-from-a-
range-i">Web link</a> |
 * Medium
 */

package leetcode;

import java.util.HashSet;
import java.util.Set;

public class Solution02554 {

    public int maxCount(int[] banned, int n, int maxSum) {
        Set<Integer> bannedSet = new HashSet<>();
        for (int num : banned) {
            bannedSet.add(num);
        }

        int sum = 0;
        int count = 0;
        for (int i = 1; i <= n; i++) {
            if (bannedSet.contains(i)) {
                continue;
            }
            int sumCandidate = sum + i;
            if (sumCandidate > maxSum) {
                return count;
            }
            sum = sumCandidate;
            count++;
        }

        return count;
    }

}
```

434. `Solution02558.java`

```
/**
 * 2558. Take Gifts From the Richest Pile
 * <a href="https://leetcode.com/problems/take-gifts-from-the-richest-pile">Web link</a>
|
 * Easy
 */

package leetcode;

import java.util.PriorityQueue;

public class Solution02558 {

    public long pickGifts(int[] gifts, int k) {
        long total = 0L;
        PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) -> b - a);
        for (int gift : gifts) {
            maxHeap.add(gift);
            total += gift;
        }
        int remaining = k;
        while (remaining > 0) {
            int max = maxHeap.poll();
            if (max == 1) {
                break;
            }
            int sqrt = (int) Math.sqrt(max);
            maxHeap.add(sqrt);
            total -= (max - sqrt);
            remaining--;
        }
        return total;
    }

}
```

435. `Solution02559.java`

```
/**
 * 2559. Count Vowel Strings in Ranges
 * <a href="https://leetcode.com/problems/count-vowel-strings-in-ranges">Web link</a> |
 * Medium
 */

package leetcode;

public class Solution02559 {

    public int[] vowelStrings(String[] words, int[][] queries) {
        final int wordsLen = words.length;
        int[] countAt = new int[wordsLen + 1];
        for (int i = 0; i < wordsLen; i++) {
            countAt[i + 1] = countAt[i] + (isVowelString(words[i]) ? 1 : 0);
        }
        final int queriesLen = queries.length;
        int[] result = new int[queriesLen];
        for (int i = 0; i < queriesLen; i++) {
            result[i] = countAt[queries[i][1] + 1] - countAt[queries[i][0]];
        }
        return result;
    }

    private boolean isVowelString(String word) {
        final int len = word.length();
        if (len == 0) {
            return false;
        }
        if (!isVowelChar(word.charAt(0))) {
            return false;
        }
        if (len == 1) {
            return true;
        }
        return isVowelChar(word.charAt(len - 1));
    }

    private boolean isVowelChar(char ch) {
        return ch == 'a' ||
                ch == 'e' ||
                ch == 'i' ||
                ch == 'o' ||
                ch == 'u';
    }

}
```

436. `Solution02563.java`

```
/**
 * 2563. Count the Number of Fair Pairs
 * <a href="https://leetcode.com/problems/count-the-number-of-fair-pairs">Web link</a> |
 * Medium
 */

package leetcode;

import java.util.Arrays;

public class Solution02563 {

    public long countFairPairs(int[] nums, int lower, int upper) {
        Arrays.sort(nums);
        long ans = 0;
        for (int i = 0; i < nums.length - 1; i++) {
            int low = lowerBound(nums, i + 1, nums.length, lower - nums[i]);
            int up = upperBound(nums, i + 1, nums.length, upper - nums[i]);
            ans += up - low;
        }
        return ans;
    }

    private int lowerBound(int[] nums, int start, int end, int target) {
        while (start < end) {
            int mid = start + (end - start) / 2;
            if (nums[mid] >= target) end = mid;
            else start = mid + 1;
        }
        return start;
    }

    private int upperBound(int[] nums, int start, int end, int target) {
        while (start < end) {
            int mid = start + (end - start) / 2;
            if (nums[mid] > target) end = mid;
            else start = mid + 1;
        }
        return start;
    }

}
```

437. `Solution02577.java`

```
/**
 * 2577. Minimum Time to Visit a Cell In a Grid
 * <a href="https://leetcode.com/problems/minimum-time-to-visit-a-cell-in-a-grid">Web
link</a> |
 * Medium
 */

package leetcode;

import java.util.Comparator;
import java.util.PriorityQueue;
import java.util.Queue;

public class Solution02577 {

    public int minimumTime(int[][] grid) {
        int m = grid.length;
        int n = grid[0].length;

        // Check if it's impossible to reach the destination from the start
        if (grid[0][1] > 1 && grid[1][0] > 1)
            return -1;

        int[][] time = new int[m + 1][n + 1];
        int[][] vis = new int[m + 1][n + 1];

        for (int i = 0; i <= m; i++) {
            for (int j = 0; j <= n; j++) {
                vis[i][j] = 0;
                time[i][j] = Integer.MAX_VALUE;
            }
        }

        Queue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[0]));
        pq.add(new int[]{0, 0, 0});  // Start from the top-left corner

        int[] x = {0, 0, 1, -1};
        int[] y = {1, -1, 0, 0};

        while (!pq.isEmpty()) {
            int[] p = pq.poll();
            int t = p[0];
            int curx = p[1];
            int cury = p[2];

            // Return time if we reach the bottom-right corner
            if (curx == m - 1 && cury == n - 1) {
                return t;
            }

            if (vis[curx][cury] == 1) continue;
```

```
            vis[curx][cury] = 1;

            for (int i = 0; i < 4; i++) {
                int newx = curx + x[i];
                int newy = cury + y[i];

                if (newx >= 0 && newx < m && newy >= 0 && newy < n) {
                    int needtime = t + 1;

                    // Adjust time based on grid values
                    if (grid[newx][newy] > t + 1) {
                        int diff = grid[newx][newy] - t;
                        if (diff % 2 == 1)
                            needtime = grid[newx][newy];
                        else
                            needtime = grid[newx][newy] + 1;
                    }

                    if (needtime < time[newx][newy]) {
                        time[newx][newy] = needtime;
                        pq.add(new int[]{needtime, newx, newy});
                    }
                }
            }
        }

        return -1;
    }

}
```

438. `Solution02582.java`

```
package leetcode;

/**
 * 2582. Pass the Pillow
 * <a href="https://leetcode.com/problems/pass-the-pillow">Web link</a> |
 * Easy
 */

public class Solution02582 {

    public int passThePillow(int n, int time) {
        int idx = 1;
        int direction = 1;
        for (int currentTime = 1; currentTime <= time; currentTime++) {
            if (idx == n) {
                direction = -1;
            } else if (idx == 1) {
                direction = 1;
            }
            idx += direction;
        }
        return idx;
    }

    public int passThePillowOptimized(int n, int time) {
        int cycle = 2 * (n - 1);
        int effectiveTime = time % cycle;
        if (effectiveTime < n) {
            return effectiveTime + 1;
        } else {
            return n - effectiveTime % n - 1;
        }
    }

}
```

```
    /*

}
    int current = start;
    while (true) {
        if (current == 1) {
            return 1;
        }
        if (current % 2 == 0) {
            current = n - current / 2;
        } else {
            current = n - (current + 1) / 2;
        }
    }
}

     */
```

439. `Solution02583.java`

```
/**
 * 2583. Kth Largest Sum in a Binary Tree
 * <a href="https://leetcode.com/problems/kth-largest-sum-in-a-binary-tree">Web link</a>
|
 * Medium
 */

package leetcode;

import leetcode.common.TreeNode;

import java.util.ArrayDeque;
import java.util.PriorityQueue;
import java.util.Queue;

public class Solution02583 {

    public long kthLargestLevelSum(TreeNode root, int k) {
        PriorityQueue<Long> results = new PriorityQueue<>();
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.add(root);

        while (!queue.isEmpty()) {
            int n = queue.size();
            long sum = 0;

            for (int i = 0; i < n; i++) {
                TreeNode node = queue.poll();
                sum += node.val;

                if (node.left != null) queue.add(node.left);
                if (node.right != null) queue.add(node.right);
            }
            if (results.size() < k) {
                results.offer(sum);
            } else {
                if (sum > results.peek()) {
                    results.poll();
                    results.offer(sum);
                }
            }
        }

        if (k > results.size()) {
            return -1;
        }

        return results.peek();
    }

}
```

440. `Solution02593.java`

```
/**
 * 2593. Find Score of an Array After Marking All Elements
 * <a href="https://leetcode.com/problems/find-score-of-an-array-after-marking-all-
elements">Web link</a> |
 * Medium
 */

package leetcode;

import java.util.Arrays;
import java.util.Comparator;
import java.util.HashSet;
import java.util.Set;

public class Solution02593 {

    public long findScore(int[] nums) {
        int n = nums.length;
        int[][] arr = new int[n][2];

        for (int i = 0; i < n; i++) {
            arr[i][0] = nums[i];
            arr[i][1] = i;
        }

        Arrays.sort(arr, Comparator.comparingInt((int[] a) -> a[0]).thenComparingInt(a ->
a[1]));

        Set<Integer> set = new HashSet<>();
        long score = 0;

        for (int[] a : arr) {
            int val = a[0];
            int idx = a[1];

            if (set.add(idx)) {
                score += val;
                set.add(idx + 1);
                set.add(idx - 1);
            }
        }
        return score;
    }

}
```

441. `Solution02597.java`

```
package leetcode;

import java.util.*;

/**
 * 2597. The Number of Beautiful Subsets
 * <a href="https://leetcode.com/problems/the-number-of-beautiful-subsets">Web link</a> |
 * Medium
 */

public class Solution02597 {
    public int beautifulSubsets(int[] nums, int k) {
        Map<Integer, Integer> count = new HashMap<>();
        for (int num : nums) {
            count.put(num, count.getOrDefault(num, 0) + 1);
        }
        List<Map<Integer, Integer>> groups = new ArrayList<>();
        Set<Integer> visited = new HashSet<>();

        for (int num : count.keySet()) {
            if (visited.contains(num))
                continue;
            Map<Integer, Integer> group = new HashMap<>();
            while (count.containsKey(num - k)) {
                num -= k;
            }
            while (count.containsKey(num)) {
                group.put(num, count.get(num));
                visited.add(num);
                num += k;
            }
            groups.add(group);
        }

        int result = 1;

        for (Map<Integer, Integer> group : groups) {
            int min = group.keySet().stream().min(Integer::compareTo).orElse(0);
            result *= calculateSubsetCombinations(min, k, group);
        }

        return result - 1;
    }

    private int calculateSubsetCombinations(int min, int k, Map<Integer, Integer> group)
{
        if (!group.containsKey(min)) {
            return 1;
        }
        int skip = calculateSubsetCombinations(min + k, k, group);
        int include = ((int) Math.pow(2, group.get(min)) - 1) *
```

```
calculateSubsetCombinations(min + 2 * k, k, group);
        return skip + include;
    }

}
```

442. `Solution02601.java`

```
/**
 * 2601. Prime Subtraction Operation
 * <a href="https://leetcode.com/problems/prime-subtraction-operation">Web link</a> |
 * Medium
 */

package leetcode;

public class Solution02601 {

    public boolean primeSubOperation(int[] nums) {
        for (int i = 0; i < nums.length; i++) {
            int bound = (i == 0) ? nums[i] : nums[i] - nums[i - 1];

            if (bound <= 0) return false;

            int largestPrime = 0;
            for (int j = bound - 1; j >= 2; j--) {
                if (checkPrime(j)) {
                    largestPrime = j;
                    break;
                }
            }

            nums[i] -= largestPrime;
        }
        return true;
    }

    private boolean checkPrime(int x) {
        if (x <= 1) return false;
        for (int i = 2; i <= Math.sqrt(x); i++) {
            if (x % i == 0) {
                return false;
            }
        }
        return true;
    }

}
```

443. `Solution02610.java`

```
package leetcode;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * https://leetcode.com/problems/convert-an-array-into-a-2d-array-with-conditions/
 * 2610. Convert an Array Into a 2D Array With Conditions
 *
 */
public class Solution02610 {

    // Time: O(n)
    // Space: O(n)
    public List<List<Integer>> findMatrix(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();

        Map<Integer, Integer> map = new HashMap<>();

        for (int num : nums) {
            final int currentFrequency = map.getOrDefault(num, 0);
            if (currentFrequency >= result.size()) {
                result.add(new ArrayList<>());
            }
            result.get(currentFrequency).add(num);
            map.put(num, currentFrequency + 1);
        }

        return result;
    }
}
```

444. `Solution02641.java`

```
/**
 * 2641. Cousins in Binary Tree II
 * <a href="https://leetcode.com/problems/cousins-in-binary-tree-ii">Web link</a> |
 * Medium
 */

package leetcode;

import leetcode.common.TreeNode;

public class Solution02641 {

    public TreeNode replaceValueInTree(TreeNode root) {
        dfs(new TreeNode[]{root});
        root.val = 0;
        return root;
    }

    private void dfs(TreeNode[] arr) {
        if (arr.length == 0) return;

        int sum = 0;
        for (TreeNode node : arr) {
            if (node == null) continue;
            if (node.left != null) sum += node.left.val;
            if (node.right != null) sum += node.right.val;
        }

        TreeNode[] childArr = new TreeNode[arr.length * 2];
        int index = 0;

        for (TreeNode node : arr) {
            int curSum = 0;
            if (node.left != null) curSum += node.left.val;
            if (node.right != null) curSum += node.right.val;

            if (node.left != null) {
                node.left.val = sum - curSum;
                childArr[index++] = node.left;
            }
            if (node.right != null) {
                node.right.val = sum - curSum;
                childArr[index++] = node.right;
            }
        }

        dfs(java.util.Arrays.copyOf(childArr, index));
    }

}
```

445. `Solution02657.java`

```
/**
 * 2657. Find the Prefix Common Array of Two Arrays
 * <a href="https://leetcode.com/problems/find-the-prefix-common-array-of-two-arrays">Web
link</a> |
 * Medium
 */

package leetcode;

public class Solution02657 {

    public int[] findThePrefixCommonArray(int[] A, int[] B) {
        int n = A.length;
        int[] freq = new int[n + 1];
        int[] result = new int[n];
        int commonCount = 0;
        for (int i = 0; i < n; i++) {
            if (++freq[A[i]] == 2) commonCount++;
            if (++freq[B[i]] == 2) commonCount++;
            result[i] = commonCount;
        }
        return result;
    }

}
```

446. `Solution02678.java`

```
package leetcode;

/**
 * 2678. Number of Senior Citizens
 * <a href="https://leetcode.com/problems/number-of-senior-citizens">Web link</a> |
 * Easy
 */

public class Solution02678 {

    public int countSeniors(String[] details) {
        int count = 0;
        for (String detail : details) {
            if (detail.charAt(11) > '6' ||
                (detail.charAt(11) == '6' && detail.charAt(12) != '0')) {
                count++;
            }
        }
        return count;
    }

}
```

447. `Solution02683.java`

```
/**
 * 2683. Neighboring Bitwise XOR
 * <a href="https://leetcode.com/problems/neighboring-bitwise-xor">Web link</a> |
 * Medium
 */

package leetcode;

public class Solution02683 {

    public boolean doesValidArrayExist(int[] derived) {
        int xr = 0;
        for (int x : derived) {
            xr = xr ^ x;
        }
        return xr == 0;
    }

}
```

448. `Solution02684.java`

```
/**
 * 2684. Maximum Number of Moves in a Grid
 * <a href="https://leetcode.com/problems/maximum-number-of-moves-in-a-grid">Web link</a>
|
 * Medium
 */

package leetcode;

public class Solution02684 {

    public int maxMoves(int[][] grid) {
        int m = grid.length;
        int n = grid[0].length;
        int res = 0;
        int[] dp = new int[m];

        for (int j = 1; j < n; j++) {
            int leftTop = 0;
            boolean found = false;

            for (int i = 0; i < m; i++) {
                int cur = -1;
                int nxtLeftTop = dp[i];

                if (i - 1 >= 0 && leftTop != -1 && grid[i][j] > grid[i - 1][j - 1]) {
                    cur = Math.max(cur, leftTop + 1);
                }

                if (dp[i] != -1 && grid[i][j] > grid[i][j - 1]) {
                    cur = Math.max(cur, dp[i] + 1);
                }

                if (i + 1 < m && dp[i + 1] != -1 && grid[i][j] > grid[i + 1][j - 1]) {
                    cur = Math.max(cur, dp[i + 1] + 1);
                }

                dp[i] = cur;
                found = found || (dp[i] != -1);
                leftTop = nxtLeftTop;
            }

            if (!found) break;
            res = j;
        }

        return res;
    }

}
```

449. `Solution02696.java`

```
package leetcode;

import java.util.ArrayDeque;

/**
 * 2696. Minimum String Length After Removing Substrings
 * <a href="https://leetcode.com/problems/minimum-string-length-after-removing-
substrings">Web link</a> |
 * Easy
 */

public class Solution02696 {

    public int minLength(String s) {
        final int strLen = s.length();
        if (strLen < 2) {
            return strLen;
        }
        ArrayDeque<Character> stack = new ArrayDeque<>();
        for (int i = 0; i < strLen; i++) {
            char currentChar = s.charAt(i);
            if (currentChar == 'B' && !stack.isEmpty() && stack.peek() == 'A') {
                stack.pop();
            } else if (currentChar == 'D' && !stack.isEmpty() && stack.peek() == 'C') {
                stack.pop();
            } else {
                stack.push(currentChar);
            }
        }
        return stack.size();
    }

}
```

450. `Solution02699.java`

```
package leetcode;

import java.util.*;

/**
 * 2699. Modify Graph Edge Weights
 * <a href="https://leetcode.com/problems/modify-graph-edge-weights">Web link</a> |
 * Hard
 */

public class Solution02699 {

    public int[][] modifiedGraphEdges(int n, int[][] edges, int source, int destination,
int target) {
        List<int[]>[] adjacencyList = new ArrayList[n];
        for (int i = 0; i < n; i++) {
            adjacencyList[i] = new ArrayList<>();
        }
        for (int i = 0; i < edges.length; i++) {
            int nodeA = edges[i][0], nodeB = edges[i][1];
            adjacencyList[nodeA].add(new int[]{nodeB, i});
            adjacencyList[nodeB].add(new int[]{nodeA, i});
        }

        int[][] distances = new int[n][2];
        Arrays.fill(distances[source], 0);
        for (int i = 0; i < n; i++) {
            if (i != source) {
                distances[i][0] = distances[i][1] = Integer.MAX_VALUE;
            }
        }

        runDijkstra(adjacencyList, edges, distances, source, 0, 0);
        int difference = target - distances[destination][0];
        if (difference < 0) return new int[][]{};
        runDijkstra(adjacencyList, edges, distances, source, difference, 1);
        if (distances[destination][1] < target) return new int[][]{};

        for (int[] edge : edges) {
            if (edge[2] == -1) edge[2] = 1;
        }
        return edges;
    }

    private void runDijkstra(List<int[]>[] adjacencyList, int[][] edges, int[][]
distances, int source, int difference, int run) {
        int n = adjacencyList.length;
        PriorityQueue<int[]> priorityQueue = new PriorityQueue<>
(Comparator.comparingInt(a -> a[1]));
        priorityQueue.add(new int[]{source, 0});
        distances[source][run] = 0;
```

```
        while (!priorityQueue.isEmpty()) {
            int[] current = priorityQueue.poll();
            int currentNode = current[0];
            int currentDistance = current[1];

            if (currentDistance > distances[currentNode][run]) continue;

            for (int[] neighbor : adjacencyList[currentNode]) {
                int nextNode = neighbor[0], edgeIndex = neighbor[1];
                int weight = edges[edgeIndex][2];

                if (weight == -1) weight = 1; // Initially consider -1 as 1

                if (run == 1 && edges[edgeIndex][2] == -1) {

                    int newWeight = difference + distances[nextNode][0] -
distances[currentNode][1];
                    if (newWeight > weight) {
                        edges[edgeIndex][2] = weight = newWeight;
                    }
                }

                if (distances[nextNode][run] > distances[currentNode][run] + weight) {
                    distances[nextNode][run] = distances[currentNode][run] + weight;
                    priorityQueue.add(new int[]{nextNode, distances[nextNode][run]});
                }
            }
        }
    }

}
```

451. `Solution02707.java`

```
package leetcode;

import java.util.Collections;
import java.util.HashSet;
import java.util.Set;

/**
 * 2707. Extra Characters in a String
 * <a href="https://leetcode.com/problems/extra-characters-in-a-string">Web link</a> |
 * Medium
 */

public class Solution02707 {

    public int minExtraChar(String s, String[] dictionary) {
        int strLen = s.length();
        Set<String> dictSet = new HashSet<>(strLen);
        Collections.addAll(dictSet, dictionary);
        int[] delCount = new int[strLen + 1];
        for (int right = 1; right <= strLen; right++) {
            delCount[right] = 1 + delCount[right - 1];
            for (int left = right; left > 0; left--) {
                String word = s.substring(left - 1, right);
                if (dictSet.contains(word)) {
                    delCount[right] = Math.min(delCount[right], delCount[left - 1]);
                }
            }
        }
        return delCount[strLen];
    }

}
```

452. `Solution02709.java`

```
package leetcode;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

/**
 * 2709. Greatest Common Divisor Traversal
 * https://leetcode.com/problems/greatest-common-divisor-traversal/
 */

public class Solution02709 {

    // Time: O(n * log(n))
    // Space: O(n)
    public boolean canTraverseAllPairs(int[] nums) {
        final int n = nums.length;
        UnionFind uf = new UnionFind(n);

        Map<Integer, Integer> factorIndex = new HashMap<>();
        for (int i = 0; i < n; i++) {
            int num = nums[i];
            int f = 2;
            while (f * f <= num) {
                if (num % f == 0) {
                    if (factorIndex.containsKey(f)) {
                        uf.union(i, factorIndex.get(f));
                    } else {
                        factorIndex.put(f, i);
                    }
                    while (num % f == 0) {
                        num /= f;
                    }
                }
                f++;
            }
            if (num > 1) {
                if (factorIndex.containsKey(num)) {
                    uf.union(i, factorIndex.get(num));
                } else {
                    factorIndex.put(num, i);
                }
            }
        }
        return uf.count == 1;
    }

    static class UnionFind {
        private final int[] parent;
        private final int[] size;
        int count;
```

```
        public UnionFind(int n) {
            parent = new int[n];
            for (int i = 0; i < n; i++) {
                parent[i] = i;
            }
            size = new int[n];
            Arrays.fill(size, 1);
            count = n;
        }

        public int find(int x) {
            if (parent[x] != x) {
                parent[x] = find(parent[x]);
            }
            return parent[x];
        }

        public void union(int x, int y) {
            int rootX = find(x);
            int rootY = find(y);
            if (rootX == rootY) {
                return;
            }
            if (size[rootX] < size[rootY]) {
                parent[rootX] = rootY;
                size[rootY] += size[rootX];
            } else {
                parent[rootY] = rootX;
                size[rootX] += size[rootY];
            }
            count--;
        }
    }

}
```

453. `Solution02751.java`

```
package leetcode;

import java.util.*;

/**
 * 2751. Robot Collisions
 * <a href="https://leetcode.com/problems/robot-collisions">Web link</a> |
 * Hard
 */

public class Solution02751 {

    public List<Integer> survivedRobotsHealths(int[] positions, int[] healths, String
directions) {
        Map<Integer, Integer> indexMap = new HashMap<>();
        for (int i = 0; i < positions.length; i++) {
            indexMap.put(positions[i], i);
        }
        Arrays.sort(positions);

        Deque<Integer> stack = new ArrayDeque<>();
        for (int position : positions) {
            int index = indexMap.get(position);
            if (directions.charAt(index) == 'R') {
                stack.push(index);
            } else {
                while (!stack.isEmpty() && healths[index] > 0) {
                    int i2 = stack.pop();
                    if (healths[index] > healths[i2]) {
                        healths[i2] = 0;
                        healths[index] -= 1;
                    } else if (healths[index] < healths[i2]) {
                        healths[index] = 0;
                        healths[i2] -= 1;
                        stack.push(i2);
                    } else {
                        healths[index] = 0;
                        healths[i2] = 0;
                    }
                }
            }
        }

        List<Integer> result = new ArrayList<>();
        for (int health : healths) {
            if (health > 0) {
                result.add(health);
            }
        }
        return result;
    }
```

```
}
```

454. `Solution02762.java`

```
/**
 * 2762. Continuous Subarrays
 * <a href="https://leetcode.com/problems/continuous-subarrays">Web link</a> |
 * Medium
 */

package leetcode;

import java.util.SortedMap;
import java.util.TreeMap;

public class Solution02762 {

    public long continuousSubarrays(int[] nums) {
        long count = 0;
        int start = 0;
        SortedMap<Integer, Integer> hash = new TreeMap<>();
        for (int end = 0; end < nums.length; end++) {
            hash.put(nums[end], hash.getOrDefault(nums[end], 0) + 1);
            while (hash.lastKey() - hash.firstKey() > 2) {
                hash.put(nums[start], hash.get(nums[start]) - 1);
                if (hash.get(nums[start]) == 0) {
                    hash.remove(nums[start]);
                }
                start++;
            }
            count += end - start + 1;
        }
        return count;
    }

}
```

455. `Solution02779.java`

```
/**
 * 2779. Maximum Beauty of an Array After Applying Operation
 * <a href="https://leetcode.com/problems/maximum-beauty-of-an-array-after-applying-
operation">Web link</a> |
 * Medium
 */

package leetcode;

import java.util.Arrays;

public class Solution02779 {

    public int maximumBeauty(int[] nums, int k) {
        Arrays.sort(nums);
        int maxBeauty = 0;
        int right = 0;

        for (int left = 0; left < nums.length; left++) {
            while (right < nums.length && nums[right] - nums[left] <= 2 * k) {
                right++;
            }
            maxBeauty = Math.max(maxBeauty, right - left);
        }
        return maxBeauty;
    }

}
```

456. `Solution02807.java`

```
package leetcode;

import leetcode.common.ListNode;

import java.math.BigInteger;

/**
 * 2807. Insert Greatest Common Divisors in Linked List
 * <a href="https://leetcode.com/problems/insert-greatest-common-divisors-in-linked-
list">Web link</a> |
 * Medium
 */

public class Solution02807 {

    public ListNode insertGreatestCommonDivisors(ListNode head) {
        ListNode dummy = new ListNode(0, head);
        ListNode previous = head;
        ListNode current = head.next;
        while (current != null) {
            int gcd = gcd(previous.val, current.val);
            previous.next = new ListNode(gcd, current);
            previous = current;
            current = current.next;
        }
        return dummy.next;
    }

    private int gcd(int value1, int value2) {
        BigInteger bi1 = BigInteger.valueOf(value1);
        BigInteger bi2 = BigInteger.valueOf(value2);
        BigInteger gcd = bi1.gcd(bi2);
        return gcd.intValue();
    }

}
```

457. `Solution02812.java`

```
package leetcode;

import java.util.*;

/**
 * 2812. Find the Safest Path in a Grid
 * <a href="https://leetcode.com/problems/find-the-safest-path-in-a-grid">Web link</a>
 * Medium
 */

public class Solution02812 {

    public int maximumSafenessFactor(List<List<Integer>> grid) {
        Cell[][] cellGrid = fillDistances(grid);
        return traverseGridCells(cellGrid);
    }

    private Cell[][] fillDistances(List<List<Integer>> grid) {
        int n = grid.size();

        Queue<Cell> queue = new ArrayDeque<>();
        boolean[][] visited = new boolean[n][n];

        for (int row = 0; row < n; row++) {
            for (int col = 0; col < n; col++) {
                if (grid.get(row).get(col) == 1) {
                    queue.offer(new Cell(row, col, 0));
                    visited[row][col] = true;
                }
            }
        }

        int dist = 0;

        Cell[][] gridCells = new Cell[n][n];
        while (!queue.isEmpty()) {
            int numToPop = queue.size();
            for (int i = 0; i < numToPop; i++) {
                Cell room = queue.poll();
                final int row = room.row;
                final int col = room.col;

                gridCells[row][col] = new Cell(row, col, dist);

                addCell(gridCells, row + 1, col, queue, visited, dist);
                addCell(gridCells, row - 1, col, queue, visited, dist);
                addCell(gridCells, row, col + 1, queue, visited, dist);
                addCell(gridCells, row, col - 1, queue, visited, dist);
            }
            dist++;
        }
```

```
        return gridCells;
    }

    private void addCell(Cell[][] rooms, int row, int col, Queue<Cell> deque, boolean[][]
visited, int dist) {
        if (row < 0 || row >= rooms.length ||
            col < 0 || col >= rooms[0].length ||
            visited[row][col]) {
            return;
        }

        deque.offer(new Cell(row, col, dist + 1));
        visited[row][col] = true;
    }

    public int traverseGridCells(Cell[][] gridCells) {
        final Cell zeroCell = gridCells[0][0];
        if (zeroCell == null) {
            return 0;
        }

        int n = gridCells.length;

        boolean[][] visited = new boolean[n][n];
        Queue<Cell> deque = new PriorityQueue<>((a, b) -> b.distance - a.distance);
        deque.offer(zeroCell);
        visited[0][0] = true;

        while (!deque.isEmpty()) {
            Cell cell = deque.poll();
            int row = cell.row;
            int col = cell.col;
            int distance = cell.distance;

            if (row == n - 1 && col == n - 1) {
                return cell.distance;
            }

            visitDirection(gridCells, distance, row, col + 1, visited, deque);
            visitDirection(gridCells, distance, row + 1, col, visited, deque);
            visitDirection(gridCells, distance, row - 1, col, visited, deque);
            visitDirection(gridCells, distance, row, col - 1, visited, deque);
        }

        return 0;
    }

    private static void visitDirection(Cell[][] grid, int distance, int row, int col,
boolean[][] visited, Queue<Cell> deque) {
        int n = grid.length;
        if (row < 0 || row >= n ||
            col < 0 || col >= n ||
            visited[row][col]) {
            return;
```

```
        }
        deque.offer(new Cell(row, col, Math.min(distance, grid[row][col].distance)));
        visited[row][col] = true;
    }


    private static class Cell {
        private final int row;
        private final int col;
        private final int distance;

        public Cell(int row, int col, int distance) {
            this.row = row;
            this.col = col;
            this.distance = distance;
        }

        @Override
        public String toString() {
            return "Cell{" +
                   "row=" + row +
                   ", col=" + col +
                   ", distance=" + distance +
                   '}';
        }
    }

    public int maximumSafenessFactorAlternative(List<List<Integer>> grid) {
        int[][] directions = new int[][]{{0, -1}, {0, 1}, {-1, 0}, {1, 0}};
        int n = grid.size();
        int[][] gridArr = new int[n][n];
        int[][] safety = new int[n][n]; //closest distance to thief
        Queue<int[]> queue = new LinkedList<>();
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (grid.get(i).get(j) == 1) {
                    gridArr[i][j] = 1;
                    safety[i][j] = 0;
                    queue.add(new int[]{i, j, 0});
                } else {
                    safety[i][j] = Integer.MAX_VALUE;
                }
            }
        }
        if (gridArr[0][0] == 1 || gridArr[n - 1][n - 1] == 1) return 0;
        //bfs from each thief, update the smallest safety for all cells
        while (!queue.isEmpty()) {
            int[] curr = queue.poll();
            int currX = curr[0];
            int currY = curr[1];
            int currSafety = curr[2];
            for (int[] dir : directions) {
                int nextX = currX + dir[0];
                int nextY = currY + dir[1];
```

```
                if (isValid(nextX, nextY, n) && safety[nextX][nextY] > currSafety + 1) {
                    queue.add(new int[]{nextX, nextY, currSafety + 1});
                    safety[nextX][nextY] = currSafety + 1;
                }
            }
        }
        //bfs from start
        Queue<int[]> path = new PriorityQueue<>((a, b) -> Integer.compare(b[2], a[2]));
        path.add(new int[]{0, 0, safety[0][0]});
        gridArr[0][0] = 2;
        while (!path.isEmpty()) {
            int[] curr = path.poll();
            int currX = curr[0];
            int currY = curr[1];
            int currSafety = curr[2];
            if (currX == n - 1 && currY == n - 1) return currSafety;
            for (int[] dir : directions) {
                int nextX = currX + dir[0];
                int nextY = currY + dir[1];
                if (isValid(nextX, nextY, n) && gridArr[nextX][nextY] != 2) {
                    path.add(new int[]{nextX, nextY, Math.min(currSafety, safety[nextX]
[nextY])});
                    gridArr[nextX][nextY] = 2;
                }
            }
        }
        return 0;
    }

    private boolean isValid(int i, int j, int n) {
        if (i < 0 || i >= n || j < 0 || j >= n) return false;
        return true;
    }

}
```

458. `Solution02816.java`

```
package leetcode;

import leetcode.common.ListNode;

/**
 * 2816. Double a Number Represented as a Linked List
 * <a href="https://leetcode.com/problems/double-a-number-represented-as-a-linked-
list">Web link</a> |
 * Medium
 */

public class Solution02816 {

    public ListNode doubleIt(ListNode head) {
        ListNode dummy = new ListNode(0, head);
        reverseList(dummy);
        doubleList(dummy);
        reverseList(dummy);
        return dummy.next;
    }

    private void doubleList(ListNode dummy) {
        int carry = 0;
        ListNode prev = null;
        ListNode curr = dummy.next;
        while (curr != null) {
            curr.val = curr.val * 2 + carry;
            carry = curr.val / 10;
            curr.val = curr.val % 10;
            prev = curr;
            curr = prev.next;
        }
        if (carry > 0) {
            prev.next = new ListNode(carry);
        }
    }

    private void reverseList(ListNode dummy) {
        ListNode prev = null;
        ListNode curr = dummy.next;
        while (curr != null) {
            ListNode next = curr.next;
            curr.next = prev;
            prev = curr;
            curr = next;
        }
        dummy.next = prev;
    }

}
```

459. `Solution02825.java`

```
/**
 * 2825. Make String a Subsequence Using Cyclic Increments
 * <a href="https://leetcode.com/problems/make-string-a-subsequence-using-cyclic-
increments">Web link</a> |
 * Medium
 */

package leetcode;

public class Solution02825 {

    public boolean canMakeSubsequence(String str1, String str2) {
        int i = 0;
        int j = 0;

        while (i < str1.length() && j < str2.length()) {
            final char ch1 = str1.charAt(i);
            final char ch2 = str2.charAt(j);
            if (ch1 == ch2 ||
                    ch1 == (ch2 == 'a' ? 'z' : (char) (ch2 - 1))) {
                j++;
            }
            i++;
        }

        return j == str2.length();
    }


}
```

460. `Solution02864.java`

```
package leetcode;

/**
 * 2864. Maximum Odd Binary Number
 * https://leetcode.com/problems/maximum-odd-binary-number/
 * Easy
 */

public class Solution02864 {

    // Time complexity: O(n)
    // Space complexity: O(n)
    public String maximumOddBinaryNumber(String s) {
        int n = s.length();
        if (n == 1) {
            return s;
        }
        int count1 = 0;
        for (int i = 0; i < n; i++) {
            if (s.charAt(i) == '1') {
                count1++;
            }
        }
        return new StringBuilder()
                .repeat("1", count1 - 1)
                .repeat("0", n - count1)
                .append("1")
                .toString();
    }

}
```

461. `Solution02870.java`

```
package leetcode;

import java.util.HashMap;
import java.util.Map;

/**
 * <a href="https://leetcode.com/problems/minimum-number-of-operations-to-make-array-
empty/">
 * 2870. Minimum Number of Operations to Make Array Empty
 * </a>
 * <p>
 * You are given a 0-indexed array nums consisting of positive integers.
 * <p>
 * There are two types of operations that you can apply on the array any number of times:
 * <p>
 * Choose two elements with equal values and delete them from the array.
 * Choose three elements with equal values and delete them from the array.
 * Return the minimum number of operations required to make the array empty, or -1 if it
is not possible.
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [2,3,3,2,2,4,2,3,4]
 * Output: 4
 * Explanation: We can apply the following operations to make the array empty:
 * - Apply the first operation on the elements at indices 0 and 3. The resulting array is
nums = [3,3,2,4,2,3,4].
 * - Apply the first operation on the elements at indices 2 and 4. The resulting array is
nums = [3,3,4,3,4].
 * - Apply the second operation on the elements at indices 0, 1, and 3. The resulting
array is nums = [4,4].
 * - Apply the first operation on the elements at indices 0 and 1. The resulting array is
nums = [].
 * It can be shown that we cannot make the array empty in less than 4 operations.
 * Example 2:
 * <p>
 * Input: nums = [2,1,2,2,3,3]
 * Output: -1
 * Explanation: It is impossible to empty the array.
 * <p>
 * <p>
 * Constraints:
 * <p>
 * 2 <= nums.length <= 105
 * 1 <= nums[i] <= 106
 */
public class Solution02870 {

    public int minOperations(int[] nums) {
        Map<Integer, Integer> map = new HashMap<>();
        int result = 0;
```

```
        for (int num : nums) {
            map.merge(num, 1, Integer::sum);
        }
        for (int count : map.values()) {
            if (count == 1) {
                return -1;
            } else {
                result += count / 3;
                int remainderOf3 = count % 3;
                if (remainderOf3 > 0) {
                    // remainderOf3 == 1
                    // step 1 not possible, but we can avoid removing 3, and remove 2
twice, so plus 1 step
                    // remainderOf3 == 2
                    // can remove 2, 1 step
                    result += 1;
                }
            }
        }
        return result;
    }

    public int minOperationsCeil(int[] nums) {
        Map<Integer, Integer> map = new HashMap<>();
        int result = 0;
        for (int num : nums) {
            map.merge(num, 1, Integer::sum);
        }
        for (int count : map.values()) {
            if (count == 1) {
                return -1;
            } else {
                result += Math.ceil((double) count / 3);
            }
        }
        return result;
    }

}
```

462. `Solution02872.java`

```
/**
 * 2872. Maximum Number of K-Divisible Components
 * <a href="https://leetcode.com/problems/maximum-number-of-k-divisible-components">Web
link</a> |
 * Hard
 */

package leetcode;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Solution02872 {

    public int maxKDivisibleComponents(int n, int[][] edges, int[] values, int k) {
        List<List<Integer>> tree = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            tree.add(new ArrayList<>());
        }
        for (int[] edge : edges) {
            tree.get(edge[0]).add(edge[1]);
            tree.get(edge[1]).add(edge[0]);
        }

        long totalSum = Arrays.stream(values)
                .asLongStream()
                .sum();

        int[] result = new int[1];
        boolean[] visited = new boolean[n];
        postorder(0, tree, values, visited, totalSum, k, result);

        return result[0];
    }

    private long postorder(int node, List<List<Integer>> tree, int[] values,
                           boolean[] visited, long totalSum, int k, int[] result) {
        visited[node] = true;
        long subtreeSum = values[node];
        for (int neighbor : tree.get(node)) {
            if (!visited[neighbor]) {
                subtreeSum += postorder(neighbor, tree, values, visited, totalSum, k,
result);
            }
        }

        if (subtreeSum % k == 0 && (totalSum - subtreeSum) % k == 0) {
            result[0]++;
            return 0;
        }
```

```
        return subtreeSum;
    }

}
```

463. `Solution02914.java`

```
package leetcode;

/**
 * 2914. Minimum Number of Changes to Make Binary String Beautiful
 * <a href="https://leetcode.com/problems/minimum-number-of-changes-to-make-binary-
string-beautiful">Web link</a> |
 * Medium
 */

public class Solution02914 {

    public int minChanges(String s) {
        int count = 0;
        for (int i = 1; i < s.length(); i += 2) {
            if (s.charAt(i - 1) != s.charAt(i)) {
                count++;
            }
        }
        return count;
    }

}
```

464. `Solution02924.java`

```
/**
 * 2924. Find Champion II
 * <a href="https://leetcode.com/problems/find-champion-ii">Web link</a> |
 * Medium
 */

package leetcode;

public class Solution02924 {

    public int findChampion(int n, int[][] edges) {
        int[] inDegree = new int[n]; // Track the number of incoming edges for each node.

        for (int[] edge : edges) {
            int v = edge[1];
            inDegree[v]++; // Increment in-degree for destination node.
        }

        int champion = 0; // To count nodes with zero in-degree.
        int ans = 0; // Store the node with zero in-degree.

        for (int i = 0; i < n; i++) {
            if (inDegree[i] == 0) { // Check for zero in-degree.
                champion++;
                ans = i;
            }
        }

        // If there's more than one champion, return -1.
        if (champion > 1)
            return -1;
        return ans;
    }

}
```

465. `Solution02938.java`

```
package leetcode;

/**
 * 2938. Separate Black and White Balls
 * <a href="https://leetcode.com/problems/separate-black-and-white-balls">Web link</a> |
 * Medium
 */

public class Solution02938 {

    public long minimumSteps(String s) {
        long swaps = 0;
        int blackPosition = 0;
        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) == '0') {
                swaps += blackPosition;
            } else {
                ++blackPosition;
            }
        }
        return swaps;
    }

}
```

466. `Solution02940.java`

```
/**
 * 2940. Find Building Where Alice and Bob Can Meet
 * <a href="https://leetcode.com/problems/find-building-where-alice-and-bob-can-meet">Web
link</a> |
 * Hard
 */

package leetcode;

import java.util.*;

public class Solution02940 {

    public int[] leftmostBuildingQueries(int[] heights, int[][] queries) {
        int heightsLen = heights.length;
        int queriesLen = queries.length;
        int[] result = new int[queriesLen];
        Arrays.fill(result, -1);
        List<List<int[]>> deferred = new ArrayList<>();
        for (int i = 0; i < heightsLen; i++) {
            deferred.add(new ArrayList<>());
        }
        PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a ->
a[0]));

        for (int i = 0; i < queriesLen; ++i) {
            int a = queries[i][0];
            int b = queries[i][1];
            if (a > b) {
                int temp = a;
                a = b;
                b = temp;
            }
            if (a == b || heights[a] < heights[b]) {
                result[i] = b;
            } else {
                deferred.get(b).add(new int[]{heights[a], i});
            }
        }

        for (int i = 0; i < heightsLen; ++i) {
            pq.addAll(deferred.get(i));
            while (!pq.isEmpty() && pq.peek()[0] < heights[i]) {
                result[pq.poll()[1]] = i;
            }
        }

        return result;
    }

}
```

467. `Solution02958.java`

```
package leetcode;

import java.util.HashMap;
import java.util.Map;

/**
 * 2958. Length of Longest Subarray With at Most K Frequency
 * https://leetcode.com/problems/length-of-longest-subarray-with-at-most-k-frequency
 * Medium
 */

public class Solution02958 {

    public int maxSubarrayLength(int[] nums, int k) {
        int n = nums.length;
        int left = 0;
        int max = 0;
        Map<Integer, Integer> freq = new HashMap<>();

        for (int right = 0; right < n; right++) {
            final int currentElement = nums[right];
            int currentFreq = freq.getOrDefault(currentElement, 0);
            if (currentFreq == k) {
                while (left < right && nums[left] != currentElement) {
                    freq.compute(nums[left], (key, value) -> (value == null) ? 0 : value
- 1);

                    left++;
                }
                left++;
            } else {
                freq.put(currentElement, currentFreq + 1);
                max = Math.max(max, right - left + 1);
            }
        }

        return max;
    }

    public int maxSubarrayLengthLessCode(int[] nums, int k) {
        int n = nums.length;
        int left = 0;
        int max = 0;
        Map<Integer, Integer> freq = new HashMap<>();

        for (int right = 0; right < n; right++) {
            int currentElement = nums[right];
            freq.put(currentElement, freq.getOrDefault(currentElement, 0) + 1);

            while (freq.get(currentElement) > k) {
                freq.put(nums[left], freq.get(nums[left]) - 1);
                left++;
```

```
            }

            max = Math.max(max, right - left + 1);
        }

        return max;
    }

}
```

468. `Solution02962.java`

```
package leetcode;

/**
 * 2962. Count Subarrays Where Max Element Appears at Least K Times
 * https://leetcode.com/problems/count-subarrays-where-max-element-appears-at-least-k-times/description/
 * Medium
 */

public class Solution02962 {

    public long countSubarrays(int[] nums, int k) {
        int n = nums.length;
        long result = 0;
        int left = 0;
        int maxCount = 0;

        int max = nums[0];
        for (int i = 1; i < n; i++) {
            if (nums[i] > max) {
                max = nums[i];
            }
        }

        for (int right = 0; right < n; right++) {
            if (nums[right] == max) {
                maxCount++;
            }

            while (maxCount == k) {
                if (nums[left] == max) {
                    maxCount--;
                }
                left++;
                result += n - right;
            }

        }

        return result;
    }

}
```

469. `Solution02966.java`

```
package leetcode;

import java.util.Arrays;

/**
 * https://leetcode.com/problems/divide-array-into-arrays-with-max-
difference/description/
 * 2966. Divide Array Into Arrays With Max Difference
 * Medium
 * <p>
 * You are given an integer array nums of size n and a positive integer k.
 * <p>
 * Divide the array into one or more arrays of size 3 satisfying the following
conditions:
 * <p>
 * Each element of nums should be in exactly one array.
 * The difference between any two elements in one array is less than or equal to k.
 * Return a 2D array containing all the arrays. If it is impossible to satisfy the
conditions, return an empty array. And if there are multiple answers, return any of them.
 * <p>
 * <p>
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [1,3,4,8,7,9,3,5,1], k = 2
 * Output: [[1,1,3],[3,4,5],[7,8,9]]
 * Explanation: We can divide the array into the following arrays: [1,1,3], [3,4,5] and
[7,8,9].
 * The difference between any two elements in each array is less than or equal to 2.
 * Note that the order of elements is not important.
 * Example 2:
 * <p>
 * Input: nums = [1,3,3,2,7,3], k = 3
 * Output: []
 * Explanation: It is not possible to divide the array satisfying all the conditions.
 * <p>
 * <p>
 * Constraints:
 * <p>
 * n == nums.length
 * 1 <= n <= 105
 * n is a multiple of 3.
 * 1 <= nums[i] <= 105
 * 1 <= k <= 105
 */

public class Solution02966 {

    // Time: O(nlogn)
    // Space: O(n)
    public int[][] divideArray(int[] nums, int k) {
```

```
        Arrays.sort(nums);
        int[][] result = new int[nums.length / 3][3];
        for (int i = 0; i < nums.length; i = i + 3) {
            if (nums[i + 2] - nums[i] > k) {
                return new int[][]{};
            }
            result[i / 3][0] = nums[i];
            result[i / 3][1] = nums[i + 1];
            result[i / 3][2] = nums[i + 2];
        }
        return result;
    }

}
```

470. `Solution02971.java`

```
package leetcode;

import java.util.Arrays;

/**
 * https://leetcode.com/problems/find-polygon-with-the-largest-perimeter/
 * 2971. Find Polygon With the Largest Perimeter
 * Medium
 */

public class Solution02971 {

    public long largestPerimeter(int[] nums) {
        Arrays.sort(nums);
        long result = -1;
        long total = 0;
        for (int num : nums) {
            if (total > num) {
                result = total + num;
            }
            total += num;
        }
        return result;
    }

}
```

471. `Solution02976.java`

```
package leetcode;

import java.util.*;

/**
 * 2976. Minimum Cost to Convert String I
 * <a href="https://leetcode.com/problems/minimum-cost-to-convert-string-i">Web link</a>
|
 * Medium
 */

public class Solution02976 {

    public long minimumCost(String source, String target, char[] original, char[]
changed, int[] cost) {
        int n = source.length();

        List<Connection>[] adj = new List[26];
        long minCost = 0L;
        Map<SourceTarget, Long> memo = new HashMap<>();

        for (int i = 0; i < 26; i++) {
            adj[i] = new ArrayList<>();
        }

        for (int i = 0; i < original.length; i++) {
            adj[original[i] - 'a'].add(new Connection(changed[i] - 'a', cost[i]));
        }

        for (var i = 0; i < n; i++) {
            SourceTarget key = new SourceTarget(source.charAt(i), target.charAt(i));
            long charCost = dijkstra(adj, key, memo);
            if (charCost == -1) {
                return -1;
            }
            minCost += charCost;
        }
        return minCost;
    }

    private long dijkstra(List<Connection>[] adj, SourceTarget key, Map<SourceTarget,
Long> memo) {
        if (memo.containsKey(key)) {
            return memo.get(key);
        }

        boolean[] seen = new boolean[26];
        long charCost = 0L;
        Queue<Connection> queue = new PriorityQueue<>
(Comparator.comparingInt(Connection::distance));
        queue.offer(new Connection(key.source() - 'a', 0));
```

```
        while (!queue.isEmpty()) {
            Connection a = queue.poll();
            int currentTargetIndex = a.targetIndex();
            int costSoFar = a.distance();

            if ((char) (currentTargetIndex + 'a') == key.target()) {
                charCost += costSoFar;
                memo.put(key, charCost);
                return charCost;
            }
            if (seen[currentTargetIndex]) {
                continue;
            }
            seen[currentTargetIndex] = true;
            for (Connection nei : adj[currentTargetIndex]) {
                if (!seen[nei.targetIndex()]) {
                    queue.offer(new Connection(nei.targetIndex(), costSoFar +
nei.distance()));
                }
            }
        }
        return -1;
    }

    private record Connection(int targetIndex, int distance) {
    }

    private record SourceTarget(char source, char target) {
    }

}
```

472. `Solution02981.java`

```
/**
 * 2981. Find Longest Special Substring That Occurs Thrice I
 * <a href="https://leetcode.com/problems/find-longest-special-substring-that-occurs-
thrice-i">Web link</a> |
 * Medium
 */

package leetcode;

import java.util.HashMap;
import java.util.Map;

public class Solution02981 {

    public int maximumLength(String s) {
        Map<String, Integer> count = new HashMap<>();

        for (int start = 0; start < s.length(); start++) {
            for (int end = start + 1; end <= s.length(); end++) {
                if (s.charAt(start) != s.charAt(end - 1)) {
                    break;
                }
                String substring = s.substring(start, end);
                count.put(substring, count.getOrDefault(substring, 0) + 1);
            }
        }

        int ans = 0;
        for (Map.Entry<String, Integer> entry : count.entrySet()) {
            if (entry.getValue() >= 3) {
                ans = Math.max(ans, entry.getKey().length());
            }
        }

        return ans == 0 ? -1 : ans;
    }

}
```

473. `Solution02996.java`

```
package leetcode;

import java.util.HashSet;
import java.util.Set;

/*
* https://leetcode.com/problems/smallest-missing-integer-greater-than-sequential-prefix-sum/description/
* 2996. Smallest Missing Integer Greater Than Sequential Prefix Sum
*
* You are given a 0-indexed array of integers nums.

A prefix nums[0..i] is sequential if, for all 1 <= j <= i, nums[j] = nums[j - 1] + 1. In particular, the prefix consisting only of nums[0] is sequential.

Return the smallest integer x missing from nums such that x is greater than or equal to the sum of the longest sequential prefix.

Example 1:

Input: nums = [1,2,3,2,5]
Output: 6
Explanation: The longest sequential prefix of nums is [1,2,3] with a sum of 6. 6 is not in the array, therefore 6 is the smallest missing integer greater than or equal to the sum of the longest sequential prefix.
Example 2:

Input: nums = [3,4,5,1,12,14,13]
Output: 15
Explanation: The longest sequential prefix of nums is [3,4,5] with a sum of 12. 12, 13, and 14 belong to the array while 15 does not. Therefore 15 is the smallest missing integer greater than or equal to the sum of the longest sequential prefix.


Constraints:

1 <= nums.length <= 50
1 <= nums[i] <= 50
 */
public class Solution02996 {

    // Time: O(n)
    // Space: O(n)
    public int missingInteger(int[] nums) {
        int previous = nums[0];
        int maxSum = nums[0];

        Set<Integer> set = new HashSet<>();
        for (int num : nums) {
            set.add(num);
        }
```

```
        for (int i = 1; i < nums.length; i++) {
            int num = nums[i];
            if (num == previous + 1) {
                maxSum += num;
            } else {
                break;
            }
            previous = num;
        }

        int maxMissing = maxSum;
        while (set.contains(maxMissing)) {
            maxMissing++;
        }

        return maxMissing;
    }

}
```

474. `Solution02997.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/minimum-number-of-operations-to-make-array-xor-equal-to-k/description/
 * 2997. Minimum Number of Operations to Make Array XOR Equal to K
 * <p>
 * You are given a 0-indexed integer array nums and a positive integer k.
 * <p>
 * You can apply the following operation on the array any number of times:
 * <p>
 * Choose any element of the array and flip a bit in its binary representation. Flipping a bit means changing a 0 to 1 or vice versa.
 * Return the minimum number of operations required to make the bitwise XOR of all elements of the final array equal to k.
 * <p>
 * Note that you can flip leading zero bits in the binary representation of elements. For example, for the number (101)2 you can flip the fourth bit and obtain (1101)2.
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [2,1,3,4], k = 1
 * Output: 2
 * Explanation: We can do the following operations:
 * - Choose element 2 which is 3 == (011)2, we flip the first bit and we obtain (010)2 == 2. nums becomes [2,1,2,4].
 * - Choose element 0 which is 2 == (010)2, we flip the third bit and we obtain (110)2 = 6. nums becomes [6,1,2,4].
 * The XOR of elements of the final array is (6 XOR 1 XOR 2 XOR 4) == 1 == k.
 * It can be shown that we cannot make the XOR equal to k in less than 2 operations.
 * Example 2:
 * <p>
 * Input: nums = [2,0,2,0], k = 0
 * Output: 0
 * Explanation: The XOR of elements of the array is (2 XOR 0 XOR 2 XOR 0) == 0 == k. So no operation is needed.
 * <p>
 * <p>
 * Constraints:
 * <p>
 * 1 <= nums.length <= 105
 * 0 <= nums[i] <= 106
 * 0 <= k <= 106
 */
public class Solution02997 {

    // Time: O(n)
    // Space: O(1)
    public int minOperations(int[] nums, int k) {
        for (int a : nums) {
            k ^= a;
```

```
        }
        return Integer.bitCount(k);
    }

}
```

475. `Solution02998.java`

```
package leetcode;

import java.util.*;

/*
* https://leetcode.com/problems/minimum-number-of-operations-to-make-x-and-y-
equal/description/
* 2998. Minimum Number of Operations to Make X and Y Equal
* Medium
* You are given two positive integers x and y.

In one operation, you can do one of the four following operations:

Divide x by 11 if x is a multiple of 11.
Divide x by 5 if x is a multiple of 5.
Decrement x by 1.
Increment x by 1.
Return the minimum number of operations required to make x and y equal.

Example 1:

Input: x = 26, y = 1
Output: 3
Explanation: We can make 26 equal to 1 by applying the following operations:
1. Decrement x by 1
2. Divide x by 5
3. Divide x by 5
It can be shown that 3 is the minimum number of operations required to make 26 equal to
1.
Example 2:

Input: x = 54, y = 2
Output: 4
Explanation: We can make 54 equal to 2 by applying the following operations:
1. Increment x by 1
2. Divide x by 11
3. Divide x by 5
4. Increment x by 1
It can be shown that 4 is the minimum number of operations required to make 54 equal to
2.
Example 3:

Input: x = 25, y = 30
Output: 5
Explanation: We can make 25 equal to 30 by applying the following operations:
1. Increment x by 1
2. Increment x by 1
3. Increment x by 1
4. Increment x by 1
5. Increment x by 1
It can be shown that 5 is the minimum number of operations required to make 25 equal to
```

```
30.


Constraints:

1 <= x, y <= 104
 */
public class Solution02998 {

    // Time: O(n)
    // Space: O(n)
    public int minimumOperationsToMakeEqual(int x, int y) {
        if (x == y) {
            return 0;
        }
        if (x < y) {
            return y - x;
        }

        int maxNumberOfSteps = x - y;

        maxNumberOfSteps = Math.min(maxNumberOfSteps, 1 + x % 11 +
minimumOperationsToMakeEqual(x / 11, y));
        maxNumberOfSteps = Math.min(maxNumberOfSteps, 1 + 11 - x % 11 +
minimumOperationsToMakeEqual(x / 11 + 1, y));

        maxNumberOfSteps = Math.min(maxNumberOfSteps, 1 + x % 5 +
minimumOperationsToMakeEqual(x / 5, y));
        maxNumberOfSteps = Math.min(maxNumberOfSteps, 1 + 5 - x % 5 +
minimumOperationsToMakeEqual(x / 5 + 1, y));

        return maxNumberOfSteps;
    }

    // Time: O(n)
    // Space: O(n)
    public int minimumOperationsToMakeEqualBfs(int x, int y) {
        if (x == y) {
            return 0;
        }
        if (x < y) {
            return y - x;
        }

        Queue<Integer> queue = new LinkedList<>();
        queue.add(x);
        Set<Integer> visited = new HashSet<>();
        int level = 1;
        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int i = 0; i < size; i++) {
                int node = queue.poll();
                List<Integer> adjNodes = getAdjNodes(node);
                for (int adjNode : adjNodes) {
```

```
                    if (adjNode == y) return level;
                    if (!visited.contains(adjNode)) {
                        queue.add(adjNode);
                        visited.add(adjNode);
                    }
                }
            }
            level++;
        }
        return level;
    }

    private List<Integer> getAdjNodes(int node) {
        List<Integer> adjNodes = new ArrayList<>();
        adjNodes.add(node - 1);
        adjNodes.add(node + 1);
        if (node % 11 == 0) adjNodes.add(node / 11);
        if (node % 5 == 0) adjNodes.add(node / 5);
        return adjNodes;
    }

}
```

476. `Solution03005.java`

```
package leetcode;

/**
 * 3005. Count Elements With Maximum Frequency
 * <a href="https://leetcode.com/problems/count-elements-with-maximum-frequency/">...</a>
 * Easy
 */

public class Solution03005 {

    // Time: O(n)
    // Space: O(n)
    public int maxFrequencyElements(int[] nums) {
        int[] freq = new int[101];
        for (int num : nums) {
            freq[num]++;
        }
        int maxFreq = 0;
        int totalFrequences = 0;
        for (int f : freq) {
            if (f > maxFreq) {
                maxFreq = f;
                totalFrequences = f;
            } else if (f == maxFreq) {
                totalFrequences += f;
            }
        }
        return totalFrequences;
    }

}
```

477. `Solution03010.java`

```
package leetcode;

/**
 * 3010. Divide an Array Into Subarrays With Minimum Cost I
 * https://leetcode.com/problems/divide-an-array-into-subarrays-with-minimum-cost-i
 * Easy
 * You are given an array of integers nums of length n.
 * <p>
 * The cost of an array is the value of its first element. For example, the cost of
[1,2,3] is 1 while the cost of [3,4,1] is 3.
 * <p>
 * You need to divide nums into 3 disjoint contiguous subarrays.
 * <p>
 * Return the minimum possible sum of the cost of these subarrays.
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [1,2,3,12]
 * Output: 6
 * Explanation: The best possible way to form 3 subarrays is: [1], [2], and [3,12] at a
total cost of 1 + 2 + 3 = 6.
 * The other possible ways to form 3 subarrays are:
 * - [1], [2,3], and [12] at a total cost of 1 + 2 + 12 = 15.
 * - [1,2], [3], and [12] at a total cost of 1 + 3 + 12 = 16.
 * Example 2:
 * <p>
 * Input: nums = [5,4,3]
 * Output: 12
 * Explanation: The best possible way to form 3 subarrays is: [5], [4], and [3] at a
total cost of 5 + 4 + 3 = 12.
 * It can be shown that 12 is the minimum cost achievable.
 * Example 3:
 * <p>
 * Input: nums = [10,3,1,1]
 * Output: 12
 * Explanation: The best possible way to form 3 subarrays is: [10,3], [1], and [1] at a
total cost of 10 + 1 + 1 = 12.
 * It can be shown that 12 is the minimum cost achievable.
 * <p>
 * Constraints:
 * <p>
 * 3 <= n <= 50
 * 1 <= nums[i] <= 50
 */
public class Solution03010 {

    // Time: O(n)
    // Space: O(1)
    public int minimumCost(int[] nums) {
        int min1 = nums[0];
        int min2 = nums[1];
```

```
        int min3 = nums[2];

        for (int i = 3; i < nums.length; i++) {
            if (nums[i] < min2 || nums[i] < min3) {
                if (min2 > min3) {
                    min2 = nums[i];
                } else {
                    min3 = nums[i];
                }
            }
        }
        return min1 + min2 + min3;
    }

}
```

478. `Solution03011.java`

```
package leetcode;

/**
 * 3011. Find if Array Can Be Sorted
 * https://leetcode.com/problems/find-if-array-can-be-sorted
 * Medium
 * You are given a 0-indexed array of positive integers nums.
 * <p>
 * In one operation, you can swap any two adjacent elements if they have the same number
of
 * set bits
 * . You are allowed to do this operation any number of times (including zero).
 * <p>
 * Return true if you can sort the array, else return false.
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [8,4,2,30,15]
 * Output: true
 * Explanation: Let's look at the binary representation of every element. The numbers 2,
4, and 8 have one set bit each with binary representation "10", "100", and "1000"
respectively. The numbers 15 and 30 have four set bits each with binary representation
"1111" and "11110".
 * We can sort the array using 4 operations:
 * - Swap nums[0] with nums[1]. This operation is valid because 8 and 4 have one set bit
each. The array becomes [4,8,2,30,15].
 * - Swap nums[1] with nums[2]. This operation is valid because 8 and 2 have one set bit
each. The array becomes [4,2,8,30,15].
 * - Swap nums[0] with nums[1]. This operation is valid because 4 and 2 have one set bit
each. The array becomes [2,4,8,30,15].
 * - Swap nums[3] with nums[4]. This operation is valid because 30 and 15 have four set
bits each. The array becomes [2,4,8,15,30].
 * The array has become sorted, hence we return true.
 * Note that there may be other sequences of operations which also sort the array.
 * Example 2:
 * <p>
 * Input: nums = [1,2,3,4,5]
 * Output: true
 * Explanation: The array is already sorted, hence we return true.
 * Example 3:
 * <p>
 * Input: nums = [3,16,8,4,2]
 * Output: false
 * Explanation: It can be shown that it is not possible to sort the input array using any
number of operations.
 * <p>
 * Constraints:
 * <p>
 * 1 <= nums.length <= 100
 * 1 <= nums[i] <= 28
 */
```

```java
public class Solution03011 {

    // Time: O(n)
    // Space: O(1)
    public boolean canSortArray(int[] nums) {
        int lastGroupHigh = 0;
        int high = nums[0];
        boolean bitChange;
        for (int i = 1; i < nums.length; i++) {
            bitChange = Integer.bitCount(high) != Integer.bitCount(nums[i]);
            if (bitChange) {
                lastGroupHigh = high;
            }
            if (nums[i] > high) {
                high = nums[i];
            }
            if (nums[i] < lastGroupHigh) {
                return false;
            }
        }
        return true;
    }

}
```

479. `Solution03012.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 3012. Minimize Length of Array Using Operations
 * https://leetcode.com/problems/minimize-length-of-array-using-operations
 * Medium
 * <p>
 * You are given a 0-indexed integer array nums containing positive integers.
 * <p>
 * Your task is to minimize the length of nums by performing the following operations any
number of times (including zero):
 * <p>
 * Select two distinct indices i and j from nums, such that nums[i] > 0 and nums[j] > 0.
 * Insert the result of nums[i] % nums[j] at the end of nums.
 * Delete the elements at indices i and j from nums.
 * Return an integer denoting the minimum length of nums after performing the operation
any number of times.
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [1,4,3,1]
 * Output: 1
 * Explanation: One way to minimize the length of the array is as follows:
 * Operation 1: Select indices 2 and 1, insert nums[2] % nums[1] at the end and it
becomes [1,4,3,1,3], then delete elements at indices 2 and 1.
 * nums becomes [1,1,3].
 * Operation 2: Select indices 1 and 2, insert nums[1] % nums[2] at the end and it
becomes [1,1,3,1], then delete elements at indices 1 and 2.
 * nums becomes [1,1].
 * Operation 3: Select indices 1 and 0, insert nums[1] % nums[0] at the end and it
becomes [1,1,0], then delete elements at indices 1 and 0.
 * nums becomes [0].
 * The length of nums cannot be reduced further. Hence, the answer is 1.
 * It can be shown that 1 is the minimum achievable length.
 * Example 2:
 * <p>
 * Input: nums = [5,5,5,10,5]
 * Output: 2
 * Explanation: One way to minimize the length of the array is as follows:
 * Operation 1: Select indices 0 and 3, insert nums[0] % nums[3] at the end and it
becomes [5,5,5,10,5,5], then delete elements at indices 0 and 3.
 * nums becomes [5,5,5,5].
 * Operation 2: Select indices 2 and 3, insert nums[2] % nums[3] at the end and it
becomes [5,5,5,5,0], then delete elements at indices 2 and 3.
 * nums becomes [5,5,0].
 * Operation 3: Select indices 0 and 1, insert nums[0] % nums[1] at the end and it
becomes [5,5,0,0], then delete elements at indices 0 and 1.
 * nums becomes [0,0].
 * The length of nums cannot be reduced further. Hence, the answer is 2.
```

```
 * It can be shown that 2 is the minimum achievable length.
 * Example 3:
 * <p>
 * Input: nums = [2,3,4]
 * Output: 1
 * Explanation: One way to minimize the length of the array is as follows:
 * Operation 1: Select indices 1 and 2, insert nums[1] % nums[2] at the end and it
becomes [2,3,4,3], then delete elements at indices 1 and 2.
 * nums becomes [2,3].
 * Operation 2: Select indices 1 and 0, insert nums[1] % nums[0] at the end and it
becomes [2,3,1], then delete elements at indices 1 and 0.
 * nums becomes [1].
 * The length of nums cannot be reduced further. Hence, the answer is 1.
 * It can be shown that 1 is the minimum achievable length.
 * <p>
 * Constraints:
 * <p>
 * 1 <= nums.length <= 105
 * 1 <= nums[i] <= 109
 */
public class Solution03012 {

    // Time: O(nlogn)
    // Space: O(1)
    public int minimumArrayLength(int[] nums) {
        int n = nums.length;

        Arrays.sort(nums);

        if (n > 2 && nums[0] != nums[1]) {
            return 1;
        }

        int gcd = nums[0];
        for (int val : nums) {
            gcd = gcd(gcd, val);
        }

        int count = 0;
        for (int val : nums) {
            if (val == gcd) {
                count++;
            }
        }
        return Math.max(1, (count + 1) / 2);
    }

    private int gcd(int a, int b) {
        while (b != 0) {
            int temp = b;
            b = a % b;
            a = temp;
        }
        return a;
```

```
    }

}
```

480. `Solution03014.java`

```
package leetcode;

/**
 * 3014. Minimum Number of Pushes to Type Word I
 * <a href="https://leetcode.com/problems/minimum-number-of-pushes-to-type-word-i">Web
link</a>
 * Easy
 */

public class Solution03014 {

    public int minimumPushes(String word) {
        int n = word.length();
        int result = 0;
        while (n > 0) {
            result += n;
            n -= 8;
        }
        return result;
    }

}
```

481. `Solution03016.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 3016. Minimum Number of Pushes to Type Word II
 * <a href="https://leetcode.com/problems/minimum-number-of-pushes-to-type-word-ii">Web
link</a>
 * Medium
 */

public class Solution03016 {

    public int minimumPushes(String word) {
        int n = word.length();
        if (n < 8) {
            return n;
        }
        int[] freq = new int[26];
        for (int i = 0; i < n; i++) {
            freq[word.charAt(i) - 'a']++;
        }
        Arrays.sort(freq);
        int currentCount = 0;
        int distinctLetterCount = 0;
        for (int i = freq.length - 1; i >= 0; i--) {
            int numberOfPresses = 1 + distinctLetterCount / 8;
            currentCount += (freq[i] * numberOfPresses);
            distinctLetterCount++;
        }
        return currentCount;
    }

}
```

482. `Solution03024.java`

```
package leetcode;

import java.util.Arrays;

/**
 * https://leetcode.com/problems/type-of-triangle-ii/
 * 3024. Type of Triangle II
 * Easy
 * <p>
 * You are given a 0-indexed integer array nums of size 3 which can form the sides of a
triangle.
 * <p>
 * A triangle is called equilateral if it has all sides of equal length.
 * A triangle is called isosceles if it has exactly two sides of equal length.
 * A triangle is called scalene if all its sides are of different lengths.
 * Return a string representing the type of triangle that can be formed or "none" if it
cannot form a triangle.
 * <p>
 * Example 1:
 * <p>
 * Input: nums = [3,3,3]
 * Output: "equilateral"
 * Explanation: Since all the sides are of equal length, therefore, it will form an
equilateral triangle.
 * Example 2:
 * <p>
 * Input: nums = [3,4,5]
 * Output: "scalene"
 * Explanation:
 * nums[0] + nums[1] = 3 + 4 = 7, which is greater than nums[2] = 5.
 * nums[0] + nums[2] = 3 + 5 = 8, which is greater than nums[1] = 4.
 * nums[1] + nums[2] = 4 + 5 = 9, which is greater than nums[0] = 3.
 * Since the sum of the two sides is greater than the third side for all three cases,
therefore, it can form a triangle.
 * As all the sides are of different lengths, it will form a scalene triangle.
 * <p>
 * Constraints:
 * <p>
 * nums.length == 3
 * 1 <= nums[i] <= 100
 */
public class Solution03024 {

    // Time complexity: O(1)
    // Space complexity: O(1)
    public String triangleType(int[] nums) {
        Arrays.sort(nums);
        if (nums[0] + nums[1] <= nums[2]) {
            return "none";
        }
        if (nums[0] == nums[1] && nums[1] == nums[2]) {
```

```
            return "equilateral";
        }
        if (nums[0] == nums[1] || nums[1] == nums[2]) {
            return "isosceles";
        }
        return "scalene";
    }

}
```

483. `Solution03025.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/find-the-number-of-ways-to-place-people-i
 * 3025. Find the Number of Ways to Place People I
 * Medium
 * <p>
 * You are given a 2D array points of size n x 2 representing integer coordinates of some points on a 2D-plane, where points[i] = [xi, yi].
 * <p>
 * We define the right direction as positive x-axis (increasing x-coordinate) and the left direction as negative x-axis (decreasing x-coordinate). Similarly, we define the up direction as positive y-axis (increasing y-coordinate) and the down direction as negative y-axis (decreasing y-coordinate)
 * <p>
 * You have to place n people, including Chisato and Takina, at these points such that there is exactly one person at every point. Chisato wants to be alone with Takina, so Chisato will build a rectangular fence with Chisato's position as the upper left corner and Takina's position as the lower right corner of the fence (Note that the fence might not enclose any area, i.e. it can be a line). If any person other than Chisato and Takina is either inside the fence or on the fence, Chisato will be sad.
 * <p>
 * Return the number of pairs of points where you can place Chisato and Takina, such that Chisato does not become sad on building the fence.
 * <p>
 * Note that Chisato can only build a fence with Chisato's position as the upper left corner, and Takina's position as the lower right corner. For example, Chisato cannot build either of the fences in the picture below with four corners (1, 1), (1, 3), (3, 1), and (3, 3), because:
 * <p>
 * With Chisato at (3, 3) and Takina at (1, 1), Chisato's position is not the upper left corner and Takina's position is not the lower right corner of the fence.
 * With Chisato at (1, 3) and Takina at (1, 1), Takina's position is not the lower right corner of the fence.
 * <p>
 * Example 1:
 * <p>
 * Input: points = [[1,1],[2,2],[3,3]]
 * Output: 0
 * Explanation: There is no way to place Chisato and Takina such that Chisato can build a fence with Chisato's position as the upper left corner and Takina's position as the lower right corner. Hence we return 0.
 * Example 2:
 * <p>
 * Input: points = [[6,2],[4,4],[2,6]]
 * Output: 2
 * Explanation: There are two ways to place Chisato and Takina such that Chisato will not be sad:
 * - Place Chisato at (4, 4) and Takina at (6, 2).
 * - Place Chisato at (2, 6) and Takina at (4, 4).
 * You cannot place Chisato at (2, 6) and Takina at (6, 2) because the person at (4, 4)
```

```
will be inside the fence.
 * Example 3:
 * <p>
 * Input: points = [[3,1],[1,3],[1,1]]
 * Output: 2
 * Explanation: There are two ways to place Chisato and Takina such that Chisato will not
be sad:
 * - Place Chisato at (1, 1) and Takina at (3, 1).
 * - Place Chisato at (1, 3) and Takina at (1, 1).
 * You cannot place Chisato at (1, 3) and Takina at (3, 1) because the person at (1, 1)
will be on the fence.
 * Note that it does not matter if the fence encloses any area, the first and second
fences in the image are valid.
 * <p>
 * Constraints:
 * <p>
 * 2 <= n <= 50
 * points[i].length == 2
 * 0 <= points[i][0], points[i][1] <= 50
 * All points[i] are distinct.
 */
public class Solution03025 {

    // Time complexity: O(n^2)
    // Space complexity: O(1)
    public int numberOfPairs(int[][] points) {
        int n = points.length;
        int totalValid = 0;

        for (int i = 0; i < n; i++) {
            int[] p1 = points[i];

            for (int j = i + 1; j < n; j++) {
                int[] p2 = points[j];
                boolean valid = checkPointGood(points, p1, p2);
                if (valid) {
                    totalValid++;
                } else {
                    boolean isValid2 = checkPointGood(points, p2, p1);
                    if (isValid2) {
                        totalValid++;
                    }
                }
            }
        }

        return totalValid;
    }

    private static boolean checkPointGood(int[][] points, int[] p1, int[] p2) {
        int p1x = p1[0];
        int p1y = p1[1];

        int p2x = p2[0];
```

```
        int p2y = p2[1];
        if ((p1x > p2x && p1y < p2y) || // diagonal left top to right bottom
            (p1x > p2x && p1y == p2y) || // horizontal left to right
            (p1x == p2x && p1y < p2y)) { // vertical top to bottom
            boolean isInside = false;
            for (int[] p : points) {
                int px = p[0];
                int py = p[1];
                if (p != p1 && p != p2 &&
                    px >= p2x && px <= p1x && // x is between p1 and p2
                    py <= p2y && py >= p1y) { // y is between p1 and p2
                    isInside = true;
                    break;
                }
            }
            return !isInside;
        }
        return false;
    }

}
```

484. `Solution03038.java`

```
package leetcode;

/**
 * https://leetcode.com/problems/maximum-number-of-operations-with-the-same-score-i/
 * 3038. Maximum Number of Operations With the Same Score I
 * Easy
 */
public class Solution03038 {

    // Time: O(n)
    // Space: O(1)
    public int maxOperations(int[] nums) {
        if (nums.length < 2) {
            return 0;
        }
        int maxOperations = 1;
        int sum = nums[0] + nums[1];
        for (int i = 3; i < nums.length; i = i + 2) {
            if (nums[i - 1] + nums[i] == sum) {
                maxOperations++;
            } else {
                break;
            }
        }
        return maxOperations;
    }

}
```

485. `Solution03039.java`

```
package leetcode;

import java.util.HashMap;
import java.util.Map;

/**
 * https://leetcode.com/problems/apply-operations-to-make-string-empty/
 * 3039. Apply Operations to Make String Empty
 * Medium
 */
public class Solution03039 {

    // Time: O(n)
    // Space: O(n)
    public String lastNonEmptyString(String s) {
        Map<Character, Integer> freqMap = new HashMap<>();
        for (char ch : s.toCharArray()) {
            freqMap.merge(ch, 1, Integer::sum);
        }
        int max = freqMap.values().stream()
                .mapToInt(intObj -> intObj)
                .max()
                .orElse(0);
        if (max == 1) {
            return s;
        }
        Map<Character, Integer> onlyMaxFreqMap = new HashMap<>();
        for (Map.Entry<Character, Integer> entry : freqMap.entrySet()) {
            if (entry.getValue() == max) {
                onlyMaxFreqMap.put(entry.getKey(), entry.getValue());
            }
        }
        StringBuilder stringBuilder = new StringBuilder();
        for (char ch : s.toCharArray()) {
            Integer count = onlyMaxFreqMap.get(ch);
            if (count != null) {
                if (count > 1) {
                    onlyMaxFreqMap.put(ch, count - 1);
                } else {
                    stringBuilder.append(ch);
                }
            }
        }
        return stringBuilder.toString();
    }

}
```

486. `Solution03042.java`

```
/**
 * 3042. Count Prefix and Suffix Pairs I
 * <a href="https://leetcode.com/problems/count-prefix-and-suffix-pairs-i">Web link</a>
 * Easy
 */

package leetcode;

public class Solution03042 {

    public int countPrefixSuffixPairs(String[] words) {
        int result = 0;
        for (int i = 0; i < words.length; i++) {
            result += countPrefixAndSuffix(i, words);
        }
        return result;
    }

    private int countPrefixAndSuffix(int i, String[] words) {
        String wordToCheck = words[i];
        int count = 0;
        for (int j = i + 1; j < words.length; j++) {
            String word = words[j];
            if (i < j &&
                    word.startsWith(wordToCheck) &&
                    word.endsWith(wordToCheck)) {
                count++;
            }
        }
        return count;
    }

}
```

487. `Solution03043.java`

```
package leetcode;

import java.util.HashMap;
import java.util.Map;

/**
 * 3043. Find the Length of the Longest Common Prefix
 * <a href="https://leetcode.com/problems/find-the-length-of-the-longest-common-
prefix">Web link</a>
 * Medium
 */

public class Solution03043 {

    public int longestCommonPrefix(int[] arr1, int[] arr2) {
        Map<String, Integer> prefixMap = new HashMap<>();

        // Step 1: Build the prefix map for arr1
        for (int num : arr1) {
            String strNum = Integer.toString(num);
            for (int i = 1; i <= strNum.length(); i++) {
                String prefix = strNum.substring(0, i);
                prefixMap.put(prefix, prefixMap.getOrDefault(prefix, 0) + 1);
            }
        }

        int maxLength = 0;

        // Step 2: Check for common prefixes in arr2
        for (int num : arr2) {
            String strNum = Integer.toString(num);
            for (int i = 1; i <= strNum.length(); i++) {
                String prefix = strNum.substring(0, i);
                if (prefixMap.containsKey(prefix)) {
                    maxLength = Math.max(maxLength, i);
                }
            }
        }

        return maxLength;
    }

}
```

488. `Solution03065.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 3065. Minimum Operations to Exceed Threshold Value I
 * https://leetcode.com/problems/minimum-operations-to-exceed-threshold-value-i/
 * Easy
 */
public class Solution03065 {

    // Time: O(nlogn)
    // Space: O(1)
    public int minOperations(int[] nums, int k) {
        Arrays.sort(nums);
        int n = nums.length;
        for (int i = 0; i < n; i++) {
            if (nums[i] >= k) {
                return i;
            }
        }
        return n;
    }

    // Time: O(n)
    // Space: O(n)
    public int minOperationsCount(int[] nums, int k) {
        int steps = 0;
        for (int num : nums) {
            if (num < k) {
                steps++;
            }
        }

        return steps;
    }

}
```

489. `Solution03066.java`

```
package leetcode;

import java.util.ArrayDeque;
import java.util.Arrays;
import java.util.PriorityQueue;
import java.util.Queue;

/**
 * 3066. Minimum Operations to Exceed Threshold Value II
 * https://leetcode.com/problems/minimum-operations-to-exceed-threshold-value-ii/
 * Medium
 */
public class Solution03066 {

    public int minOperations(int[] nums, int k) {
        PriorityQueue<Long> queue = new PriorityQueue<>();
        for (int num : nums) {
            queue.offer((long) num);
        }
        int steps = 0;
        while (queue.peek() < k) {
            long first = queue.poll();
            long second = queue.poll();
            long amount = first * 2L + second;
            queue.offer(amount);
            steps++;
        }
        return steps;
    }

    public int minOperationsDraft(int[] nums, int k) {
        Arrays.sort(nums);
        int n = nums.length;

        int steps = 0;
        Queue<Long> queue = new ArrayDeque<>();
        int i = 0;
        while (i < n || !queue.isEmpty()) {
            long first;
            if (i == n) {
                first = queue.poll();
            } else {
                first = nums[i];
                if (!queue.isEmpty() && queue.peek() < first) {
                    first = queue.poll();
                } else {
                    i++;
                }
            }
            if (first >= k) {
                return steps;
```

```
            }
            long second;
            if (i == n) { // nothing left in main array for checking
                second = queue.poll();
            } else {
                second = nums[i];
                if (!queue.isEmpty() && queue.peek() < second) {
                    second = queue.poll();
                } else {
                    i++;
                }
            }
            long amount = Math.min(first, second) * 2L + Math.max(first, second);
            queue.offer(amount);
            steps++;
        }
        return steps;
    }
}
```

490. `Solution03068.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 3068. Find the Maximum Sum of Node Values
 * <a href="https://leetcode.com/problems/find-the-maximum-sum-of-node-values">Web
link</a>
 * Hard
 */

public class Solution03068 {
    public long maximumValueSum(int[] nums, int k, int[][] edges) {
        int n = nums.length;
        int[] delta = new int[n];

        long sum = 0;
        for (int i = 0; i < n; i++) {
            delta[i] = (nums[i] ^ k) - nums[i];
            sum += nums[i];
        }

        Arrays.sort(delta);

        for (int i = n - 1; i > 0; i -= 2) {
            int deltaOf2 = delta[i] + delta[i - 1];
            if (deltaOf2 > 0) {
                sum += deltaOf2;
            } else {
                break;
            }
        }

        return sum;
    }

    public long maximumValueSumOptimized(int[] nums, int k, int[][] edges) {
        long sum = 0;
        boolean isOdd = false;
        int maxNegative = Integer.MIN_VALUE;
        int minPositive = Integer.MAX_VALUE;
        for (int num : nums) {
            int delta = (num ^ k) - num;
            if (delta > 0) {
                sum += delta;
                minPositive = Math.min(minPositive, delta);
                isOdd = !isOdd;
            } else {
                maxNegative = Math.max(maxNegative, delta);
            }
            sum += num;
```

```
        }

        if (!isOdd) {
            return sum;
        }

        return Math.max(sum - minPositive, sum + maxNegative);
    }

}
```

491. `Solution03075.java`

```
package leetcode;

import java.util.Arrays;

/**
 * 3075. Maximize Happiness of Selected Children
 * https://leetcode.com/problems/maximize-happiness-of-selected-children/description/
 * Medium
 */

public class Solution03075 {

    public long maximumHappinessSum(int[] happiness, int k) {
        Arrays.sort(happiness);
        int n = happiness.length;
        int numberToDecrease = 0;
        long sum = 0L;
        for (int i = n - 1; i >= 0 && numberToDecrease < k; i--, numberToDecrease++) {
            int candidateToSum = happiness[i] - numberToDecrease;
            if (candidateToSum > 0) {
                sum += candidateToSum;
            } else {
                break;
            }
        }
        return sum;
    }

}
```

492. `Solution03079.java`

```
package leetcode;

/**
 * 3079. Find the Sum of Encrypted Integers
 * https://leetcode.com/problems/find-the-sum-of-encrypted-integers/description/
 * Easy
 */
public class Solution03079 {

    public int sumOfEncryptedInt(int[] nums) {
        int sum = 0;
        for (int num : nums) {
            sum += encrypt(num);
        }
        return sum;
    }

    private int encrypt(int num) {
        int maxDigit = 0;
        int length = 0;

        while (num > 0) {
            int digit = num % 10;
            maxDigit = Math.max(maxDigit, digit);
            num /= 10;
            length++;
        }

        int encrypted = 0;
        for (int i = 0; i < length; i++) {
            encrypted = encrypted * 10 + maxDigit;
        }
        return encrypted;
    }

    public int sumOfEncryptedIntViaString(int[] nums) {
        int sum = 0;
        for (int num : nums) {
            String str = String.valueOf(num);
            char max = '0';
            for (char i : str.toCharArray()) {
                if (i > max) {
                    max = i;
                }
            }
            StringBuilder sb = new StringBuilder(str.length());
            sb.repeat(max, str.length());
            int digit = Integer.parseInt(sb.toString());
            sum += digit;
        }
        return sum;
```

```
    }

}
```

493. `Solution03080.java`

```
package leetcode;

import javafx.util.Pair;

import java.util.Comparator;
import java.util.PriorityQueue;

/**
 * 3080. Mark Elements on Array by Performing Queries
 * <a href="https://leetcode.com/problems/mark-elements-on-array-by-performing-
queries">Web link</a>
 * Medium
 */
public class Solution03080 {

    public long[] unmarkedSumArray(int[] nums, int[][] queries) {
        Comparator<Pair<Integer, Integer>> pairComparator = (pair1, pair2) -> {
            int keyComparison = pair1.getKey().compareTo(pair2.getKey());
            if (keyComparison != 0) {
                return keyComparison;
            } else {
                return pair1.getValue().compareTo(pair2.getValue());
            }
        };
        PriorityQueue<Pair<Integer, Integer>> pq = new PriorityQueue<>(pairComparator);
        long totalSum = 0;
        for (int i = 0; i < nums.length; i++) {
            int num = nums[i];
            totalSum += num;
            pq.add(new Pair<>(num, i));
        }
        long[] result = new long[queries.length];
        for (int i = 0; i < queries.length; i++) {
            int[] command = queries[i];
            int indexForMark = command[0];
            if (nums[indexForMark] > 0) {
                totalSum -= nums[indexForMark];
            }
            nums[indexForMark] = -1;
            int elementsToMark = command[1];
            while (elementsToMark > 0) {
                if (pq.isEmpty()) {
                    break;
                }
                Pair<Integer, Integer> pair = pq.poll();
                int indexOfElementFromQueue = pair.getValue();
                int actualValueWithUpdates = nums[indexOfElementFromQueue];
                if (actualValueWithUpdates >= 0) { // not marked yet
                    totalSum -= actualValueWithUpdates;
                    nums[indexOfElementFromQueue] = -1;
                    elementsToMark--;
```

```
                }
            }
            result[i] = totalSum;
        }
        return result;
    }

}
```

494. `Solution03081.java`

```
package leetcode;

/**
 * 3081. Replace Question Marks in String to Minimize Its Value
 * <a href="https://leetcode.com/problems/replace-question-marks-in-string-to-minimize-
its-value">Web link</a>
 * Medium
 */
public class Solution03081 {

    public String minimizeStringValue(String s) {
        int[] count = new int[26];
        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) != '?') {
                count[s.charAt(i) - 'a']++;
            }
        }
        int[] charactersToAdd = new int[26];
        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) == '?') {
                int minIndex = -1;
                int minCount = Integer.MAX_VALUE;
                for (int j = 0; j < 26; j++) {
                    if (count[j] < minCount) {
                        minCount = count[j];
                        minIndex = j;
                    }
                }
                charactersToAdd[minIndex]++;
                count[minIndex]++;
            }
        }
        StringBuilder sb = new StringBuilder();
        int currentIndex = 0;
        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) == '?') {
                while (charactersToAdd[currentIndex] == 0) {
                    currentIndex++;
                }
                sb.append((char) ('a' + currentIndex));
                charactersToAdd[currentIndex]--;
            } else {
                sb.append(s.charAt(i));
            }
        }
        return sb.toString();
    }

}
```

495. `Solution03095.java`

```
package leetcode;

/**
 * 3095. Shortest Subarray With OR at Least K I
 * https://leetcode.com/problems/shortest-subarray-with-or-at-least-k-i/
 * Easy
 */

public class Solution03095 {

    public int minimumSubarrayLength(int[] nums, int k) {
        int n = nums.length;
        int min = Integer.MAX_VALUE;

        for (int left = 0; left < n; left++) {
            long or = 0;
            for (int right = left; right < n; right++) {
                or |= nums[right];
                if (or >= k) {
                    min = Math.min(min, right - left + 1);
                    break;
                }
            }
        }

        return min == Integer.MAX_VALUE ? -1 : min;
    }

}
```

496. `Solution03096.java`

```
package leetcode;

/**
 * 3096. Minimum Levels to Gain More Points
 * <a href="https://leetcode.com/problems/minimum-levels-to-gain-more-points/">Web
link</a> |
 * Medium
 */

public class Solution03096 {

    public int minimumLevels(int[] possible) {
        int maxScore = 0;
        for (int j : possible) {
            maxScore += j == 1 ? 1 : -1;
        }

        int danielScore = 0;
        int bobScore = maxScore;
        int maxPlays = possible.length - 1; // bob should play at least 1 level

        for (int i = 0; i < maxPlays; i++) {
            if (possible[i] == 1) {
                danielScore++;
                bobScore--;
            } else {
                danielScore--;
                bobScore++;
            }
            if (danielScore > bobScore) {
                return i + 1;
            }
        }

        return -1;
    }

}
```

497. `Solution03097.java`

```
package leetcode;

/**
 * 3097. Shortest Subarray With OR at Least K II
 * https://leetcode.com/problems/shortest-subarray-with-or-at-least-k-ii/
 * Medium
 */

public class Solution03097 {

    // The size of the bits array is determined by the maximum number of bits in an
integer
    private static final int BITS_ARRAY_SIZE = 32;

    public int minimumSubarrayLength(int[] nums, int k) {
        int arrayLength = nums.length;
        int minimumLength = arrayLength + 1;
        int[] bitCounts = new int[BITS_ARRAY_SIZE];
        int windowStartIndex = 0;

        for (int windowEndIndex = 0; windowEndIndex < arrayLength; windowEndIndex++) {
            updateBitCounts(bitCounts, nums[windowEndIndex], 1); // Add the current
number to the window

            // While the OR of the nums in the window is greater than or equal to the k
            while (windowStartIndex <= windowEndIndex && convertBitsToNumber(bitCounts)
>= k) {
                // Update the minimum length if the current window size is smaller
                minimumLength = Math.min(minimumLength, windowEndIndex - windowStartIndex
+ 1);

                // Remove the number at the start of the window
                updateBitCounts(bitCounts, nums[windowStartIndex], -1);
                windowStartIndex++;
            }
        }

        // If no subarray was found that meets the condition, return -1
        return minimumLength != arrayLength + 1 ? minimumLength : -1;
    }

    private void updateBitCounts(int[] bitCounts, int number, int change) {
        // For each bit in the number, if it's set, increment or decrement the
corresponding count in the bitCounts array
        for (int bitIndex = 0; bitIndex < BITS_ARRAY_SIZE; bitIndex++) {
            if (((number >> bitIndex) & 1) != 0) {
                bitCounts[bitIndex] += change;
            }
        }
    }
```

```
    private int convertBitsToNumber(int[] bitCounts) {
        // Convert the bitCounts array back to a number by setting the corresponding bit
for each non-zero count
        int number = 0;
        for (int bitIndex = 0; bitIndex < BITS_ARRAY_SIZE; bitIndex++) {
            if (bitCounts[bitIndex] != 0) {
                number |= 1 << bitIndex;
            }
        }
        return number;
    }

    public int minimumSubarrayLengthOptimized(int[] nums, int k) {
        int currentOrValue = 0;
        int minimumLength = Integer.MAX_VALUE;

        for (int currentIndex = 0; currentIndex < nums.length; currentIndex++) {
            currentOrValue = currentOrValue | nums[currentIndex];

            if (currentOrValue >= k) {
                int tempOrValue = nums[currentIndex];
                int previousIndex = currentIndex - 1;

                // If the current number itself is greater than or equal to the k, return
1
                if (nums[currentIndex] >= k) {
                    return 1;
                }

                // Try to find a subarray ending at the current index with OR value
greater than or equal to the k
                while (tempOrValue < k) {
                    int newOrValue = tempOrValue | nums[previousIndex];

                    if (newOrValue >= k) {
                        currentOrValue = tempOrValue;
                        minimumLength = Math.min(minimumLength, currentIndex -
previousIndex + 1);
                        break;
                    }

                    tempOrValue = newOrValue;
                    previousIndex--;
                }
            }
        }

        // If no subarray was found that meets the condition, return -1
        return minimumLength == Integer.MAX_VALUE ? -1 : minimumLength;
    }

}
```

498. `Solution03110.java`

```
package leetcode;

/**
 * 3110. Score of a String
 * https://leetcode.com/problems/score-of-a-string/
 * Easy
 */
public class Solution03110 {

    public int scoreOfString(String s) {
        int n = s.length();
        int score = 0;
        for (int i = 1; i < n; i++) {
            score += Math.abs(s.charAt(i) - s.charAt(i - 1));
        }
        return score;
    }

}
```

499. `Solution03111.java`

```
package leetcode;

import java.util.Arrays;
import java.util.Comparator;

/**
 * 3111. Minimum Rectangles to Cover Points
 * https://leetcode.com/problems/minimum-rectangles-to-cover-points/
 * Medium
 */
public class Solution03111 {

    public int minRectanglesToCoverPoints(int[][] points, int w) {
        Arrays.sort(points, Comparator.comparingInt(a -> a[0]));
        final int n = points.length;
        int rectCount = 1;
        int lastX = points[0][0] + w;
        for (int i = 1; i < n; i++) {
            int x = points[i][0];
            if (x > lastX) {
                rectCount++;
                lastX = x + w;
            }
        }
        return rectCount;
    }

}
```

500. `Solution03127.java`

```
package leetcode;

/**
 * 3127. Make a Square with the Same Color
 * <a href="https://leetcode.com/problems/make-a-square-with-the-same-color">Web link</a>
|
 * Easy
 */

public class Solution03127 {

    public boolean canMakeSquare(char[][] grid) {
        final int n = grid.length;
        for (int i = 1; i < n; i++) {
            for (int j = 1; j < n; j++) {
                int blackCount = amountInSquare(grid, i, j, 'B');
                int whiteCount = amountInSquare(grid, i, j, 'W');
                if (blackCount >= 3 || whiteCount >= 3) {
                    return true;
                }
            }
        }
        return false;
    }

    private int amountInSquare(char[][] grid, int i, int j, char color) {
        int amount = 0;
        if (grid[i - 1][j - 1] == color) {
            amount++;
        }
        if (grid[i - 1][j] == color) {
            amount++;
        }
        if (grid[i][j - 1] == color) {
            amount++;
        }
        if (grid[i][j] == color) {
            amount++;
        }
        return amount;
    }

}
```

501. `Solution03128.java`

```
package leetcode;

/**
 * 3128. Right Triangles
 * https://leetcode.com/problems/right-triangles/
 * Medium
 */
public class Solution03128 {

    public long numberOfRightTriangles(int[][] grid) {
        int numRows = grid.length;
        int numCols = grid[0].length;

        int[] rowCounts = new int[numRows];
        int[] colCounts = new int[numCols];

        for (int row = 0; row < numRows; row++) {
            for (int col = 0; col < numCols; col++) {
                if (grid[row][col] == 1) {
                    rowCounts[row]++;
                    colCounts[col]++;
                }
            }
        }

        long count = 0;
        for (int row = 0; row < numRows; row++) {
            for (int col = 0; col < numCols; col++) {
                if (grid[row][col] == 1) {
                    count += (long) (rowCounts[row] - 1) * (colCounts[col] - 1);
                }
            }
        }

        return count;
    }

}
```

502. `Solution03133.java`

```
/**
 * 3133. Minimum Array End
 * <a href="https://leetcode.com/problems/minimum-array-end">Web link</a>
 * Medium
 */

package leetcode;

public class Solution03133 {

    public long minEnd(int n, int x) {
        long result = x;
        long iX = 1;
        long iN = 1;
        while (iN < n) {
            if ((iX & x) == 0) {
                if ((iN & (n - 1)) > 0) {
                    result |= iX;
                }
                iN = iN << 1;
            }
            iX = iX << 1;
        }
        return result;
    }

}
```

503. `Solution03152.java`

```
/**
 * 3152. Special Array II
 * <a href="https://leetcode.com/problems/special-array-ii">Web link</a>
 * Medium
 */

package leetcode;

public class Solution03152 {

    public boolean[] isArraySpecial(int[] nums, int[][] queries) {
        int n = nums.length;
        int[] cum = new int[n];
        boolean[] ans = new boolean[queries.length];

        for (int i = 1; i < n; i++) {
            cum[i] = cum[i - 1];
            if (nums[i] % 2 == nums[i - 1] % 2) {
                cum[i]++;
            }
        }

        for (int i = 0; i < queries.length; i++) {
            int[] query = queries[i];
            int x = query[0];
            int y = query[1];
            int falseCount = cum[y] - (x > 0 ? cum[x] : 0);
            if (falseCount <= 0 || x == y) {
                ans[i] = true;
            }
        }

        return ans;
    }

}
```

504. `Solution03163.java`

```
package leetcode;

/**
 * 3163. String Compression III
 * <a href="https://leetcode.com/problems/string-compression-iii">Web link</a>
 * Medium
 */

public class Solution03163 {

    public String compressedString(String word) {
        StringBuilder sb = new StringBuilder();
        int strLen = word.length();
        char lastChar = word.charAt(0);
        int lastCharCount = 1;
        for (int pos = 1; pos < strLen; pos++) {
            char currentChar = word.charAt(pos);
            if (currentChar == lastChar && lastCharCount < 9) {
                lastCharCount++;
            } else {
                sb.append(lastCharCount).append(lastChar);
                lastChar = currentChar;
                lastCharCount = 1;
            }
        }
        sb.append(lastCharCount).append(lastChar);
        return sb.toString();
    }

}
```

505. `Solution03203.java`

```
/**
 * 3203. Find Minimum Diameter After Merging Two Trees
 * <a href="https://leetcode.com/problems/find-minimum-diameter-after-merging-two-
trees">Web link</a>
 * Hard
 */

package leetcode;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.List;
import java.util.Queue;

public class Solution03203 {

    public int minimumDiameterAfterMerge(int[][] edges1, int[][] edges2) {
        int d1 = calculateDiameter(edges1);
        int d2 = calculateDiameter(edges2);


        return Math.max(d1, Math.max(d2, (d1 + 1) / 2 + (d2 + 1) / 2 + 1));
    }

    private int calculateDiameter(int[][] edges) {
        int n = edges.length + 1;
        int[] degree = new int[n];
        List<List<Integer>> adj = new ArrayList<>();

        // initialize adjacency list
        for (int i = 0; i < n; i++) {
            adj.add(new ArrayList<>());
        }

        //preparing edges
        for (int[] edge : edges) {
            int u = edge[0], v = edge[1];
            adj.get(u).add(v);
            adj.get(v).add(u);
            degree[u]++;
            degree[v]++;
        }
        //ye graph nhi tree h toh 1 is the minimum
        Queue<Integer> queue = new ArrayDeque<>();
        for (int i = 0; i < n; i++) {
            if (degree[i] == 1) {
                queue.offer(i);
            }
        }

        //main use of kahns algo yahape
```

```
        int level = 0;
        int remainingNodes = n;

        while (remainingNodes > 2) {
            int size = queue.size();
            remainingNodes -= size;
            level++;

            for (int i = 0; i < size; i++) {
                int node = queue.poll();
                for (int neighbor : adj.get(node)) {
                    if (--degree[neighbor] == 1) {
                        queue.offer(neighbor);
                    }
                }
            }
        }

        return (remainingNodes == 2) ? 2 * level + 1 : 2 * level;
    }

}
```

506. `Solution03217.java`

```
package leetcode;

import leetcode.common.ListNode;

import java.util.HashSet;
import java.util.Set;

/**
 * 3217. Delete Nodes From Linked List Present in Array
 * <a href="https://leetcode.com/problems/delete-nodes-from-linked-list-present-in-
array">Web link</a>
 * Medium
 */

public class Solution03217 {

    public ListNode modifiedList(int[] nums, ListNode head) {
        Set<Integer> numsSet = new HashSet<>(nums.length);
        for (int num : nums) {
            numsSet.add(num);
        }
        ListNode dummy = new ListNode(-1, head);
        ListNode prev = dummy;
        ListNode curr = dummy.next;
        while (curr != null) {
            if (numsSet.contains(curr.val)) {
                prev.next = curr.next;
            } else {
                prev = curr;
            }
            curr = curr.next;
        }
        return dummy.next;
    }

}
```

507. `Solution03223.java`

```
/**
 * 3223. Minimum Length of String After Operations
 * <a href="https://leetcode.com/problems/minimum-length-of-string-after-operations">Web
link</a>
 * Medium
 */

package leetcode;

public class Solution03223 {

    public int minimumLength(String s) {
        int n = s.length();
        int removableLen = 0;
        int[] freq = new int[26];

        for (char c : s.toCharArray()) {
            freq[c - 'a']++;
        }

        for (int frequency : freq) {
            if (frequency % 2 != 0) {
                removableLen += frequency - 1;
            } else if (frequency != 0) {
                removableLen += frequency - 2;
            }
        }

        return n - removableLen;
    }

}
```

508. `Solution03243.java`

```
/**
 * 3243. Shortest Distance After Road Addition Queries I
 * <a href="https://leetcode.com/problems/shortest-distance-after-road-addition-queries-
i">Web link</a>
 * Medium
 */

package leetcode;

import java.util.ArrayList;
import java.util.List;

public class Solution03243 {

    public int[] shortestDistanceAfterQueries(int n, int[][] queries) {
        int[] distances = new int[n];
        for (int i = 0; i < n; ++i) {
            distances[i] = n - 1 - i;
        }

        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i < n; ++i) {
            graph.add(new ArrayList<>());
        }

        for (int i = 0; i + 1 < n; ++i) {
            graph.get(i + 1).add(i);
        }

        int[] answer = new int[queries.length];
        int queryIdx = 0;

        for (int[] query : queries) {
            int source = query[0];
            int target = query[1];

            graph.get(target).add(source);
            distances[source] = Math.min(distances[source], distances[target] + 1);
            updateDistances(graph, source, distances);

            answer[queryIdx++] = distances[0];
        }

        return answer;
    }

    private void updateDistances(List<List<Integer>> graph, int current, int[] distances)
{
        int newDist = distances[current] + 1;

        for (int neighbor : graph.get(current)) {
```

```
            if (distances[neighbor] <= newDist) continue;

            distances[neighbor] = newDist;
            updateDistances(graph, neighbor, distances);
        }
    }

}
```

509. `Solution03254.java`

```
/**
 * 3254. Find the Power of K-Size Subarrays I
 * <a href="https://leetcode.com/problems/find-the-power-of-k-size-subarrays-i">Web
link</a>
 * Medium
 */

package leetcode;

public class Solution03254 {

    public int[] resultsArray(int[] nums, int k) {
        int count = 0;
        int max = nums[0];
        int resIndex = 0;

        for (int i = 0; i < k - 1; i++) {
            if (nums[i] == nums[i + 1] - 1) {
                max++;
            } else {
                count = i + 1;
                max = nums[i + 1];
            }
        }

        int[] res = new int[nums.length - k + 1];
        if (count > 0) {
            res[resIndex++] = -1;
        } else {
            res[resIndex++] = max;
        }

        for (int i = k; i < nums.length; i++) {
            if (nums[i] == nums[i - 1] + 1) {
                max++;
                count--;
            } else {
                count = k - 1;
                max = nums[i];
            }

            if (count > 0) {
                res[resIndex++] = -1;
            } else {
                res[resIndex++] = max;
            }
        }

        return res;
    }
```

```
}
```

510. `Solution03264.java`

```
package leetcode;

import java.util.Comparator;
import java.util.PriorityQueue;

/**
 * 3264. Final Array State After K Multiplication Operations I
 * <a href="https://leetcode.com/problems/final-array-state-after-k-multiplication-
operations-i">Web link</a>
 * Easy
 */

public class Solution03264 {

    public int[] getFinalState(int[] nums, int k, int multiplier) {
        int n = nums.length;
        final Comparator<int[]> comparator = (a, b) -> {
            if (a[0] == b[0]) {
                return a[1] - b[1];
            }
            return a[0] - b[0];
        };
        PriorityQueue<int[]> pq = new PriorityQueue<>(comparator);
        for (int i = 0; i < n; i++) {
            pq.offer(new int[]{nums[i], i});
        }

        for (int i = 0; i < k; i++) {
            int[] minPair = pq.poll();
            int value = minPair[0];
            int index = minPair[1];

            nums[index] = value * multiplier;

            pq.offer(new int[]{nums[index], index});
        }

        return nums;
    }

}
```